Project Euler 1–255
Algorithms and Cross-Language Implementations

Source-based technical commentary

November 2026

Abstract

This document covers Project Euler problems 1 through 255. The original explanations and assessments describe historical C attempts, including unfinished programs and defects; their source links point to the preserved reorganization branch. They must not be mistaken for assessments of the subsequently corrected implementations.

Reading the corrections: Red, explicitly labelled sections explain the repaired or completed C implementation. The main C files now own their algorithms and call reusable mathematical helpers in `support/c/euler_common.c`. Historical programs remain under `solutions/c/variants/historical/`. Problems 13 and 17 have new C implementations as well as their preserved earlier analyses.

Current answer coverage contains 255 C programs, 255 Rust programs, 120 Gforth programs, and 191 Factor programs. Passing the fixed reference inputs is evidence of correctness for those inputs, not a proof of a general-purpose algorithm. The 120-entry Gforth run recorded in `docs/VALIDATION_FORTH_1_120.md` passes 115 entries with a 60-second limit per solver. The remaining five are timeouts; the run contains no wrong answers or runtime errors.

Contents

1 Multiples of 3 or 5 — Problem 1 **12**

2 Even Fibonacci Numbers — Problem 2 **13**

3 Largest Prime Factor — Problem 3 **14**

4 Largest Palindrome Product — Problem 4 **15**

5 Smallest Multiple — Problem 5 **16**

6 Sum Square Difference — Problem 6 **17**

7 10 001st Prime — Problem 7 **18**

8 Largest Product in a Series — Problem 8 **19**

9 Special Pythagorean Triplet — Problem 9 **20**

10 Summation of Primes — Problem 10 **21**

11 Largest Product in a Grid — Problem 11 **22**

12 Highly Divisible Triangular Number — Problem 12 **23**

13 Large Sum — Problem 13 **24**

14 Longest Collatz Sequence — Problem 14 **25**

15 Power Digit Sum — Problem 16 **26**

16 Number Letter Counts — Problem 17 **27**

17 Counting Sundays — Problem 19 **28**

18 Factorial Digit Sum — Problem 20 **29**

19 Amicable Numbers — Problem 21 **30**

20 Names Scores — Problem 22 **31**

21 1000-digit Fibonacci Number — Problem 25 **32**

22 Quadratic Primes — Problem 27 **33**

23 Number Spiral Diagonals — Problem 28 **34**

24 Distinct Powers — Problem 29 **35**

25 Digit Fifth Powers — Problem 30 **36**

26 Digit Factorials — Problem 34 **37**

27 Circular Primes — Problem 35 **38**

28 Integer Right Triangles — Problem 39 **39**

29 Champernowne's Constant — Problem 40 **40**

30 Pentagon Numbers — Problem 44 **41**

31 Goldbach's Other Conjecture — Problem 46 **42**

32 Distinct Primes Factors — Problem 47 **43**

33 Self Powers — Problem 48 **44**

34 Consecutive Prime Sum — Problem 50 **45**

35 Lychrel Numbers — Problem 55 **46**

36 Totient Maximum — Problem 69 **47**

37 Square Digit Chains — Problem 92 **48**

38 Cross-language algorithm comparisons **49**

38.1 Problem 9: Special Pythagorean Triplet 49

38.2 Problem 11: Largest Product in a Grid 50

38.3 Problem 28: Number Spiral Diagonals 50

38.4 Problem 39: Integer Right Triangles 51

38.5 Exact arithmetic: Factor versus C and Gforth 52

38.6 Profiling interpretation 52

39 Problem 154: counting divisible coefficients in Pascal's pyramid **54**

39.1 Replace divisibility with two valuation tests 54

39.2 Split one trinomial coefficient into two binomial coefficients 54

39.3 C implementation 54

39.4 Rust implementation 55

39.5 Cross-language comparison and missing ports 55

39.6 Small checks catch symmetry errors 56

40 Compact walkthroughs for problems 1–255 **57**

40.1 Problem 1: Multiples of 3 or 5 57

40.2 Problem 2: Even Fibonacci Numbers 58

40.3 Problem 3: Largest Prime Factor 59

40.4 Problem 4: Largest Palindrome Product 60

40.5 Problem 5: Smallest Multiple 62

40.6 Problem 6: Sum Square Difference 63

40.7 Problem 7: 10 001st Prime 64

40.8 Problem 8: Largest Product in a Series 65

40.9 Problem 9: Special Pythagorean Triplet 66
40.10Problem 10: Summation of Primes 68
40.11Problem 11: Largest Product in a Grid 69
40.12Problem 12: Highly Divisible Triangular Number 70
40.13Problem 13: Large Sum 72
40.14Problem 14: Longest Collatz Sequence 73
40.15Problem 15: Lattice Paths 74
40.16Problem 16: Power Digit Sum 75
40.17Problem 17: Number Letter Counts 76
40.18Problem 18: Maximum Path Sum I 77
40.19Problem 19: Counting Sundays 78
40.20Problem 20: Factorial Digit Sum 80
40.21Problem 21: Amicable Numbers 81
40.22Problem 22: Names Scores 82
40.23Problem 23: Non-Abundant Sums 84
40.24Problem 24: Lexicographic Permutations 85
40.25Problem 25: 1000-digit Fibonacci Number 86
40.26Problem 26: Reciprocal Cycles 88
40.27Problem 27: Quadratic Primes 89
40.28Problem 28: Number Spiral Diagonals 91
40.29Problem 29: Distinct Powers 92
40.30Problem 30: Digit Fifth Powers 93
40.31Problem 31: Coin Sums 94
40.32Problem 32: Pandigital Products 96
40.33Problem 33: Digit Cancelling Fractions 97
40.34Problem 34: Digit Factorials 98
40.35Problem 35: Circular Primes 100
40.36Problem 36: Double-base Palindromes 101
40.37Problem 37: Truncatable Primes 102
40.38Problem 38: Pandigital Multiples 104
40.39Problem 39: Integer Right Triangles 105
40.40Problem 40: Champernowne's Constant 107
40.41Problem 41: Pandigital Prime 108
40.42Problem 42: Coded Triangle Numbers 109

40.43Problem 43: Sub-string Divisibility . . . 110
40.44Problem 44: Pentagon Numbers . . . 112
40.45Problem 45: Triangular, Pentagonal, and Hexagonal . . . 113
40.46Problem 46: Goldbach's Other Conjecture . . . 114
40.47Problem 47: Distinct Primes Factors . . . 116
40.48Problem 48: Self Powers . . . 117
40.49Problem 49: Prime Permutations . . . 118
40.50Problem 50: Consecutive Prime Sum . . . 120
40.51Problem 51: Prime Digit Replacements . . . 121
40.52Problem 52: Permuted Multiples . . . 123
40.53Problem 53: Combinatoric Selections . . . 124
40.54Problem 54: Poker Hands . . . 126
40.55Problem 55: Lychrel Numbers . . . 127
40.56Problem 56: Powerful Digit Sum . . . 128
40.57Problem 57: Square Root Convergents . . . 130
40.58Problem 58: Spiral Primes . . . 131
40.59Problem 59: XOR Decryption . . . 133
40.60Problem 60: Prime Pair Sets . . . 134
40.61Problem 61: Cyclical Figurate Numbers . . . 136
40.62Problem 62: Cubic Permutations . . . 137
40.63Problem 63: Powerful Digit Counts . . . 139
40.64Problem 64: Odd Period Square Roots . . . 140
40.65Problem 65: Convergents of e . . . 141
40.66Problem 66: Diophantine Equation . . . 143
40.67Problem 67: Maximum Path Sum II . . . 144
40.68Problem 68: Magic 5-gon Ring . . . 145
40.69Problem 69: Totient Maximum . . . 147
40.70Problem 70: Totient Permutation . . . 148
40.71Problem 71: Ordered Fractions . . . 149
40.72Problem 72: Counting Fractions . . . 150
40.73Problem 73: Counting Fractions in a Range . . . 152
40.74Problem 74: Digit Factorial Chains . . . 153
40.75Problem 75: Singular Integer Right Triangles . . . 154
40.76Problem 76: Counting Summations . . . 156

40.77 Problem 77: Prime Summations . . . 157
40.78 Problem 78: Coin Partitions . . . 159
40.79 Problem 79: Passcode Derivation . . . 160
40.80 Problem 80: Square Root Digital Expansion . . . 162
40.81 Problem 81: Path Sum: Two Ways . . . 163
40.82 Problem 82: Path Sum: Three Ways . . . 165
40.83 Problem 83: Path Sum: Four Ways . . . 166
40.84 Problem 84: Monopoly Odds . . . 168
40.85 Problem 85: Counting Rectangles . . . 169
40.86 Problem 86: Cuboid Route . . . 171
40.87 Problem 87: Prime Power Triples . . . 172
40.88 Problem 88: Product-sum Numbers . . . 174
40.89 Problem 89: Roman Numerals . . . 175
40.90 Problem 90: Cube Digit Pairs . . . 176
40.91 Problem 91: Right Triangles with Integer Coordinates . . . 178
40.92 Problem 92: Square Digit Chains . . . 179
40.93 Problem 93: Arithmetic Expressions . . . 180
40.94 Problem 94: Almost Equilateral Triangles . . . 182
40.95 Problem 95: Amicable Chains . . . 183
40.96 Problem 96: Su Doku . . . 185
40.97 Problem 97: Large Non-Mersenne Prime . . . 186
40.98 Problem 98: Anagramic Squares . . . 187
40.99 Problem 99: Largest Exponential . . . 189
40.100 Problem 100: Arranged Probability . . . 190
40.101 Problem 101: Optimum Polynomial . . . 191
40.102 Problem 102: Triangle Containment . . . 193
40.103 Problem 103: Special Subset Sums: Optimum . . . 194
40.104 Problem 104: Pandigital Fibonacci Ends . . . 195
40.105 Problem 105: Special Subset Sums: Testing . . . 197
40.106 Problem 106: Special Subset Sums: Meta-testing . . . 198
40.107 Problem 107: Minimal Network . . . 200
40.108 Problem 108: Diophantine Reciprocals I . . . 201
40.109 Problem 109: Darts . . . 202
40.110 Problem 110: Diophantine Reciprocals II . . . 204

40.111 Problem 111: Primes with Runs . . . 205
40.112 Problem 112: Bouncy Numbers . . . 206
40.113 Problem 113: Non-bouncy Numbers . . . 207
40.114 Problem 114: Counting Block Combinations I . . . 209
40.115 Problem 115: Counting Block Combinations II . . . 210
40.116 Problem 116: Red, Green or Blue Tiles . . . 211
40.117 Problem 117: Red, Green, and Blue Tiles . . . 212
40.118 Problem 118: Pandigital Prime Sets . . . 214
40.119 Problem 119: Digit Power Sum . . . 215
40.120 Problem 120: Square Remainders . . . 217
40.121 Problem 121: Disc Game Prize Fund . . . 218
40.122 Problem 122: Efficient Exponentiation . . . 219
40.123 Problem 123: Prime Square Remainders . . . 220
40.124 Problem 124: Ordered Radicals . . . 221
40.125 Problem 125: Palindromic Sums . . . 222
40.126 Problem 126: Cuboid Layers . . . 223
40.127 Problem 127: abc-hits . . . 224
40.128 Problem 128: Hexagonal Tile Differences . . . 225
40.129 Problem 129: Repunit Divisibility . . . 226
40.130 Problem 130: Composites with Prime Repunit Property . . . 227
40.131 Problem 131: Prime Cube Partnership . . . 228
40.132 Problem 132: Large Repunit Factors . . . 229
40.133 Problem 133: Repunit Nonfactors . . . 230
40.134 Problem 134: Prime Pair Connection . . . 231
40.135 Problem 135: Same Differences . . . 232
40.136 Problem 136: Singleton Difference . . . 233
40.137 Problem 137: Fibonacci Golden Nuggets . . . 234
40.138 Problem 138: Special Isosceles Triangles . . . 235
40.139 Problem 139: Pythagorean Tiles . . . 236
40.140 Problem 140: Modified Fibonacci Golden Nuggets . . . 237
40.141 Problem 141: Square Progressive Numbers . . . 238
40.142 Problem 142: Perfect Square Collection . . . 239
40.143 Problem 143: Torricelli Triangles . . . 240
40.144 Problem 144: Laser Beam Reflections . . . 241

40.145 Problem 145: Reversible Numbers . . . 242
40.146 Problem 146: Investigating a Prime Pattern . . . 243
40.147 Problem 147: Rectangles in Cross-hatched Grids . . . 244
40.148 Problem 148: Exploring Pascal's Triangle . . . 245
40.149 Problem 149: Maximum-sum Subsequence . . . 246
40.150 Problem 150: Sub-triangle Sums . . . 247
40.151 Problem 151: A Preference for A5 . . . 248
40.152 Problem 152: Sums of Square Reciprocals . . . 249
40.153 Problem 153: Investigating Gaussian Integers . . . 250
40.154 Problem 154: Exploring Pascal's Pyramid . . . 251
40.155 Problem 155: Counting Capacitor Circuits . . . 252
40.156 Problem 156: Counting Digits . . . 253
40.157 Problem 157: Base-10 Diophantine Reciprocal . . . 254
40.158 Problem 158: Lexicographical Neighbours . . . 255
40.159 Problem 159: Digital Root Sums of Factorisations . . . 256
40.160 Problem 160: Factorial Trailing Digits . . . 257
40.161 Problem 161: Triominoes . . . 258
40.162 Problem 162: Hexadecimal Numbers . . . 259
40.163 Problem 163: Cross-hatched Triangles . . . 260
40.164 Problem 164: Three Consecutive Digital Sum Limit . . . 261
40.165 Problem 165: Intersections . . . 262
40.166 Problem 166: Criss Cross . . . 262
40.167 Problem 167: Investigating Ulam Sequences . . . 263
40.168 Problem 168: Number Rotations . . . 264
40.169 Problem 169: Sums of Powers of Two . . . 265
40.170 Problem 170: Pandigital Concatenating Products . . . 266
40.171 Problem 171: Square Sum of the Digital Squares . . . 267
40.172 Problem 172: Few Repeated Digits . . . 268
40.173 Problem 173: Hollow Square Laminae I . . . 269
40.174 Problem 174: Hollow Square Laminae II . . . 270
40.175 Problem 175: Fractions and Sum of Powers of Two . . . 271
40.176 Problem 176: Common Cathetus Right-angled Triangles . . . 272
40.177 Problem 177: Integer Angled Quadrilaterals . . . 273
40.178 Problem 178: Step Numbers . . . 274

40.179 Problem 179: Consecutive Positive Divisors . . . 275
40.180 Problem 180: Golden Triplets . . . 276
40.181 Problem 181: Grouping Two Different Coloured Objects . . . 277
40.182 Problem 182: RSA Encryption . . . 278
40.183 Problem 183: Maximum Product of Parts . . . 279
40.184 Problem 184: Triangles Containing the Origin . . . 280
40.185 Problem 185: Number Mind . . . 281
40.186 Problem 186: Connectedness of a Network . . . 282
40.187 Problem 187: Semiprimes . . . 283
40.188 Problem 188: Hyperexponentiation . . . 284
40.189 Problem 189: Tri-colouring a Triangular Grid . . . 285
40.190 Problem 190: Maximising a Weighted Product . . . 286
40.191 Problem 191: Prize Strings . . . 287
40.192 Problem 192: Best Approximations . . . 288
40.193 Problem 193: Squarefree Numbers . . . 289
40.194 Problem 194: Coloured Configurations . . . 290
40.195 Problem 195: 60-degree Triangle Inscribed Circles . . . 291
40.196 Problem 196: Prime Triplets . . . 292
40.197 Problem 197: A Recursively Defined Sequence . . . 293
40.198 Problem 198: Ambiguous Numbers . . . 293
40.199 Problem 199: Iterative Circle Packing . . . 294
40.200 Problem 200: Prime-proof Squbes . . . 295
40.201 Problem 201: Subsets with a Unique Sum . . . 296
40.202 Problem 202: Laserbeam . . . 297
40.203 Problem 203: Squarefree Binomial Coefficients . . . 298
40.204 Problem 204: Generalised Hamming Numbers . . . 299
40.205 Problem 205: Dice Game . . . 300
40.206 Problem 206: Concealed Square . . . 301
40.207 Problem 207: Integer Partition Equations . . . 302
40.208 Problem 208: Robot Walks . . . 303
40.209 Problem 209: Circular Logic . . . 304
40.210 Problem 210: Obtuse Angled Triangles . . . 306
40.211 Problem 211: Divisor Square Sum . . . 306
40.212 Problem 212: Combined Volume of Cuboids . . . 307

40.213 Problem 213: Flea Circus . . . 308
40.214 Problem 214: Totient Chains . . . 309
40.215 Problem 215: Crack-free Walls . . . 310
40.216 Problem 216: The Primality of 2n^2 - 1 . . . 311
40.217 Problem 217: Balanced Numbers . . . 311
40.218 Problem 218: Perfect Right-angled Triangles . . . 313
40.219 Problem 219: Skew-cost Coding . . . 314
40.220 Problem 220: Heighway Dragon . . . 315
40.221 Problem 221: Alexandrian Integers . . . 316
40.222 Problem 222: Sphere Packing . . . 316
40.223 Problem 223: Almost Right-angled Triangles I . . . 317
40.224 Problem 224: Almost Right-angled Triangles II . . . 318
40.225 Problem 225: Tribonacci Non-divisors . . . 319
40.226 Problem 226: A Scoop of Blancmange . . . 320
40.227 Problem 227: The Chase . . . 321
40.228 Problem 228: Minkowski Sums . . . 322
40.229 Problem 229: Four Representations Using Squares . . . 323
40.230 Problem 230: Fibonacci Words . . . 324
40.231 Problem 231: Prime Factorisation of Binomial Coefficients . . . 325
40.232 Problem 232: The Race . . . 326
40.233 Problem 233: Lattice Points on a Circle . . . 327
40.234 Problem 234: Semidivisible Numbers . . . 328
40.235 Problem 235: An Arithmetic Geometric Sequence . . . 329
40.236 Problem 236: Luxury Hampers . . . 330
40.237 Problem 237: Tours on a 4 \times N Playing Board . . . 331
40.238 Problem 238: Infinite String Tour . . . 332
40.239 Problem 239: Twenty-two Foolish Primes . . . 333
40.240 Problem 240: Top Dice . . . 334
40.241 Problem 241: Perfection Quotients . . . 335
40.242 Problem 242: Odd Triplets . . . 336
40.243 Problem 243: Resilience . . . 337
40.244 Problem 244: Sliders . . . 338
40.245 Problem 245: Coresilience . . . 339
40.246 Problem 246: Tangents to an Ellipse . . . 340

40.247 Problem 247: Squares Under a Hyperbola . . . 340
40.248 Problem 248: Euler's Totient Function Equals 13! . . . 341
40.249 Problem 249: Prime Subset Sums . . . 342
40.250 Problem 250: 250250 . . . 343
40.251 Problem 251: Cardano Triplets . . . 344
40.252 Problem 252: Convex Holes . . . 345
40.253 Problem 253: Tidying Up A . . . 346
40.254 Problem 254: Sums of Digit Factorials . . . 347
40.255 Problem 255: Rounded Square Roots . . . 348

41 Mathematical visualization atlas **350**

41.1 Integer geometry and spirals . . . 350
41.2 Grids and dynamic programming . . . 350
41.3 Discrete counting and networks . . . 350
41.4 Pascal's pyramid . . . 350

A Shared Project Euler library API **362**

A.1 C API . . . 362
A.2 Rust API . . . 363
A.3 Gforth API . . . 364
A.4 Factor API . . . 367

1 Multiples of 3 or 5 — Problem 1

Source: `https://github.com/KOOI/projectEuler/blob/reorganize/solutions-by-language/solutions/c/multiples_of_3_or_5_1.c`

Assessment: Correct for the specified bound.

Mathematical goal

The problem asks for the sum of integers below 1000 divisible by three or five:

$$S = \sum_{n=1}^{999} n \, \mathbf{1}_{(3|n)\vee(5|n)}.$$

The disjunction counts multiples of both divisors once.

How the C code works

The loop variable `i` runs from 0 to 999. The remainder tests `i%3==0` and `i%5==0` select the eligible integers. The logical OR combines the conditions; the accumulator `sum` starts at zero and receives each selected integer exactly once. The result is printed with `printf`.

Correctness and limitations

The code is correct for the stated bound and produces `...3168`. Including zero has no effect. The procedure takes $O(N)$ time and $O(1)$ auxiliary space. Inclusion–exclusion would be faster for very large limits, but it is not used here.

Correction to historical C — Problem 1

Correction: The existing direct enumeration already expressed the desired disjunction correctly. The corrected code retains this method and emits only the numeric answer.

2 Even Fibonacci Numbers — Problem 2

Source: `https://github.com/KOOI/projectEuler/blob/reorganize/solutions-by-language/solutions/c/even_fibonacci_numbers_2.c`

Assessment: Undefined behavior.

Mathematical goal

The Fibonacci recurrence is $F_n = F_{n-1} + F_{n-2}$. The required sum includes even terms at most four million:

$$S = \sum_{\substack{F_n \le 4\,000\,000 \\ 2 \mid F_n}} F_n.$$

How the C code works

The program allocates `fib[33]`, initializes the first two entries to one and fills the array using `fib[i-2]+fib[i-1]`. A second loop checks evenness with `fib[k]%2==0`, accumulating qualifying entries in `sum`. This separates generating the sequence from filtering it.

Correctness and limitations

The declared `unsigned long sum` is never initialized before `sum += fib[k]`, so the behavior is undefined and the output cannot be trusted. It should start at zero. A value-bound loop would also be clearer than a fixed term count. The intended mathematical result is `...3732`, not a reliable result of this source.

Correction to historical C — Problem 2

Correction: The original accumulator was never initialized. The corrected routine begins with zero and generates Fibonacci terms until their value exceeds four million, rather than trusting a fixed array length.

3 Largest Prime Factor — Problem 3

Source: `https://github.com/KOOI/projectEuler/blob/reorganize/solutions-by-language/solutions/c/largest_prime_factor_3.c`

Assessment: Core method implemented.

Mathematical goal

The goal is to determine the largest prime factor of 600851475143. Removing factors one by one isolates the greatest factor remaining at the end.

How the C code works

The function `factorize` starts with candidate divisor `d=2`. If `n%d==0`, it divides `n` by `d` and tries the same divisor again, removing repeated factors. Otherwise it moves to 3 and then successive odd candidates. Once the loop condition `d<n` becomes false, it prints `d`.

Correctness and limitations

This factor-stripping approach yields the expected factor `...6857` for the supplied value. `unsigned long` needs to be at least 64 bits to represent the input portably. Testing candidates only up to the square root of the remaining number could improve efficiency.

Correction to historical C — Problem 3

Correction: The corrected trial-division routine uses a 64-bit integer and stops factoring once a candidate divisor exceeds the square root of the remaining number.

4 Largest Palindrome Product — Problem 4

Source: `https://github.com/KOOI/projectEuler/blob/reorganize/solutions-by-language/solutions/c/largest_palindrome_product_4.c`

Assessment: Dependent on generated input.

Mathematical goal

Find the maximum ab for three-digit integers a, b such that the decimal digits of ab form a palindrome.

How the C code works

The program opens `palindrome`, reads each candidate number into `temp`, and tests `a` from 100 to 999. If `temp%a==0`, it tests `b` in the same range and checks `a*b==temp`. A successful product greater than the running `max` replaces it. The CPU time is measured using `clock`.

Correctness and limitations

The source assumes the input file contains a complete, correctly separated list of palindromes. The historical palindrome generator does not reliably separate output records, so the data path must be verified. The calculated quotient `q=temp/a` is unused; checking whether `q` is a three-digit integer would remove the inner loop. The expected maximum is `...6609`, but it is not verified by the existing file workflow.

Correction to historical C — Problem 4

Correction: The new implementation enumerates products of three-digit integers and tests each product as a palindrome. It no longer relies on the historical palindrome input file or an unused second factor search.

5 Smallest Multiple — Problem 5

Source: `https://github.com/KOOI/projectEuler/blob/reorganize/solutions-by-language/solutions/c/smallest_multiple_5.c`

Assessment: Logically valid but extremely slow.

Mathematical goal

The desired integer is $\text{lcm}(1, 2, \ldots, 20)$: the least positive value divisible by every integer from 1 to 20.

How the C code works

The variable `mult` begins at one. The `for` loop tests `mult%i` for each divisor up to `limit=20`. Whenever a test fails, `mult` increases and the divisor index is reset so that the new candidate is checked again. After all divisibility tests pass, the code prints the candidate.

Correctness and limitations

The first qualifying candidate is `...2560`, but checking essentially every preceding integer is computationally expensive. A much faster implementation would multiply the highest prime powers not exceeding 20. The surrounding `done` loop is not needed to express the search.

Correction to historical C — Problem 5

Correction: Unbounded candidate-by-candidate searching is replaced by successive least common multiples computed using the Euclidean greatest common divisor.

6 Sum Square Difference — Problem 6

Source: `https://github.com/KOOI/projectEuler/blob/reorganize/solutions-by-language/solutions/c/sum_square_difference_6.c`

Assessment: Correct for the input.

Mathematical goal

For $N = 100$ compute

$$\left(\sum_{k=1}^{N} k\right)^2 - \sum_{k=1}^{N} k^2.$$

How the C code works

The helper `squareSum` adds the integers from zero through N before squaring the accumulated total. The helper `sumSquare` squares each integer individually and adds the squares. `main` subtracts the results and prints the intermediate quantities and difference.

Correctness and limitations

The algorithm computes `...4150` for $N = 100$. It takes $O(N)$ time and constant extra space. Closed-form expressions for the two sums would make the calculation $O(1)$, and wider integer types would allow larger N.

Correction to historical C — Problem 6

Correction: The corrected code evaluates the exact closed forms for the ordinary sum and the sum of squares. The original pair of accumulation loops was also mathematically sound for the specified input.

7 10 001st Prime — Problem 7

Source: `https://github.com/KOOI/projectEuler/blob/reorganize/solutions-by-language/solutions/c/10_001st_prime_7.c`

Assessment: Does not reliably solve the named problem.

Mathematical goal

The aim is to identify p_{10001}, the prime in position 10001 of the ordered sequence $2, 3, 5, 7, \ldots$.

How the C code works

The program seeds a table with 2 and 3, examines successive odd candidates using trial division up to a rounded square root, and writes accepted candidates to a file named `primes`. Comments discuss using the table for other Euler problems as well.

Correctness and limitations

The hard-coded `countDone=78497` is unrelated to the 10001st prime, and the desired table entry is never printed. Both the generation loop and the output loop access index `countDone` in an array of that length, which is out of bounds. Candidate-reset behavior during trial division also needs correction. The expected prime is `...4743`; this file does not establish it.

Correction to historical C — Problem 7

Correction: The array bound overflow and unrelated prime-count target are removed. The corrected search counts primes until the 10001st candidate is found.

8 Largest Product in a Series — Problem 8

Source: `https://github.com/KOOI/projectEuler/blob/reorganize/solutions-by-language/solutions/c/largest_product_in_a_series_8.c`

Assessment: Unimplemented.

Mathematical goal

Given the decimal digits d_i, find a window of the prescribed length w maximizing $\prod_{j=0}^{w-1} d_{i+j}$.

How the C code works

The main file contains only two header inclusions. It defines no input array, search loop, product calculation, or `main` function. A historical variant in `solutions/c/variants/code/` records manual observations about products of large digits, but does not implement the algorithm.

Correctness and limitations

This is a placeholder rather than a runnable solution. A complete implementation would scan every length-w window, compute the product, and maintain the maximum. A rolling product is possible with special handling for zero digits.

Correction to historical C — Problem 8

Correction: The historical main C file was only a stub. The completed algorithm examines every 13-digit window of the exact 1000-digit input and retains the largest product.

9 Special Pythagorean Triplet — Problem 9

Source: `https://github.com/KOOI/projectEuler/blob/reorganize/solutions-by-language/solutions/c/special_pythagorean_triplet_9.c`

Assessment: Search implemented.

Mathematical goal

Find positive integers $a < b < c$ satisfying $a^2 + b^2 = c^2$ and $a + b + c = 1000$, then calculate abc.

How the C code works

The outer loop enumerates `a` and the inner loop enumerates `b`. Rather than search all three values independently, it computes `c=sum-a-b`, making the perimeter constraint automatic. It tests `a*a+b*b==c*c`. A match sets `found` and prints the triplet and product.

Correctness and limitations

The search finds $(a, b, c) = (200, 375, 425)$, giving `...5000`. The `break` exits only the inner loop, so the outer loop continues until its bound before the enclosing `while` stops. A no-solution case would repeat forever because there is no explicit failure exit.

Correction to historical C — Problem 9

Correction: The perimeter-constrained nested search is retained, but the corrected routine terminates immediately after finding the valid triple.

10 Summation of Primes — Problem 10

Source: `https://github.com/KOOI/projectEuler/blob/reorganize/solutions-by-language/solutions/c/summation_of_primes_10.c`

Assessment: Computes a different sum.

Mathematical goal

The problem asks for the sum of primes strictly below two million:

$$S = \sum_{\substack{p < 2\,000\,000 \\ p \text{ prime}}} p.$$

How the C code works

The program reads an integer `n` from its command-line argument and loops from `i=1` through `n`, adding each integer to `sum`. It thus computes the arithmetic series $n(n+1)/2$. There is no primality test.

Correctness and limitations

This C program does not solve the prime-summation problem. Using `int` also overflows for the intended bound. A sieve of Eratosthenes combined with a 64-bit accumulator would solve the actual problem. The mathematical answer is `...8922`, not the sum produced by the present algorithm.

Correction to historical C — Problem 10

Correction: The old program summed all integers, not primes, and used an undersized accumulator. The corrected implementation uses a sieve of Eratosthenes and a 64-bit prime sum.

11 Largest Product in a Grid — Problem 11

Source: `https://github.com/KOOI/projectEuler/blob/reorganize/solutions-by-language/solutions/c/largest_product_in_a_grid_11.c`

Assessment: Undefined indexing in diagonal loops.

Mathematical goal

Find the largest product of four adjacent entries along any row, column, or diagonal in the fixed 20×20 grid.

How the C code works

The code stores the grid as `long grid[20][20]` and makes four directional sweeps. For each starting location it multiplies four entries and compares the result with `max`. The horizontal and vertical sweeps use appropriate bounds for their directions.

Correctness and limitations

Both diagonal expressions contain `j-1`, `j-2`, and `j-3` even when `j` begins at zero, producing negative indices and undefined behavior. The diagonal starting-column bounds must be repaired. The unused macro `#define N = 4;` is not a valid numeric definition for use in expressions. The known maximum is `...0674` but is not safely demonstrated by this source.

Correction to historical C — Problem 11

Correction: The old diagonal sweeps accessed negative column indices. The corrected implementation checks all four directions with explicit endpoint bounds before multiplying grid entries.

12 Highly Divisible Triangular Number — Problem 12

Source: `https://github.com/KOOI/projectEuler/blob/reorganize/solutions-by-language/solutions/c/highly_divisible_triangular_number_12.c`

Assessment: Partial: triangular numbers only.

Mathematical goal

A triangular number is $T_n = n(n+1)/2$. The challenge seeks the first one with more than 500 positive divisors.

How the C code works

Starting from `tri=1`, the loop adds every integer from 2 through 500. The printed value is $T_{500} = 125250$. The variable `j` is incremented but has no effect on the result.

Correctness and limitations

The source never counts the divisors of a triangular number and never tests a divisor-count threshold, so it does not solve the full problem. Efficient factorization of n and $n + 1$, with their known coprimality, can support a complete divisor-count search.

Correction to historical C — Problem 12

Correction: The original source only computed the 500th triangular number. The corrected search generates successive triangular numbers and explicitly counts their positive divisors until the threshold is exceeded.

13 Large Sum — Problem 13

Source: `https://github.com/KOOI/projectEuler/blob/feature/euler-c-rust-forth/solutions/c/large_sum_13.c`

Mathematical goal

Sum the one hundred provided 50-digit integers, then report the first ten decimal digits of their exact sum.

Historical approach

The original repository contains a short shell script attempting to sum file rows using `awk`. Floating-point addition cannot reliably retain every digit of this sum.

Corrected C implementation

Correction: The C solution reads each value from the versioned `lsum` input, accumulates it exactly with GNU MP arbitrary-precision integers, and extracts the leading ten decimal digits. The historical shell attempt remains preserved.

Validation

The C, Rust and Forth implementations all returned `...6230` for the repository's original 100-number input in the complete CI validation baseline.

14 Longest Collatz Sequence — Problem 14

Source: `https://github.com/KOOI/projectEuler/blob/reorganize/solutions-by-language/solutions/c/longest_collatz_sequence_14.c`

Assessment: Search implemented; reporting format unsafe.

Mathematical goal

For every starting integer $2 \leq n < 10^6$, iterate $n \mapsto n/2$ when even, or $n \mapsto 3n + 1$ when odd. Find the start whose chain reaches 1 after the greatest number of terms.

How the C code works

The helper `collatz` counts the current number as the first term and advances until reaching 1. `main` compares every length with `max` and records the corresponding start in `maxi`. It also prints each candidate's length.

Correctness and limitations

The implementation redundantly calls `collatz(i)` up to three times. The final `printf` uses `%i` for `unsigned long long` values, causing undefined behavior; use `%llu`. Intermediate values can exceed the starting limit and may overflow. The expected starting value is `...7799`, but the final formatting must be repaired.

Correction to historical C — Problem 14

Correction: The corrected implementation calculates each chain length only once per starting value and returns the winning start without the original mismatched printf conversion specifiers.

15 Power Digit Sum — Problem 16

Source: `https://github.com/KOOI/projectEuler/blob/reorganize/solutions-by-language/solutions/c/power_digit_sum_16.c`

Assessment: GMP-based algorithm implemented.

Mathematical goal

Compute the sum of the base-ten digits of 2^{1000}. GNU MP arbitrary-precision integers allow the power to be represented exactly.

How the C code works

The call `mpz_pow_ui` computes 2^{1000}. At each iteration the code calculates `giant mod a` where `a` runs through powers of ten, then divides by `a/10` to isolate the corresponding digit. The digit is added into the GMP variable `result`, which is initialized to zero by `mpz_init`.

Correctness and limitations

The 305 iterations exceed the 302-digit input, so later positions contribute zero. The expected sum is `...1366`. Compilation requires GMP headers and `-lgmp`. A decimal-string conversion would be simpler, but it is not the method implemented here.

Correction to historical C — Problem 16

Correction: GNU MP is retained for exact exponentiation, but the corrected code converts the resulting integer to decimal text and sums digit values directly.

16 Number Letter Counts — Problem 17

Source: `https://github.com/KOOI/projectEuler/blob/feature/euler-c-rust-forth/solutions/c/number_letter_counts_17.c`

Mathematical goal

For integers 1 through 1000 inclusive, count letters in their British English names, excluding spaces and hyphens; non-round hundreds use the word *and*.

Historical approach

The original LaTeX analysis at `solutions/latex/number_letter_counts_17.tex` counts recurring words mathematically. There was no corresponding main C implementation.

Corrected C implementation

Correction: The new C solution tabulates the lengths of words for one through nineteen and the multiples of ten, adds seven letters for *hundred* and three for *and* where applicable, handles *one thousand* separately, and sums across all 1000 numbers.

Validation

The corrected C code matched the expected value `...1124` in local testing.

17 Counting Sundays — Problem 19

Source: `https://github.com/KOOI/projectEuler/blob/reorganize/solutions-by-language/solutions/c/counting_sundays_19.c`

Assessment: Calendar simulation implemented.

Mathematical goal

Count months whose first day is Sunday between January 1901 and December 2000, inclusive.

How the C code works

The global state tracks year, month `j`, weekday `i`, day of month and `score`. It starts on Tuesday, January 1, 1901. `nextDay` cycles through weekdays, and `nextMonth` advances to the first of the next month and increments `score` when that day is Sunday. The main loop handles February and months with 30 or 31 days.

Correctness and limitations

The test `year%4==0` omits the full Gregorian century rule, but covers leap years in the specified 1901–2000 interval, including 2000. Formal function arguments are unused, and the stopping boundary deserves tests. The intended answer is `...171`.

Correction to historical C — Problem 19

Correction: The corrected calendar traversal uses the full Gregorian leap-year rule and counts Sundays at the start of each month only within the requested date interval.

18 Factorial Digit Sum — Problem 20

Source: `https://github.com/KOOI/projectEuler/blob/reorganize/solutions-by-language/solutions/c/factorial_digit_sum_20.c`

Assessment: GMP digit sum implemented; output label wrong.

Mathematical goal

Find the sum of the decimal digits of 100!.

How the C code works

The call `mpz_fac_ui(factorial,100)` constructs the factorial exactly. A modulus with powers of ten retains the lowest digits; division by the next lower power isolates one digit. Each digit is added to `result`, an arbitrary-precision accumulator.

Correctness and limitations

The loop runs beyond the decimal length, but the surplus digits are zero. The expected sum is `...648`. The output message incorrectly describes digits of 2^{100}, copied from problem 16; it should name 100!. Compiling requires GMP and `-lgmp`.

Correction to historical C — Problem 20

Correction: GNU MP still computes 100 factorial exactly. The corrected digit-sum routine eliminates the copied message that incorrectly described a power of two.

19 Amicable Numbers — Problem 21

Source: `https://github.com/KOOI/projectEuler/blob/reorganize/solutions-by-language/solutions/c/amicable_numbers_21.c`

Assessment: Pair-validation logic needs correction.

Mathematical goal

Let $d(n)$ denote the sum of positive proper divisors of n. An amicable pair has $d(a) = b$, $d(b) = a$, and $a \neq b$. Sum the amicable numbers under 10000.

How the C code works

The helper `d` tries divisors from 2 through $n - 1$, beginning its sum at one. The `amicable` loop computes `a=d(j)` and `b=d(a)` for each candidate `j`, then adds `a+b` when its tests pass and the total differs from the previous pair sum.

Correctness and limitations

The condition `d(a)==b` is true by construction, but the source never requires `b==j`. The candidate seed therefore need not belong to the pair being added. `prevsum` only filters consecutive duplicates, and partner bounds are not enforced. The correct total is `...1626`; the code needs stronger pair checks before claiming it.

Correction to historical C — Problem 21

Correction: Each candidate below the requested limit is counted only when it has a distinct proper-divisor-sum partner that maps back to the candidate. The earlier consecutive-sum deduplication is removed.

20 Names Scores — Problem 22

Source: `https://github.com/KOOI/projectEuler/blob/reorganize/solutions-by-language/solutions/c/names_scores_22.c`

Historical assessment: Full workflow attempted; parser fragile.

Mathematical goal

Alphabetize the names in the input file, sum each name's letter values $A = 1, \ldots, Z = 26$, multiply by its one-based sorted position, and sum all the scores.

How the historical C code works

The file `names.txt` is parsed into a fixed array of 5163 names. Several adjacent-swap sorting passes examine successive character positions. After sorting, uppercase ASCII letters are valued by subtracting 64, each name score is multiplied by `j+1`, and the grand total is accumulated.

Historical correctness and limitations

Parsing by byte positions and fixed name lengths is fragile; the multi-pass sort should be replaced by lexicographic comparison with `strcmp` and `qsort`. On input failure the old code calls `fclose` on a null pointer. The expected score is `...8282`, but the old program should not be assumed correct merely because the reference answer is known.

Correction to historical C — Problem 22

Correction: The fixed-pass string sort and fragile byte seeking are replaced with bounded parsing of the quoted data, standard lexicographic sorting, and positional scoring. The corrected C implementation, independent Rust implementation and Forth implementation each passed the expected answer `...8282` using the original repository dataset of 5163 names.

21 1000-digit Fibonacci Number — Problem 25

Source: `https://github.com/KOOI/projectEuler/blob/reorganize/solutions-by-language/solutions/c/1000_digit_fibonacci_number_25.c`

Assessment: GMP search with indexing and portability defects.

Mathematical goal

Find the first Fibonacci index n with $F_n \geq 10^{999}$, which is equivalent to reaching 1000 decimal digits.

How the C code works

GMP integers `a` and `b` start at one. The nested helper `fibonacci` adds these numbers and swaps them to advance the sequence. The threshold `top=10^999` is constructed with `mpz_ui_pow_ui`. A quotient `lentest=a/top` is compared with one to detect the first term above the threshold. A counter `i` increments for each generated term.

Correctness and limitations

A nested function is a GNU C extension, not ISO C. Because the initial two sequence terms are already present, the counter starting at zero does not equal the standard Fibonacci index. Several `printf` calls use `%i` for `unsigned long` values. The memory cleanup is placed after an unreachable `return`. The correct index is `...4782`; the program's counter must be adjusted before treating it as that answer.

Correction to historical C

Correction: The corrected GNU MP recurrence advances the standard Fibonacci index until the value has 1000 digits. It removes the historical indexing and unreachable cleanup errors.

22 Quadratic Primes — Problem 27

Source: `https://github.com/KOOI/projectEuler/blob/reorganize/solutions-by-language/solutions/c/quadratic_primes_27.c`

Assessment: Exhaustive search with primality edge cases.

Mathematical goal

Find coefficients a, b with $|a| < 1000$, $|b| \leq 1000$, maximizing the length of the prime-producing run of $f(n) = n^2 + an + b$ for $n = 0, 1, \ldots$. Report ab.

How the C code works

Two nested loops enumerate coefficient pairs. The helper `ipow` computes the square, and `isPrime` decides whether each evaluated polynomial value continues the run. The `score` increases while values appear prime. The first nonprime ends the run, updates `highscore` and the best coefficient pair if needed, and resets `n` and `score`.

Correctness and limitations

The local primality checker misclassifies 2 as nonprime, because its divisor loop does not execute and its default return value denotes nonprime. It calls `sqrt` before handling negatives. The algorithm also explores many coefficients with b nonprime, although $b = f(0)$ must be prime for a nonempty run. The expected coefficient product is `...9231`; it should be verified only after repairing the primality test.

Correction to historical C

Correction: The primality test now correctly accepts two and excludes negative and nonprime values, including zero and one. The coefficient search starts at n equals zero.

23 Number Spiral Diagonals — Problem 28

Source: `https://github.com/KOOI/projectEuler/blob/reorganize/solutions-by-language/solutions/c/number_spiral_diagonals_28.c`

Assessment: Closed-form idea, but no accumulated sum.

Mathematical goal

Determine the total of both diagonals in a 1001×1001 clockwise number spiral. Each successive odd-width ring contributes four corner values.

How the C code works

Four functions, `A` through `D`, attempt to express the corner sequences algebraically. `main` chooses a width `n=5`, calls the four functions, adds the central one, and prints the result, thereby avoiding a full matrix representation.

Correctness and limitations

Every helper overwrites its global `sumA`, `sumB`, `sumC` or `sumD` on each loop iteration instead of adding a sequence of terms. The width is five, not 1001. The print statement uses integer `%ld` conversions with `long double` arguments, which is undefined behavior. The expected total is `...1001`, but this source does not compute it.

Correction to historical C

Correction: All four corners of every odd-width ring are accumulated through width 1001. The old code only retained the last term and used invalid floating-point print conversions.

24 Distinct Powers — Problem 29

Source: `https://github.com/KOOI/projectEuler/blob/reorganize/solutions-by-language/solutions/c/distinct_powers_29.c`

Assessment: Diagnostic power table, not uniqueness count.

Mathematical goal

Count the distinct integers among a^b with $2 \leq a \leq 100$ and $2 \leq b \leq 100$. Different pairs can produce the same number, for example $4^3 = 8^2$.

How the C code works

The helper `ipow` uses exponentiation by squaring: it multiplies the accumulated result when an exponent bit is set, shifts the exponent, and squares the base. `main` prints a small table for bases and exponents 2 through 6. This demonstrates power generation but not deduplication.

Correctness and limitations

No distinct-values set or counter is present. A signed `int` cannot represent general powers up to 100^{100}, though the small demonstration fits. A corrected algorithm could normalize bases to primitive powers and count exponent combinations, or use GMP values and a set. The expected distinct count is `...9183`, not an output of the existing program.

Correction to historical C

Correction: The former sample table is replaced by exact GNU MP powers that are sorted and deduplicated to count distinct results.

25 Digit Fifth Powers — Problem 30

Source: `https://github.com/KOOI/projectEuler/blob/reorganize/solutions-by-language/solutions/c/digit_fifth_powers_30.c`

Assessment: Direct exhaustive solution.

Mathematical goal

Find nontrivial positive integers whose value equals the sum of the fifth powers of their decimal digits, then sum those integers.

How the C code works

The helper `powi` evaluates integer powers by repeated multiplication. For each candidate `j` from 2 to one million, seven expressions using division and remainder extract decimal digits from units through millions. Their fifth powers are summed and compared to `j`. Matching candidates are printed and added to `answer`.

Correctness and limitations

This search covers all candidates that can meet the condition, and leading zero digits contribute zero. The expected sum is `...3839`. The upper bound is conservative; using the maximum digit-power sum gives a tighter bound. The unused variable `k` and outdated comment about fourth powers do not change the computation.

Correction to historical C

Correction: The original digit-power test is retained with a mathematically bounded search domain rather than an arbitrary ceiling.

26 Digit Factorials — Problem 34

Source: `https://github.com/KOOI/projectEuler/blob/reorganize/solutions-by-language/solutions/c/digit_factorials_34.c`

Assessment: Direct search with an unjustified upper bound.

Mathematical goal

Find numbers equal to the sum of the factorials of their decimal digits and sum them, excluding the trivial cases 1 and 2.

How the C code works

The helper `fact` calculates a digit's factorial by multiplying integers from one through that digit, with special cases for zero and negative input. The outer loop tests `j` from 100 through 99999. Remainder and division expressions isolate the units through ten-thousands digits, and the code sums their factorials into `t`. If `t==j`, the candidate contributes to the final `sum`.

Correctness and limitations

This range happens to include 145 and 40585, whose total is `...0730`, but the source does not prove that 99999 is a sufficient upper bound. A rigorous bound comes from comparing a d-digit number with $d \cdot 9!$; candidates of six or seven digits cannot be dismissed merely by intuition. The program should derive its stopping criterion and avoid recalculating the same digit factorials.

Correction to historical C

Correction: The search now covers every candidate up to seven times nine factorial, instead of the unjustified five-digit ceiling.

27 Circular Primes — Problem 35

Source: `https://github.com/KOOI/projectEuler/blob/reorganize/solutions-by-language/solutions/c/circular_primes_35.c`

Assessment: Unfinished scaffolding.

Mathematical goal

A prime is circular when every cyclic rotation of its decimal digits is also prime. Count all circular primes below one million.

How the C code works

The source describes separate ranges by digit length and sketches functions `a`, `b`, `c` and `d` to examine rotations. The actual `main` sets up a few variables, opens `circprimes`, checks for file-open failure, and closes it. No rotations are calculated and no answer is printed.

Correctness and limitations

The helper functions have empty bodies despite declaring integer return types. The initialized `sum=9` is neither updated nor printed. A complete solution needs prime membership tests for every rotation, with attention to leading zeros and repeated cyclic forms. The known count is `...55`, not a result established by this placeholder.

Correction to historical C

Correction: The empty historical helper stubs have been replaced with a complete sieve and cyclic digit-rotation search for every prime below one million.

28 Integer Right Triangles — Problem 39

Source: `https://github.com/KOOI/projectEuler/blob/reorganize/solutions-by-language/solutions/c/integer_right_triangles_39.c`

Assessment: Restricted search with counting flaws.

Mathematical goal

For perimeters $p \leq 1000$, count integer-sided right triangles $a < b < c$ with $a^2 + b^2 = c^2$ and $a + b + c = p$. Determine which perimeter has the most distinct triples.

How the C code works

The global `p` begins at 12. Nested loops search restricted intervals of sides `a`, `b` and `c`, while `testvalid` checks the Pythagorean equation using `pow` and the perimeter sum. Each hit increases `score`. After a perimeter is tested, `highscore` may be updated; the code advances to another `p` and measures CPU time.

Correctness and limitations

The code advances `p` by four and later by eight, which skips many candidate perimeters. The side-loop bounds are interdependent and can omit triples; incrementing `c` after a match can skip additional triples. `testvalid` lacks a return for one possible branch, and the program prints the highest score rather than its perimeter. The expected winning perimeter is `...840`, but this search is not a reliable proof.

Correction to historical C

Correction: The corrected algorithm examines all perimeters up to 1000 without skipping candidates and reports the maximizing perimeter.

29 Champernowne's Constant — Problem 40

Source: `https://github.com/KOOI/projectEuler/blob/reorganize/solutions-by-language/solutions/c/champernownes_constant_40.c`

Assessment: Partial digit-generation experiment.

Mathematical goal

Champernowne's constant is the concatenation 0.12345678910111213…. The task asks for the product of digits at specified powers-of-ten positions.

How the C code works

The program allocates an integer digit array of length `end=\texttt{...210}`. Separate branches attempt to fill the array with one-, two- and three-digit numbers, updating counters `secondpow` and `thirdpow` as the decimal length changes. At the end it prints the digit positions and the generated string.

Correctness and limitations

The program does not calculate the requested digit product or reach the millionth position. Its outer loop permits `n==end` while the later branches write `d[n+1]` or `d[n+2]`, causing out-of-bounds access. It also uses a custom index convention that needs verification against the first few concatenated digits. The historical `v2` alternative is likewise exploratory. The target product is `...210`, not a confirmed result of this code.

Correction to historical C

Correction: The corrected implementation generates decimal digits until position one million and multiplies the required digit values without a too-small array.

30 Pentagon Numbers — Problem 44

Source: `https://github.com/KOOI/projectEuler/blob/reorganize/solutions-by-language/solutions/c/pentagon_numbers_44.c`

Assessment: Search has a repeated-condition bug.

Mathematical goal

Pentagonal numbers satisfy $P_n = n(3n-1)/2$. Find two pentagonal numbers whose sum and absolute difference are both pentagonal, minimizing their positive difference.

How the C code works

The helper `P` evaluates the formula, and `isPentagon` searches a precomputed array for an exact match, returning zero on success. The program generates 5000 pentagonal numbers, checks all ordered pairs in nested loops, and tracks a variable `D` initialized to 100.

Correctness and limitations

Both nested conditionals test `testdif`; neither tests `testsum`. As written, the key requirement that the sum is pentagonal is missing. The initial bound `D=100` also prevents recording a valid larger difference. The double loop is quadratic in the number of candidates, each with a linear scan for membership. The known minimum is `...2660`; it is not computed correctly by this source.

Correction to historical C

Correction: Both the sum and difference must be pentagonal; the revised search checks both predicates and minimizes a positive difference without the erroneous initial bound.

31 Goldbach's Other Conjecture — Problem 46

Source: `https://github.com/KOOI/projectEuler/blob/reorganize/solutions-by-language/solutions/c/goldbachs_other_conjecture_46.c`

Assessment: Conceptual search, unsafe bounds and missing declarations.

Mathematical goal

The conjecture under test is that every odd composite integer z can be written $z = p + 2m^2$ for a prime p and positive integer m. Find the least counterexample.

How the C code works

The program reads prime numbers from `million_primes`, generates odd composite candidates using the external helper `isPrime`, then searches pairs (p, m) for the required decomposition. It calls external `powi` to square `m`, advances to the next candidate when a representation is found, and times the search.

Correctness and limitations

The source does not declare its external helper prototypes or link them directly; it must be built with `support/c/functions/isPrime.c` and `powi.c`. It reads without guarding the million-element array capacity and loops using `i<=1000000`, an out-of-range index. Advancing `i` inside nested loops can also invalidate the current candidate. The expected counterexample is `...5777`; the current implementation cannot be regarded as a safe computation of it.

Correction to historical C

Correction: The corrected odd-composite search tests prime-plus-twice-a-square representations directly, eliminating unsafe prime-file indexing and nested-loop index mutations.

32 Distinct Primes Factors — Problem 47

Source: `https://github.com/KOOI/projectEuler/blob/reorganize/solutions-by-language/solutions/c/distinct_primes_factors_47.c`

Assessment: Search implemented with external factor counter.

Mathematical goal

Find the first four consecutive integers each having exactly four distinct prime divisors. For example, repetitions of the same prime count once.

How the C code works

The main loop tests starts from `i=4` upward. For each candidate start, it checks whether `distinctpfactors(i)` through `distinctpfactors(i+3)` all equal `score=4`. If all four tests pass, the program prints the sequence and exits the loop. It asserts that 60 has three distinct prime factors as a basic check.

Correctness and limitations

The function `distinctpfactors` lives in `support/c/functions/distinctpfactors.c` and is not declared in this source. Linking and a prototype are required; the helper's fixed-size factor array and counting logic also merit tests. The search limit of twenty million is arbitrary. The expected first integer is `...4043`, but the exact code and helper should be tested before certifying the result.

Correction to historical C

Correction: Prime factors are divided out completely before counting distinct factors, then four consecutive four-factor integers are sought.

33 Self Powers — Problem 48

Source: `https://github.com/KOOI/projectEuler/blob/reorganize/solutions-by-language/solutions/c/self_powers_48.c`

Assessment: Modular computation implemented.

Mathematical goal

Compute the last ten decimal digits of $\sum_{n=1}^{1000} n^n$. Only the residue modulo 10^{10} is needed:

$$S \bmod 10^{10} = \left(\sum_{n=1}^{1000} n^n\right) \bmod 10^{10}.$$

How the C code works

GMP constructs the modulus `digitspan=10^10`. The loop evaluates each `n^n` with `mpz_ui_pow_ui`, reduces it modulo `digitspan`, and adds it into `result`. Finally the accumulated sum is reduced modulo the same modulus and printed.

Correctness and limitations

The loop runs from 999 down to 1, omitting the 1000th term. This omission does not change the last ten digits because 1000^{1000} is divisible by 10^{10}. The expected residue is `...6700`. The code computes full huge powers before reducing them; modular exponentiation would be significantly faster. GMP headers and `-lgmp` are required.

Correction to historical C

Correction: The corrected modular GNU MP calculation includes the term for 1000, which was previously omitted but happens to contribute zero modulo ten to the tenth power.

34 Consecutive Prime Sum — Problem 50

Source: `https://github.com/KOOI/projectEuler/blob/reorganize/solutions-by-language/solutions/c/consecutive_prime_sum_50.c`

Assessment: Unsafe exploratory attempt.

Mathematical goal

Find the prime below one million that can be expressed as a sum of the greatest number of consecutive primes.

How the C code works

The program loads an initial segment of primes from `million_primes` into `primeList` and contains a loop intended to sum consecutive windows from an offset until a demonstration sum is reached. A local `isPrime` helper and result-tracking variables are present, but the main primality and scoring blocks are commented out.

Correctness and limitations

The array has only ten elements while the window loop reads at least 22, causing out-of-bounds access. `test` is read before initialization in the enclosing condition, and `fgets(line,8,fp)` can write eight bytes into a seven-byte array. A file-open failure is not handled. The program does not produce a valid answer; the expected prime is `...7651`. The alternative versions in `variants` are separate attempts.

Correction to historical C

Correction: A sieve and prefix sums replace the historical ten-element array, its out-of-bounds reads, and the uninitialized accumulator.

35 Lychrel Numbers — Problem 55

Source: `https://github.com/KOOI/projectEuler/blob/reorganize/solutions-by-language/solutions/c/lychrel_numbers_55.c`

Assessment: Reverse-add search limited by integer overflow.

Mathematical goal

For each positive integer below 10000, repeatedly reverse its decimal digits and add. Count candidates that do not reach a palindrome within 50 iterations.

How the C code works

The helper `reverse` extracts base-ten digits using remainder and division, building the reversed number. `isPalindrome` compares a number with its reversal. For each starting value, the main loop performs up to 50 reverse-and-add operations, clears the `lychrel` flag if a palindrome appears, and increments `score` otherwise.

Correctness and limitations

The outer loop includes 10000 itself (`a<10001`), one beyond the stated strict upper bound. More importantly, repeated reverse-and-add operations can exceed the capacity of `unsigned long long`; unsigned overflow silently changes the sequence. Arbitrary-precision integers are needed for a trustworthy 50-iteration test. The expected candidate count is `...249`, but the program's fixed-width arithmetic may not reproduce it.

Correction to historical C

Correction: GNU MP replaces fixed-width integers to avoid overflow during fifty reverse-and-add attempts; only starting values strictly below 10000 are counted.

36 Totient Maximum — Problem 69

Source: `https://github.com/KOOI/projectEuler/blob/reorganize/solutions-by-language/solutions/c/totient_maximum_69.c`

Assessment: Totient-ratio search implemented.

Mathematical goal

Euler's totient $\varphi(n)$ counts positive integers at most n coprime to n. Find $n \leq 10^6$ maximizing $n/\varphi(n)$. For distinct prime divisors p of n,

$$\varphi(n) = n \prod_{p \mid n} \left(1 - \frac{1}{p}\right).$$

How the C code works

The helper `phi` begins at `res=i` and, whenever a prime factor is found, updates `res -= res/p`, removing repeated copies of that factor from the working integer. `main` loops over candidate `n` values from 2 up to one million, calculates the floating-point ratio, and records the best position in `maxpos`.

Correctness and limitations

The factorization-based totient computation is appropriate, and the expected maximizing integer is `...0510`. The code recomputes `phi(n)` after deciding the ratio improved, doubling work for those iterations. Its diagnostic print compares two equal values because `maxval` was just updated. A product of successive small primes offers a faster mathematical route. The printed maximum remains valid if the helper is evaluated correctly.

Correction to historical C

Correction: The corrected implementation multiplies successive primes while staying at most one million. The original totient factorization was useful but recomputed values unnecessarily.

37 Square Digit Chains — Problem 92

Source: `https://github.com/KOOI/projectEuler/blob/reorganize/solutions-by-language/solutions/c/square_digit_chains_92.c`

Assessment: Unimplemented.

Mathematical goal

Starting from an integer n, repeatedly replace it with the sum of the squares of its decimal digits. Count starting values below ten million whose chain reaches 89 rather than 1.

How the C code works

The source includes a comment describing the digit-square transformation and example chains. Its `main` contains no calculations, no loop over starting numbers, and no counter; it immediately returns zero.

Correctness and limitations

This is a problem statement preserved as a C placeholder, not an algorithm. An implementation can precompute the destination of each possible digit-square sum and reuse that result across all starting values. There is no output from the current source that answers the problem; the mathematical count is `...1146`.

Correction to historical C

Correction: The formerly empty C program now evaluates repeated decimal digit-square sums and counts exactly those starts below ten million that reach 89.

38 Cross-language algorithm comparisons

The original 37-problem collection has a dated validation baseline for C, Rust, and Gforth. C and Rust now cover problems 1 through 255. Factor has 191 validated entry points, and Gforth has 120. A 60-second-per-solver Gforth run passes 115 entries and times out on five. The *Factor programming language* is a concatenative, stack-based language with its own VM; it is neither Gforth nor the GNU `factor` integer-factorization command. The excerpts below reflect the algorithms in this development branch, not the uncorrected historical programs.

The C, Rust, and Gforth algorithms now reside in their individual problem files. Shared libraries contain reusable mathematical and input operations, with no problem-number dispatcher. Problem-specific search state and helper routines stay with their owning algorithm. The code excerpts below identify those owners.

38.1 Problem 9: Special Pythagorean Triplet

The perimeter constraint eliminates one search variable:

$$c = 1000 - a - b, \qquad a < b < c, \qquad a^2 + b^2 = c^2.$$

All versions test the same conditions but spell the computation differently.

C (`solutions/c/special_pythagorean_triplet_9.c`):

```
int c=1000-a-b;
if(c>b&&a*a+b*b==c*c)return a*b*c;
```

Rust (`solutions/rust/src/bin/p9.rs`):

```
let c=1000-a-b;
if c>b&&a*a+b*b==c*c {answer=a*b*c;break;}
```

Gforth (`solutions/forth/p9.fs`, `pythagorean-product`):

```
1000 a - b - to c
c b > a a * b b * + c c * = and
if a b * c * else 0 then ;
```

Factor (`solutions/factor/p9.factor`):

```
1000 a - b - :> c
c b > [
    a sq b sq + c sq = [
        a b * c * answer!
    ] when
] when
```

Unlike the C early return, this initial Factor version continues scanning the fixed candidate range after finding a solution. Both remain bounded searches, but the extra work prevents using these timings as a pure language benchmark.

38.2 Problem 11: Largest Product in a Grid

For each grid coordinate (x, y), the four direction vectors are $(1, 0)$, $(0, 1)$, $(1, 1)$ and $(-1, 1)$. A four-cell window is legal only when its endpoint $(x + 3d_x, y + 3d_y)$ stays in the 20×20 grid:

$$P(x, y, d_x, d_y) = \prod_{k=0}^{3} G[y + kd_y, x + kd_x].$$

Checking the endpoint first eliminates the negative-index undefined behavior in the historical C attempt.

C:

```
const int dx[]={1,0,1,-1},dy[]={0,1,1,1};
int xx=x+3*dx[d],yy=y+3*dy[d];
if(xx<0||xx>=20||yy<0||yy>=20)continue;
long long product=1;
for(int k=0;k<4;k++)
    product*=grid[y+k*dy[d]][x+k*dx[d]];
```

Rust:

```
let dirs=[(1,0),(0,1),(1,1),(-1,1)];
let xx=x+3*dx; let yy=y+3*dy;
if xx<0||xx>=20||yy<0||yy>=20 {continue;}
```

Gforth, using a flat cell array:

```
grid y i dy * + 20 * x i dx * + + cells + @
product * to product
```

Factor, using sequence access:

```
y step dy * + 20 *
x step dx * + +
grid nth result * result!
```

These are excerpts of inner loops; the complete source validates all four directions and all starting points. The Factor and C variants read the committed `tests/data/p011.txt` fixture at runtime. The Rust version embeds it, so I/O work differs.

38.3 Problem 28: Number Spiral Diagonals

For odd side length $s \geq 3$, the four corners on its newest ring have sum

$$\Delta(s) = 4s^2 - 6(s - 1).$$

The final sum is $1 + \Delta(3) + \Delta(5) + \cdots + \Delta(1001)$; no 1001×1001 array is required.

C:

```
for(long long side=3;side<=1001;side+=2)
    sum+=4*side*side-6*(side-1);
```

Rust:

```
for side in (3i64..=1001).step_by(2) {
    sum+=4*side*side-6*(side-1);
}
```

Gforth:

```
i i * 4 * i 1- 6 * - result + to result
```

Factor:

```
ring 2 * 1 + :> side
side sq 4 * side 1 - 6 * -
answer + answer!
```

All four perform a fixed number of ring updates. Their language-specific loop mechanics and VM/process startup can dominate the short computation.

38.4 Problem 39: Integer Right Triangles

The C and Rust implementations test side combinations for all perimeters $p \leq 1000$. Gforth and Factor instead solve the two equations for b:

$$b = \frac{p(p-2a)}{2(p-a)}, \qquad c = p - a - b,$$

then accept only exact integer divisions satisfying $a < b < c$. This reduces a nested side search to one candidate b per pair (p, a).

C:

```
int c=p-a-b;
if(a*a+b*b==c*c)count++;
```

Rust:

```
let c=p-a-b;
if a*a+b*b==c*c {count+=1;}
```

Gforth:

```
j j i 2 * - * to numerator
2 j i - * to denominator
numerator denominator mod 0= if
    numerator denominator / to b
    ...
then
```

Factor:

```
p p a 2 * - * :> numerator
p a - 2 * :> denominator
numerator denominator mod zero? [
    numerator denominator /i :> b
    p a - b - :> c
    a b < b c < and [ count 1 + count! ] when
] when
```

The Gforth excerpt includes an ellipsis where the full source applies its positivity and ordering tests. Differences in asymptotic work mean that these four implementations must not be ranked as if they executed identical algorithms.

38.5 Exact arithmetic: Factor versus C and Gforth

For problem 16 the Factor code expresses its entire big-integer computation as a composition of existing arbitrary-precision words:

```
: p16 ( -- n ) 2 1000 ^ digit-sum ;
```

Problem 20 likewise composes a range product with decimal digit summation. C uses GMP and Gforth uses a project-specific base-10000 digit library. These implementations trade library code size, native compilation, data representation, and startup costs against different amounts of hand-written algorithm code.

38.6 Profiling interpretation

The runner executes each selected binary or VM script *once*, without warming up; it checks that same invocation's exact decimal output against `tests/answers.json`. The wall time includes startup and output capture, but not compilation. It records process version, source revision and selected coverage; no error bars or confidence intervals are inferred from one measurement. Only complete passing declared-coverage language suites get total-suite bars. In the recorded Gforth run, the five timed-out entries are marked explicitly and no Gforth total-suite bar is drawn.

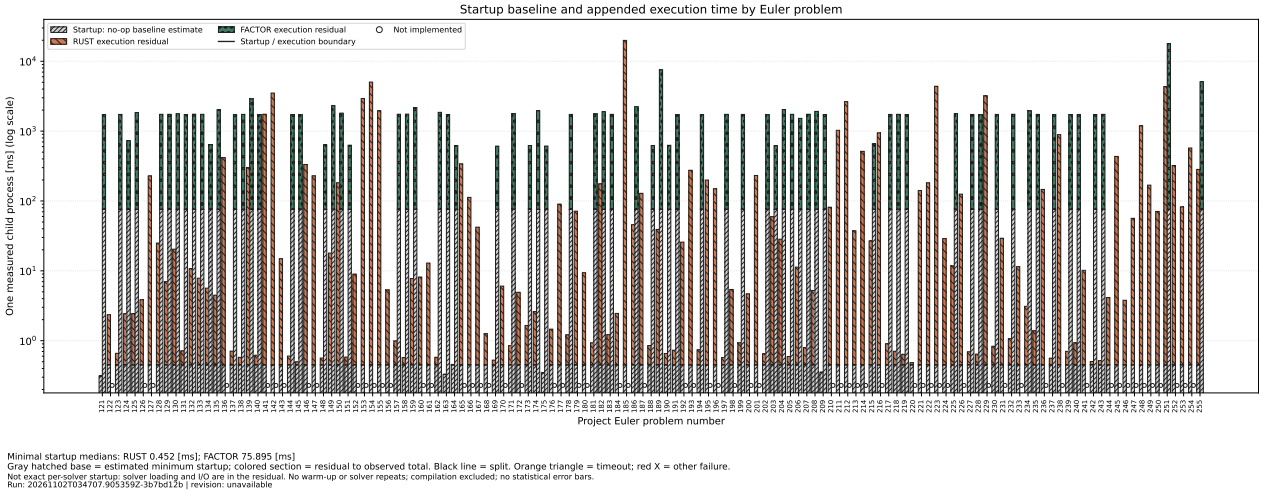


Figure 1: The recorded profile run preceding the ownership refactor. Passing solvers have measured bars; timeouts use orange triangles, while wrong answers and runtime errors would use red crosses. The run identifier printed in the figure ties it to its CSV and metadata.

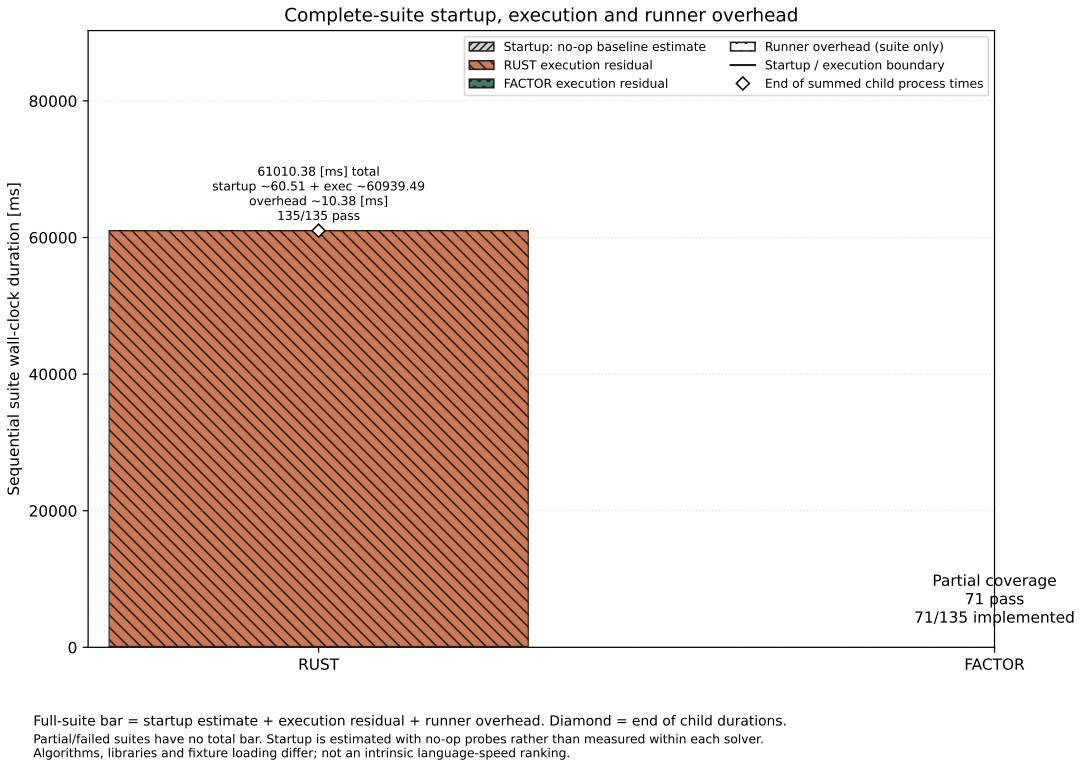


Figure 2: Complete-suite status for the same run. An incomplete suite receives no duration bar, so timeout work cannot be mistaken for a passing total.

39 Problem 154: counting divisible coefficients in Pascal's pyramid

Problem 154 asks for one layer of Pascal's pyramid. The coefficient of $x^a y^b z^c$ in $(x+y+z)^N$, where $a+b+c=N$, is

$$T(N;a,b,c)=\frac{N!}{a!b!c!}.$$

Here $N=200000$. We must count the coefficients divisible by $10^{12}=2^{12}5^{12}$. The implementation never constructs a coefficient that large. It counts factors of 2 and 5 instead.

The organization of this explanation follows Tam Nguyen's Project Euler 154 note: derive the valuation test, reduce the search by symmetry, and check the logic on a smaller layer. The code and the two-stage optimization below come from this repository.

39.1 Replace divisibility with two valuation tests

For a prime p, let $v_p(m)$ be the exponent of p in m. Then

$$v_p(T)=v_p(N!)-v_p(a!)-v_p(b!)-v_p(c!).$$

The coefficient qualifies exactly when $v_2(T)\ge 12$ and $v_5(T)\ge 12$. Both programs build prefix tables for $v_2(n!)$ and $v_5(n!)$. The recurrence is small:

$$v_p(n!)=v_p((n-1)!)+v_p(n).$$

After this $O(N\log N)$ setup, either valuation of a binomial coefficient is an $O(1)$ subtraction.

39.2 Split one trinomial coefficient into two binomial coefficients

Set $i=b+c$ and $j=b$. Then $a=N-i$ and $c=i-j$, so

$$T(N;N-i,j,i-j)=\binom{N}{i}\binom{i}{j}.$$

Valuations turn the product into a sum:

$$v_p(T)=v_p\left(\binom{N}{i}\right)+v_p\left(\binom{i}{j}\right).$$

This factorization creates a useful fast path. If $\binom{N}{i}$ already has at least twelve factors of both 2 and 5, then all $i+1$ splits of i into j and $i-j$ qualify. The program adds the whole row without inspecting its members.

For the remaining rows, exchanging b and c does not change the coefficient. The inner loop examines half of the row and gives a nonsymmetric pair weight two. When $b=c$, the pair has weight one.

39.3 C implementation

The C version stores the two prefix tables in flat arrays. Its helper computes the valuation of a binomial coefficient.

```
static unsigned choose154(const unsigned *factorial, unsigned n, unsigned k) {
    return factorial[n] - factorial[k] - factorial[n - k];
}

for (unsigned i = 0; i <= LAYER; ++i) {
    unsigned outer2 = choose154(twos, LAYER, i);
    unsigned outer5 = choose154(fives, LAYER, i);
    if (outer2 >= 12 && outer5 >= 12) {
        answer += i + 1;
        continue;
    }
    for (unsigned j = 0; j <= (i + 1) / 2; ++j) {
        if (outer2 + choose154(twos, i, j) >= 12 &&
            outer5 + choose154(fives, i, j) >= 12) {
            ++answer;
            if (j < i / 2) ++answer;
        }
    }
}
```

The arrays contain at most $N + 1$ values. The code uses unsigned subtraction only where the factorial valuation is monotone, so the result cannot become negative.

39.4 Rust implementation

Rust expresses the same algorithm with slices and vectors. The loop bounds, the row fast path, and the symmetry weights match the C version.

```
fn choose(v: &[u32], n: usize, k: usize) -> u32 {
    v[n] - v[k] - v[n - k]
}

for i in 0..=N {
    let (a, b) = (choose(&twos, N, i), choose(&fives, N, i));
    if a >= 12 && b >= 12 {
        answer += (i + 1) as u64;
        continue;
    }
    for j in 0..=(i + 1) / 2 {
        if a + choose(&twos, i, j) >= 12
            && b + choose(&fives, i, j) >= 12 {
            answer += 1;
            if j < i / 2 { answer += 1 }
        }
    }
}
```

The Rust types state two bounds that C leaves implicit. Table indices use `usize`, and the final count uses `u64`. The valuation entries fit in `u32` by a wide margin.

39.5 Cross-language comparison and missing ports

C and Rust use the same mathematical decomposition and asymptotic work. C manages the table allocation and release. Rust owns the tables through `Vec<u32>` and releases them when `solve` returns.

Neither version uses the shared prime or combinatoric API because the two valuation tables are specific to this problem.

The repository has no validated Gforth or Factor implementation for problem 154. Their scaffold files stay outside answer coverage. This report does not replace those missing programs with pseudocode or reference-answer constants.

39.6 Small checks catch symmetry errors

For a reduced layer, enumerate every ordered triple (a, b, c) with $a + b + c = n$. Compare the direct multinomial valuation with the two-binomial decomposition. Then compare the full ordered count with the weighted half-row count. These checks detect the two common mistakes: dropping the middle pair when i is odd and counting the $b = c$ case twice.

The generated figures in Section 41 show three views of the same object. The coefficient surface explains why central entries grow fastest. The parity plot shows what remains when divisibility is reduced to one factor of 2. Problem 154 applies the stronger two-prime test at layer 200000.

40 Compact walkthroughs for problems 1–255

This section is generated from the answer manifest and the current source files. Each entry records the implementation strategy, shows a short excerpt from every available language, and names any shared helper calls. The excerpt is an entry point into the source, not a replacement for the complete program.

40.1 Problem 1: Multiples of 3 or 5

The program computes the exact answer for Multiples of 3 or 5.

C. Source: `solutions/c/multiples_of_3_or_5_1.c`.

```
static long long solve(void) {
  long long s = 0;
  for (int n = 1; n < 1000; n++)
    if (n % 3 == 0 || n % 5 == 0)
      s += n;
  return s;
}

int main(void) {
  long long answer = solve();
  if (answer < 0)
    return 1;
  printf("%lld\n", answer);
  return 0;
```

Rust. Source: `solutions/rust/src/bin/p1.rs`.

```
fn solve() -> i64 {
    (1i64..1000).filter(|v| v % 3 == 0 || v % 5 == 0).sum()
}

fn main() {
    println!("{}", solve());
}
```

Gforth. Source: `solutions/forth/p1.fs`.

```
: p1 ( -- answer )
  0 1000 0 do i 3 mod 0= i 5 mod 0= or if i + then loop ;

[undefined] euler-testing [if] p1 . cr bye [then]
```

Factor. Source: `solutions/factor/p1.factor`.

```
:: p1 ( -- n )
    0 :> result!
    0 999 [a..b] [| n |
        n 3 mod zero? n 5 mod zero? or [
```

```
            result n + result!
        ] when
    ] each
    result ;
p1 number>string print
```

Comparison. C exposes direct bounded arithmetic. Rust exposes direct bounded arithmetic. Gforth exposes modular arithmetic. Factor exposes modular arithmetic. All listed programs print the same exact answer string from the manifest. The excerpts can differ in algorithm, data representation, and startup cost.

40.2 Problem 2: Even Fibonacci Numbers

The program computes the exact answer for Even Fibonacci Numbers.

C. Source: `solutions/c/even_fibonacci_numbers_2.c`.

```
static long long solve(void) {
  long long a = 1, b = 2, s = 0;
  while (b <= 4000000) {
    if (!(b % 2))
      s += b;
    long long t = a + b;
    a = b;
    b = t;
  }
  return s;
}

int main(void) {
  long long answer = solve();
```

Rust. Source: `solutions/rust/src/bin/p2.rs`.

```
fn solve() -> i64 {
    {
        let (mut a, mut b, mut sum) = (1i64, 2i64, 0i64);
        while b <= 4_000_000 {
            if b % 2 == 0 {
                sum += b;
            }
            (a, b) = (b, a + b);
        }
        sum
    }
}

fn main() {
```

Gforth. Source: `solutions/forth/p2.fs`.

```
: p2 ( -- answer )
  1 2 0 0 { a b sum next }
  begin b 4000000 <= while
    b 2 mod 0= if sum b + to sum then
    a b + to next b to a next to b
  repeat sum ;

[undefined] euler-testing [if] p2 . cr bye [then]
```

Factor. Source: `solutions/factor/p2.factor`.

```
:: p2 ( -- n )
    1 :> a! 2 :> b! 0 :> result!
    [ b 4000000 <= ] [
        b 2 mod zero? [ result b + result! ] when
        a b + :> next
        b a!
        next b!
    ] while
    result ;
p2 number>string print
```

Comparison. C exposes direct bounded arithmetic. Rust exposes direct bounded arithmetic. Gforth exposes modular arithmetic. Factor exposes modular arithmetic. All listed programs print the same exact answer string from the manifest. The excerpts can differ in algorithm, data representation, and startup cost.

40.3 Problem 3: Largest Prime Factor

The program computes the exact answer for Largest Prime Factor.

C. Source: `solutions/c/largest_prime_factor_3.c`.

```
static long long solve(void) {
  uint64_t n = 600851475143ULL, p = 2, last = 1;
  while (p <= n / p) {
    if (n % p == 0) {
      last = p;
      n /= p;
    } else
      p += (p == 2 ? 1 : 2);
  }
  return (long long)(n > 1 ? n : last);
}

int main(void) {
  long long answer = solve();
```

Rust. Source: `solutions/rust/src/bin/p3.rs`.

```
fn solve() -> i64 {
    {
        let mut n = 600_851_475_143u64;
        let mut d = 2u64;
        let mut last = 1u64;
        while d <= n / d {
            if n % d == 0 {
                last = d;
                n /= d;
            } else {
                d += if d == 2 { 1 } else { 2 };
            }
        }
        (if n > 1 { n } else { last }) as i64
```

Gforth. Source: `solutions/forth/p3.fs`.

```
: p3 ( -- answer )
  600851475143 2 1 { n d last }
  begin d n d / <= while
    n d mod 0= if d to last n d / to n
    else d 2 = if 3 else d 2 + then to d then
  repeat n 1 > if n else last then ;

[undefined] euler-testing [if] p3 . cr bye [then]
```

Factor. Source: `solutions/factor/euler/extra/extra.factor`.

```
:: p3 ( -- n )
    600851475143 :> remaining! 2 :> divisor! 1 :> largest!
    [ divisor sq remaining <= ] [
        remaining divisor mod zero?
        [ divisor largest! remaining divisor /i remaining! ]
        [ divisor 2 = [ 3 ] [ divisor 2 + ] if divisor! ] if
    ] while remaining 1 > [ remaining ] [ largest ] if ;
```

Comparison. C exposes prime generation or testing. Rust exposes prime generation or testing. Gforth exposes prime generation or testing, and modular arithmetic. Factor exposes modular arithmetic. All listed programs print the same exact answer string from the manifest. The excerpts can differ in algorithm, data representation, and startup cost.

40.4 Problem 4: Largest Palindrome Product

The program computes the exact answer for Largest Palindrome Product.

C. Source: `solutions/c/largest_palindrome_product_4.c`.

```
static long long solve(void) {
  int best = 0;
  for (int a = 100; a <= 999; a++)
    for (int b = a; b <= 999; b++) {
```

```
      int product = a * b;
      if (product > best && palindrome(product))
        best = product;
    }
  return best;
}

int main(void) {
  long long answer = solve();
  if (answer < 0)
```

Rust. Source: `solutions/rust/src/bin/p4.rs`.

```
fn solve() -> i64 {
    {
        let mut best = 0u32;
        for a in 100..=999 {
            for b in a..=999 {
                let prod = a * b;
                if prod > best && is_palindrome(prod) {
                    best = prod;
                }
            }
        }
        i64::from(best)
    }
}
```

Gforth. Source: `solutions/forth/p4.fs`.

```
: decimal-palindrome? 0 0 { n x reversed } ( n -- flag )
  n to x 0 to reversed
  begin x 0> while
    reversed 10 * x 10 mod + to reversed
    x 10 / to x
  repeat reversed n = ;

: p4 ( -- answer )
  0 { best }
  1000 100 do
    1000 i do
      i j * dup best > if
        dup decimal-palindrome? if to best else drop then
      else drop then
```

Factor. Source: `solutions/factor/euler/extra/extra.factor`.

```
:: p4 ( -- n )
    0 :> best!
    100 999 [a..b] [| a |
        a 999 [a..b] [| b |
            a b * :> product
            product best > product decimal-palindrome? and
            [ product best! ] when
        ] each
```

```
    ] each best ;
```

Comparison. C exposes direct bounded arithmetic. Shared calls: `palindrome`. Rust exposes direct bounded arithmetic. Shared calls: `is_palindrome`. Gforth exposes modular arithmetic. Factor exposes direct bounded arithmetic. Shared calls: `decimal-palindrome?`. All listed programs print the same exact answer string from the manifest. The excerpts can differ in algorithm, data representation, and startup cost.

40.5 Problem 5: Smallest Multiple

The program computes the exact answer for Smallest Multiple.

C. Source: `solutions/c/smallest_multiple_5.c`.

```
static long long solve(void) {
  unsigned result = 1;
  for (unsigned n = 2; n <= 20; n++)
    result = result / gcd(result, n) * n;
  return result;
}

int main(void) {
  long long answer = solve();
  if (answer < 0)
    return 1;
  printf("%lld\n", answer);
  return 0;
}
```

Rust. Source: `solutions/rust/src/bin/p5.rs`.

```
fn solve() -> i64 {
    (2..=20u64).fold(1u64, |acc, v| acc / gcd(acc, v) * v) as i64
}

fn main() {
    println!("{}", solve());
}
```

Gforth. Source: `solutions/forth/p5.fs`.

```
: p5 ( -- answer )
  1 { result }
  21 2 do result result i integer-gcd / i * to result loop result ;

[undefined] euler-testing [if] p5 . cr bye [then]
```

Factor. Source: `solutions/factor/p5.factor`.

```
: p5 ( -- n ) 20 [1..b] 1 [ lcm ] reduce ;
p5 number>string print
```

Comparison. C exposes direct bounded arithmetic. Shared calls: `gcd`. Rust exposes direct bounded arithmetic. Shared calls: `gcd`. Gforth exposes direct bounded arithmetic. Shared calls: `integer-gcd`. Factor exposes direct bounded arithmetic. All listed programs print the same exact answer string from the manifest. The excerpts can differ in algorithm, data representation, and startup cost.

40.6 Problem 6: Sum Square Difference

The program computes the exact answer for Sum Square Difference.

C. Source: `solutions/c/sum_square_difference_6.c`.

```
static long long solve(void) {
  long long sum = 100LL * 101 / 2, sq = 100LL * 101 * 201 / 6;
  return sum * sum - sq;
}

int main(void) {
  long long answer = solve();
  if (answer < 0)
    return 1;
  printf("%lld\n", answer);
  return 0;
}
```

Rust. Source: `solutions/rust/src/bin/p6.rs`.

```
fn solve() -> i64 {
    {
        let sum = 100i64 * 101 / 2;
        sum * sum - 100 * 101 * 201 / 6
    }
}

fn main() {
    println!("{}", solve());
}
```

Gforth. Source: `solutions/forth/p6.fs`.

```
: p6 ( -- answer )
  100 101 * 2 / dup * 100 101 * 201 * 6 / - ;

[undefined] euler-testing [if] p6 . cr bye [then]
```

Factor. Source: `solutions/factor/p6.factor`.

```
: p6 ( -- n )
    100 101 * 2 /i sq
    100 101 * 201 * 6 /i - ;
p6 number>string print
```

Comparison. C exposes direct bounded arithmetic. Rust exposes direct bounded arithmetic. Gforth exposes direct bounded arithmetic. Factor exposes direct bounded arithmetic. All listed programs print the same exact answer string from the manifest. The excerpts can differ in algorithm, data representation, and startup cost.

40.7 Problem 7: 10 001st Prime

The program computes the exact answer for 10 001st Prime.

C. Source: `solutions/c/10_001st_prime_7.c`.

```
static long long solve(void) {
  unsigned count = 0, n = 1;
  while (count < 10001) {
    n++;
    if (prime(n))
      count++;
  }
  return n;
}

int main(void) {
  long long answer = solve();
  if (answer < 0)
    return 1;
```

Rust. Source: `solutions/rust/src/bin/p7.rs`.

```
fn solve() -> i64 {
    {
        let mut found = 0;
        let mut candidate = 1u64;
        while found < 10001 {
            candidate += 1;
            if prime(candidate) {
                found += 1;
            }
        }
        candidate as i64
    }
}
```

Gforth. Source: `solutions/forth/p7.fs`.

```
: p7 ( -- answer )
  0 1 { count value }
```

```
  begin count 10001 < while
    value 1+ to value
    value is-prime? if count 1+ to count then
  repeat value ;

[undefined] euler-testing [if] p7 . cr bye [then]
```

Factor. Source: `solutions/factor/euler/extra/extra.factor`.

```
:: p7 ( -- n )
    1 :> value! 0 :> found!
    [ found 10001 < ] [
        value 1 + value!
        value prime? [ found 1 + found! ] when
    ] while value ;
```

Comparison. C exposes prime generation or testing. Shared calls: `prime`. Rust exposes prime generation or testing. Gforth exposes prime generation or testing. Shared calls: `is-prime?`. Factor exposes prime generation or testing. All listed programs print the same exact answer string from the manifest. The excerpts can differ in algorithm, data representation, and startup cost.

40.8 Problem 8: Largest Product in a Series

The program computes the exact answer for Largest Product in a Series.

C. Source: `solutions/c/largest_product_in_a_series_8.c`.

```
static long long solve(void) {
  FILE *f = fopen("tests/data/p008.txt", "r");
  if (!f)
    return -1;
  char digits[1024];
  unsigned n = 0;
  int ch;
  while ((ch = fgetc(f)) != EOF)
    if (ch >= '0' && ch <= '9') {
      if (n >= sizeof digits) {
        fclose(f);
        return -1;
      }
      digits[n++] = (char)ch;
```

Rust. Source: `solutions/rust/src/bin/p8.rs`.

```
fn solve() -> i64 {
    {
        let digits: Vec<u64> = include_str!("../../../../tests/data/p008.txt")
            .bytes()
            .filter(|b| b.is_ascii_digit())
            .map(|b| u64::from(b - b'0'))
            .collect();
```

```
        assert_eq!(digits.len(), 1000);
        digits
            .windows(13)
            .map(|w| w.iter().copied().product::<u64>())
            .max()
            .unwrap() as i64
    }
```

Gforth. Source: `solutions/forth/p8.fs`.

```
: p8 ( -- answer )
  s" tests/data/p008.txt" 1200 read-fixture 0 0 0 0 { buffer length digits count best product }
  1000 allocate throw to digits
  0 to count 0 to best
  length 0 do
    buffer i + c@ dup [char] 0 >= over [char] 9 <= and if
      [char] 0 - digits count + c! count 1+ to count
    else drop then
  loop
  count 1000 <> abort" p008 digit fixture must have 1000 digits"
  988 0 do
    1 to product
    13 0 do product digits j i + + c@ * to product loop
    product best > if product to best then
```

Factor. Source: `solutions/factor/euler/extra/extra.factor`.

```
:: p8 ( -- n )
    "tests/data/p008.txt" utf8 file-contents
    [ dup 48 >= swap 57 <= and ] filter [ 48 - ] map :> digits
    0 :> best!
    0 digits length 13 - [a..b] [| at |
        1 :> product!
        0 12 [a..b] [| delta |
            at delta + digits nth product * product!
        ] each
        product best max best!
    ] each best ;
```

Comparison. C exposes direct bounded arithmetic. Rust exposes fixture parsing. Gforth exposes fixture parsing. Shared calls: `read-fixture`. Factor exposes fixture parsing. All listed programs print the same exact answer string from the manifest. The excerpts can differ in algorithm, data representation, and startup cost.

40.9 Problem 9: Special Pythagorean Triplet

The program computes the exact answer for Special Pythagorean Triplet.

C. Source: `solutions/c/special_pythagorean_triplet_9.c`.

```
static long long solve(void) {
```

```
  for (int a = 1; a < 334; a++)
    for (int b = a + 1; b < 500; b++) {
      int c = 1000 - a - b;
      if (c > b && a * a + b * b == c * c)
        return a * b * c;
    }
  return -1;
}

int main(void) {
  long long answer = solve();
  if (answer < 0)
    return 1;
```

Rust. Source: `solutions/rust/src/bin/p9.rs`.

```
fn solve() -> i64 {
    {
        let mut answer = 0;
        for a in 1..334i64 {
            for b in a + 1..500 {
                let c = 1000 - a - b;
                if c > b && a * a + b * b == c * c {
                    answer = a * b * c;
                    break;
                }
            }
            if answer != 0 {
                break;
            }
```

Gforth. Source: `solutions/forth/p9.fs`.

```
: pythagorean-product 0 { a b c } ( a b -- result-or-zero )
  1000 a - b - to c
  c b > a a * b b * + c c * = and if a b * c * else 0 then ;

: p9 ( -- answer )
  334 1 do
    500 i 1+ do
      j i pythagorean-product dup 0<> if unloop unloop exit else drop then
    loop
  loop -1 ;

[undefined] euler-testing [if] p9 . cr bye [then]
```

Factor. Source: `solutions/factor/p9.factor`.

```
:: p9 ( -- n )
    0 :> answer!
    1 333 [a..b] [| a |
        a 1 + 499 [a..b] [| b |
            1000 a - b - :> c
            c b > [
                a sq b sq + c sq = [
```

```
                    a b * c * answer!
                ] when
            ] when
        ] each
    ] each
    answer ;
p9 number>string print
```

Comparison. C exposes direct bounded arithmetic. Rust exposes direct bounded arithmetic. Gforth exposes direct bounded arithmetic. Factor exposes direct bounded arithmetic. All listed programs print the same exact answer string from the manifest. The excerpts can differ in algorithm, data representation, and startup cost.

40.10 Problem 10: Summation of Primes

The program computes the exact answer for Summation of Primes.

C. Source: `solutions/c/summation_of_primes_10.c`.

```
static long long solve(void) {
  unsigned char *p = sieve(2000000);
  if (!p)
    return -1;
  long long sum = 0;
  for (int n = 2; n < 2000000; n++)
    if (p[n])
      sum += n;
  free(p);
  return sum;
}

int main(void) {
  long long answer = solve();
```

Rust. Source: `solutions/rust/src/bin/p10.rs`.

```
fn solve() -> i64 {
    {
        let p = sieve(2_000_000);
        (2..2_000_000).filter(|&v| p[v]).map(|v| v as i64).sum()
    }
}

fn main() {
    println!("{}", solve());
}
```

Gforth. Source: `solutions/forth/p10.fs`.

```
: p10 ( -- answer )
  2000000 sieve-build 0 { bytes answer }
```

```
  0 to answer
  2000000 2 do bytes i + c@ 0<> if answer i + to answer then loop
  bytes free throw answer ;

[undefined] euler-testing [if] p10 . cr bye [then]
```

Factor. Source: `solutions/factor/p10.factor`.

```
: p10 ( -- n ) 1999999 primes-upto sum ;
p10 number>string print
```

Comparison. C exposes prime generation or testing. Shared calls: `sieve`. Rust exposes prime generation or testing. Shared calls: `sieve`. Gforth exposes prime generation or testing. Shared calls: `sieve-build`. Factor exposes prime generation or testing. All listed programs print the same exact answer string from the manifest. The excerpts can differ in algorithm, data representation, and startup cost.

40.11 Problem 11: Largest Product in a Grid

The program computes the exact answer for Largest Product in a Grid.

C. Source: `solutions/c/largest_product_in_a_grid_11.c`.

```
static long long solve(void) {
  FILE *f = fopen("tests/data/p011.txt", "r");
  if (!f)
    return -1;
  int grid[20][20];
  for (int y = 0; y < 20; y++)
    for (int x = 0; x < 20; x++)
      if (fscanf(f, "%d", &grid[y][x]) != 1) {
        fclose(f);
        return -1;
      }
  fclose(f);
  long long best = 0;
  const int dx[] = {1, 0, 1, -1}, dy[] = {0, 1, 1, 1};
```

Rust. Source: `solutions/rust/src/bin/p11.rs`.

```
fn solve() -> i64 {
    {
        let v: Vec<i64> = include_str!("../../../../tests/data/p011.txt")
            .split_whitespace()
            .map(|v| v.parse().unwrap())
            .collect();
        assert_eq!(v.len(), 400);
        let dirs = [(1, 0), (0, 1), (1, 1), (-1, 1)];
        let mut best = 0;
        for y in 0..20i32 {
            for x in 0..20i32 {
```

```
                for (dx, dy) in dirs {
                    let xx = x + 3 * dx;
                    let yy = y + 3 * dy;
```

Gforth. Source: `solutions/forth/p11.fs`.

```
: grid-product 0 0 0 { grid x y dx dy xx yy product } ( grid x y dx dy -- product )
  x dx 3 * + to xx y dy 3 * + to yy
  xx 0< xx 20 >= or yy 0< or yy 20 >= or if 0 exit then
  1 to product
  4 0 do
    grid y i dy * + 20 * x i dx * + + cells + @ product * to product
  loop product ;

: p11 ( -- answer )
  s" tests/data/p011.txt" 4096 read-fixture
  0 0 0 0 0 { buf length grid count value active best }
  400 cells allocate throw to grid
  0 to count 0 to value false to active
  length 0 do
```

Factor. Source: `solutions/factor/p11.factor`.

```
:: p11 ( -- n )
    read-grid :> grid
    0 :> best!
    { { 1 0 } { 0 1 } { 1 1 } { -1 1 } } :> directions
    0 19 [a..b] [| y |
        0 19 [a..b] [| x |
            directions [| d |
                0 d nth :> dx
                1 d nth :> dy
                x 3 dx * + :> xx
                y 3 dy * + :> yy
                xx 0 >= xx 20 < and
                yy 0 >= and yy 20 < and [
                    grid x y dx dy grid-product :> candidate
```

Comparison. C exposes direct bounded arithmetic. Rust exposes fixture parsing. Gforth exposes fixture parsing. Shared calls: `read-fixture`. Factor exposes fixture parsing. Shared calls: `decimal-cell`. All listed programs print the same exact answer string from the manifest. The excerpts can differ in algorithm, data representation, and startup cost.

40.12 Problem 12: Highly Divisible Triangular Number

The program computes the exact answer for Highly Divisible Triangular Number.

C. Source: `solutions/c/highly_divisible_triangular_number_12.c`.

```
static long long solve(void) {
  for (uint64_t n = 1;; n++)
```

```
    if (count_divisors(n * (n + 1) / 2) > 500)
      return (long long)(n * (n + 1) / 2);
}

int main(void) {
  long long answer = solve();
  if (answer < 0)
    return 1;
  printf("%lld\n", answer);
  return 0;
}
```

Rust. Source: `solutions/rust/src/bin/p12.rs`.

```
fn solve() -> i64 {
    {
        let mut n = 1u64;
        loop {
            let tri = n * (n + 1) / 2;
            if divisor_count(tri) > 500 {
                break tri as i64;
            }
            n += 1;
        }
    }
}

fn main() {
```

Gforth. Source: `solutions/forth/p12.fs`.

```
: p12 ( -- answer )
  1 0 { n tri }
  begin n n 1+ * 2 / to tri
    tri divisor-count 500 > if tri exit then
    n 1+ to n
  again ;

[undefined] euler-testing [if] p12 . cr bye [then]
```

Factor. Source: `solutions/factor/euler/extra/extra.factor`.

```
:: p12 ( -- n )
    1 :> n! 1 :> triangle!
    [ triangle p12-divisors 500 <= ] [
        n 1 + n! triangle n + triangle!
    ] while triangle ;
```

Comparison. C exposes direct bounded arithmetic. Shared calls: `count_divisors`. Rust exposes direct bounded arithmetic. Gforth exposes direct bounded arithmetic. Shared calls: `divisor-count`. Factor exposes direct bounded arithmetic. All listed programs print the same exact answer string from the manifest. The excerpts can differ in algorithm, data representation, and startup cost.

40.13 Problem 13: Large Sum

The program computes the exact answer for Large Sum.

C. Source: `solutions/c/large_sum_13.c`.

```
static long long solve(void) {
  FILE *f = fopen("lsum", "r");
  if (!f)
    return -1;
  mpz_t total, tmp;
  mpz_inits(total, tmp, NULL);
  char line[256];
  int count = 0;
  while (fgets(line, sizeof line, f))
    if (mpz_set_str(tmp, line, 10) == 0) {
      mpz_add(total, total, tmp);
      count++;
    }
  fclose(f);
```

Rust. Source: `solutions/rust/src/bin/p13.rs`.

```
fn solve() -> i64 {
    {
        let mut total = BigUint::zero();
        let mut lines = 0;
        for line in include_str!("../../../../lsum").lines() {
            let number = line.trim();
            if number.is_empty() {
                continue;
            }
            total += number.parse::<BigUint>().unwrap();
            lines += 1;
        }
        assert_eq!(lines, 100);
        total.to_string()[..10].parse::<i64>().unwrap()
```

Gforth. Source: `solutions/forth/p13.fs`.

```
: bto-u 0 { b result } ( b -- u )
  0 to result
  b blen@ 0 ?do
    result big-base * b blen@ 1- i - b blimb@ + to result
  loop result ;

: p13 ( -- answer )
  s" lsum" r/o open-file throw
  0 0 0 0 0 { f total next line count more }
  16 bnew to total 16 bnew to next
  128 allocate throw to line 0 to count
  begin line 128 f read-line throw to more more while
    line over next bfrom-dec total next baddabs!
    count 1+ to count drop
```

Factor. Source: `solutions/factor/euler/extra/extra.factor`.

```
: p13 ( -- n ) "lsum" read-numbers sum
    number>string 0 10 rot subseq decimal-cell ;
```

Comparison. C exposes exact large-integer arithmetic. Rust exposes exact large-integer arithmetic, and fixture parsing. Gforth exposes exact large-integer arithmetic, and fixture parsing. Shared calls: `baddabs!`, `bdecimal-length`, `bdiv-small!`, `bfree`, `bfrom-dec`, `blen@`, `blimb@`, `bnew`. Factor exposes fixture parsing. Shared calls: `decimal-cell`, `read-numbers`. All listed programs print the same exact answer string from the manifest. The excerpts can differ in algorithm, data representation, and startup cost.

40.14 Problem 14: Longest Collatz Sequence

The program computes the exact answer for Longest Collatz Sequence.

C. Source: `solutions/c/longest_collatz_sequence_14.c`.

```
static long long solve(void) {
  unsigned best_n = 1, best_len = 0;
  for (unsigned start = 1; start < 1000000; start++) {
    uint64_t n = start;
    unsigned len = 1;
    while (n != 1) {
      n = (n % 2) ? 3 * n + 1 : n / 2;
      len++;
    }
    if (len > best_len) {
      best_len = len;
      best_n = start;
    }
  }
```

Rust. Source: `solutions/rust/src/bin/p14.rs`.

```
fn solve() -> i64 {
    {
        let mut best_len = 0;
        let mut best_start = 0;
        for start in 1u64..1_000_000 {
            let (mut v, mut length) = (start, 1);
            while v != 1 {
                v = if v % 2 == 0 { v / 2 } else { 3 * v + 1 };
                length += 1;
            }
            if length > best_len {
                best_len = length;
                best_start = start;
            }
```

Gforth. Source: `solutions/forth/p14.fs`.

```
: p14 ( -- answer )
  1 0 0 0 { best-start best-length n length }
  1000000 1 do
    i to n 1 to length
    begin n 1 <> while
      n 2 mod 0= if n 2 / to n else n 3 * 1+ to n then
      length 1+ to length
    repeat
    length best-length > if length to best-length i to best-start then
  loop best-start ;

[undefined] euler-testing [if] p14 . cr bye [then]
```

Factor. Source: `solutions/factor/euler/extra/extra.factor`.

```
:: p14 ( -- n )
    1 :> best! 1 :> index!
    2 999999 [a..b] [| n |
        n p14-collatz :> candidate
        candidate best > [ candidate best! n index! ] when
    ] each index ;
```

Comparison. C exposes direct bounded arithmetic. Rust exposes direct bounded arithmetic. Gforth exposes modular arithmetic. Factor exposes direct bounded arithmetic. All listed programs print the same exact answer string from the manifest. The excerpts can differ in algorithm, data representation, and startup cost.

40.15 Problem 15: Lattice Paths

A vertex receives every path arriving from above or from its left. Updating one row in place counts all right/down paths to (20,20).

C. Source: `solutions/c/lattice_paths_15.c`.

```
int main(void) {
    uint64_t paths[21];
    for (unsigned x = 0; x <= 20; ++x) paths[x] = 1;
    for (unsigned y = 1; y <= 20; ++y)
        for (unsigned x = 1; x <= 20; ++x) paths[x] += paths[x - 1];
    printf("%" PRIu64 "\n", paths[20]);
    return 0;
}
```

Rust. Source: `solutions/rust/src/bin/p15.rs`.

```
fn solve() -> i64 {
    {
        let mut ways = [1u64; 21];
        for _ in 1..=20 {
            for j in 1..=20 {
                ways[j] += ways[j - 1];
```

```
            }
        }
        ways[20] as i64
    }
}

fn main() {
    println!("{}", solve());
```

Gforth. Source: `solutions/forth/p15.fs`.

```
: p15 ( -- n )
  21 cells allocate throw { ways }
  21 0 do 1 ways i cells + ! loop
  20 0 do 21 1 do ways i 1- cells + @ ways i cells + +! loop loop
  20 ways prefix@ ways free throw ;

[undefined] euler-testing [if] p15 . cr bye [then]
```

Factor. Source: `solutions/factor/euler/extra/extra.factor`.

```
: p15 ( -- n ) 40 factorial-number 20 factorial-number sq /i ;
```

Comparison. C exposes direct bounded arithmetic. Rust exposes direct bounded arithmetic. Gforth exposes direct bounded arithmetic. Shared calls: `prefix@`. Factor exposes exact large-integer arithmetic. Shared calls: `factorial-number`. All listed programs print the same exact answer string from the manifest. The excerpts can differ in algorithm, data representation, and startup cost.

40.16 Problem 16: Power Digit Sum

The program computes the exact answer for Power Digit Sum.

C. Source: `solutions/c/power_digit_sum_16.c`.

```
static long long solve(void) {
  mpz_t n;
  mpz_init(n);
  mpz_ui_pow_ui(n, 2, 1000);
  long long s = digit_sum(n);
  mpz_clear(n);
  return s;
}

int main(void) {
  long long answer = solve();
  if (answer < 0)
    return 1;
  printf("%lld\n", answer);
```

Rust. Source: `solutions/rust/src/bin/p16.rs`.

```
fn solve() -> i64 {
    digit_sum(&BigUint::from(2u8).pow(1000))
}

fn main() {
    println!("{}", solve());
}
```

Gforth. Source: `solutions/forth/p16.fs`.

```
: p16 ( -- answer )
  64 bnew { n } 1 n bset-u
  1000 0 do 2 n bmul-small! loop n bdigitsum n bfree ;

[undefined] euler-testing [if] p16 . cr bye [then]
```

Factor. Source: `solutions/factor/p16.factor`.

```
: p16 ( -- n ) 2 1000 ^ digit-sum ;
p16 number>string print
```

Comparison. C exposes exact large-integer arithmetic. Shared calls: `digit_sum`. Rust exposes exact large-integer arithmetic. Gforth exposes exact large-integer arithmetic. Shared calls: `bdigitsum`, `bfree`, `bmul-small!`, `bnew`, `bset-u`. Factor exposes direct bounded arithmetic. Shared calls: `digit-sum`. All listed programs print the same exact answer string from the manifest. The excerpts can differ in algorithm, data representation, and startup cost.

40.17 Problem 17: Number Letter Counts

The program computes the exact answer for Number Letter Counts.

C. Source: `solutions/c/number_letter_counts_17.c`.

```
static int number_letters(unsigned n) {
  static const int small[] = {0, 3, 3, 5, 4, 4, 3, 5, 5, 4,
                              3, 6, 6, 8, 8, 7, 7, 9, 8, 8};
  static const int tens[] = {0, 0, 6, 6, 5, 5, 5, 7, 6, 6};
  if (n == 1000)
    return 11;
  int length = 0;
  if (n >= 100) {
    length += small[n / 100] + 7;
    n %= 100;
    if (n)
      length += 3;
  }
  if (n >= 20) {
```

Rust. Source: `solutions/rust/src/bin/p17.rs`.

```
fn letters(mut n: usize) -> u32 {
    let small = [0, 3, 3, 5, 4, 4, 3, 5, 5, 4, 3, 6, 6, 8, 8, 7, 7, 9, 8, 8];
    let tens = [0, 0, 6, 6, 5, 5, 5, 7, 6, 6];
    if n == 1000 {
        return 11;
    }
    let mut count = 0;
    if n >= 100 {
        count += small[n / 100] + 7;
        n %= 100;
        if n != 0 {
            count += 3;
        }
    }
```

Gforth. Source: `solutions/forth/p17.fs`.

```
create letter-small
  0 , 3 , 3 , 5 , 4 , 4 , 3 , 5 , 5 , 4 ,
  3 , 6 , 6 , 8 , 8 , 7 , 7 , 9 , 8 , 8 ,

create letter-tens 0 , 0 , 6 , 6 , 5 , 5 , 5 , 7 , 6 , 6 ,

: small-letters ( n -- count ) cells letter-small + @ ;

: tens-letters ( n -- count ) cells letter-tens + @ ;

: count-letters 0 { n answer } ( n -- count )
  n 1000 = if 11 exit then 0 to answer
  n 100 >= if
    n 100 / small-letters 7 + answer + to answer
```

Factor. Source: `solutions/factor/euler/advanced/advanced.factor`.

```
: p17 ( -- n ) 1 1000 [a..b] [ english-count ] map sum ;
```

Comparison. C exposes direct bounded arithmetic. Rust exposes modular arithmetic. Gforth exposes modular arithmetic. Factor exposes direct bounded arithmetic. All listed programs print the same exact answer string from the manifest. The excerpts can differ in algorithm, data representation, and startup cost.

40.18 Problem 18: Maximum Path Sum I

Replace each entry, bottom upwards, by itself plus its better child. The official fifteen-row input is reproduced here as integer data.

C. Source: `solutions/c/maximum_path_sum_i_18.c`.

```
int main(void) {
    unsigned triangle[15][15] = {
```

```
        {75},
        {95,64},
        {17,47,82},
        {18,35,87,10},
        {20,4,82,47,65},
        {19,1,23,75,3,34},
        {88,2,77,73,7,63,67},
        {99,65,4,28,6,16,70,92},
        {41,41,26,56,83,40,80,70,33},
        {41,48,72,33,47,32,37,16,94,29},
        {53,71,44,65,25,43,91,52,97,51,14},
        {70,11,33,28,77,73,17,78,39,68,17,57},
```

Rust. Source: `solutions/rust/src/bin/p18.rs`.

```
fn solve() -> i64 {
    triangle_max(matrix(include_str!(
        "../../../../tests/data/p018_triangle.txt"
    ))) as i64
}

fn main() {
    println!("{}", solve());
}
```

Gforth. Source: `solutions/forth/p18.fs`.

```
: p18 ( -- n ) s" tests/data/p018_triangle.txt" 15 triangle-path-max ;

[undefined] euler-testing [if] p18 . cr bye [then]
```

Factor. Source: `solutions/factor/euler/extra/extra.factor`.

```
: p18 ( -- n ) "tests/data/p018_triangle.txt" max-triangle-path ;
```

Comparison. C exposes direct bounded arithmetic. Rust exposes fixture parsing. Shared calls: `matrix`. Gforth exposes direct bounded arithmetic. Shared calls: `triangle-path-max`. Factor exposes direct bounded arithmetic. Shared calls: `max-triangle-path`. All listed programs print the same exact answer string from the manifest. The excerpts can differ in algorithm, data representation, and startup cost.

40.19 Problem 19: Counting Sundays

The program computes the exact answer for Counting Sundays.

C. Source: `solutions/c/counting_sundays_19.c`.

```
static int leap(int year) {
  return year % 4 == 0 && (year % 100 != 0 || year % 400 == 0);
}
```

```
static long long solve(void) {
  int w = 1, total = 0;
  static const int days[] = {31, 28, 31, 30, 31, 30, 31, 31, 30, 31, 30, 31};
  for (int y = 1900; y <= 2000; y++)
    for (int m = 0; m < 12; m++) {
      if (y >= 1901 && w == 0)
        total++;
      w = (w + days[m] + (m == 1 && leap(y))) % 7;
    }
  return total;
```

Rust. Source: `solutions/rust/src/bin/p19.rs`.

```
fn leap(y: i32) -> bool {
    y % 4 == 0 && (y % 100 != 0 || y % 400 == 0)
}

fn solve() -> i64 {
    {
        let mut day = 1;
        let mut count = 0;
        let days = [31, 28, 31, 30, 31, 30, 31, 31, 30, 31, 30, 31];
        for year in 1900..=2000 {
            for month in 0..12 {
                if year >= 1901 && day == 0 {
                    count += 1;
                }
```

Gforth. Source: `solutions/forth/p19.fs`.

```
: leap? { year } ( year -- flag )
  year 4 mod 0= year 100 mod 0<> year 400 mod 0= or and ;

create month-days 31 , 28 , 31 , 30 , 31 , 30 , 31 , 31 , 30 , 31 , 30 , 31 ,

: p19 ( -- answer )
  1 0 0 { weekday answer days }
  2001 1900 do
    12 0 do
      j 1901 >= weekday 0= and if answer 1+ to answer then
      month-days i cells + @ to days
      i 1 = j leap? and if days 1+ to days then
      weekday days + 7 mod to weekday
    loop
```

Factor. Source: `solutions/factor/euler/extra/extra.factor`.

```
:: p19 ( -- n )
    ! 1900-01-01 was Monday (day-of-week 1); advance through months.
    1 :> dow! 0 :> answer!
    1900 2000 [a..b] [| year |
        1 12 [a..b] [| month |
            year 1901 >= dow 0 = and [ answer 1 + answer! ] when
            month { 4 6 9 11 } member?
```

```
            [ 30 ] [ month 2 = [
              year 4 mod zero?
              year 100 mod zero? not year 400 mod zero? or and [ 29 ] [ 28 ] if
            ] [ 31 ] if ] if
          :> length
          dow length + 7 mod dow!
        ] each
```

Comparison. C exposes direct bounded arithmetic. Rust exposes direct bounded arithmetic. Gforth exposes modular arithmetic. Factor exposes modular arithmetic. All listed programs print the same exact answer string from the manifest. The excerpts can differ in algorithm, data representation, and startup cost.

40.20 Problem 20: Factorial Digit Sum

The program computes the exact answer for Factorial Digit Sum.

C. Source: `solutions/c/factorial_digit_sum_20.c`.

```
static long long solve(void) {
  mpz_t n;
  mpz_init(n);
  mpz_fac_ui(n, 100);
  long long s = digit_sum(n);
  mpz_clear(n);
  return s;
}

int main(void) {
  long long answer = solve();
  if (answer < 0)
    return 1;
  printf("%lld\n", answer);
```

Rust. Source: `solutions/rust/src/bin/p20.rs`.

```
fn solve() -> i64 {
    {
        let mut fact = BigUint::one();
        for n in 2u32..=100 {
            fact *= n;
        }
        digit_sum(&fact)
    }
}

fn main() {
    println!("{}", solve());
}
```

Gforth. Source: `solutions/forth/p20.fs`.

```
: p20 ( -- answer )
  64 bnew { n } 1 n bset-u
  101 2 do i n bmul-small! loop n bdigitsum n bfree ;

[undefined] euler-testing [if] p20 . cr bye [then]
```

Factor. Source: `solutions/factor/p20.factor`.

```
: p20 ( -- n ) 100 [1..b] product digit-sum ;
p20 number>string print
```

Comparison. C exposes exact large-integer arithmetic. Shared calls: `digit_sum`. Rust exposes exact large-integer arithmetic. Gforth exposes exact large-integer arithmetic. Shared calls: `bdigitsum`, `bfree`, `bmul-small!`, `bnew`, `bset-u`. Factor exposes direct bounded arithmetic. Shared calls: `digit-sum`. All listed programs print the same exact answer string from the manifest. The excerpts can differ in algorithm, data representation, and startup cost.

40.21 Problem 21: Amicable Numbers

The program computes the exact answer for Amicable Numbers.

C. Source: `solutions/c/amicable_numbers_21.c`.

```
static long long solve(void) {
  unsigned sum = 0;
  for (unsigned n = 2; n < 10000; n++) {
    unsigned pair = divisor_sum(n);
    if (pair != n && pair > 0 && divisor_sum(pair) == n)
      sum += n;
  }
  return sum;
}

int main(void) {
  long long answer = solve();
  if (answer < 0)
    return 1;
```

Rust. Source: `solutions/rust/src/bin/p21.rs`.

```
fn solve() -> i64 {
    (2u32..10000)
        .filter(|&a| {
            let b = divisor_sum(a);
            b != a && b > 0 && divisor_sum(b) == a
        })
        .map(i64::from)
        .sum()
}

fn main() {
```

```
    println!("{}", solve());
}
```

Gforth. Source: `solutions/forth/p21.fs`.

```
: p21 ( -- answer )
  0 0 { sum pair }
  10000 2 do
    i proper-divisor-sum to pair
    pair i <> pair 0> and if
      pair proper-divisor-sum i = if sum i + to sum then
    then
  loop sum ;

[undefined] euler-testing [if] p21 . cr bye [then]
```

Factor. Source: `solutions/factor/euler/extra/extra.factor`.

```
:: p21 ( -- n )
    9999 divisor-sum-sieve :> sums
    0 :> total!
    2 9999 [a..b] [| a |
        a sums nth :> b
        b 0 > b 10000 < and a b = not and [
            b sums nth a = [ total a + total! ] when
        ] when
    ] each total ;
```

Comparison. C exposes direct bounded arithmetic. Shared calls: `divisor_sum`. Rust exposes direct bounded arithmetic. Shared calls: `divisor_sum`. Gforth exposes direct bounded arithmetic. Shared calls: `proper-divisor-sum`. Factor exposes prime generation or testing. Shared calls: `divisor-sum-sieve`. All listed programs print the same exact answer string from the manifest. The excerpts can differ in algorithm, data representation, and startup cost.

40.22 Problem 22: Names Scores

The program computes the exact answer for Names Scores.

C. Source: `solutions/c/names_scores_22.c`.

```
static long long solve(void) {
  FILE *f = fopen("names.txt", "r");
  if (!f)
    return -1;
  char *names[6000];
  size_t count = 0;
  int ch;
  while ((ch = fgetc(f)) != EOF) {
    if (ch != '"')
      continue;
    char buf[128];
```

```
    size_t len = 0;
    while ((ch = fgetc(f)) != EOF && ch != '"') {
      if (len + 1 >= sizeof buf) {
```

Rust. Source: `solutions/rust/src/bin/p22.rs`.

```
fn solve() -> i64 {
    {
        let mut names: Vec<&str> = include_str!("../../../../names.txt")
            .split(',')
            .map(|s| s.trim().trim_matches('"'))
            .collect();
        assert_eq!(names.len(), 5163);
        names.sort_unstable();
        names
            .iter()
            .enumerate()
            .map(|(i, name)| {
                let score: i64 = name
                    .bytes()
```

Gforth. Source: `solutions/forth/p22.fs`.

```
create names-address 5200 cells allot

create names-length 5200 cells allot

: name-addr ( index -- addr ) cells names-address + @ ;

: name-len ( index -- len ) cells names-length + @ ;

: name-swap 0 0 { a b addr len } ( index index -- )
  a name-addr to addr a name-len to len
  b name-addr a cells names-address + !
  b name-len a cells names-length + !
  addr b cells names-address + ! len b cells names-length + ! ;
```

Factor. Source: `solutions/factor/euler/extra/extra.factor`.

```
:: p22 ( -- n )
    "names.txt" read-upper-words sort :> names
    0 :> total!
    names [| name index |
        name [ 64 - ] map sum index 1 + * total + total!
    ] each-index total ;
```

Comparison. C exposes sorting or set deduplication. Rust exposes fixture parsing, and sorting or set deduplication. Gforth exposes fixture parsing, sorting or set deduplication, and recursive search. Shared calls: `read-fixture`. Factor exposes fixture parsing, and sorting or set deduplication. Shared calls: `read-upper-words`. All listed programs print the same exact answer string from the manifest. The excerpts can differ in algorithm, data representation, and startup cost.

40.23 Problem 23: Non-Abundant Sums

The statement supplies 28123 as an inclusive sufficient bound. A divisor sieve identifies abundant numbers, then all their pair sums.

C. Source: `solutions/c/non_abundant_sums_23.c`.

```
int main(void) {
    enum { LIMIT = 28123 };
    unsigned *divisors = euler_calloc(LIMIT + 1, sizeof *divisors);
    unsigned *abundant = euler_malloc((LIMIT + 1) * sizeof *abundant);
    unsigned char *represented = euler_calloc(LIMIT + 1, 1);
    for (unsigned d = 1; d <= LIMIT / 2; ++d)
        for (unsigned n = 2 * d; n <= LIMIT; n += d) divisors[n] += d;
    size_t count = 0;
    for (unsigned n = 1; n <= LIMIT; ++n)
        if (divisors[n] > n) abundant[count++] = n;
    for (size_t i = 0; i < count; ++i)
        for (size_t j = i; j < count; ++j) {
            unsigned sum = abundant[i] + abundant[j];
            if (sum > LIMIT) break;
```

Rust. Source: `solutions/rust/src/bin/p23.rs`.

```
fn solve() -> i64 {
    {
        let divs = proper_divisor_sums(28123);
        let abundant: Vec<usize> = (1..=28123).filter(|&x| divs[x] > x).collect();
        let mut represented = vec![false; 28124];
        for (i, &a) in abundant.iter().enumerate() {
            for &b in &abundant[i..] {
                if a + b > 28123 {
                    break;
                }
                represented[a + b] = true;
            }
        }
        (1..=28123).filter(|&x| !represented[x]).sum::<usize>() as i64
```

Gforth. Source: `solutions/forth/p23.fs`.

```
: p23 ( -- n )
  28124 allocate throw 28124 cells allocate throw 0 0 0
  { marked abundant count answer sum }
  marked 28124 erase
  28124 1 do i proper-divisor-sum i > if
    i abundant count cells + ! count 1+ to count
  then loop
  count 0 do
    count i do
      i abundant prefix@ j abundant prefix@ + to sum
      sum 28123 > if leave then
      1 marked sum + c!
    loop
  loop
```

Factor. Source: `solutions/factor/euler/extra/extra.factor`.

```
:: p23 ( -- n )
    28123 divisor-sum-sieve :> sums
    12 28123 [a..b] [| n | n sums nth n > ] filter :> abundant
    28124 f <array> :> representable
    abundant [| a |
        abundant [| b |
            a b + 28123 <= [ t a b + representable set-nth ] when
        ] each
    ] each
    1 28123 [a..b] [| n | n representable nth not ] filter sum ;
```

Comparison. C exposes prime generation or testing. Shared calls: `euler_calloc`, `euler_malloc`. Rust exposes direct bounded arithmetic. Shared calls: `proper_divisor_sums`. Gforth exposes direct bounded arithmetic. Shared calls: `prefix@`, `proper-divisor-sum`. Factor exposes prime generation or testing. Shared calls: `divisor-sum-sieve`. All listed programs print the same exact answer string from the manifest. The excerpts can differ in algorithm, data representation, and startup cost.

40.24 Problem 24: Lexicographic Permutations

The remaining permutations split into equal factorial-sized blocks. Unrank zero-based index 999999, deleting each chosen digit.

C. Source: `solutions/c/lexicographic_permutations_24.c`.

```
int main(void) {
    unsigned factorial[11] = {1}, digits[10];
    for (unsigned i = 1; i <= 10; ++i) factorial[i] = i * factorial[i - 1];
    for (unsigned i = 0; i < 10; ++i) digits[i] = i;
    unsigned rank = 1000000 - 1;
    uint64_t answer = 0;
    for (unsigned remaining = 10; remaining; --remaining) {
        unsigned index = rank / factorial[remaining - 1];
        rank %= factorial[remaining - 1];
        answer = answer * 10 + digits[index];
        for (unsigned i = index; i + 1 < remaining; ++i) digits[i] = digits[i + 1];
    }
    printf("%" PRIu64 "\n", answer);
    return 0;
```

Rust. Source: `solutions/rust/src/bin/p24.rs`.

```
fn solve() -> i64 {
    {
        let mut pool: Vec<u64> = (0..10).collect();
        let mut rank = 999999usize;
        let mut value = 0u64;
        for remaining in (1..=10).rev() {
            let factorial = (1..remaining).product::<usize>();
            let idx = rank / factorial;
            rank %= factorial;
            value = value * 10 + pool.remove(idx);
```

```
        }
        value as i64
    }
}
```

Gforth. Source: `solutions/forth/p24.fs`.

```
: p24 ( -- n )
  10 cells allocate throw 999999 0 0 0 { digits rank result index remaining }
  10 0 do i digits i cells + ! loop
  10 0 do
    10 i - to remaining
    rank remaining 1- factorial / to index
    rank remaining 1- factorial mod to rank
    result 10 * index digits prefix@ + to result
    remaining 1- index ?do
      i 1+ digits prefix@ i digits prefix!
    loop
  loop digits free throw result ;

[undefined] euler-testing [if] p24 . cr bye [then]
```

Factor. Source: `solutions/factor/euler/extra/extra.factor`.

```
:: p24 ( -- n )
    999999 :> rank! 0 :> answer!
    10 f <array> :> used
    0 9 [a..b] [| index |
        9 index - factorial-number :> block
        rank block /i :> wanted
        rank block mod rank!
        0 :> found!
        0 9 [a..b] [| digit |
            digit used nth not [
                found wanted = [
                    answer 10 * digit + answer!
                    t digit used set-nth
                    found 1 + found!
```

Comparison. C exposes exact large-integer arithmetic. Rust exposes exact large-integer arithmetic. Gforth exposes modular arithmetic, and exact large-integer arithmetic. Shared calls: `factorial`, `prefix!`, `prefix@`. Factor exposes modular arithmetic, and exact large-integer arithmetic. Shared calls: `factorial-number`. All listed programs print the same exact answer string from the manifest. The excerpts can differ in algorithm, data representation, and startup cost.

40.25 Problem 25: 1000-digit Fibonacci Number

The program computes the exact answer for 1000-digit Fibonacci Number.

C. Source: `solutions/c/1000_digit_fibonacci_number_25.c`.

```
static long long solve(void) {
  mpz_t a, b, limit, tmp;
  mpz_inits(a, b, limit, tmp, NULL);
  mpz_set_ui(a, 1);
  mpz_set_ui(b, 1);
  mpz_ui_pow_ui(limit, 10, 999);
  unsigned i = 2;
  while (mpz_cmp(b, limit) < 0) {
    mpz_add(tmp, a, b);
    mpz_set(a, b);
    mpz_set(b, tmp);
    i++;
  }
  mpz_clears(a, b, limit, tmp, NULL);
```

Rust. Source: `solutions/rust/src/bin/p25.rs`.

```
fn solve() -> i64 {
    {
        let mut a = BigUint::one();
        let mut b = BigUint::one();
        let limit = BigUint::from(10u8).pow(999);
        let mut index = 2;
        while b < limit {
            let next = &a + &b;
            a = b;
            b = next;
            index += 1;
        }
        index
    }
```

Gforth. Source: `solutions/forth/p25.fs`.

```
: p25 ( -- answer )
  1 bnew 1 bnew 1 bnew 0 0 { a b temp old index }
  1 a bset-u 1 b bset-u 2 to index
  begin b bdecimal-length 1000 < while
    a b temp badd a to old b to a temp to b old to temp
    index 1+ to index
  repeat a bfree b bfree temp bfree index ;

\ Names scores: in-memory sort of quoted, comma-separated records.

[undefined] euler-testing [if] p25 . cr bye [then]
```

Factor. Source: `solutions/factor/p25.factor`.

```
:: p25 ( -- n )
    1 :> a! 1 :> b! 2 :> index!
    [ b number>string length 1000 < ] [
        a b + :> next
        b a! next b!
        index 1 + index!
```

```
    ] while
    index ;
p25 number>string print
```

Comparison. C exposes exact large-integer arithmetic. Rust exposes exact large-integer arithmetic. Gforth exposes dynamic programming or memoization, exact large-integer arithmetic, and sorting or set deduplication. Shared calls: `badd`, `bdecimal-length`, `bfree`, `bnew`, `bset-u`. Factor exposes direct bounded arithmetic. All listed programs print the same exact answer string from the manifest. The excerpts can differ in algorithm, data representation, and startup cost.

40.26 Problem 26: Reciprocal Cycles

Long division repeats exactly when a remainder repeats. Remainder zero means a terminating decimal, whose recurring cycle has length zero.

C. Source: `solutions/c/reciprocal_cycles_26.c`.

```
static unsigned cycle_length(unsigned denominator) {
    int first[1000];
    for (unsigned i = 0; i < denominator; ++i) first[i] = -1;
    unsigned remainder = 1 % denominator, position = 0;
    while (remainder && first[remainder] < 0) {
        first[remainder] = (int)position++;
        remainder = 10 * remainder % denominator;
    }
    return remainder ? position - (unsigned)first[remainder] : 0;
}

int main(void) {
    unsigned best = 0, answer = 0;
    for (unsigned d = 2; d < 1000; ++d) {
```

Rust. Source: `solutions/rust/src/bin/p26.rs`.

```
fn solve() -> i64 {
    {
        let mut best = (0usize, 0i64);
        for denominator in 2usize..1000 {
            let mut seen = vec![usize::MAX; denominator];
            let mut r = 1usize % denominator;
            let mut position = 0;
            while r != 0 && seen[r] == usize::MAX {
                seen[r] = position;
                position += 1;
                r = (r * 10) % denominator;
            }
            let length = if r == 0 { 0 } else { position - seen[r] };
            if length > best.0 {
```

Gforth. Source: `solutions/forth/p26.fs`.

```
: p26 ( -- n )
  1000 cells allocate throw 0 0 0 0 0 { seen best answer remainder position cycle }
  1000 2 do
    i 0 do -1 i seen prefix! loop
    1 i mod to remainder 0 to position
    begin remainder 0<> if remainder seen prefix@ -1 = else false then while
      position remainder seen prefix!
      position 1+ to position
      remainder 10 * i mod to remainder
    repeat
    remainder 0= if 0 else position remainder seen prefix@ - then to cycle
    cycle best > if cycle to best i to answer then
  loop seen free throw answer ;
```

Factor. Source: `solutions/factor/euler/extra/extra.factor`.

```
:: p26 ( -- n )
    0 :> longest! 0 :> best!
    2 999 [a..b] [| d |
        d -1 <array> :> seen
        0 :> pos! 1 d mod :> remainder!
        [ remainder 0 > remainder seen nth -1 = and ] [
            pos remainder seen set-nth
            pos 1 + pos!
            remainder 10 * d mod remainder!
        ] while
        remainder 0 = [ 0 ] [ pos remainder seen nth - ] if :> cycle
        cycle longest > [ cycle longest! d best! ] when
    ] each best ;
```

Comparison. C exposes direct bounded arithmetic. Rust exposes direct bounded arithmetic. Gforth exposes modular arithmetic. Shared calls: `prefix!`, `prefix@`. Factor exposes modular arithmetic. All listed programs print the same exact answer string from the manifest. The excerpts can differ in algorithm, data representation, and startup cost.

40.27 Problem 27: Quadratic Primes

The program computes the exact answer for Quadratic Primes.

C. Source: `solutions/c/quadratic_primes_27.c`.

```
static long long solve(void) {
  int best = 0, result = 0;
  for (int a = -999; a < 1000; a++)
    for (int b = -999; b < 1000; b++) {
      int n = 0;
      while (1) {
        long long v = (long long)n * n + (long long)a * n + b;
        if (v < 2 || !prime((uint64_t)v))
          break;
        n++;
      }
```

```
      if (n > best) {
        best = n;
        result = a * b;
```

Rust. Source: `solutions/rust/src/bin/p27.rs`.

```
fn solve() -> i64 {
    {
        let mut longest = 0;
        let mut product = 0;
        for a in -999i64..1000 {
            for b in -999i64..1000 {
                let mut len = 0;
                loop {
                    let candidate = len * len + a * len + b;
                    if candidate < 2 || !prime(candidate as u64) {
                        break;
                    }
                    len += 1;
                }
```

Gforth. Source: `solutions/forth/p27.fs`.

```
: quadratic-run 0 0 { a b n value } ( a b -- count )
  0 to n begin n n * a n * + b + to value value is-prime?
  while n 1+ to n repeat n ;

: p27 ( -- answer )
  0 0 0 { best product score }
  1000 2 do i is-prime? if
    1000 -999 do
      i j quadratic-run to score score best > if score to best i j * to product then
    loop
  then loop product ;

[undefined] euler-testing [if] p27 . cr bye [then]
```

Factor. Source: `solutions/factor/euler/extra/extra.factor`.

```
:: p27 ( -- n )
    0 :> best-length! 0 :> best-product!
    -999 999 [a..b] [| a |
        2 1000 [a..b] [| b |
            b prime? [
                0 :> n!
                [ n sq a n * + b + dup 1 > swap prime? and ] [ n 1 + n! ] while
                n best-length > [ n best-length! a b * best-product! ] when
            ] when
        ] each
    ] each best-product ;
```

Comparison. C exposes prime generation or testing. Shared calls: `prime`. Rust exposes prime generation or testing. Shared calls: `prime`. Gforth exposes prime generation or testing. Shared calls:

`is-prime?`. Factor exposes prime generation or testing. All listed programs print the same exact answer string from the manifest. The excerpts can differ in algorithm, data representation, and startup cost.

40.28 Problem 28: Number Spiral Diagonals

The program computes the exact answer for Number Spiral Diagonals.

C. Source: `solutions/c/number_spiral_diagonals_28.c`.

```
static long long solve(void) {
  long long sum = 1;
  for (long long side = 3; side <= 1001; side += 2)
    sum += 4 * side * side - 6 * (side - 1);
  return sum;
}

int main(void) {
  long long answer = solve();
  if (answer < 0)
    return 1;
  printf("%lld\n", answer);
  return 0;
}
```

Rust. Source: `solutions/rust/src/bin/p28.rs`.

```
fn solve() -> i64 {
    {
        let mut sum = 1i64;
        for side in (3i64..=1001).step_by(2) {
            sum += 4 * side * side - 6 * (side - 1);
        }
        sum
    }
}

fn main() {
    println!("{}", solve());
}
```

Gforth. Source: `solutions/forth/p28.fs`.

```
: p28 ( -- answer )
  1 { result }
  1002 3 do i 2 mod 1 = if
    i i * 4 * i 1- 6 * - result + to result
  then loop result ;

[undefined] euler-testing [if] p28 . cr bye [then]
```

Factor. Source: `solutions/factor/p28.factor`.

```
:: p28 ( -- n )
    1 :> answer!
    1 500 [a..b] [| ring |
        ring 2 * 1 + :> side
        side sq 4 * side 1 - 6 * -
        answer + answer!
    ] each
    answer ;
p28 number>string print
```

Comparison. C exposes direct bounded arithmetic. Rust exposes direct bounded arithmetic. Gforth exposes modular arithmetic. Factor exposes direct bounded arithmetic. All listed programs print the same exact answer string from the manifest. The excerpts can differ in algorithm, data representation, and startup cost.

40.29 Problem 29: Distinct Powers

The program computes the exact answer for Distinct Powers.

C. Source: `solutions/c/distinct_powers_29.c`.

```
static long long solve(void) {
  char *values[9801];
  size_t idx = 0;
  mpz_t v;
  mpz_init(v);
  for (unsigned a = 2; a <= 100; a++)
    for (unsigned b = 2; b <= 100; b++) {
      mpz_ui_pow_ui(v, a, b);
      values[idx++] = mpz_get_str(NULL, 10, v);
    }
  mpz_clear(v);
  qsort(values, idx, sizeof values[0], namecmp);
  long long count = 0;
  for (size_t i = 0; i < idx; i++)
```

Rust. Source: `solutions/rust/src/bin/p29.rs`.

```
fn solve() -> i64 {
    {
        let mut values = HashSet::new();
        for a in 2u32..=100 {
            for b in 2..=100 {
                values.insert(BigUint::from(a).pow(b));
            }
        }
        values.len() as i64
    }
}

fn main() {
    println!("{}", solve());
```

Gforth. Source: `solutions/forth/p29.fs`.

```
: primitive-root 0 0 0 0 { n base power exp current-exp } ( n -- root exp )
  n to base 1 to exp
  n 1+ 2 ?do i to power 1 to current-exp
    begin power n < while power i * to power current-exp 1+ to current-exp repeat
    power n = if i to base current-exp to exp leave then
  loop base exp ;

create power-seen 61000 allot

: p29 ( -- answer )
  power-seen 61000 erase 0 0 0 0 { count root exp key }
  101 2 do i primitive-root to exp to root
    101 2 do root 601 * exp i * + to key
      power-seen key + c@ 0= if 1 power-seen key + c! count 1+ to count then
```

Factor. Source: `solutions/factor/euler/extra/extra.factor`.

```
:: p29 ( -- n )
    H{ } clone :> powers
    2 100 [a..b] [| base |
        2 100 [a..b] [| exp |
            t base exp ^ powers set-at
        ] each
    ] each powers assoc-size ;
```

Comparison. C exposes exact large-integer arithmetic, and sorting or set deduplication. Rust exposes exact large-integer arithmetic, and sorting or set deduplication. Gforth exposes direct bounded arithmetic. Factor exposes sorting or set deduplication. All listed programs print the same exact answer string from the manifest. The excerpts can differ in algorithm, data representation, and startup cost.

40.30 Problem 30: Digit Fifth Powers

The program computes the exact answer for Digit Fifth Powers.

C. Source: `solutions/c/digit_fifth_powers_30.c`.

```
static long long solve(void) {
  unsigned pow5[10];
  for (unsigned n = 0; n < 10; n++) {
    pow5[n] = 1;
    for (int k = 0; k < 5; k++)
      pow5[n] *= n;
  }
  unsigned sum = 0;
  for (unsigned n = 2; n <= 354294; n++)
    if (digit_power_sum(n, pow5) == n)
      sum += n;
  return sum;
}
```

Rust. Source: `solutions/rust/src/bin/p30.rs`.

```
fn solve() -> i64 {
    {
        let p5: [u32; 10] = std::array::from_fn(|v| (v as u32).pow(5));
        (2u32..=354294)
            .filter(|&n| digit_power_sum(n, &p5) == n)
            .map(i64::from)
            .sum()
    }
}

fn main() {
    println!("{}", solve());
}
```

Gforth. Source: `solutions/forth/p30.fs`.

```
: p30 ( -- answer )
  0 { sum }
  354295 2 do i 5 digit-pow-sum i = if sum i + to sum then loop sum ;

[undefined] euler-testing [if] p30 . cr bye [then]
```

Factor. Source: `solutions/factor/euler/extra/extra.factor`.

```
:: p30 ( -- n )
    0 :> total!
    2 354294 [a..b] [| n |
        n number>string [ 48 - 5 ^ ] map sum n =
        [ total n + total! ] when
    ] each total ;
```

Comparison. C exposes direct bounded arithmetic. Shared calls: `digit_power_sum`. Rust exposes direct bounded arithmetic. Shared calls: `digit_power_sum`. Gforth exposes direct bounded arithmetic. Shared calls: `digit-pow-sum`. Factor exposes direct bounded arithmetic. All listed programs print the same exact answer string from the manifest. The excerpts can differ in algorithm, data representation, and startup cost.

40.31 Problem 31: Coin Sums

Processing coin values outside the amount loop counts multisets once. ways[s] includes zero or more copies of every coin processed so far.

C. Source: `solutions/c/coin_sums_31.c`.

```
int main(void) {
    static const unsigned coins[] = {1,2,5,10,20,50,100,200};
    uint64_t ways[201] = {1};
    for (size_t i = 0; i < sizeof coins / sizeof *coins; ++i)
        for (unsigned amount = coins[i]; amount <= 200; ++amount)
```

```
            ways[amount] += ways[amount - coins[i]];
    printf("%" PRIu64 "\n", ways[200]);
    return 0;
}
```

Rust. Source: `solutions/rust/src/bin/p31.rs`.

```
fn solve() -> i64 {
    {
        let mut ways = [0u64; 201];
        ways[0] = 1;
        for coin in [1, 2, 5, 10, 20, 50, 100, 200] {
            for amount in coin..=200 {
                ways[amount] += ways[amount - coin];
            }
        }
        ways[200] as i64
    }
}

fn main() {
```

Gforth. Source: `solutions/forth/p31.fs`.

```
create p31-coins 1 , 2 , 5 , 10 , 20 , 50 , 100 , 200 ,

: p31 ( -- n )
  201 cells allocate throw 0 0 { ways coin result }
  ways 201 cells erase 1 ways !
  8 0 do i p31-coins prefix@ to coin
    201 coin do i coin - ways prefix@ i cells ways + +! loop
  loop
  200 ways prefix@ to result ways free throw result ;

[undefined] euler-testing [if] p31 . cr bye [then]
```

Factor. Source: `solutions/factor/euler/extra/extra.factor`.

```
:: p31 ( -- n )
    201 0 <array> :> ways
    1 0 ways set-nth
    { 1 2 5 10 20 50 100 200 } [| coin |
        coin 200 [a..b] [| amount |
            amount ways nth amount coin - ways nth + amount ways set-nth
        ] each
    ] each 200 ways nth ;
```

Comparison. C exposes direct bounded arithmetic. Rust exposes direct bounded arithmetic. Gforth exposes direct bounded arithmetic. Shared calls: `prefix@`. Factor exposes direct bounded arithmetic. All listed programs print the same exact answer string from the manifest. The excerpts can differ in algorithm, data representation, and startup cost.

40.32 Problem 32: Pandigital Products

A nine-digit identity must use 1x4=4 or 2x3=4 digits. Consequently the smaller factor is below 100 and the product below 10000. Mark products so identities sharing a product contribute only once.

C. Source: `solutions/c/pandigital_products_32.c`.

```
static bool add_digits(unsigned n, unsigned *mask) {
    do {
        unsigned digit = n % 10, bit = 1u << digit;
        if (!digit || (*mask & bit)) return false;
        *mask |= bit;
        n /= 10;
    } while (n);
    return true;
}

int main(void) {
    bool seen[10000] = {false};
    unsigned sum = 0;
    for (unsigned a = 1; a < 100; ++a)
```

Rust. Source: `solutions/rust/src/bin/p32.rs`.

```
fn solve() -> i64 {
    {
        let mut found = vec![false; 10000];
        let mut sum = 0;
        for a in 1..100u64 {
            for b in a + 1..=9999 / a {
                let product = a * b;
                let text = format!("{a}{b}{product}");
                if text.len() == 9
                    && pandigital_1_to_9(text.parse().unwrap())
                    && !found[product as usize]
                {
                    found[product as usize] = true;
                    sum += product;
```

Gforth. Source: `solutions/forth/p32.fs`.

```
: p32 ( -- n )
  10000 allocate throw 0 0 0 { seen total product joined }
  seen 10000 erase
  100 1 do
    10000 i / 1+ i 1+ do
      i j * to product
      j i append-number product append-number to joined
      joined 100000000 >= joined 999999999 <= and if
        joined pandigital-nine? seen product + c@ 0= and if
          1 seen product + c! total product + to total
        then
      then
    loop
  loop seen free throw total ;
```

Factor. Source: `solutions/factor/euler/extra/extra.factor`.

```
:: p32 ( -- n )
    10000 f <array> :> seen
    0 :> total!
    1 99 [a..b] [| a |
        a 1 + 9999 a /i [a..b] [| b |
            a b * :> product
            a b concat-number product concat-number pandigital-nine?
            product 9999 <= and [
                product seen nth not [
                    total product + total!
                    t product seen set-nth
                ] when
            ] when
        ] each
```

Comparison. C exposes direct bounded arithmetic. Rust exposes direct bounded arithmetic. Shared calls: `pandigital_1_to_9`. Gforth exposes direct bounded arithmetic. Shared calls: `append-number`, `pandigital-nine?`. Factor exposes direct bounded arithmetic. Shared calls: `concat-number`, `pandigital-nine?`. All listed programs print the same exact answer string from the manifest. The excerpts can differ in algorithm, data representation, and startup cost.

40.33 Problem 33: Digit Cancelling Fractions

Check every proper two-digit fraction and both positions of its common nonzero digit. Equality is an exact cross multiplication.

C. Source: `solutions/c/digit_cancelling_fractions_33.c`.

```
int main(void) {
    uint64_t numerator = 1, denominator = 1;
    for (unsigned n = 10; n < 100; ++n)
        for (unsigned d = n + 1; d < 100; ++d) {
            unsigned nd[2] = {n / 10, n % 10}, dd[2] = {d / 10, d % 10};
            bool valid = false;
            for (unsigned i = 0; i < 2; ++i)
                for (unsigned j = 0; j < 2; ++j)
                    if (nd[i] && nd[i] == dd[j] && dd[1 - j] &&
                        n * dd[1 - j] == d * nd[1 - i]) valid = true;
            if (valid) { numerator *= n; denominator *= d; }
        }
    printf("%" PRIu64 "\n", denominator / euler_gcd(numerator, denominator));
    return 0;
```

Rust. Source: `solutions/rust/src/bin/p33.rs`.

```
fn solve() -> i64 {
    {
        let mut numerator = 1u64;
        let mut denominator = 1u64;
        for a in 10u64..100 {
            for b in a + 1..100 {
```

```
                let ad = [a / 10, a % 10];
                let bd = [b / 10, b % 10];
                let mut valid = false;
                for i in 0..2 {
                    for j in 0..2 {
                        if ad[i] > 0
                            && ad[i] == bd[j]
                            && bd[1 - j] > 0
```

Gforth. Source: `solutions/forth/p33.fs`.

```
: digit-cancel? 0 0 0 0 0 { numerator denominator nt no dt du valid } ( n d -- flag )
  numerator 10 / to nt numerator 10 mod to no
  denominator 10 / to dt denominator 10 mod to du
  false to valid
  nt dt = nt 0<> and no denominator * numerator du * = and if true to valid then
  nt du = nt 0<> and no denominator * numerator dt * = and if true to valid then
  no dt = no 0<> and nt denominator * numerator du * = and if true to valid then
  no du = no 0<> and nt denominator * numerator dt * = and if true to valid then
  valid ;

: p33 ( -- n ) 1 1 { product-n product-d }
  100 10 do
    100 i 1+ do
      j i digit-cancel? if product-n j * to product-n product-d i * to product-d then
```

Factor. Source: `solutions/factor/euler/extra/extra.factor`.

```
:: p33 ( -- n )
    1 :> numerator! 1 :> denominator!
    10 99 [a..b] [| n |
        n 1 + 99 [a..b] [| d |
            n d p33-cancel? [ numerator n * numerator! denominator d * denominator! ] when
        ] each
    ] each
    denominator dup numerator gcd nip /i ;
```

Comparison. C exposes direct bounded arithmetic. Shared calls: `euler_gcd`. Rust exposes direct bounded arithmetic. Shared calls: `gcd`. Gforth exposes modular arithmetic. Shared calls: `integer-gcd`. Factor exposes direct bounded arithmetic. All listed programs print the same exact answer string from the manifest. The excerpts can differ in algorithm, data representation, and startup cost.

40.34 Problem 34: Digit Factorials

The program computes the exact answer for Digit Factorials.

C. Source: `solutions/c/digit_factorials_34.c`.

```
static long long solve(void) {
  unsigned f[10];
```

```
  for (int n = 0; n < 10; n++)
    f[n] = factorial(n);
  unsigned sum = 0;
  for (unsigned n = 3; n <= 2540160; n++)
    if (digit_power_sum(n, f) == n)
      sum += n;
  return sum;
}

int main(void) {
  long long answer = solve();
  if (answer < 0)
```

Rust. Source: `solutions/rust/src/bin/p34.rs`.

```
fn solve() -> i64 {
    {
        let table: [u32; 10] = std::array::from_fn(|v| factorial(v as u32));
        (3u32..=2_540_160)
            .filter(|&n| digit_power_sum(n, &table) == n)
            .map(i64::from)
            .sum()
    }
}

fn main() {
    println!("{}", solve());
}
```

Gforth. Source: `solutions/forth/p34.fs`.

```
: digit-factorial-sum 0 0 { n value result } ( n -- sum )
  n to value 0 to result
  begin value 10 mod factorial result + to result value 10 / to value
    value 0= if result exit then
  again ;

: p34 ( -- answer )
  0 { sum }
  2540161 3 do i digit-factorial-sum i = if sum i + to sum then loop sum ;

[undefined] euler-testing [if] p34 . cr bye [then]
```

Factor. Source: `solutions/factor/euler/extra/extra.factor`.

```
:: p34 ( -- n )
    0 9 [a..b] [ factorial-number ] map :> facts
    0 :> total!
    3 2540160 [a..b] [| n |
        n number>string [ 48 - facts nth ] map sum n =
        [ total n + total! ] when
    ] each total ;
```

Comparison. C exposes exact large-integer arithmetic. Shared calls: `digit_power_sum`, `factorial`. Rust exposes exact large-integer arithmetic. Shared calls: `digit_power_sum`, `factorial`. Gforth exposes modular arithmetic, and exact large-integer arithmetic. Shared calls: `factorial`. Factor exposes exact large-integer arithmetic. Shared calls: `factorial-number`. All listed programs print the same exact answer string from the manifest. The excerpts can differ in algorithm, data representation, and startup cost.

40.35 Problem 35: Circular Primes

The program computes the exact answer for Circular Primes.

C. Source: `solutions/c/circular_primes_35.c`.

```
static long long solve(void) {
  unsigned char *p = sieve(1000000);
  if (!p)
    return -1;
  unsigned count = 0;
  for (unsigned n = 2; n < 1000000; n++)
    if (p[n]) {
      unsigned power = 1;
      while (power <= n / 10)
        power *= 10;
      unsigned rotated = n;
      int good = 1;
      do {
        rotated = rotated / 10 + (rotated % 10) * power;
```

Rust. Source: `solutions/rust/src/bin/p35.rs`.

```
fn solve() -> i64 {
    {
        let p = sieve(1_000_000);
        let mut count = 0;
        for n in 2u32..1_000_000 {
            if !p[n as usize] {
                continue;
            }
            let mut magnitude = 1u32;
            while magnitude <= n / 10 {
                magnitude *= 10;
            }
            let mut rotated = n;
            let mut good = true;
```

Gforth. Source: `solutions/forth/p35.fs`.

```
: p35 ( -- answer )
  1000000 sieve-build 0 0 0 0 { primes count rotated power good }
  0 to count
  1000000 2 do primes i + c@ 0<> if
    1 to power begin power i 10 / <= while power 10 * to power repeat
    i to rotated true to good
```

```
    begin rotated 10 / rotated 10 mod power * + to rotated
      primes rotated + c@ 0= if false to good then
      rotated i = good 0= or
    until
    good if count 1+ to count then
  then loop primes free throw count ;

[undefined] euler-testing [if] p35 . cr bye [then]
```

Factor. Source: `solutions/factor/euler/extra/extra.factor`.

```
:: p35 ( -- n )
    999999 euler-sieve :> primes
    0 :> answer!
    2 999999 [a..b] [| n |
        n primes nth [
            t :> valid!
            1 n decimal-digits 1 - [a..b] [| shift |
                n shift p35-rotate primes nth not [ f valid! ] when
            ] each
            valid [ answer 1 + answer! ] when
        ] when
    ] each answer ;
```

Comparison. C exposes prime generation or testing. Shared calls: `sieve`. Rust exposes prime generation or testing. Shared calls: `sieve`. Gforth exposes prime generation or testing, and modular arithmetic. Shared calls: `sieve-build`. Factor exposes prime generation or testing. Shared calls: `decimal-digits`, `euler-sieve`. All listed programs print the same exact answer string from the manifest. The excerpts can differ in algorithm, data representation, and startup cost.

40.36 Problem 36: Double-base Palindromes

Reverse the ordinary, unpadded representation independently in bases ten and two. Binary palindromes other than zero are necessarily odd.

C. Source: `solutions/c/double_base_palindromes_36.c`.

```
static bool palindrome(unsigned n, unsigned base) {
    unsigned original = n, reversed = 0;
    do { reversed = base * reversed + n % base; n /= base; } while (n);
    return reversed == original;
}

int main(void) {
    unsigned sum = 0;
    for (unsigned n = 1; n < 1000000; n += 2)
        if (palindrome(n, 10) && palindrome(n, 2)) sum += n;
    printf("%u\n", sum);
    return 0;
}
```

Rust. Source: `solutions/rust/src/bin/p36.rs`.

```
fn solve() -> i64 {
    (1u64..1_000_000)
        .step_by(2)
        .filter(|&v| palindrome(v, 10) && palindrome(v, 2))
        .sum::<u64>() as i64
}

fn main() {
    println!("{}", solve());
}
```

Gforth. Source: `solutions/forth/p36.fs`.

```
: palindrome-base? 0 0 { n base reversed x } ( n base -- flag )
  n to x 0 to reversed
  begin reversed base * x base mod + to reversed x base / to x x 0= until
  reversed n = ;

: p36 ( -- n ) 0 { total }
  1000000 1 do i 10 palindrome-base? i 2 palindrome-base? and if total i + to total then 2 +loop
     total ;

[undefined] euler-testing [if] p36 . cr bye [then]
```

Factor. Source: `solutions/factor/euler/extra/extra.factor`.

```
:: p36 ( -- n )
    0 :> total!
    1 999999 [a..b] [| n |
        n decimal-palindrome? n 2 reverse-base n = and
        [ total n + total! ] when
    ] each total ;
```

Comparison. C exposes direct bounded arithmetic. Rust exposes direct bounded arithmetic. Shared calls: `palindrome`. Gforth exposes modular arithmetic. Factor exposes direct bounded arithmetic. Shared calls: `decimal-palindrome?`. All listed programs print the same exact answer string from the manifest. The excerpts can differ in algorithm, data representation, and startup cost.

40.37 Problem 37: Truncatable Primes

Grow all right-truncatable primes from 2,3,5,7. Appending any digit except 1,3,7,9 makes a composite. Test each surviving node's left truncations; the recursion explores every right-truncatable prime.

C. Source: `solutions/c/truncatable_primes_37.c`.

```
static uint64_t sum;
static unsigned found;

static void visit(uint64_t prime) {
    uint64_t power = 1;
```

```
    while (power <= prime / 10) power *= 10;
    bool left_prime = prime >= 10;
    for (; power > 1; power /= 10)
        if (!euler_is_prime(prime % power)) { left_prime = false; break; }
    if (left_prime) { sum += prime; ++found; }
    static const unsigned digits[] = {1,3,7,9};
    if (prime > (UINT64_MAX - 9) / 10) euler_fail("Truncatable-prime search overflow");
    for (size_t i = 0; i < sizeof digits / sizeof *digits; ++i) {
        uint64_t next = 10 * prime + digits[i];
```

Rust. Source: `solutions/rust/src/bin/p37.rs`.

```
fn solve() -> i64 {
    {
        fn traverse(n: u64, sum: &mut u64, count: &mut usize) {
            let mut power = 1;
            while power <= n / 10 {
                power *= 10;
            }
            if n >= 10 && {
                let mut p = power;
                let mut ok = true;
                while p > 1 {
                    if !prime(n % p) {
                        ok = false;
                        break;
```

Gforth. Source: `solutions/forth/p37.fs`.

```
: right-truncatable? { n } ( n -- flag )
  begin n 0> while n is-prime? 0= if false exit then n 10 / to n repeat true ;

: left-truncatable? 0 { n power } ( n -- flag )
  1 to power begin power n 10 / <= while power 10 * to power repeat
  begin power 1 > while
    n power mod to n n is-prime? 0= if false exit then
    power 10 / to power
  repeat true ;

: p37 ( -- n ) 0 0 { total count }
  1000000 11 do
    i right-truncatable? i left-truncatable? and if total i + to total count 1+ to count then
    count 11 = if total unloop exit then
```

Factor. Source: `solutions/factor/euler/extra/extra.factor`.

```
:: p37 ( -- n )
    999999 euler-sieve :> sieve
    0 :> found! 0 :> answer! 11 :> n!
    [ found 11 < ] [
        n sieve p37-truncatable? [ answer n + answer! found 1 + found! ] when
        n 2 + n!
    ] while answer ;
```

Comparison. C exposes prime generation or testing. Shared calls: `euler_fail`, `euler_is_prime`. Rust exposes prime generation or testing. Gforth exposes prime generation or testing, and modular arithmetic. Shared calls: `is-prime?`. Factor exposes prime generation or testing. Shared calls: `euler-sieve`. All listed programs print the same exact answer string from the manifest. The excerpts can differ in algorithm, data representation, and startup cost.

40.38 Problem 38: Pandigital Multiples

With at least two multiples, the starting integer has at most four digits. Concatenate its successive multiples until nine digits appear.

C. Source: `solutions/c/pandigital_multiples_38.c`.

```
int main(void) {
    uint64_t best = 0;
    for (unsigned n = 1; n < 10000; ++n) {
        unsigned used = 0, digits = 0;
        uint64_t joined = 0;
        for (unsigned multiplier = 1; digits < 9; ++multiplier) {
            char text[16];
            snprintf(text, sizeof text, "%u", n * multiplier);
            bool valid = true;
            for (const char *p = text; *p; ++p) {
                unsigned d = (unsigned)(*p - '0'), bit = 1u << d;
                if (!d || (used & bit) || digits == 9) { valid = false; break; }
                used |= bit;
                ++digits;
```

Rust. Source: `solutions/rust/src/bin/p38.rs`.

```
fn solve() -> i64 {
    {
        let mut best = 0u64;
        for start in 1..10000 {
            let mut joined = String::new();
            let mut n = 1;
            while joined.len() < 9 {
                joined.push_str(&(start * n).to_string());
                n += 1;
            }
            if joined.len() == 9 && n > 2 {
                let value = joined.parse().unwrap();
                if pandigital_1_to_9(value) {
                    best = best.max(value);
```

Gforth. Source: `solutions/forth/p38.fs`.

```
: p38 ( -- n ) 0 0 0 0 0 { best joined multiplier base product }
  10000 1 do
    0 to joined 1 to multiplier
    begin joined decimal-digits 9 < while
      i multiplier * to product
      joined product append-number to joined
```

```
      multiplier 1+ to multiplier
    repeat
    multiplier 2 > joined 100000000 >= and joined 999999999 <= and if
      joined pandigital-nine? if best joined max to best then
    then
  loop best ;

[undefined] euler-testing [if] p38 . cr bye [then]
```

Factor. Source: `solutions/factor/euler/extra/extra.factor`.

```
:: p38 ( -- n )
    0 :> best!
    1 9999 [a..b] [| start |
        0 :> number! 1 :> factor!
        [ number decimal-digits 9 < ] [
            number start factor * concat-number number!
            factor 1 + factor!
        ] while
        factor 2 > number pandigital-nine? and
        [ number best max best! ] when
    ] each best ;
```

Comparison. C exposes direct bounded arithmetic. Rust exposes direct bounded arithmetic. Gforth exposes direct bounded arithmetic. Shared calls: `append-number`, `decimal-digits`, `pandigital-nine?`. Factor exposes direct bounded arithmetic. Shared calls: `concat-number`, `decimal-digits`, `pandigital-nine?`. All listed programs print the same exact answer string from the manifest. The excerpts can differ in algorithm, data representation, and startup cost.

40.39 Problem 39: Integer Right Triangles

The program computes the exact answer for Integer Right Triangles.

C. Source: `solutions/c/integer_right_triangles_39.c`.

```
static long long solve(void) {
  int best = 0, best_p = 0;
  for (int p = 12; p <= 1000; p++) {
    int count = 0;
    for (int a = 1; a < p / 3; a++)
      for (int b = a + 1; b < (p - a) / 2; b++) {
        int c = p - a - b;
        if (a * a + b * b == c * c)
          count++;
      }
    if (count > best) {
      best = count;
      best_p = p;
    }
```

Rust. Source: `solutions/rust/src/bin/p39.rs`.

```
fn solve() -> i64 {
    {
        let mut best_p = 0;
        let mut best_count = 0;
        for p in 12i64..=1000 {
            let mut count = 0;
            for a in 1..p / 3 {
                for b in a + 1..(p - a) / 2 {
                    let c = p - a - b;
                    if a * a + b * b == c * c {
                        count += 1;
                    }
                }
            }
```

Gforth. Source: `solutions/forth/p39.fs`.

```
: p39 ( -- answer )
  0 0 0 0 0 0 0 { best-perimeter best-count count numerator denominator b c }
  1001 12 do 0 to count
    i 3 / 1 do
      j j i 2 * - * to numerator 2 j i - * to denominator
      numerator 0> numerator denominator mod 0= and if
        numerator denominator / to b j i - b - to c
        i b < b c < and if count 1+ to count then
      then
    loop count best-count > if count to best-count i to best-perimeter then
  loop best-perimeter ;

[undefined] euler-testing [if] p39 . cr bye [then]
```

Factor. Source: `solutions/factor/p39.factor`.

```
:: p39 ( -- n )
    0 :> best-count!
    0 :> best-perimeter!
    6 500 [a..b] [| half |
        half 2 * :> p
        0 :> count!
        1 p 3 /i [a..b] [| a |
            p p a 2 * - * :> numerator
            p a - 2 * :> denominator
            numerator denominator mod zero? [
                numerator denominator /i :> b
                p a - b - :> c
                a b < b c < and [
                    count 1 + count!
```

Comparison. C exposes direct bounded arithmetic. Rust exposes direct bounded arithmetic. Gforth exposes modular arithmetic. Factor exposes modular arithmetic. All listed programs print the same exact answer string from the manifest. The excerpts can differ in algorithm, data representation, and startup cost.

40.40 Problem 40: Champernowne's Constant

The program computes the exact answer for Champernowne's Constant.

C. Source: `solutions/c/champernownes_constant_40.c`.

```
static long long solve(void) {
  unsigned targets[] = {1, 10, 100, 1000, 10000, 100000, 1000000};
  int pos = 0;
  unsigned digit_pos = 0;
  long long product = 1;
  char buf[24];
  for (unsigned n = 1; pos < 7; n++) {
    sprintf(buf, "%u", n);
    for (char *p = buf; *p; p++) {
      digit_pos++;
      if (digit_pos == targets[pos]) {
        product *= *p - '0';
        pos++;
        if (pos == 7)
```

Rust. Source: `solutions/rust/src/bin/p40.rs`.

```
fn solve() -> i64 {
    {
        let positions = [1, 10, 100, 1000, 10000, 100000, 1_000_000];
        let mut at = 0;
        let mut digit = 0;
        let mut product = 1i64;
        for n in 1.. {
            for c in n.to_string().bytes() {
                digit += 1;
                if digit == positions[at] {
                    product *= i64::from(c - b'0');
                    at += 1;
                    if at == positions.len() {
                        break;
```

Gforth. Source: `solutions/forth/p40.fs`.

```
create champernowne-targets 1 , 10 , 100 , 1000 , 10000 , 100000 , 1000000 ,

: p40 ( -- answer )
  0 0 1 1 0 0 { at position product n divisor digit }
  begin at 7 < while
    1 to divisor begin divisor n 10 / <= while divisor 10 * to divisor repeat
    begin divisor 0> while
      n divisor / 10 mod to digit position 1+ to position
      position champernowne-targets at cells + @ = if
        product digit * to product at 1+ to at
        at 7 = if product exit then
      then
      divisor 10 / to divisor
    repeat n 1+ to n
```

Factor. Source: `solutions/factor/euler/extra/extra.factor`.

```
:: p40 ( -- n )
    { 1 10 100 1000 10000 100000 1000000 }
    [ p40-digit ] map product ;
```

Comparison. C exposes direct bounded arithmetic. Rust exposes direct bounded arithmetic. Gforth exposes modular arithmetic. Factor exposes direct bounded arithmetic. All listed programs print the same exact answer string from the manifest. The excerpts can differ in algorithm, data representation, and startup cost.

40.41 Problem 41: Pandigital Prime

Enumerate permutations in decreasing numerical order. Digit lengths whose triangular digit sum is divisible by three cannot give a prime.

C. Source: `solutions/c/pandigital_prime_41.c`.

```
static uint64_t descending(unsigned length, unsigned remaining,
                           unsigned used, uint64_t value) {
    if (!remaining) return euler_is_prime(value) ? value : 0;
    for (unsigned digit = length; digit; --digit)
        if (!(used & (1u << digit))) {
            uint64_t answer = descending(length, remaining - 1,
                                        used | (1u << digit), 10 * value + digit);
            if (answer) return answer;
        }
    return 0;
}

int main(void) {
    for (unsigned length = 9; length; --length) {
```

Rust. Source: `solutions/rust/src/bin/p41.rs`.

```
fn solve() -> i64 {
    {
        let mut best = 0u64;
        for len in [7usize, 4] {
            let mut p: Vec<u8> = (1..=len as u8).collect();
            loop {
                let n = p.iter().fold(0u64, |v, &d| v * 10 + d as u64);
                if prime(n) {
                    best = best.max(n);
                }
                if !next_permutation(&mut p) {
                    break;
                }
            }
```

Gforth. Source: `solutions/forth/p41.fs`.

```
: pandigital-seven? 0 0 0 { n mask digit bit } ( n -- flag )
  0 to mask 7 0 do
    n 10 mod to digit digit 1 < digit 7 > or if false unloop exit then
    1 digit lshift to bit mask bit and if false unloop exit then
    mask bit or to mask n 10 / to n
  loop n 0= mask 254 = and ;

: p41 ( -- n ) 7654321 { number }
  begin number 1234567 >= while
    number pandigital-seven? if number is-prime? if number exit then then
    number 2 - to number
  repeat abort" No 7-pandigital prime" ;

[undefined] euler-testing [if] p41 . cr bye [then]
```

Factor. Source: `solutions/factor/euler/extra/extra.factor`.

```
:: p41 ( -- n )
    7654321 :> n! 0 :> answer!
    [ n 1234567 >= answer 0 = and ] [
        n pandigital-seven? n prime? and [ n answer! ] when
        n 2 - n!
    ] while answer ;
```

Comparison. C exposes prime generation or testing. Shared calls: `euler_is_prime`. Rust exposes prime generation or testing. Shared calls: `next_permutation`. Gforth exposes prime generation or testing, and modular arithmetic. Shared calls: `is-prime?`. Factor exposes prime generation or testing. Shared calls: `pandigital-seven?`. All listed programs print the same exact answer string from the manifest. The excerpts can differ in algorithm, data representation, and startup cost.

40.42 Problem 42: Coded Triangle Numbers

Read the official quoted words, sum their A=1,...,Z=26 scores, and recognize triangular scores by the exact square test 8s+1=(2n+1)^2.

C. Source: `solutions/c/coded_triangle_numbers_42.c`.

```
static int next_nonspace(FILE *file) {
    int ch;
    do { ch = fgetc(file); } while (ch != EOF && isspace((unsigned char)ch));
    return ch;
}

int main(void) {
    FILE *file = euler_open("tests/data/p042_words.txt");
    unsigned words = 0, triangles = 0;
    int ch = next_nonspace(file);
    for (;;) {
        if (ch != '"') euler_fail("Expected a quoted word in p042_words.txt");
        unsigned score = 0, length = 0;
        while ((ch = fgetc(file)) != EOF && ch != '"') {
```

Rust. Source: `solutions/rust/src/bin/p42.rs`.

```
fn solve() -> i64 {
    {
        let words = include_str!("../../../../tests/data/p042_words.txt");
        words
            .split(',')
            .filter(|w| {
                let w = w.trim().trim_matches('"');
                let sum: u64 = w.bytes().map(|b| (b - b'A' + 1) as u64).sum();
                is_square(8 * sum + 1)
            })
            .count() as i64
    }
}
```

Gforth. Source: `solutions/forth/p42.fs`.

```
: p42 ( -- n )
  s" tests/data/p042_words.txt" 100000 read-fixture
  0 0 0 0 { raw length score count ch root }
  0 to score 0 to count
  length 0 do raw i + c@ to ch
    ch [char] A >= ch [char] Z <= and if
      score ch [char] A - 1+ + to score
    else ch 34 = score 0> and if
      score 8 * 1+ integer-sqrt to root
      root root * score 8 * 1+ = if count 1+ to count then
      0 to score
    then then
  loop raw free throw count ;
```

Factor. Source: `solutions/factor/euler/extra/extra.factor`.

```
:: p42 ( -- n )
    "tests/data/p042_words.txt" read-upper-words
    [| word | word [ 64 - ] map sum :> score
      score 8 * 1 + euler-isqrt :> root
      root sq score 8 * 1 + = ] count ;
```

Comparison. C exposes direct bounded arithmetic. Shared calls: `euler_isqrt`, `euler_open`. Rust exposes fixture parsing. Gforth exposes fixture parsing. Shared calls: `integer-sqrt`, `read-fixture`. Factor exposes fixture parsing. Shared calls: `euler-isqrt`, `read-upper-words`. All listed programs print the same exact answer string from the manifest. The excerpts can differ in algorithm, data representation, and startup cost.

40.43 Problem 43: Sub-string Divisibility

Extend a 0..9 permutation, checking each three-digit divisibility condition as soon as its final digit is available.

C. Source: `solutions/c/sub_string_divisibility_43.c`.

```
static uint64_t search(unsigned position, unsigned used, uint64_t value) {
    static const unsigned divisors[] = {2,3,5,7,11,13,17};
    if (position == 10) return value;
    uint64_t sum = 0;
    for (unsigned digit = position ? 0 : 1; digit < 10; ++digit) {
        if (used & (1u << digit)) continue;
        uint64_t next = 10 * value + digit;
        if (position >= 3 && (next % 1000) % divisors[position - 3]) continue;
        sum += search(position + 1, used | (1u << digit), next);
    }
    return sum;
}

int main(void) {
```

Rust. Source: `solutions/rust/src/bin/p43.rs`.

```
fn solve() -> i64 {
    {
        fn search(at: usize, used: u16, n: u64) -> u64 {
            const DIV: [u64; 7] = [2, 3, 5, 7, 11, 13, 17];
            if at == 10 {
                return n;
            }
            let mut total = 0;
            for d in (if at == 0 { 1 } else { 0 })..10 {
                if used & (1 << d) != 0 {
                    continue;
                }
                let next = n * 10 + d;
                if at >= 3 && next % 1000 % DIV[at - 3] != 0 {
```

Gforth. Source: `solutions/forth/p43.fs`.

```
: p43-search 0 0 { position used value total digit } ( position used number -- sum )
  position 10 = if value exit then
  10 0 do
    i to digit
    position 0<> digit 0<> or
    1 digit lshift used and 0= and if
      value 10 * digit +
      position 3 >= if
        dup 1000 mod
        position 3 - case
          0 of 2 endof 1 of 3 endof 2 of 5 endof
          3 of 7 endof 4 of 11 endof 5 of 13 endof 6 of 17 endof
        endcase mod 0<>
      else false then
```

Factor. Source: `solutions/factor/euler/advanced/advanced.factor`.

```
:: p43 ( -- n )
    0 :> result!
```

```
    0 999 [a..b] [| seed |
        seed 17 mod zero? [ seed 3 0 p43-extend result + result! ] when
    ] each result ;
```

Comparison. C exposes direct bounded arithmetic. Rust exposes recursive search. Gforth exposes modular arithmetic, and recursive search. Factor exposes modular arithmetic. All listed programs print the same exact answer string from the manifest. The excerpts can differ in algorithm, data representation, and startup cost.

40.44 Problem 44: Pentagon Numbers

The program computes the exact answer for Pentagon Numbers.

C. Source: `solutions/c/pentagon_numbers_44.c`.

```
static long long solve(void) {
  long long best = 0;
  for (uint64_t j = 2; j < 5000; j++) {
    uint64_t pj = j * (3 * j - 1) / 2;
    for (uint64_t k = j - 1; k > 0; k--) {
      uint64_t pk = k * (3 * k - 1) / 2, diff = pj - pk;
      if (best && (long long)diff >= best)
        break;
      if (pentagonal(diff) && pentagonal(pj + pk))
        best = (long long)diff;
    }
    if (best && 3 * j - 2 > (uint64_t)best)
      break;
  }
```

Rust. Source: `solutions/rust/src/bin/p44.rs`.

```
fn solve() -> i64 {
    {
        let mut best = None;
        for j in 2u64..5000 {
            let pj = j * (3 * j - 1) / 2;
            for k in (1..j).rev() {
                let pk = k * (3 * k - 1) / 2;
                let diff = pj - pk;
                if best.is_some_and(|current| diff >= current) {
                    break;
                }
                if pentagonal(diff) && pentagonal(pj + pk) {
                    best = Some(diff);
                }
```

Gforth. Source: `solutions/forth/p44.fs`.

```
: p44 ( -- answer )
  0 0 0 0 { best pj pk difference }
```

```
  5000 2 do
    i i 3 * 1- * 2 / to pj
    \ A descending +LOOP includes its limit. Use 1, never the invalid P_0.
    1 i 1- do
      i i 3 * 1- * 2 / to pk pj pk - to difference
      best 0<> difference best >= and if leave then
      difference pentagonal? if
        pj pk + pentagonal? if difference to best then
      then
    -1 +loop
  loop best ;
```

Factor. Source: `solutions/factor/euler/advanced/advanced.factor`.

```
:: p44 ( -- n )
    0 :> best!
    2 4000 [a..b] [| k |
        k 3 k * 1 - * 2 /i :> large
        1 k 1 - [a..b] [| j |
            j 3 j * 1 - * 2 /i :> small
            large small - :> gap
            best zero? gap best < or [
                gap 5 polygonal? large small + 5 polygonal? and
                [ gap best! ] when
            ] when
        ] each
    ] each best ;
```

Comparison. C exposes direct bounded arithmetic. Shared calls: `pentagonal`. Rust exposes direct bounded arithmetic. Gforth exposes direct bounded arithmetic. Shared calls: `pentagonal?`. Factor exposes direct bounded arithmetic. Shared calls: `polygonal?`. All listed programs print the same exact answer string from the manifest. The excerpts can differ in algorithm, data representation, and startup cost.

40.45 Problem 45: Triangular, Pentagonal, and Hexagonal

H_n=T_(2n-1), so test only hexagonal numbers after H_143=40755. A positive x is pentagonal exactly when 24x+1 is a square whose positive square root is congruent to five modulo six.

C. Source: `solutions/c/triangular_pentagonal_and_hexagonal_45.c`.

```
int main(void) {
    for (uint64_t n = 144; ; ++n) {
        if (n > UINT64_MAX / 2) euler_fail("Hexagonal index overflow");
        if (n > UINT64_MAX / (2 * n - 1)) euler_fail("Hexagonal search overflow");
        uint64_t value = n * (2 * n - 1);
        if (value > (UINT64_MAX - 1) / 24) euler_fail("Pentagonal test overflow");
        uint64_t discriminant = 24 * value + 1, root = euler_isqrt(discriminant);
        if (root * root == discriminant && (root + 1) % 6 == 0) {
            printf("%" PRIu64 "\n", value);
            return 0;
        }
```

```
    }
}
```

Rust. Source: `solutions/rust/src/bin/p45.rs`.

```
fn solve() -> i64 {
    {
        let mut n = 144u64;
        loop {
            let hex = n * (2 * n - 1);
            let discr = 24 * hex + 1;
            let root = isqrt(discr);
            if root * root == discr && (root + 1) % 6 == 0 {
                break hex as i64;
            }
            n += 1;
        }
    }
}
```

Gforth. Source: `solutions/forth/p45.fs`.

```
: p45 ( -- n ) 144 { index }
  begin index index 2* 1- * dup pentagonal? if exit then drop index 1+ to index again ;

[undefined] euler-testing [if] p45 . cr bye [then]
```

Factor. Source: `solutions/factor/euler/extra/extra.factor`.

```
:: p45 ( -- n )
    144 :> n! 0 :> answer!
    [ answer zero? ] [
        n 2 * 1 - n * :> hexagonal
        hexagonal 5 polygonal? [ hexagonal answer! ] when
        n 1 + n!
    ] while answer ;
```

Comparison. C exposes direct bounded arithmetic. Shared calls: `euler_fail`, `euler_isqrt`. Rust exposes direct bounded arithmetic. Shared calls: `isqrt`. Gforth exposes direct bounded arithmetic. Shared calls: `pentagonal?`. Factor exposes direct bounded arithmetic. Shared calls: `polygonal?`. All listed programs print the same exact answer string from the manifest. The excerpts can differ in algorithm, data representation, and startup cost.

40.46 Problem 46: Goldbach's Other Conjecture

The program computes the exact answer for Goldbach's Other Conjecture.

C. Source: `solutions/c/goldbachs_other_conjecture_46.c`.

```
static long long solve(void) {
  for (unsigned n = 9;; n += 2)
    if (!prime(n)) {
      int found = 0;
      for (unsigned p = 2; p < n; p++)
        if (prime(p)) {
          unsigned half = (n - p) / 2;
          if ((n - p) % 2)
            continue;
          unsigned root = (unsigned)sqrt((double)half);
          if (root * root == half) {
            found = 1;
            break;
          }
```

Rust. Source: `solutions/rust/src/bin/p46.rs`.

```
fn solve() -> i64 {
    {
        let mut n = 9;
        loop {
            if !prime(n) {
                let mut representation = false;
                for p in 2..n {
                    if !prime(p) {
                        continue;
                    }
                    if (n - p) % 2 != 0 {
                        continue;
                    }
                    let half = (n - p) / 2;
```

Gforth. Source: `solutions/forth/p46.fs`.

```
: p46 ( -- answer )
  9 0 0 0 { n representation half root }
  begin n is-prime? 0= if
    false to representation
    n 2 do i is-prime? if n i - 2 mod 0= if
      n i - 2 / to half half integer-sqrt to root
      root root * half = if true to representation leave then
    then then loop
    representation 0= if n exit then
  then n 2 + to n again ;

[undefined] euler-testing [if] p46 . cr bye [then]
```

Factor. Source: `solutions/factor/euler/extra/extra.factor`.

```
:: p46 ( -- n )
    9 :> value! 0 :> answer!
    [ answer 0 = ] [
        value prime? not [
            t :> unrepresented!
```

```
            2 value 1 - [a..b] [| x |
                x prime? value x - 2 mod zero? and [
                    value x - 2 /i euler-isqrt :> root
                    root sq value x - 2 /i = [ f unrepresented! ] when
                ] when
            ] each
            unrepresented [ value answer! ] when
        ] when
        value 2 + value!
```

Comparison. C exposes prime generation or testing. Shared calls: `prime`. Rust exposes prime generation or testing. Shared calls: `prime`. Gforth exposes prime generation or testing, and modular arithmetic. Shared calls: `integer-sqrt`, `is-prime?`. Factor exposes prime generation or testing, and modular arithmetic. Shared calls: `euler-isqrt`. All listed programs print the same exact answer string from the manifest. The excerpts can differ in algorithm, data representation, and startup cost.

40.47 Problem 47: Distinct Primes Factors

The program computes the exact answer for Distinct Primes Factors.

C. Source: `solutions/c/distinct_primes_factors_47.c`.

```
static long long solve(void) {
  unsigned streak = 0;
  for (unsigned n = 2; n < 1000000; n++) {
    if (distinct_factors(n) == 4) {
      if (++streak == 4)
        return n - 3;
    } else
      streak = 0;
  }
  return -1;
}

int main(void) {
  long long answer = solve();
```

Rust. Source: `solutions/rust/src/bin/p47.rs`.

```
fn solve() -> i64 {
    {
        let mut consecutive = 0;
        let mut answer = 0;
        for n in 2u32..1_000_000 {
            if distinct_factors(n) == 4 {
                consecutive += 1;
                if consecutive == 4 {
                    answer = n - 3;
                    break;
                }
            } else {
                consecutive = 0;
            }
```

Gforth. Source: `solutions/forth/p47.fs`.

```
: distinct-prime-factors 0 0 0 { n value divisor count } ( n -- distinct count )
  n to value 2 to divisor 0 to count
  begin divisor value divisor / <= while
    value divisor mod 0= if count 1+ to count
      begin value divisor mod 0= while value divisor / to value repeat
    then divisor 1+ to divisor
  repeat value 1 > if count 1+ to count then count ;

: p47 ( -- answer )
  0 { streak }
  1000000 2 do i distinct-prime-factors 4 = if
    streak 1+ to streak streak 4 = if i 3 - unloop exit then
  else 0 to streak then loop -1 ;
```

Factor. Source: `solutions/factor/euler/extra/extra.factor`.

```
:: p47 ( -- n )
    0 :> streak! 2 :> n!
    [ streak 4 < ] [
        n p47-distinct 4 = [ streak 1 + streak! ] [ 0 streak! ] if
        n 1 + n!
    ] while n 4 - ;
```

Comparison. C exposes prime generation or testing. Shared calls: `distinct_factors`. Rust exposes prime generation or testing. Gforth exposes prime generation or testing, and modular arithmetic. Factor exposes direct bounded arithmetic. All listed programs print the same exact answer string from the manifest. The excerpts can differ in algorithm, data representation, and startup cost.

40.48 Problem 48: Self Powers

The program computes the exact answer for Self Powers.

C. Source: `solutions/c/self_powers_48.c`.

```
static long long solve(void) {
  mpz_t sum, base, term, mod;
  mpz_inits(sum, base, term, mod, NULL);
  mpz_ui_pow_ui(mod, 10, 10);
  for (unsigned n = 1; n <= 1000; n++) {
    mpz_set_ui(base, n);
    mpz_powm_ui(term, base, n, mod);
    mpz_add(sum, sum, term);
    mpz_mod(sum, sum, mod);
  }
  /* sum < 10^10. Decimal conversion avoids unsigned-long truncation
   * on LLP64 targets such as 64-bit Windows. */
  char decimal[32];
  mpz_get_str(decimal, 10, sum);
```

Rust. Source: `solutions/rust/src/bin/p48.rs`.

```
fn solve() -> i64 {
    {
        let modulus = BigUint::from(10u64).pow(10);
        let mut total = BigUint::zero();
        for n in 1u32..=1000 {
            let b = BigUint::from(n);
            total = (total + b.modpow(&b, &modulus)) % &modulus;
        }
        total.to_string().parse::<i64>().unwrap()
    }
}

fn main() {
    println!("{}", solve());
```

Gforth. Source: `solutions/forth/p48.fs`.

```
: p48 ( -- answer )
  0 { sum }
  1001 1 do i i 10000000000 power-mod sum + 10000000000 mod to sum loop sum ;

[undefined] euler-testing [if] p48 . cr bye [then]
```

Factor. Source: `solutions/factor/euler/extra/extra.factor`.

```
:: p48 ( -- n )
    0 :> total!
    1 1000 [a..b] [| n |
        n n 10000000000 modular-pow total + 10000000000 mod total!
    ] each total ;
```

Comparison. C exposes modular arithmetic, and exact large-integer arithmetic. Rust exposes exact large-integer arithmetic. Gforth exposes modular arithmetic. Shared calls: `power-mod`. Factor exposes modular arithmetic. Shared calls: `modular-pow`. All listed programs print the same exact answer string from the manifest. The excerpts can differ in algorithm, data representation, and startup cost.

40.49 Problem 49: Prime Permutations

Exhaust all increasing three-term four-digit arithmetic progressions. A five-bit counter per decimal digit gives an order-free signature.

C. Source: `solutions/c/prime_permutations_49.c`.

```
static uint64_t signature(unsigned value) {
    uint64_t result = 0;
    do { result += UINT64_C(1) << (5 * (value % 10)); value /= 10; } while (value);
    return result;
}
```

```
int main(void) {
    unsigned char *prime = euler_sieve(9999);
    for (unsigned a = 1000; a < 10000; ++a) if (prime[a])
        for (unsigned b = a + 1; 2 * b - a < 10000; ++b) {
            unsigned c = 2 * b - a;
            if (a == 1487 && b == 4817) continue; /* The supplied example. */
            if (prime[b] && prime[c] && signature(a) == signature(b) &&
                signature(a) == signature(c)) {
```

Rust. Source: `solutions/rust/src/bin/p49.rs`.

```
fn solve() -> i64 {
    {
        let p = sieve(9999);
        let mut answer = 0;
        'outer: for a in 1000..10000usize {
            if !p[a] {
                continue;
            }
            for b in a + 1..10000 {
                let c = 2 * b - a;
                if c >= 10000 {
                    break;
                }
                if a == 1487 && b == 4817 {
```

Gforth. Source: `solutions/forth/p49.fs`.

```
: p49 ( -- n )
  9999 sieve-build 0 0 0 0 { primes b c answer sig }
  10000 1000 do primes i + c@ if
    i digit-signature to sig
    10000 i 1+ ?do
      j 2* i - to c
      c 10000 >= if leave then
      primes j + c@ primes c + c@ and if
        i 1487 = j 4817 = and 0= if
          sig j digit-signature = sig c digit-signature = and if
            i 100000000 * j 10000 * + c + to answer
            primes free throw answer unloop unloop exit
          then
        then
```

Factor. Source: `solutions/factor/euler/extra/extra.factor`.

```
:: p49 ( -- n )
    9999 euler-sieve :> sieve
    0 :> answer!
    1000 9999 [a..b] [| a |
        a sieve nth [
            a 1 + 9999 [a..b] [| b |
                b 2 * a - :> c
                c 10000 < [
                    b sieve nth c sieve nth and
```

```
                    a digit-signature b digit-signature = and
                    a digit-signature c digit-signature = and
                    a 1487 = b 4817 = and not and [
                        a 100000000 * b 10000 * + c + answer!
                    ] when
```

Comparison. C exposes prime generation or testing. Shared calls: `euler_sieve`. Rust exposes prime generation or testing. Shared calls: `decimal_signature`, `sieve`. Gforth exposes prime generation or testing. Shared calls: `digit-signature`, `sieve-build`. Factor exposes prime generation or testing. Shared calls: `digit-signature`, `euler-sieve`. All listed programs print the same exact answer string from the manifest. The excerpts can differ in algorithm, data representation, and startup cost.

40.50 Problem 50: Consecutive Prime Sum

The program computes the exact answer for Consecutive Prime Sum.

C. Source: `solutions/c/consecutive_prime_sum_50.c`.

```
static long long solve(void) {
  unsigned char *p = sieve(1000000);
  if (!p)
    return -1;
  unsigned count = 0;
  uint64_t prefix[80001] = {0};
  for (unsigned n = 2; n < 1000000; n++)
    if (p[n]) {
      prefix[count + 1] = prefix[count] + n;
      count++;
    }
  unsigned best_len = 0, best_prime = 0;
  for (unsigned i = 0; i < count; i++) {
    for (unsigned j = i + best_len + 1; j <= count; j++) {
```

Rust. Source: `solutions/rust/src/bin/p50.rs`.

```
fn solve() -> i64 {
    {
        let p = sieve(1_000_000);
        let mut prefix = vec![0u64];
        for (n, &is_prime) in p.iter().enumerate().skip(2).take(999998) {
            if is_prime {
                prefix.push(*prefix.last().unwrap() + n as u64);
            }
        }
        let (mut length, mut best) = (0usize, 0i64);
        for i in 0..prefix.len() - 1 {
            for j in (i + length + 1)..prefix.len() {
                let sum = prefix[j] - prefix[i];
                if sum >= 1_000_000 {
```

Gforth. Source: `solutions/forth/p50.fs`.

```
: p50 ( -- answer )
  1000000 sieve-build 0 0 0 0 0 0 0 { primes count best-length best-prime start-index end-index
    value prefix }
  80001 cells allocate throw to prefix
  0 to count 0 to best-length 0 to best-prime 0 0 prefix prefix!
  1000000 2 do primes i + c@ 0<> if
    count prefix prefix@ i + count 1+ prefix prefix! count 1+ to count
  then loop
  0 to start-index
  begin start-index count < while
    start-index best-length + 1+ to end-index
    begin end-index count <= while
      end-index prefix prefix@ start-index prefix prefix@ - to value
      value 1000000 >= if
        \ LEAVE is for counted loops, not BEGIN loops.
```

Factor. Source: `solutions/factor/euler/advanced/advanced.factor`.

```
:: p50 ( -- n )
    999999 euler-sieve :> sieve
    V{ } clone :> primes
    2 999999 [a..b] [| n | n sieve nth [ n primes push ] when ] each
    primes length 1 + 0 <array> :> prefix
    0 0 prefix set-nth
    1 primes length [a..b] [| i |
        i 1 - prefix nth i 1 - primes nth + i prefix set-nth
    ] each
    0 :> bestlen! 0 :> bestvalue!
    0 primes length 1 - [a..b] [| a |
        a bestlen + 1 + :> b!
        [ b primes length <= ] [
            b prefix nth a prefix nth - :> value
```

Comparison. C exposes prime generation or testing. Shared calls: `sieve`. Rust exposes prime generation or testing. Shared calls: `sieve`. Gforth exposes prime generation or testing. Shared calls: `prefix!`, `prefix@`, `sieve-build`. Factor exposes prime generation or testing. Shared calls: `euler-sieve`. All listed programs print the same exact answer string from the manifest. The excerpts can differ in algorithm, data representation, and startup cost.

40.51 Problem 51: Prime Digit Replacements

Search increasing digit lengths, then primes. A replacement mask may select only positions that originally contain the same digit. Test all ten replacements, excluding a leading zero, using a full-length sieve.

C. Source: `solutions/c/prime_digit_replacements_51.c`.

```
int main(void) {
    unsigned lower = 10, upper = 99;
    for (unsigned length = 2; length < 10; ++length) {
        unsigned char *prime = euler_sieve(upper);
```

```
        for (unsigned n = lower; n <= upper; ++n) if (prime[n]) {
            unsigned digits[10], powers[10], value = n, power = 1;
            for (unsigned i = 0; i < length; ++i) {
                digits[i] = value % 10; value /= 10;
                powers[i] = power; power *= 10;
            }
            for (unsigned mask = 1; mask < (1u << length); ++mask) {
                unsigned selected_digit = 10, coefficient = 0;
                bool consistent = true;
                for (unsigned i = 0; i < length; ++i) if (mask & (1u << i)) {
```

Rust. Source: `solutions/rust/src/bin/p51.rs`.

```
fn solve() -> i64 {
    {
        let mut answer = None;
        'length: for len in 2..=7 {
            let upper = 10usize.pow(len as u32) - 1;
            let lower = 10usize.pow(len as u32 - 1);
            let p = sieve(upper);
            for n in lower..=upper {
                if !p[n] {
                    continue;
                }
                let d: Vec<usize> = (0..len).map(|k| (n / 10usize.pow(k as u32)) % 10).collect();

                for mask in 1..(1 << len) {
                    let selected: Vec<usize> = (0..len).filter(|k| mask & (1 << k) != 0).collect
    ();
```

Gforth. Source: `solutions/forth/p51.fs`.

```
: p51-family? 0 0 0 0 0 0 0 { number mask primes selected coefficient digit base count first
    candidate } ( n mask sieve -- flag )
  10 to selected 0 to coefficient
  number decimal-digits 0 do
    mask 1 i lshift and if
      number i pow10 / 10 mod to digit
      selected 10 = if digit to selected else
        selected digit <> if false unloop exit then
      then
      coefficient i pow10 + to coefficient
    then
  loop
  number selected coefficient * - to base
  0 to count 0 to first
  10 0 do
```

Factor. Source: `solutions/factor/euler/advanced/advanced.factor`.

```
:: p51 ( -- n )
    999999 euler-sieve :> sieve
    0 :> answer!
    11 999999 [a..b] [| n |
        answer zero? n sieve nth and [
```

```
            1 2 n decimal-digits ^ 1 - [a..b] [| mask |
                answer zero? [
                    n mask sieve p51-family [ n answer! ] when
                ] when
            ] each
        ] when
    ] each answer ;
```

Comparison. C exposes prime generation or testing. Shared calls: `euler_sieve`. Rust exposes prime generation or testing. Shared calls: `sieve`. Gforth exposes prime generation or testing, and modular arithmetic. Shared calls: `decimal-digits`, `pow10`, `sieve-build`. Factor exposes prime generation or testing. Shared calls: `decimal-digits`, `euler-sieve`. All listed programs print the same exact answer string from the manifest. The excerpts can differ in algorithm, data representation, and startup cost.

40.52 Problem 52: Permuted Multiples

For each digit length, n and 6n must stay in the same decimal interval. Compare exact digit multiplicities for n,2n,...,6n.

C. Source: `solutions/c/permuted_multiples_52.c`.

```
static uint64_t signature(uint64_t value) {
    uint64_t result = 0;
    do { result += UINT64_C(1) << (5 * (value % 10)); value /= 10; } while (value);
    return result;
}

int main(void) {
    uint64_t lower = 1, upper = 9;
    for (;;) {
        for (uint64_t n = lower; n <= upper / 6; ++n) {
            bool same = true;
            uint64_t original = signature(n);
            for (unsigned multiplier = 2; multiplier <= 6; ++multiplier)
                if (signature(n * multiplier) != original) { same = false; break; }
```

Rust. Source: `solutions/rust/src/bin/p52.rs`.

```
fn solve() -> i64 {
    {
        let mut lower = 1u64;
        let mut upper = 9u64;
        loop {
            let answer = (lower..=upper / 6).find(|&n| {
                let signature = decimal_signature(n);
                (2..=6).all(|k| decimal_signature(k * n) == signature)
            });
            if let Some(v) = answer {
                break v as i64;
            }
            lower = upper + 1;
```

```
            upper = upper * 10 + 9;
```

Gforth. Source: `solutions/forth/p52.fs`.

```
: p52 ( -- n ) 1 9 0 0 { lower upper signature candidate }
  begin upper 6 / 1+ lower ?do
      i digit-signature to signature
      true 7 2 do signature i j * digit-signature = and loop
      if i unloop exit then
    loop upper 1+ to lower upper 10 * 9 + to upper
  again ;

[undefined] euler-testing [if] p52 . cr bye [then]
```

Factor. Source: `solutions/factor/euler/extra/extra.factor`.

```
:: p52 ( -- n )
    1 :> magnitude! 0 :> answer!
    [ answer zero? ] [
        magnitude magnitude 10 * 1 - [a..b] [| n |
            n 6 * magnitude 10 * < answer zero? and [
                n digit-signature :> signature
                2 6 [a..b] [| factor |
                    n factor * digit-signature signature =
                ] all? [ n answer! ] when
            ] when
        ] each
        magnitude 10 * magnitude!
    ] while answer ;
```

Comparison. C exposes direct bounded arithmetic. Shared calls: `euler_fail`. Rust exposes direct bounded arithmetic. Shared calls: `decimal_signature`. Gforth exposes direct bounded arithmetic. Shared calls: `digit-signature`. Factor exposes direct bounded arithmetic. Shared calls: `digit-signature`. All listed programs print the same exact answer string from the manifest. The excerpts can differ in algorithm, data representation, and startup cost.

40.53 Problem 53: Combinatoric Selections

Pascal's recurrence is monotone in its nonnegative inputs. Saturating coefficients at 1000001 preserves precisely the required comparison.

C. Source: `solutions/c/combinatoric_selections_53.c`.

```
int main(void) {
    enum { THRESHOLD = 1000000 };
    unsigned row[101] = {1}, count = 0;
    for (unsigned n = 1; n <= 100; ++n) {
        for (unsigned r = n; r; --r) {
            unsigned sum = row[r] + row[r - 1];
            row[r] = sum > THRESHOLD ? THRESHOLD + 1 : sum;
        }
```

```
        for (unsigned r = 0; r <= n; ++r) count += row[r] > THRESHOLD;
    }
    printf("%u\n", count);
    return 0;
}
```

Rust. Source: `solutions/rust/src/bin/p53.rs`.

```
fn solve() -> i64 {
    {
        let mut row = [0u64; 101];
        row[0] = 1;
        let mut count = 0;
        for n in 1..=100 {
            for j in (1..=n).rev() {
                row[j] = (row[j] + row[j - 1]).min(1_000_001);
            }
            count += (0..=n).filter(|&j| row[j] > 1_000_000).count();
        }
        count as i64
    }
}
```

Gforth. Source: `solutions/forth/p53.fs`.

```
: p53 ( -- n )
  101 cells allocate throw 0 0 { row count value }
  row 101 cells erase 1 row !
  101 1 do
    i 1+ 1 do
      j i - 1+ to value
      value row prefix@ value 1- row prefix@ + 1000001 min
      value row prefix!
    loop
    i 1+ 0 do i row prefix@ 1000000 > if count 1+ to count then loop
  loop row free throw count ;

[undefined] euler-testing [if] p53 . cr bye [then]
```

Factor. Source: `solutions/factor/euler/extra/extra.factor`.

```
:: p53 ( -- n )
    101 0 <array> :> coeff
    1 0 coeff set-nth
    0 :> answer!
    1 100 [a..b] [| n |
        n 1 [a..b] [| k |
            k coeff nth k 1 - coeff nth + 1000001 min
            k coeff set-nth
        ] each
        0 n [a..b] [| k |
            k coeff nth 1000000 > [ answer 1 + answer! ] when
        ] each
    ] each answer ;
```

Comparison. C exposes direct bounded arithmetic. Rust exposes direct bounded arithmetic. Gforth exposes direct bounded arithmetic. Shared calls: `prefix!`, `prefix@`. Factor exposes direct bounded arithmetic. All listed programs print the same exact answer string from the manifest. The excerpts can differ in algorithm, data representation, and startup cost.

40.54 Problem 54: Poker Hands

Rank a hand by its category followed by all required tie breakers. Base-15 packing makes integer comparison agree with poker ordering. A,2,3,4,5 is treated as the five-high straight.

C. Source: `solutions/c/poker_hands_54.c`.

```
static unsigned rank_value(char ch) {
    const char *ranks = "23456789TJQKA";
    const char *found = strchr(ranks, ch);
    if (!found) euler_fail("Invalid poker rank");
    return (unsigned)(found - ranks) + 2;
}

static uint64_t hand_value(const unsigned ranks[5], const char suits[5]) {
    unsigned count[15] = {0}, key[5] = {0}, category = 0;
    bool flush = true;
    for (unsigned i = 0; i < 5; ++i) {
        ++count[ranks[i]];
        if (suits[i] != suits[0]) flush = false;
    }
```

Rust. Source: `solutions/rust/src/bin/p54.rs`.

```
fn poker_value(cards: &[&str]) -> u64 {
    let mut counts = [0usize; 15];
    let mut flush = true;
    let suit = cards[0].as_bytes()[1];
    for card in cards {
        let b = card.as_bytes();
        let value = b"23456789TJQKA"
            .iter()
            .position(|&c| c == b[0])
            .expect("rank")
            + 2;
        counts[value] += 1;
        flush &= b[1] == suit;
    }
```

Gforth. Source: `solutions/forth/p54.fs`.

```
: p54-rank { char } ( ascii -- rank )
  char [char] 2 >= char [char] 9 <= and if char [char] 0 - exit then
  char case [char] T of 10 endof [char] J of 11 endof
    [char] Q of 12 endof [char] K of 13 endof [char] A of 14 endof
    abort" Invalid card rank" endcase ;

: p54-value 0 0 0 0 0 0 0 0 0 0 0 0 0
```

```
  { text counts ordered pairs keys suit flush unique trip four pair-count straight category rank
     } ( ptr -- rank value )
  15 cells allocate throw to counts
  5 cells allocate throw to ordered
  2 cells allocate throw to pairs
  5 cells allocate throw to keys
  counts 15 cells erase keys 5 cells erase
  true to flush 0 to unique 0 to trip 0 to four 0 to pair-count 0 to straight
```

Factor. Source: `solutions/factor/euler/advanced/advanced.factor`.

```
:: p54 ( -- n )
    "tests/data/p054_poker.txt" utf8 file-contents
    [ whitespace? ] split-when [ empty? not ] filter :> cards
    0 :> answer!
    0 999 [a..b] [| i |
        i 10 * :> offset
        offset offset 5 + cards subseq poker-value
        offset 5 + offset 10 + cards subseq poker-value >
        [ answer 1 + answer! ] when
    ] each answer ;

CONSTANT: english-frequency { 8 2 3 4 13 2 2 6 7 1 1 4 2 7 8 2 1 6 6 9 3 1 2 1 2 1 }
CONSTANT: english-phrases { " the " " and " " of " " to " " in "
  " that " " is " " a " " for " " with " }
```

Comparison. C exposes direct bounded arithmetic. Shared calls: `euler_fail`, `euler_open`. Rust exposes modular arithmetic, fixture parsing, and sorting or set deduplication. Gforth exposes fixture parsing. Shared calls: `prefix!`, `prefix@`, `read-fixture`. Factor exposes fixture parsing. Shared calls: `whitespace?`. All listed programs print the same exact answer string from the manifest. The excerpts can differ in algorithm, data representation, and startup cost.

40.55 Problem 55: Lychrel Numbers

The program computes the exact answer for Lychrel Numbers.

C. Source: `solutions/c/lychrel_numbers_55.c`.

```
static long long solve(void) {
  mpz_t n, rev;
  mpz_inits(n, rev, NULL);
  unsigned total = 0;
  for (unsigned start = 1; start < 10000; start++) {
    mpz_set_ui(n, start);
    int reached = 0;
    for (int i = 0; i < 50; i++) {
      char *str = mpz_get_str(NULL, 10, n);
      size_t len = strlen(str);
      for (size_t l = 0; l < len / 2; l++) {
        char t = str[l];
        str[l] = str[len - 1 - l];
        str[len - 1 - l] = t;
```

Rust. Source: `solutions/rust/src/bin/p55.rs`.

```
fn solve() -> i64 {
    {
        let mut count = 0;
        for start in 1u32..10000 {
            let mut n = BigUint::from(start);
            let mut palindrome = false;
            for _ in 0..50 {
                let reverse: String = n.to_string().chars().rev().collect();
                n += reverse.parse::<BigUint>().unwrap();
                let s = n.to_string();
                if s.bytes().eq(s.bytes().rev()) {
                    palindrome = true;
                    break;
                }
```

Gforth. Source: `solutions/forth/p55.fs`.

```
: big-reverse 0 0 { original reversed temp digit } ( big big -- )
  8 bnew to temp original temp bcopy reversed bclear
  begin temp blen@ 0> while
    10 temp bdiv-small! to digit 10 reversed bmul-small! digit reversed badd-small!
  repeat temp bfree ;

: p55 ( -- answer )
  8 bnew 8 bnew 8 bnew 0 0 0 { value reversed temp old reached count }
  0 to count
  10000 1 do i value bset-u false to reached
    50 0 do
      value reversed big-reverse value reversed temp badd
      value to old temp to value old to temp
      value reversed big-reverse
```

Factor. Source: `solutions/factor/euler/extra/extra.factor`.

```
:: p55 ( -- n ) 1 9999 [a..b] [ p55-lychrel? ] count ;
```

Comparison. C exposes exact large-integer arithmetic. Rust exposes exact large-integer arithmetic. Gforth exposes exact large-integer arithmetic. Shared calls: `babscompare`, `badd`, `badd-small!`, `bclear`, `bcopy`, `bdiv-small!`, `bfree`, `blen@`, `bmul-small!`, `bnew`, `bset-u`. Factor exposes direct bounded arithmetic. All listed programs print the same exact answer string from the manifest. The excerpts can differ in algorithm, data representation, and startup cost.

40.56 Problem 56: Powerful Digit Sum

Compute every a^b with 1 <= a,b < 100 exactly with GMP. At most 198 decimal digits are needed, since 99^99 < 100^99.

C. Source: `solutions/c/powerful_digit_sum_56.c`.

```
int main(void) {
    mpz_t value;
    mpz_init(value);
    unsigned best = 0;
    for (unsigned a = 1; a < 100; ++a)
        for (unsigned b = 1; b < 100; ++b) {
            mpz_ui_pow_ui(value, a, b);
            char decimal[200];
            mpz_get_str(decimal, 10, value);
            unsigned sum = 0;
            for (const char *p = decimal; *p; ++p) sum += (unsigned)(*p - '0');
            if (sum > best) best = sum;
        }
    mpz_clear(value);
```

Rust. Source: `solutions/rust/src/bin/p56.rs`.

```
fn solve() -> i64 {
    {
        let mut best = 0;
        for a in 1u32..100 {
            let mut power = BigUint::one();
            for _ in 1..100 {
                power *= a;
                best = best.max(digits_sum_big(&power));
            }
        }
        best as i64
    }
}
```

Gforth. Source: `solutions/forth/p56.fs`.

```
: p56 ( -- n )
  8 bnew 0 { value best }
  100 1 do
    1 value bset-u
    100 1 do
      j value bmul-small!
      value bdigitsum best max to best
    loop
  loop value bfree best ;

[undefined] euler-testing [if] p56 . cr bye [then]
```

Factor. Source: `solutions/factor/euler/extra/extra.factor`.

```
:: p56 ( -- n )
    0 :> best!
    1 99 [a..b] [| a |
        a :> value!
        1 99 [a..b] [| b |
            value a * value!
            value digit-sum best max best!
```

```
        ] each
    ] each best ;
```

Comparison. C exposes exact large-integer arithmetic. Rust exposes exact large-integer arithmetic. Shared calls: `digits_sum_big`. Gforth exposes exact large-integer arithmetic. Shared calls: `bdigitsum`, `bfree`, `bmul-small!`, `bnew`, `bset-u`. Factor exposes direct bounded arithmetic. Shared calls: `digit-sum`. All listed programs print the same exact answer string from the manifest. The excerpts can differ in algorithm, data representation, and startup cost.

40.57 Problem 57: Square Root Convergents

Starting at 3/2, advance p/q to (p+2q)/(p+q). Both integers remain exact. Decimal strings avoid GMP sizeinbase's possible one-digit overestimate. Values are below 3^1001, hence below 10^478.

C. Source: `solutions/c/square_root_convergents_57.c`.

```
int main(void) {
    mpz_t numerator, denominator, next;
    mpz_inits(numerator, denominator, next, NULL);
    mpz_set_ui(numerator, 3);
    mpz_set_ui(denominator, 2);
    unsigned count = 0;
    for (unsigned expansion = 1; expansion <= 1000; ++expansion) {
        char n[480], d[480];
        mpz_get_str(n, 10, numerator);
        mpz_get_str(d, 10, denominator);
        count += strlen(n) > strlen(d);
        mpz_mul_ui(next, denominator, 2);
        mpz_add(next, next, numerator);
        mpz_add(denominator, denominator, numerator);
```

Rust. Source: `solutions/rust/src/bin/p57.rs`.

```
fn solve() -> i64 {
    {
        let (mut top, mut bottom) = (BigUint::from(3u32), BigUint::from(2u32));
        let mut count = 0;
        for _ in 0..1000 {
            if top.to_string().len() > bottom.to_string().len() {
                count += 1;
            }
            let next = &top + &bottom * 2u32;
            bottom += &top;
            top = next;
        }
        count
    }
```

Gforth. Source: `solutions/forth/p57.fs`.

```
: p57 ( -- n )
```

```
  8 bnew 8 bnew 8 bnew 8 bnew 0 { top bottom next temp total }
  3 top bset-u 2 bottom bset-u
  1000 0 do
    top bdecimal-length bottom bdecimal-length > if total 1+ to total then
    bottom temp bcopy 2 temp bmul-small!
    top temp next badd
    top bottom temp badd
    temp bottom bcopy next top bcopy
  loop
  top bfree bottom bfree next bfree temp bfree total ;

[undefined] euler-testing [if] p57 . cr bye [then]
```

Factor. Source: `solutions/factor/euler/advanced/advanced.factor`.

```
:: p57 ( -- n )
    3 :> numerator! 2 :> denominator! 0 :> count!
    1000 [| |
        numerator decimal-digits denominator decimal-digits >
        [ count 1 + count! ] when
        numerator denominator 2 * + :> next-n
        numerator denominator + denominator!
        next-n numerator!
    ] times count ;
```

Comparison. C exposes exact large-integer arithmetic. Rust exposes exact large-integer arithmetic. Gforth exposes exact large-integer arithmetic. Shared calls: `badd`, `bcopy`, `bdecimal-length`, `bfree`, `bmul-small!`, `bnew`, `bset-u`. Factor exposes direct bounded arithmetic. Shared calls: `decimal-digits`. All listed programs print the same exact answer string from the manifest. The excerpts can differ in algorithm, data representation, and startup cost.

40.58 Problem 58: Spiral Primes

A side-s layer has corners s^2-k(s-1). Its square corner is composite; test the other three and compare the ratio using integer arithmetic.

C. Source: `solutions/c/spiral_primes_58.c`.

```
int main(void) {
    uint64_t prime_count = 0;
    for (uint64_t side = 3; ; side += 2) {
        if (side > UINT64_MAX / side) euler_fail("Spiral search overflow");
        uint64_t square = side * side, step = side - 1;
        for (unsigned corner = 1; corner <= 3; ++corner)
            prime_count += euler_is_prime(square - corner * step);
        if (10 * prime_count < 2 * side - 1) {
            printf("%" PRIu64 "\n", side);
            return 0;
        }
    }
}
```

Rust. Source: `solutions/rust/src/bin/p58.rs`.

```
fn solve() -> i64 {
    {
        let mut diagonal_primes = 0;
        for side in (3u64..).step_by(2) {
            let square = side * side;
            for corner in 1..=3 {
                if prime(square - corner * (side - 1)) {
                    diagonal_primes += 1;
                }
            }
            if 10 * diagonal_primes < 2 * side - 1 {
                return side as i64;
            }
        }
```

Gforth. Source: `solutions/forth/p58.fs`.

```
: p58 ( -- n ) 0 0 0 { prime-count square step }
  3 begin
    dup dup * to square dup 1- to step
    4 1 do square i step * - is-prime? if prime-count 1+ to prime-count then loop
    prime-count 10 * over 2* 1- < if exit then
    2 +
  again ;

[undefined] euler-testing [if] p58 . cr bye [then]
```

Factor. Source: `solutions/factor/euler/extra/extra.factor`.

```
:: p58 ( -- n )
    0 :> prime-count! 3 :> side! 0 :> answer!
    [ answer zero? ] [
        side sq :> square
        side 1 - :> step
        1 3 [a..b] [| k |
            square k step * - prime?
            [ prime-count 1 + prime-count! ] when
        ] each
        prime-count 10 * side 2 * 1 - <
        [ side answer! ] when
        side 2 + side!
    ] while answer ;
```

Comparison. C exposes prime generation or testing. Shared calls: `euler_fail`, `euler_is_prime`. Rust exposes prime generation or testing. Gforth exposes prime generation or testing. Shared calls: `is-prime?`. Factor exposes prime generation or testing. All listed programs print the same exact answer string from the manifest. The excerpts can differ in algorithm, data representation, and startup cost.

40.59 Problem 59: XOR Decryption

Exhaust all 26^3 lowercase keys. The problem identifies the plaintext as English, so rank readable candidates by letter frequencies and common whole words. This language score is a heuristic, not a proof that arbitrary English ciphertext always has a unique interpretation.

C. Source: `solutions/c/xor_decryption_59.c`.

```
enum { CIPHER_BYTES = 1455 };

static long english_score(const unsigned char *cipher, size_t length,
                          const unsigned char key[3], char *text) {
    static const int weights[26] = {
        8,2,3,4,13,2,2,6,7,1,1,4,2,7,8,2,1,6,6,9,3,1,2,1,2,1
    };
    long score = 0;
    for (size_t i = 0; i < length; ++i) {
        unsigned ch = cipher[i] ^ key[i % 3];
        if (ch < 32 || ch > 126) return LONG_MIN;
        text[i] = (char)ch;
        if (ch == ' ') score += 15;
        else if (isalpha((unsigned char)ch)) score += weights[tolower((unsigned char)ch) - 'a'];
```

Rust. Source: `solutions/rust/src/bin/p59.rs`.

```
fn p59_score(cipher: &[u64], key: [u8; 3]) -> Option<(i64, u64)> {
    const WEIGHTS: [i64; 26] = [
        8, 2, 3, 4, 13, 2, 2, 6, 7, 1, 1, 4, 2, 7, 8, 2, 1, 6, 6, 9, 3, 1, 2, 1, 2, 1,
    ];
    let mut score = 0;
    let mut text = Vec::with_capacity(cipher.len());
    let mut sum = 0;
    for (i, &c) in cipher.iter().enumerate() {
        let ch = (c as u8) ^ key[i % 3];
        if !(32..=126).contains(&ch) {
            return None;
        }
        sum += ch as u64;
        text.push(ch);
```

Gforth. Source: `solutions/forth/p59.fs`.

```
create p59-letter-weights 8 , 2 , 3 , 4 , 13 , 2 , 2 , 6 , 7 , 1 , 1 , 4 , 2 , 7 , 8 , 2 , 1 , 6
     , 6 , 9 , 3 , 1 , 2 , 1 , 2 , 1 ,

: p59-weight { ch } ( ascii -- weight )
  ch 32 = if 15 exit then
  ch [char] A >= ch [char] Z <= and if ch [char] A - p59-letter-weights swap cells + @ exit then
  ch [char] a >= ch [char] z <= and if ch [char] a - p59-letter-weights swap cells + @ exit then
  ch [char] 0 >= ch [char] 9 <= and if -3 exit then
  ch case [char] . of -1 endof [char] , of -1 endof [char] ; of -1 endof
    [char] : of -1 endof [char] ' of -1 endof 34 of -1 endof
    [char] ! of -1 endof [char] ? of -1 endof [char] ( of -1 endof
    [char] ) of -1 endof [char] - of -1 endof [char] [ of -1 endof
    [char] ] of -1 endof -12 swap endcase ;
```

```
: p59-match 0 { text length word size hits } ( ptr n word n -- count )
```

Factor. Source: `solutions/factor/euler/advanced/advanced.factor`.

```
:: p59 ( -- n )
    "tests/data/p059_cipher.txt" read-numbers :> cipher
    -1000000000000 :> best! 0 :> answer!
    97 122 [a..b] [| a |
        97 122 [a..b] [| b |
            97 122 [a..b] [| c |
                V{ } clone :> text
                cipher [| x index |
                    index 3 mod 0 = [ a ] [ index 3 mod 1 = [ b ] [ c ] if ] if
                    x bitxor text push
                ] each-index
                text english-score :> score
                score best > [
                    score best!
```

Comparison. C exposes direct bounded arithmetic. Shared calls: `euler_open`. Rust exposes fixture parsing. Shared calls: `parse_numbers`. Gforth exposes modular arithmetic. Shared calls: `fixture-integers`, `prefix@`. Factor exposes modular arithmetic, and fixture parsing. Shared calls: `read-numbers`. All listed programs print the same exact answer string from the manifest. The excerpts can differ in algorithm, data representation, and startup cost.

40.60 Problem 60: Prime Pair Sets

Build a graph whose edges mean that both concatenations are prime. Increasing candidate lists enumerate each five-clique exactly once. Discover an upper bound by doubling a prime limit; then search all primes below the discovered sum. Every better set must lie in that final graph, so the last search establishes the global minimum.

C. Source: `solutions/c/prime_pair_sets_60.c`.

```
typedef struct {
    size_t count;
    unsigned *primes;
    unsigned char *adjacent;
    size_t *scratch;
} Graph;

static uint64_t concatenate(unsigned a, unsigned b) {
    uint64_t power = 10;
    while (power <= b) power *= 10;
    return power * a + b;
}

static Graph make_graph(unsigned limit, uint64_t best) {
```

Rust. Source: `solutions/rust/src/bin/p60.rs`.

```
fn p60_concat(a: u64, b: u64) -> u64 {
    let mut power = 10;
    while power <= b {
        power *= 10;
    }
    a * power + b
}

struct PrimeGraph {
    primes: Vec<u64>,
    edges: Vec<Vec<bool>>,
}
impl PrimeGraph {
    fn new(limit: usize, best: u64) -> Self {
```

Gforth. Source: `solutions/forth/p60.fs`.

```
create p60-primes 4000 cells allot

variable p60-adj

create p60-candidates 4000 cells allot

create p60-scratch 20000 cells allot

variable p60-count

variable p60-best

: p60-free-adj ( -- )
  p60-adj @ ?dup if free throw 0 p60-adj ! then ;
```

Factor. Source: `solutions/factor/euler/advanced/advanced.factor`.

```
:: p60 ( -- n )
    99999999 1array :> best
    10000 best p60-at-limit
    best first 99999999 = [ "No graph clique" throw ] when
    best first 1 - best p60-at-limit
    best first ;
```

Comparison. C exposes prime generation or testing, and modular arithmetic. Shared calls: `euler_fail`, `euler_is_prime`, `euler_sieve`. Rust exposes prime generation or testing, and recursive search. Shared calls: `primes`. Gforth exposes prime generation or testing, modular arithmetic, and recursive search. Shared calls: `is-prime?`, `sieve-build`. Factor exposes direct bounded arithmetic. All listed programs print the same exact answer string from the manifest. The excerpts can differ in algorithm, data representation, and startup cost.

40.61 Problem 61: Cyclical Figurate Numbers

Enumerate four-digit polygonal numbers and backtrack through their two-digit overlaps. Fixing the octagonal member first removes cyclic rotations. Each polygonal type and each numeric value must occur exactly once.

C. Source: `solutions/c/cyclical_figurate_numbers_61.c`.

```
static int p61_values[6][150], p61_count[6], p61_path[6];

static int p61_search(int depth, unsigned used) {
    if (depth == 6) {
        if (p61_path[5] % 100 != p61_path[0] / 100) return 0;
        int sum = 0;
        for (int i = 0; i < 6; ++i) sum += p61_path[i];
        return sum;
    }
    int prefix = p61_path[depth - 1] % 100;
    for (int kind = 0; kind < 6; ++kind) if (!(used & (1u << kind))) {
        for (int i = 0; i < p61_count[kind]; ++i) {
            int value = p61_values[kind][i];
            if (value / 100 != prefix) continue;
```

Rust. Source: `solutions/rust/src/bin/p61.rs`.

```
fn solve() -> i64 {
    fn search(path: &mut Vec<u64>, used: u8, values: &[Vec<u64>]) -> Option<i64> {
        if path.len() == 6 {
            return (path[5] % 100 == path[0] / 100).then_some(path.iter().sum::<u64>() as i64);
        }
        let prefix = path.last().unwrap() % 100;
        for kind in 0..6 {
            if used & (1 << kind) != 0 {
                continue;
            }
            for &v in &values[kind] {
                if v / 100 == prefix && !path.contains(&v) {
                    path.push(v);
                    if let Some(result) = search(path, used | (1 << kind), values) {
```

Gforth. Source: `solutions/forth/p61.fs`.

```
create p61-values 900 cells allot

create p61-count 6 cells allot

create p61-path 6 cells allot

: p61-prepare 0 0 0 0 { sides value tally offset }
  p61-count 6 cells erase
  6 0 do
    i 3 + to sides
    200 1 do
      sides 2 - i i * * sides 4 - i * - 2 / to value
      value 1000 >= value 10000 < and value 100 mod 10 >= and if
```

```
        j p61-count prefix@ to tally
```

Factor. Source: `solutions/factor/euler/advanced/advanced.factor`.

```
:: p61 ( -- n )
    V{ } clone :> groups
    3 8 [a..b] [| sides |
        V{ } clone :> values
        1 :> n!
        [ sides 2 - n sq * sides 4 - n * - 2 /i 10000 < ] [
            sides 2 - n sq * sides 4 - n * - 2 /i :> value
            value 1000 >= value 100 mod 10 >= and [ value values push ] when
            n 1 + n!
        ] while
        values groups push
    ] each
    0 :> answer!
    5 groups nth [| value |
```

Comparison. C exposes direct bounded arithmetic. Rust exposes recursive search. Gforth exposes modular arithmetic, and recursive search. Shared calls: `prefix!`, `prefix@`. Factor exposes modular arithmetic, and recursive search. All listed programs print the same exact answer string from the manifest. The excerpts can differ in algorithm, data representation, and startup cost.

40.62 Problem 62: Cubic Permutations

Sort complete digit-length buckets of cubes by digit multiplicities. Waiting until a bucket closes is essential: five matches might later become six. Five bits per digit count cover every decimal uint64_t value.

C. Source: `solutions/c/cubic_permutations_62.c`.

```
typedef struct { uint64_t signature, cube; } P62Cube;

static uint64_t p62_signature(uint64_t n) {
    uint64_t key = 0;
    do { key += UINT64_C(1) << (5 * (n % 10)); n /= 10; } while (n);
    return key;
}

static int p62_compare(const void *left, const void *right) {
    const P62Cube *a = left, *b = right;
    return (a->signature > b->signature) - (a->signature < b->signature);
}

static uint64_t p62_bucket(P62Cube *cubes, size_t count, size_t target) {
```

Rust. Source: `solutions/rust/src/bin/p62.rs`.

```
fn solve() -> i64 {
    {
```

```
        let mut bucket = Vec::<(u64, u64)>::new();
        let mut magnitude = 10u64;
        let mut answer = 0;
        for n in 1.. {
            let cube = n * n * n;
            if cube >= magnitude {
                let mut groups: HashMap<u64, Vec<u64>> = HashMap::new();
                for &(signature, value) in &bucket {
                    groups.entry(signature).or_default().push(value);
                }
                if let Some(best) = groups
                    .values()
```

Gforth. Source: `solutions/forth/p62.fs`.

```
: p62-test 0 { signatures values count copies } ( addresses length -- minimum or zero )
  count 0 do
    0 to copies
    count 0 do
      i signatures prefix@ j signatures prefix@ = if copies 1+ to copies then
    loop
    copies 5 = if i values prefix@ unloop exit then
  loop 0 ;

: p62 ( -- n )
  15000 cells allocate throw 15000 cells allocate throw
  0 10 1 0 0 { signatures values count bucket n cube answer }
  begin
    n n * n * to cube
```

Factor. Source: `solutions/factor/euler/advanced/advanced.factor`.

```
:: p62 ( -- n )
    H{ } clone :> groups!
    1 :> candidate! 0 :> digits! 0 :> answer!
    [ answer zero? ] [
        candidate 3 ^ :> cube
        cube decimal-digits :> count
        count digits > [
            groups >alist [| pair |
                pair second first 5 = [ pair second second ] [ f ] if
            ] map [ f = not ] filter
            dup empty? not [ infimum answer! ] [ drop ] if
            H{ } clone groups!
            count digits!
        ] when
```

Comparison. C exposes sorting or set deduplication. Shared calls: `euler_fail`, `euler_malloc`. Rust exposes direct bounded arithmetic. Shared calls: `decimal_signature`. Gforth exposes direct bounded arithmetic. Shared calls: `digit-signature`, `prefix!`, `prefix@`. Factor exposes sorting or set deduplication. Shared calls: `decimal-digits`, `digit-signature`. All listed programs print the same exact answer string from the manifest. The excerpts can differ in algorithm, data representation, and startup cost.

40.63 Problem 63: Powerful Digit Counts

Only bases 1..9 can have a^n < 10^n. Compare exact powers with 10^(n-1). Once a^n falls below that threshold, multiplying by a<10 cannot recover.

C. Source: `solutions/c/powerful_digit_counts_63.c`.

```
int main(void) {
    mpz_t power, lower;
    mpz_inits(power, lower, NULL);
    unsigned count = 0;
    for (unsigned base = 1; base <= 9; ++base) {
        mpz_set_ui(power, base); mpz_set_ui(lower, 1);
        while (mpz_cmp(power, lower) >= 0) {
            ++count;
            mpz_mul_ui(power, power, base); mpz_mul_ui(lower, lower, 10);
        }
    }
    printf("%u\n", count);
    mpz_clears(power, lower, NULL);
    return 0;
```

Rust. Source: `solutions/rust/src/bin/p63.rs`.

```
fn solve() -> i64 {
    {
        let mut count = 0;
        for base in 1u32..10 {
            let mut value = BigUint::one();
            for exponent in 1..=22 {
                value *= base;
                let length = value.to_string().len();
                if length == exponent {
                    count += 1;
                }
                if length < exponent {
                    break;
                }
```

Gforth. Source: `solutions/forth/p63.fs`.

```
: p63 ( -- n )
  4 bnew 0 0 { value total exponent }
  10 1 do 1 value bset-u
    25 1 do j value bmul-small!
      value bdecimal-length i = if total 1+ to total then
      value bdecimal-length i < if leave then
    loop
  loop value bfree total ;

[undefined] euler-testing [if] p63 . cr bye [then]
```

Factor. Source: `solutions/factor/euler/extra/extra.factor`.

```
:: p63 ( -- n )
    0 :> answer!
    1 9 [a..b] [| base |
        1 :> value!
        1 25 [a..b] [| exp |
            value base * value!
            value decimal-digits exp = [ answer 1 + answer! ] when
        ] each
    ] each answer ;
```

Comparison. C exposes exact large-integer arithmetic. Rust exposes exact large-integer arithmetic. Gforth exposes exact large-integer arithmetic. Shared calls: `bdecimal-length`, `bfree`, `bmul-small!`, `bnew`, `bset-u`. Factor exposes direct bounded arithmetic. Shared calls: `decimal-digits`. All listed programs print the same exact answer string from the manifest. The excerpts can differ in algorithm, data representation, and startup cost.

40.64 Problem 64: Odd Period Square Roots

The continued-fraction state (m,d,a) is entirely integral. A nonsquare sqrt(N) completes its period when a reaches twice its integer part.

C. Source: `solutions/c/odd_period_square_roots_64.c`.

```
static unsigned p64_period(unsigned n) {
    unsigned root = (unsigned)euler_isqrt(n);
    if (root * root == n) return 0;
    unsigned m = 0, d = 1, a = root, period = 0;
    do {
        m = d * a - m;
        d = (n - m * m) / d;
        a = (root + m) / d;
        ++period;
    } while (a != 2 * root);
    return period;
}

int main(void) {
```

Rust. Source: `solutions/rust/src/bin/p64.rs`.

```
fn solve() -> i64 {
    {
        let mut count = 0;
        for n in 2u64..=10000 {
            let a0 = isqrt(n);
            if a0 * a0 == n {
                continue;
            }
            let (mut m, mut d, mut a) = (0u64, 1u64, a0);
            let mut period = 0;
            loop {
                m = d * a - m;
```

```
                d = (n - m * m) / d;
                a = (a0 + m) / d;
```

Gforth. Source: `solutions/forth/p64.fs`.

```
: p64 ( -- n ) 0 0 0 0 0 0 { total root m d a period }
  10001 2 do i integer-sqrt to root
    root root * i <> if
      0 to m 1 to d root to a 0 to period
      begin d a * m - to m i m m * - d / to d
        root m + d / to a period 1+ to period
        a root 2* = until
      period 2 mod if total 1+ to total then
    then
  loop total ;

[undefined] euler-testing [if] p64 . cr bye [then]
```

Factor. Source: `solutions/factor/euler/extra/extra.factor`.

```
:: p64 ( -- n )
    0 :> answer!
    2 10000 [a..b] [| n |
        n euler-isqrt :> root
        root sq n = not [
            0 :> m! 1 :> d! root :> a! 0 :> period! t :> continuing!
            [ continuing ] [
                d a * m - m!
                n m sq - d /i d!
                root m + d /i a!
                period 1 + period!
                a root 2 * = [ f continuing! ] when
            ] while
            period odd? [ answer 1 + answer! ] when
```

Comparison. C exposes direct bounded arithmetic. Shared calls: `euler_isqrt`. Rust exposes direct bounded arithmetic. Shared calls: `isqrt`. Gforth exposes modular arithmetic. Shared calls: `integer-sqrt`. Factor exposes direct bounded arithmetic. Shared calls: `euler-isqrt`. All listed programs print the same exact answer string from the manifest. The excerpts can differ in algorithm, data representation, and startup cost.

40.65 Problem 65: Convergents of e

Numerators obey p[k]=a[k]*p[k-1]+p[k-2], starting with 1 and 0. e has coefficients 2; 1,2,1; 1,4,1; 1,6,1; ... . GMP keeps them exact.

C. Source: `solutions/c/convergents_of_e_65.c`.

```
int main(void) {
    mpz_t previous, before, next;
    mpz_inits(previous, before, next, NULL);
```

```
    mpz_set_ui(previous, 1);
    for (unsigned k = 0; k < 100; ++k) {
        unsigned coefficient = k == 0 ? 2 : (k % 3 == 2 ? 2 * ((k + 1) / 3) : 1);
        mpz_mul_ui(next, previous, coefficient);
        mpz_add(next, next, before);
        mpz_set(before, previous); mpz_set(previous, next);
    }
    char *decimal = mpz_get_str(NULL, 10, previous);
    unsigned sum = 0;
    for (const char *s = decimal; *s; ++s) sum += (unsigned)(*s - '0');
    printf("%u\n", sum);
```

Rust. Source: `solutions/rust/src/bin/p65.rs`.

```
fn solve() -> i64 {
    {
        let (mut numerator, mut previous) = (BigUint::one(), BigUint::zero());
        for k in (0..100).rev() {
            let a = if k == 0 {
                2
            } else if k % 3 == 2 {
                2 * (k / 3 + 1)
            } else {
                1
            };
            let next = &numerator * (a as u32) + &previous;
            previous = numerator;
            numerator = next;
```

Gforth. Source: `solutions/forth/p65.fs`.

```
: p65 ( -- n )
  8 bnew 8 bnew 8 bnew 8 bnew 0 0 { numerator previous next temp a k }
  1 numerator bset-u 0 previous bset-u
  100 0 do
    99 i - to k
    k 0= if 2 to a else
      k 3 mod 2 = if k 3 / 1+ 2* to a else 1 to a then
    then
    numerator temp bcopy a temp bmul-small!
    temp previous next badd
    numerator previous bcopy next numerator bcopy
  loop
  numerator bdigitsum to a
  numerator bfree previous bfree next bfree temp bfree a ;
```

Factor. Source: `solutions/factor/euler/advanced/advanced.factor`.

```
:: p65 ( -- n )
    1 :> numerator! 0 :> denominator!
    99 0 [a..b] [| index |
        index zero? [ 2 ] [
            index 3 mod 2 = [ index 1 + 3 /i 2 * ] [ 1 ] if
        ] if :> partial
        numerator partial * denominator + :> newnum
```

```
        numerator denominator!
        newnum numerator!
    ] each numerator digit-sum ;
```

Comparison. C exposes exact large-integer arithmetic. Rust exposes exact large-integer arithmetic. Gforth exposes modular arithmetic, and exact large-integer arithmetic. Shared calls: `badd`, `bcopy`, `bdigitsum`, `bfree`, `bmul-small!`, `bnew`, `bset-u`. Factor exposes modular arithmetic. Shared calls: `digit-sum`. All listed programs print the same exact answer string from the manifest. The excerpts can differ in algorithm, data representation, and startup cost.

40.66 Problem 66: Diophantine Equation

The fundamental positive solution of x^2-D*y^2=1 occurs among the continued fraction convergents of sqrt(D). Test that identity with arbitrary integers; both x and y can exceed machine integer range long before D reaches 1000.

C. Source: `solutions/c/diophantine_equation_66.c`.

```
static void p66_minimum_x(unsigned n, mpz_t answer) {
    unsigned root = (unsigned)euler_isqrt(n), m = 0, d = 1, a = root;
    mpz_t p0, p1, p, q0, q1, q, difference, temporary;
    mpz_inits(p0, p1, p, q0, q1, q, difference, temporary, NULL);
    mpz_set_ui(p1, 1); mpz_set_ui(q0, 1);
    for (;;) {
        mpz_mul_ui(p, p1, a); mpz_add(p, p, p0);
        mpz_mul_ui(q, q1, a); mpz_add(q, q, q0);
        mpz_mul(difference, p, p);
        mpz_mul(temporary, q, q); mpz_mul_ui(temporary, temporary, n);
        mpz_sub(difference, difference, temporary);
        if (mpz_cmp_ui(difference, 1) == 0) { mpz_set(answer, p); break; }
        mpz_set(p0, p1); mpz_set(p1, p); mpz_set(q0, q1); mpz_set(q1, q);
        m = d * a - m; d = (n - m * m) / d; a = (root + m) / d;
```

Rust. Source: `solutions/rust/src/bin/p66.rs`.

```
fn solve() -> i64 {
    {
        let mut best = BigUint::zero();
        let mut answer = 0;
        for d in 2..=1000u64 {
            let a0 = isqrt(d);
            if a0 * a0 == d {
                continue;
            }
            let (mut m, mut den, mut a) = (0u64, 1u64, a0);
            let (mut p0, mut p1, mut q0, mut q1) = (
                BigUint::zero(),
                BigUint::one(),
                BigUint::one(),
```

Gforth. Source: `solutions/forth/p66.fs`.

```
: p66-period 0 0 0 0 0 { n root m denominator a period } ( D -- period )
  n integer-sqrt to root
  0 to m 1 to denominator root to a 0 to period
  begin
    denominator a * m - to m
    n m m * - denominator / to denominator
    root m + denominator / to a
    period 1+ to period
    a root 2* =
  until period ;

: p66-numerator 0 0 0 0 0 0 0 0 0 0
  { n answer p0 p1 next old root m denominator a period steps } ( D big-result -- )
  8 bnew to p0 8 bnew to p1 8 bnew to next
```

Factor. Source: `solutions/factor/euler/advanced/advanced.factor`.

```
:: p66 ( -- n )
    0 :> best! 0 :> answer!
    2 1000 [a..b] [| d |
        d euler-isqrt :> root
        root sq d = not [
            d p66-minimum-x :> x
            x best > [ x best! d answer! ] when
        ] when
    ] each answer ;
```

Comparison. C exposes exact large-integer arithmetic. Shared calls: `euler_isqrt`. Rust exposes exact large-integer arithmetic. Shared calls: `isqrt`. Gforth exposes modular arithmetic, and exact large-integer arithmetic. Shared calls: `babscompare`, `baddabs!`, `bcopy`, `bfree`, `bmul-small!`, `bnew`, `bset-u`, `integer-sqrt`. Factor exposes direct bounded arithmetic. Shared calls: `euler-isqrt`. All listed programs print the same exact answer string from the manifest. The excerpts can differ in algorithm, data representation, and startup cost.

40.67 Problem 67: Maximum Path Sum II

Bottom-up dynamic programming replaces each entry by its value plus the larger of its two reachable children. The 100-row input has 5050 entries.

C. Source: `solutions/c/maximum_path_sum_ii_67.c`.

```
static unsigned p67_maximum(unsigned *triangle, size_t rows) {
    for (size_t row = rows - 1; row > 0; --row) {
        size_t parent = row * (row - 1) / 2, child = row * (row + 1) / 2;
        for (size_t col = 0; col < row; ++col) {
            unsigned left = triangle[child + col], right = triangle[child + col + 1];
            triangle[parent + col] += left > right ? left : right;
        }
    }
    return triangle[0];
}
```

```
int main(void) {
    unsigned triangle[5050];
    FILE *file = euler_open("tests/data/p067_triangle.txt");
```

Rust. Source: `solutions/rust/src/bin/p67.rs`.

```
fn solve() -> i64 {
    triangle_max(matrix(include_str!(
        "../../../../tests/data/p067_triangle.txt"
    ))) as i64
}

fn main() {
    println!("{}", solve());
}
```

Gforth. Source: `solutions/forth/p67.fs`.

```
: p67 ( -- n ) s" tests/data/p067_triangle.txt" 100 triangle-path-max ;

[undefined] euler-testing [if] p67 . cr bye [then]
```

Factor. Source: `solutions/factor/euler/extra/extra.factor`.

```
: p67 ( -- n ) "tests/data/p067_triangle.txt" max-triangle-path ;
```

Comparison. C exposes dynamic programming or memoization. Shared calls: `euler_open`. Rust exposes fixture parsing. Shared calls: `matrix`. Gforth exposes direct bounded arithmetic. Shared calls: `triangle-path-max`. Factor exposes direct bounded arithmetic. Shared calls: `max-triangle-path`. All listed programs print the same exact answer string from the manifest. The excerpts can differ in algorithm, data representation, and startup cost.

40.68 Problem 68: Magic 5-gon Ring

A 16-digit ring must put 10 outside, since inner nodes occur twice. Enumerate five distinct inner nodes, then derive every outer node from the common sum. The first outer node is constrained to be the smallest to fix the rotation.

C. Source: `solutions/c/magic_5_gon_ring_68.c`.

```
static int p68_inner[5];
static uint64_t p68_best;

static void p68_search(int depth, unsigned used) {
    if (depth < 5) {
        for (int value = 1; value <= 9; ++value) if (!(used & (1u << value))) {
            p68_inner[depth] = value;
            p68_search(depth + 1, used | (1u << value));
```

```
        }
        return;
    }
    for (int first = 1; first <= 10; ++first) if (!(used & (1u << first))) {
        int outer[5], sum = first + p68_inner[0] + p68_inner[1];
        unsigned all = used;
```

Rust. Source: `solutions/rust/src/bin/p68.rs`.

```
fn solve() -> i64 {
    fn search(inner: &mut [i64; 5], depth: usize, used: u16, best: &mut u64) {
        if depth < 5 {
            for value in 1..=9 {
                if used & (1 << value) != 0 {
                    continue;
                }
                inner[depth] = value;
                search(inner, depth + 1, used | (1 << value), best);
            }
            return;
        }
        for first in 1..=10i64 {
            if used & (1 << (first as u16)) != 0 {
```

Gforth. Source: `solutions/forth/p68.fs`.

```
create p68-inner 5 cells allot

create p68-outer 5 cells allot

variable p68-best

: p68-search 0 0 0 0 0 0 { depth used first line all candidate representation valid } ( depth
    used -- )
  depth 5 < if
    10 1 do
      used 1 i lshift and 0= if
        i depth p68-inner prefix!
        depth 1+ used 1 i lshift or recurse
      then
    loop exit
```

Factor. Source: `solutions/factor/euler/advanced/advanced.factor`.

```
:: p68 ( -- n )
    0 1array :> best
    V{ } clone 0 best p68-extend
    best first ;
```

Comparison. C exposes direct bounded arithmetic. Rust exposes recursive search. Gforth exposes modular arithmetic, and recursive search. Shared calls: `prefix!`, `prefix@`. Factor exposes direct bounded arithmetic. All listed programs print the same exact answer string from the manifest. The excerpts can differ in algorithm, data representation, and startup cost.

40.69 Problem 69: Totient Maximum

The program computes the exact answer for Totient Maximum.

C. Source: `solutions/c/totient_maximum_69.c`.

```
static long long solve(void) {
  unsigned n = 1;
  for (unsigned p = 2;; p++)
    if (prime(p)) {
      if (n > 1000000 / p)
        break;
      n *= p;
    }
  return n;
}

int main(void) {
  long long answer = solve();
  if (answer < 0)
```

Rust. Source: `solutions/rust/src/bin/p69.rs`.

```
fn solve() -> i64 {
    {
        let mut product = 1i64;
        for p in 2i64.. {
            if prime(p as u64) {
                if product > 1_000_000 / p {
                    break;
                }
                product *= p;
            }
        }
        product
    }
}
```

Gforth. Source: `solutions/forth/p69.fs`.

```
: p69 ( -- answer )
  1 2 { result candidate }
  begin candidate is-prime? if
    result 1000000 candidate / > if result exit then result candidate * to result
  then candidate 1+ to candidate again ;

[undefined] euler-testing [if] p69 . cr bye [then]
```

Factor. Source: `solutions/factor/euler/extra/extra.factor`.

```
:: p69 ( -- n )
    1 :> product! 2 :> n!
    [ product n * 1000000 <= ] [
        n prime? [ product n * product! ] when
```

```
        n 1 + n!
    ] while product ;
```

Comparison. C exposes prime generation or testing. Shared calls: `prime`. Rust exposes prime generation or testing. Gforth exposes prime generation or testing. Shared calls: `is-prime?`. Factor exposes prime generation or testing. All listed programs print the same exact answer string from the manifest. The excerpts can differ in algorithm, data representation, and startup cost.

40.70 Problem 70: Totient Permutation

Sieve every totient below ten million, so the minimum does not depend on a semiprime conjecture. Digit counts establish permutations; cross products compare n/phi(n) exactly without floating-point rounding.

C. Source: `solutions/c/totient_permutation_70.c`.

```
static uint64_t p70_digits(uint32_t n) {
    uint64_t key = 0;
    do { key += UINT64_C(1) << (4 * (n % 10)); n /= 10; } while (n);
    return key;
}

int main(void) {
    const uint32_t limit = 10000000;
    uint32_t *phi = euler_malloc((size_t)limit * sizeof *phi);
    for (uint32_t n = 0; n < limit; ++n) phi[n] = n;
    for (uint32_t p = 2; p < limit; ++p) if (phi[p] == p)
        for (uint32_t n = p; n < limit; n += p) phi[n] -= phi[n] / p;
    uint32_t best = 0, best_phi = 1;
    for (uint32_t n = 2; n < limit; ++n) {
```

Rust. Source: `solutions/rust/src/bin/p70.rs`.

```
fn solve() -> i64 {
    {
        let phi = totients(10_000_000);
        let mut best_n = 0u64;
        let mut best_phi = 1u64;
        for n in 2usize..10_000_000 {
            let p = phi[n] as u64;
            if decimal_signature(n as u64) != decimal_signature(p) {
                continue;
            }
            if best_n == 0 || (n as u64) * best_phi < best_n * p {
                best_n = n as u64;
                best_phi = p;
            }
```

Gforth. Source: `solutions/forth/p70.fs`.

```
: p70 ( -- n )
```

```
  10000000 cells allocate throw 0 1 0 { phi best best-phi value }
  10000000 0 do i i phi prefix! loop
  10000000 2 do
    i phi prefix@ i = if
      10000000 i do
        i phi prefix@ dup j / - i phi prefix!
      j +loop
    then
  loop
  10000000 2 do
    i phi prefix@ to value
    i digit-signature value digit-signature = if
      best 0= if i to best value to best-phi else
```

Factor. Source: `solutions/factor/euler/advanced/advanced.factor`.

```
:: p70 ( -- n )
    9999999 phi-sieve :> phi
    0 :> bestnum! 1 :> bestphi!
    2 9999999 [a..b] [| n |
        n phi nth :> totient
        bestnum zero? n bestphi * bestnum totient * < or [
            n digit-signature totient digit-signature = [
                n bestnum! totient bestphi!
            ] when
        ] when
    ] each bestnum ;
```

Comparison. C exposes prime generation or testing. Shared calls: `euler_malloc`. Rust exposes direct bounded arithmetic. Shared calls: `totients`. Gforth exposes direct bounded arithmetic. Shared calls: `digit-signature`, `prefix!`, `prefix@`. Factor exposes prime generation or testing. Shared calls: `digit-signature`, `phi-sieve`. All listed programs print the same exact answer string from the manifest. The excerpts can differ in algorithm, data representation, and startup cost.

40.71 Problem 71: Ordered Fractions

For each denominator, floor((3*d-1)/7) is the largest numerator strictly left of 3/7. Keep reduced fractions and compare by exact cross products.

C. Source: `solutions/c/ordered_fractions_71.c`.

```
static uint64_t p71_numerator(unsigned limit) {
    uint64_t best_n = 0, best_d = 1;
    for (uint64_t denominator = 2; denominator <= limit; ++denominator) {
        uint64_t numerator = (3 * denominator - 1) / 7;
        if (euler_gcd(numerator, denominator) == 1 && numerator * best_d > best_n * denominator)
     {
            best_n = numerator; best_d = denominator;
        }
    }
    return best_n;
}
```

```
int main(void) {
    printf("%" PRIu64 "\n", p71_numerator(1000000));
    return 0;
```

Rust. Source: `solutions/rust/src/bin/p71.rs`.

```
fn solve() -> i64 {
    {
        let mut best_n = 0u64;
        let mut best_d = 1u64;
        for d in 2..=1_000_000u64 {
            let n = (3 * d - 1) / 7;
            if n * best_d > best_n * d {
                best_n = n;
                best_d = d;
            }
        }
        best_n as i64
    }
}
```

Gforth. Source: `solutions/forth/p71.fs`.

```
: p71 ( -- n ) 0 1 0 { best-n best-d numerator }
  1000001 2 do
    i 3 * 1- 7 / to numerator
    numerator best-d * best-n i * > if numerator to best-n i to best-d then
  loop best-n ;

[undefined] euler-testing [if] p71 . cr bye [then]
```

Factor. Source: `solutions/factor/euler/extra/extra.factor`.

```
:: p71 ( -- n )
    0 :> best-n! 1 :> best-d!
    2 1000000 [a..b] [| d |
        d 3 * 1 - 7 /i :> numerator
        numerator best-d * best-n d * >
        [ numerator best-n! d best-d! ] when
    ] each best-n ;
```

Comparison. C exposes direct bounded arithmetic. Shared calls: `euler_gcd`. Rust exposes direct bounded arithmetic. Gforth exposes direct bounded arithmetic. Factor exposes direct bounded arithmetic. All listed programs print the same exact answer string from the manifest. The excerpts can differ in algorithm, data representation, and startup cost.

40.72 Problem 72: Counting Fractions

Exactly phi(d) reduced proper fractions have denominator d. An Eratosthenes totient sieve computes all of them before their 64-bit sum is accumulated.

C. Source: `solutions/c/counting_fractions_72.c`.

```
static uint64_t p72_count(unsigned limit) {
    unsigned *phi = euler_malloc(((size_t)limit + 1) * sizeof *phi);
    for (unsigned n = 0; n <= limit; ++n) phi[n] = n;
    for (unsigned p = 2; p <= limit; ++p) if (phi[p] == p)
        for (unsigned n = p; n <= limit; n += p) phi[n] -= phi[n] / p;
    uint64_t count = 0;
    for (unsigned n = 2; n <= limit; ++n) count += phi[n];
    free(phi);
    return count;
}

int main(void) {
    printf("%" PRIu64 "\n", p72_count(1000000));
    return 0;
```

Rust. Source: `solutions/rust/src/bin/p72.rs`.

```
fn solve() -> i64 {
    {
        let p = totients(1_000_000);
        p[2..].iter().map(|&v| v as i64).sum()
    }
}

fn main() {
    println!("{}", solve());
}
```

Gforth. Source: `solutions/forth/p72.fs`.

```
: p72 ( -- n )
  1000001 cells allocate throw 0 { phi result }
  1000001 0 do i i phi prefix! loop
  1000001 2 do
    i phi prefix@ i = if
      1000001 i do
        i phi prefix@ dup j / - i phi prefix!
      j +loop
    then
  loop
  1000001 2 do result i phi prefix@ + to result loop
  phi free throw result ;

[undefined] euler-testing [if] p72 . cr bye [then]
```

Factor. Source: `solutions/factor/euler/extra/extra.factor`.

```
:: p72 ( -- n )
    1000000 phi-sieve :> phi
    2 1000000 [a..b] [| n | n phi nth ] map sum ;
```

Comparison. C exposes prime generation or testing. Shared calls: `euler_malloc`. Rust exposes direct bounded arithmetic. Shared calls: `totients`. Gforth exposes direct bounded arithmetic. Shared calls: `prefix!`, `prefix@`. Factor exposes prime generation or testing. Shared calls: `phi-sieve`. All listed programs print the same exact answer string from the manifest. The excerpts can differ in algorithm, data representation, and startup cost.

40.73 Problem 73: Counting Fractions in a Range

1/3 and 1/2 are Farey neighbours. Repeatedly inserting their mediant visits every reduced fraction between them exactly once. Only the denominators are needed to prune a branch when their sum exceeds 12000.

C. Source: `solutions/c/counting_fractions_in_a_range_73.c`.

```
static uint64_t p73_between(unsigned left_d, unsigned right_d, unsigned limit) {
    unsigned middle_d = left_d + right_d;
    if (middle_d > limit) return 0;
    return 1 + p73_between(left_d, middle_d, limit) + p73_between(middle_d, right_d, limit);
}

int main(void) {
    printf("%" PRIu64 "\n", p73_between(3, 2, 12000));
    return 0;
}
```

Rust. Source: `solutions/rust/src/bin/p73.rs`.

```
fn solve() -> i64 {
    {
        fn count(a: u32, b: u32, c: u32, d: u32) -> u64 {
            if b + d > 12000 {
                return 0;
            }
            1 + count(a, b, a + c, b + d) + count(a + c, b + d, c, d)
        }
        count(1, 3, 1, 2) as i64
    }
}

fn main() {
    println!("{}", solve());
```

Gforth. Source: `solutions/forth/p73.fs`.

```
: p73 ( -- n ) 0 1 3 4000 11999 0 0 0
  { count a b c d k next-c next-d }
  begin
    c 2 * d = if count exit then
    count 1+ to count
    12000 b + d / to k
    k c * a - to next-c
    k d * b - to next-d
    c to a d to b next-c to c next-d to d
```

```
  again ;

[undefined] euler-testing [if] p73 . cr bye [then]
```

Factor. Source: `solutions/factor/euler/extra/extra.factor`.

```
: p73 ( -- n ) 1 3 1 2 p73-count ;
```

Comparison. C exposes direct bounded arithmetic. Rust exposes direct bounded arithmetic. Gforth exposes direct bounded arithmetic. Factor exposes direct bounded arithmetic. All listed programs print the same exact answer string from the manifest. The excerpts can differ in algorithm, data representation, and startup cost.

40.74 Problem 74: Digit Factorial Chains

Cache the count of distinct terms before the first repetition. A traversal either reaches a cached tail or discovers a cycle in its own path; cycle vertices receive the cycle length, then prefix lengths propagate backwards. Every successor is <=7*9!: starts have six digits and that bound has seven.

C. Source: `solutions/c/digit_factorial_chains_74.c`.

```
static unsigned p74_next(unsigned n) {
    static const unsigned factorial[] = {1, 1, 2, 6, 24, 120, 720, 5040, 40320, 362880};
    unsigned sum = 0;
    do { sum += factorial[n % 10]; n /= 10; } while (n);
    return sum;
}

static unsigned p74_count(unsigned start_limit, unsigned wanted) {
    const size_t capacity = 7 * 362880 + 1;
    if (start_limit > capacity) euler_fail("Starting range exceeds the proven chain bound");
    unsigned *length = euler_calloc(capacity, sizeof *length);
    unsigned *seen = euler_calloc(capacity, sizeof *seen);
    unsigned *position = euler_malloc(capacity * sizeof *position);
    unsigned *path = euler_malloc(capacity * sizeof *path);
```

Rust. Source: `solutions/rust/src/bin/p74.rs`.

```
fn p74_next(mut n: usize) -> usize {
    const FAC: [usize; 10] = [1, 1, 2, 6, 24, 120, 720, 5040, 40320, 362880];
    let mut sum = 0;
    loop {
        sum += FAC[n % 10];
        n /= 10;
        if n == 0 {
            return sum;
        }
    }
}

fn solve() -> i64 {
```

```
    let len = 7 * 362880 + 1;
```

Gforth. Source: `solutions/forth/p74.fs`.

```
create p74-factorials 1 , 1 , 2 , 6 , 24 , 120 , 720 , 5040 , 40320 , 362880 ,

: p74-next { number } ( n -- sum )
  0 begin number 0> while
    number 10 mod p74-factorials swap cells + @ +
    number 10 / to number
  repeat ;

: p74 ( -- n )
  2540161 cells allocate throw 2540161 cells allocate throw
  2540161 cells allocate throw 256 cells allocate throw
  0 0 0 0 0 0 0 { length visited positions path answer start number count first cycle tail }
  length 2540161 cells erase visited 2540161 cells erase
  1000000 1 do i to start
```

Factor. Source: `solutions/factor/euler/advanced/advanced.factor`.

```
:: p74 ( -- n )
    2600000 0 <array> :> lengths
    0 :> answer!
    1 999999 [a..b] [| start |
        H{ } clone :> positions
        V{ } clone :> path
        start :> current!
        [ current lengths length < current lengths nth zero? and
          current positions key? not and ] [
            path length current positions set-at
            current path push
            current p74-digit-factorial current!
        ] while
        current lengths length < current lengths nth zero? not and [
```

Comparison. C exposes dynamic programming or memoization, and exact large-integer arithmetic. Shared calls: `euler_calloc`, `euler_fail`, `euler_malloc`. Rust exposes exact large-integer arithmetic. Gforth exposes modular arithmetic, and exact large-integer arithmetic. Shared calls: `prefix!`, `prefix@`. Factor exposes exact large-integer arithmetic, and sorting or set deduplication. All listed programs print the same exact answer string from the manifest. The excerpts can differ in algorithm, data representation, and startup cost.

40.75 Problem 75: Singular Integer Right Triangles

Coprime, opposite-parity m>n give each primitive Pythagorean triangle once, with perimeter 2*m*(m+n). Count its multiples, saturating counts at two.

C. Source: `solutions/c/singular_integer_right_triangles_75.c`.

```
static unsigned p75_unique(unsigned limit) {
    unsigned char *counts = euler_calloc((size_t)limit + 1, 1);
    for (unsigned m = 2; 2 * m * (m + 1) <= limit; ++m)
        for (unsigned n = 1; n < m; ++n) {
            if ((m + n) % 2 == 0 || euler_gcd(m, n) != 1) continue;
            unsigned perimeter = 2 * m * (m + n);
            if (perimeter > limit) break;
            for (unsigned p = perimeter; p <= limit; p += perimeter)
                if (counts[p] < 2) ++counts[p];
        }
    unsigned answer = 0;
    for (unsigned p = 1; p <= limit; ++p) answer += counts[p] == 1;
    free(counts);
    return answer;
```

Rust. Source: `solutions/rust/src/bin/p75.rs`.

```
fn solve() -> i64 {
    {
        let mut ways = vec![0u8; 1_500_001];
        for m in 2u64..=866 {
            for n in 1..m {
                if (m - n) % 2 == 0 || gcd(m, n) != 1 {
                    continue;
                }
                let perimeter = 2 * m * (m + n);
                if perimeter > 1_500_000 {
                    break;
                }
                for p in (perimeter as usize..=1_500_000).step_by(perimeter as usize) {
                    ways[p] = ways[p].saturating_add(1);
```

Gforth. Source: `solutions/forth/p75.fs`.

```
: p75 ( -- n )
  1500001 allocate throw 0 0 0 0 { counts answer perimeter m n }
  counts 1500001 erase
  867 2 do i to m
    m 1 do i to n
      m n - 2 mod 0<> m n integer-gcd 1 = and if
        m m n + * 2* to perimeter
        perimeter 1500000 <= if
          1500001 perimeter do counts i + dup c@ 1+ 2 min swap c! perimeter +loop
        then
      then
    loop
  loop
  1500001 1 do counts i + c@ 1 = if answer 1+ to answer then loop
```

Factor. Source: `solutions/factor/euler/extra/extra.factor`.

```
:: p75 ( -- n )
    1500001 0 <array> :> perimeters
    2 866 [a..b] [| m |
```

```
        1 m 1 - [a..b] [| n |
            m n - odd? m n gcd nip 1 = and [
                m m n + * 2 * :> perimeter
                perimeter 1500000 <= [
                    perimeter 1500000 perimeter <range> [| multiple |
                        multiple perimeters nth 1 + 2 min
                        multiple perimeters set-nth
                    ] each
                ] when
            ] when
        ] each
```

Comparison. C exposes prime generation or testing. Shared calls: `euler_calloc`, `euler_gcd`. Rust exposes direct bounded arithmetic. Shared calls: `gcd`. Gforth exposes modular arithmetic. Shared calls: `integer-gcd`. Factor exposes direct bounded arithmetic. All listed programs print the same exact answer string from the manifest. The excerpts can differ in algorithm, data representation, and startup cost.

40.76 Problem 76: Counting Summations

The coin-change recurrence counts unordered partitions. Excluding the part 100 removes exactly the forbidden single-term representation of 100.

C. Source: `solutions/c/counting_summations_76.c`.

```
int main(void) {
    uint64_t ways[101] = {1};
    for (unsigned part = 1; part < 100; ++part)
        for (unsigned sum = part; sum <= 100; ++sum) ways[sum] += ways[sum - part];
    printf("%" PRIu64 "\n", ways[100]);
    return 0;
}
```

Rust. Source: `solutions/rust/src/bin/p76.rs`.

```
fn solve() -> i64 {
    {
        let mut ways = [0u64; 101];
        ways[0] = 1;
        for part in 1..100 {
            for n in part..=100 {
                ways[n] += ways[n - part];
            }
        }
        ways[100] as i64
    }
}

fn main() {
```

Gforth. Source: `solutions/forth/p76.fs`.

```
: p76 ( -- n )
  101 cells allocate throw 0 { ways answer }
  ways 101 cells erase 1 ways !
  100 1 do
    101 i do i j - ways prefix@ i cells ways + +! loop
  loop
  100 ways prefix@ to answer ways free throw answer ;

[undefined] euler-testing [if] p76 . cr bye [then]
```

Factor. Source: `solutions/factor/euler/extra/extra.factor`.

```
:: p76 ( -- n )
    101 0 <array> :> ways 1 0 ways set-nth
    1 99 [a..b] [| coin |
        coin 100 [a..b] [| n |
            n ways nth n coin - ways nth + n ways set-nth
        ] each
    ] each 100 ways nth ;
```

Comparison. C exposes direct bounded arithmetic. Rust exposes direct bounded arithmetic. Gforth exposes direct bounded arithmetic. Shared calls: `prefix@`. Factor exposes direct bounded arithmetic. All listed programs print the same exact answer string from the manifest. The excerpts can differ in algorithm, data representation, and startup cost.

40.77 Problem 77: Prime Summations

Unordered prime partitions use the coin-change recurrence. Double the search bound until the first count exceeds 5000; saturating at 5001 is sufficient for this comparison and prevents unnecessary growth in the counts.

C. Source: `solutions/c/prime_summations_77.c`.

```
int main(void) {
    for (unsigned limit = 16;; limit *= 2) {
        unsigned char *prime = euler_sieve(limit);
        unsigned *ways = euler_calloc((size_t)limit + 1, sizeof *ways);
        ways[0] = 1;
        for (unsigned p = 2; p <= limit; ++p) if (prime[p])
            for (unsigned n = p; n <= limit; ++n) {
                unsigned sum = ways[n] + ways[n - p];
                ways[n] = sum > 5000 ? 5001 : sum;
            }
        unsigned answer = 0;
        for (unsigned n = 2; n <= limit; ++n) if (ways[n] > 5000) { answer = n; break; }
        free(ways); free(prime);
        if (answer) { printf("%u\n", answer); return 0; }
```

Rust. Source: `solutions/rust/src/bin/p77.rs`.

```
fn solve() -> i64 {
    {
        let mut n = 2usize;
        loop {
            let mut ways = vec![0u64; n + 1];
            ways[0] = 1;
            for p in primes(n) {
                let p = p as usize;
                for x in p..=n {
                    ways[x] += ways[x - p];
                }
            }
            if ways[n] > 5000 {
                break n as i64;
```

Gforth. Source: `solutions/forth/p77.fs`.

```
: p77 ( -- n ) 128 cells allocate throw 2 { ways n }
  2 to n begin ways 128 cells erase 1 ways !
    n 1+ 2 do i is-prime? if
      n 1+ i do i j - ways prefix@ i cells ways + +! loop
    then loop
    n ways prefix@ 5000 > if ways free throw n exit then
    n 1+ to n
  again ;

[undefined] euler-testing [if] p77 . cr bye [then]
```

Factor. Source: `solutions/factor/euler/extra/extra.factor`.

```
:: p77 ( -- n )
    2 :> n! 0 :> answer!
    [ answer zero? ] [
        n 1 + 0 <array> :> ways
        1 0 ways set-nth
        2 n [a..b] [| p |
            p prime? [
                p n [a..b] [| amount |
                    amount ways nth amount p - ways nth +
                    amount ways set-nth
                ] each
            ] when
        ] each
        n ways nth 5000 > [ n answer! ] when
```

Comparison. C exposes prime generation or testing. Shared calls: `euler_calloc`, `euler_fail`, `euler_sieve`. Rust exposes prime generation or testing. Gforth exposes prime generation or testing. Shared calls: `is-prime?`, `prefix@`. Factor exposes prime generation or testing. All listed programs print the same exact answer string from the manifest. The excerpts can differ in algorithm, data representation, and startup cost.

40.78 Problem 78: Coin Partitions

Euler's pentagonal recurrence is linear, so reducing each p(n) modulo one million preserves the divisibility test. The signs repeat +,+,-,- at the generalized pentagonal offsets k*(3*k-1)/2 and k*(3*k+1)/2.

C. Source: `solutions/c/coin_partitions_78.c`.

```
static unsigned p78_first_divisible(unsigned modulus) {
    if (modulus == 0) euler_fail("Partition modulus must be positive");
    size_t capacity = 1024;
    unsigned *partitions = euler_malloc(capacity * sizeof *partitions);
    partitions[0] = 1 % modulus;
    for (unsigned n = 1;; ++n) {
        if (n == capacity) {
            capacity *= 2;
            unsigned *larger = realloc(partitions, capacity * sizeof *partitions);
            if (!larger) euler_fail("Partition allocation failed");
            partitions = larger;
        }
        int64_t total = 0;
        for (uint64_t k = 1;; ++k) {
```

Rust. Source: `solutions/rust/src/bin/p78.rs`.

```
fn solve() -> i64 {
    {
        let modulo = 1_000_000i64;
        let mut partitions = vec![1i64];
        let mut answer = 0;
        for n in 1..100_000usize {
            let mut p = 0i64;
            let mut k = 1i64;
            loop {
                let g1 = k * (3 * k - 1) / 2;
                let g2 = k * (3 * k + 1) / 2;
                if g1 as usize > n {
                    break;
                }
```

Gforth. Source: `solutions/forth/p78.fs`.

```
: p78 ( -- n )
  100001 cells allocate throw 0 0 0 0 0 { ways value k first second sign }
  ways 100001 cells erase 1 ways !
  100000 1 do
    0 to value 1 to k
    begin k 3 k * 1- * 2 / to first
      first i <= while
      first k + to second
      k 2 mod if 1 else -1 then to sign
      i first - ways prefix@ sign * value + to value
      second i <= if i second - ways prefix@ sign * value + to value then
      k 1+ to k
    repeat
```

```
    value 1000000 mod dup 0< if 1000000 + then dup i ways prefix! 0= if
```

Factor. Source: `solutions/factor/euler/extra/extra.factor`.

```
:: p78 ( -- n )
    100001 0 <array> :> p
    1 0 p set-nth
    1 :> n! 0 :> answer!
    [ answer zero? ] [
        0 :> total! 1 :> k!
        [ k 3 k * 1 - * 2 /i n <= ] [
            k 3 k * 1 - * 2 /i :> g1
            k 3 k * 1 + * 2 /i :> g2
            k odd? [ 1 ] [ -1 ] if :> sign
            total sign n g1 - p nth * + total!
            g2 n <= [ total sign n g2 - p nth * + total! ] when
            k 1 + k!
        ] while
```

Comparison. C exposes direct bounded arithmetic. Shared calls: `euler_fail`, `euler_malloc`. Rust exposes direct bounded arithmetic. Gforth exposes modular arithmetic. Shared calls: `prefix!`, `prefix@`. Factor exposes modular arithmetic. All listed programs print the same exact answer string from the manifest. The excerpts can differ in algorithm, data representation, and startup cost.

40.79 Problem 79: Passcode Derivation

Each observed triple contributes two precedence edges. The supplied data admit a topological ordering using each digit once. This reaches the lower bound of one occurrence per observed digit, so the passcode is shortest. Repeated digits or cyclic constraints are rejected, not silently discarded.

C. Source: `solutions/c/passcode_derivation_79.c`.

```
int main(void) {
    bool edges[10][10] = {{false}}, present[10] = {false}, emitted[10] = {false};
    FILE *file = euler_open("tests/data/p079_keylog.txt");
    unsigned lines = 0;
    char token[5];
    while (fscanf(file, "%4s", token) == 1) {
        if (strlen(token) != 3) euler_fail("Expected three-digit login attempts");
        int digit[3];
        for (int i = 0; i < 3; ++i) {
            if (token[i] < '0' || token[i] > '9') euler_fail("Invalid login digit");
            digit[i] = token[i] - '0'; present[digit[i]] = true;
        }
        if (digit[0] == digit[1] || digit[0] == digit[2] || digit[1] == digit[2])
            euler_fail("Repeated digit requires a different passcode model");
```

Rust. Source: `solutions/rust/src/bin/p79.rs`.

```
fn solve() -> i64 {
    let mut edges = [[false; 10]; 10];
```

```
    let mut present = [false; 10];
    for line in include_str!("../../../../tests/data/p079_keylog.txt").lines() {
        let d: Vec<usize> = line.bytes().map(|b| (b - b'0') as usize).collect();
        assert_eq!(d.len(), 3);
        for &v in &d {
            present[v] = true;
        }
        edges[d[0]][d[1]] = true;
        edges[d[1]][d[2]] = true;
    }
    let mut emitted = [false; 10];
    let mut result = 0;
```

Gforth. Source: `solutions/forth/p79.fs`.

```
: p79 ( -- n )
  s" tests/data/p079_keylog.txt" 150 fixture-integers
  100 allocate throw 10 allocate throw 10 allocate throw
  0 0 0 0 0 0 0 0 { triples count edges present emitted total next answer triple first second
    third available }
  edges 100 erase present 10 erase emitted 10 erase
  count 0 do
    i triples prefix@ to triple
    triple 100 / to first
    triple 10 / 10 mod to second
    triple 10 mod to third
    1 present first + c! 1 present second + c! 1 present third + c!
    1 edges first 10 * second + + c!
    1 edges second 10 * third + + c!
  loop
```

Factor. Source: `solutions/factor/euler/advanced/advanced.factor`.

```
:: p79 ( -- n )
    "tests/data/p079_keylog.txt" read-numbers :> entries
    100 f <array> :> edge
    10 f <array> :> present
    entries [| code |
        code 100 /i :> a code 10 /i 10 mod :> b code 10 mod :> c
        t a present set-nth t b present set-nth t c present set-nth
        t a 10 * b + edge set-nth
        t b 10 * c + edge set-nth
    ] each
    0 :> result! 0 :> steps!
    [ present [ t = ] any? ] [
        -1 :> chosen!
        0 9 [a..b] [| digit |
```

Comparison. C exposes direct bounded arithmetic. Shared calls: `euler_open`. Rust exposes fixture parsing. Gforth exposes modular arithmetic. Shared calls: `fixture-integers`, `prefix@`. Factor exposes modular arithmetic, and fixture parsing. Shared calls: `read-numbers`. All listed programs print the same exact answer string from the manifest. The excerpts can differ in algorithm, data representation, and startup cost.

40.80 Problem 80: Square Root Digital Expansion

For nonsquare n<=100, floor(sqrt(n*10^198)) contains exactly the first 100 decimal digits of sqrt(n), including the integer digit. Integer square root in GMP avoids roundoff at the last requested digit.

C. Source: `solutions/c/square_root_digital_expansion_80.c`.

```
int main(void) {
    mpz_t scale, value, root;
    mpz_inits(scale, value, root, NULL);
    mpz_ui_pow_ui(scale, 10, 198);
    unsigned sum = 0;
    for (unsigned n = 1; n <= 100; ++n) {
        unsigned integer = (unsigned)euler_isqrt(n);
        if (integer * integer == n) continue;
        mpz_mul_ui(value, scale, n); mpz_sqrt(root, value);
        char *digits = mpz_get_str(NULL, 10, root);
        if (strlen(digits) != 100) euler_fail("Unexpected square-root digit count");
        for (unsigned i = 0; i < 100; ++i) sum += (unsigned)(digits[i] - '0');
        free(digits);
    }
```

Rust. Source: `solutions/rust/src/bin/p80.rs`.

```
fn solve() -> i64 {
    {
        let scale = BigUint::from(10u32).pow(198);
        let mut sum = 0usize;
        for n in 1..=100u64 {
            if is_square(n) {
                continue;
            }
            let root = big_isqrt(&(BigUint::from(n) * &scale));
            let text = root.to_string();
            assert_eq!(text.len(), 100);
            sum += text.bytes().map(|b| (b - b'0') as usize).sum::<usize>();
        }
        sum as i64
```

Gforth. Source: `solutions/forth/p80.fs`.

```
: p80-root-digits 0 0 0 0 0 0 { number remainder root trial answer digit found } ( n -- digit-
    sum )
  12 bnew to remainder 12 bnew to root 12 bnew to trial
  0 remainder bset-u 0 root bset-u
  100 0 do
    100 remainder bmul-small!
    i 0= if number remainder badd-small! then
    false to found
    10 0 do
      9 i - to digit
      root trial bcopy
      20 trial bmul-small!
      digit trial badd-small!
      digit trial bmul-small!
```

```
    remainder trial babscompare 0>= if
```

Factor. Source: `solutions/factor/euler/advanced/advanced.factor`.

```
:: p80 ( -- n )
    0 :> answer!
    1 100 [a..b] [| n |
        n euler-isqrt :> root
        root sq n = not [
            n 10 198 ^ * euler-isqrt digit-sum answer + answer!
        ] when
    ] each answer ;
```

Comparison. C exposes exact large-integer arithmetic. Shared calls: `euler_fail`, `euler_isqrt`. Rust exposes exact large-integer arithmetic. Shared calls: `big_isqrt`. Gforth exposes exact large-integer arithmetic. Shared calls: `babscompare`, `badd-small!`, `bcopy`, `bfree`, `bmul-small!`, `bnew`, `bset-u`, `bsubabs!`, `integer-sqrt`. Factor exposes direct bounded arithmetic. Shared calls: `digit-sum`, `euler-isqrt`. All listed programs print the same exact answer string from the manifest. The excerpts can differ in algorithm, data representation, and startup cost.

40.81 Problem 81: Path Sum: Two Ways

Right/down paths form a directed acyclic graph. A one-row dynamic program retains the best cost from above and the newly updated cost at left.

C. Source: `solutions/c/path_sum_two_ways_81.c`.

```
static void p81_read_matrix(unsigned *matrix, size_t side, const char *path) {
    FILE *file = euler_open(path);
    for (size_t row = 0; row < side; ++row) {
        for (size_t col = 0; col < side; ++col) {
            unsigned *cell = &matrix[row * side + col];
            if (fscanf(file, "%u", cell) != 1 || *cell == 0 || *cell > 1000000)
                euler_fail("Expected positive matrix entries");
            if (col + 1 < side && fgetc(file) != ',') euler_fail("Expected matrix comma");
        }
        int separator = fgetc(file);
        if (separator == '\r') separator = fgetc(file);
        if (separator != '\n' && !(row + 1 == side && separator == EOF))
            euler_fail("Unexpected matrix row length");
    }
```

Rust. Source: `solutions/rust/src/bin/p81.rs`.

```
fn solve() -> i64 {
    let input = matrix(include_str!("../../../../tests/data/p081_matrix.txt"));
    let mut cost = vec![0u64; input[0].len()];
    for (r, line) in input.iter().enumerate() {
        for (c, &value) in line.iter().enumerate() {
            if r == 0 && c > 0 {
                cost[c] = cost[c - 1];
```

```
            } else if r > 0 && c > 0 {
                cost[c] = cost[c].min(cost[c - 1]);
            }
            cost[c] += value;
        }
    }
    cost[cost.len() - 1] as i64
```

Gforth. Source: `solutions/forth/p81.fs`.

```
: p81 ( -- n )
  s" tests/data/p081_matrix.txt" 6400 fixture-integers
  80 cells allocate throw 0 0 0 { matrix count costs row col answer }
  count 6400 <> abort" Wrong matrix size"
  costs 80 cells erase
  80 0 do i to row
    80 0 do i to col
      row 0= col 0> and if col 1- costs prefix@ col costs prefix! then
      row 0> col 0> and if
        col costs prefix@ col 1- costs prefix@ min col costs prefix!
      then
      row 80 * col + matrix prefix@ col cells costs + +!
    loop
  loop
```

Factor. Source: `solutions/factor/euler/advanced/advanced.factor`.

```
:: p81 ( -- n )
    read-matrix :> costs
    6400 0 <array> :> dp
    0 79 [a..b] [| r |
        0 79 [a..b] [| c |
            r 80 * c + :> index
            r zero? c zero? and [ 0 ] [
                r zero? [ index 1 - dp nth ] [
                    c zero? [ index 80 - dp nth ] [
                        index 80 - dp nth index 1 - dp nth min
                    ] if
                ] if
            ] if
            index costs nth + index dp set-nth
```

Comparison. C exposes dynamic programming or memoization, and fixture parsing. Shared calls: `euler_calloc`, `euler_open`. Rust exposes fixture parsing. Shared calls: `matrix`. Gforth exposes direct bounded arithmetic. Shared calls: `fixture-integers`, `prefix!`, `prefix@`. Factor exposes dynamic programming or memoization, and fixture parsing. All listed programs print the same exact answer string from the manifest. The excerpts can differ in algorithm, data representation, and startup cost.

40.82 Problem 82: Path Sum: Three Ways

Process columns from left to right. Enter each row from its left neighbour, then sweep down and up to allow a vertical segment within the new column. Positive weights make any vertical reversal or repeated visit unnecessary.

C. Source: `solutions/c/path_sum_three_ways_82.c`.

```
static void p82_read_matrix(unsigned *matrix, size_t side, const char *path) {
    FILE *file = euler_open(path);
    for (size_t row = 0; row < side; ++row) {
        for (size_t col = 0; col < side; ++col) {
            unsigned *cell = &matrix[row * side + col];
            if (fscanf(file, "%u", cell) != 1 || *cell == 0 || *cell > 1000000)
                euler_fail("Expected positive matrix entries");
            if (col + 1 < side && fgetc(file) != ',') euler_fail("Expected matrix comma");
        }
        int separator = fgetc(file);
        if (separator == '\r') separator = fgetc(file);
        if (separator != '\n' && !(row + 1 == side && separator == EOF))
            euler_fail("Unexpected matrix row length");
    }
```

Rust. Source: `solutions/rust/src/bin/p82.rs`.

```
fn solve() -> i64 {
    let input = matrix(include_str!("../../../../tests/data/p082_matrix.txt"));
    let side = input.len();
    let mut cost: Vec<u64> = (0..side).map(|row| input[row][0]).collect();
    for c in 1..side {
        for r in 0..side {
            cost[r] += input[r][c];
        }
        for r in 1..side {
            cost[r] = cost[r].min(cost[r - 1] + input[r][c]);
        }
        for r in (0..side - 1).rev() {
            cost[r] = cost[r].min(cost[r + 1] + input[r][c]);
        }
```

Gforth. Source: `solutions/forth/p82.fs`.

```
: p82 ( -- n )
  s" tests/data/p082_matrix.txt" 6400 fixture-integers
  80 cells allocate throw 0 0 0 { matrix count costs column row answer }
  count 6400 <> abort" Wrong matrix size"
  80 0 do i 80 * matrix prefix@ i costs prefix! loop
  80 1 do i to column
    80 0 do
      i 80 * column + matrix prefix@ i cells costs + +!
    loop
    80 1 do
      i 1- costs prefix@ i 80 * column + matrix prefix@ +
      i costs prefix@ min i costs prefix!
    loop
```

```
    79 0 do
```

Factor. Source: `solutions/factor/euler/advanced/advanced.factor`.

```
:: p82 ( -- n )
    read-matrix :> matrix
    80 0 <array> :> dist
    0 79 [a..b] [| r | r 80 * matrix nth r dist set-nth ] each
    1 79 [a..b] [| c |
        0 79 [a..b] [| r |
            r dist nth r 80 * c + matrix nth +
            r dist set-nth
        ] each
        1 79 [a..b] [| r |
            r 1 - dist nth r 80 * c + matrix nth +
            r dist nth min r dist set-nth
        ] each
        78 0 [a..b] [| r |
```

Comparison. C exposes fixture parsing. Shared calls: `euler_malloc`, `euler_open`. Rust exposes fixture parsing. Shared calls: `matrix`. Gforth exposes direct bounded arithmetic. Shared calls: `fixture-integers`, `prefix!`, `prefix@`. Factor exposes fixture parsing. All listed programs print the same exact answer string from the manifest. The excerpts can differ in algorithm, data representation, and startup cost.

40.83 Problem 83: Path Sum: Four Ways

Dijkstra settles the cheapest unvisited vertex before relaxing its four neighbours. The 80x80 grid is small enough for the simple O(V^2) minimum scan; uint64_t costs retain exact sums of positive input weights.

C. Source: `solutions/c/path_sum_four_ways_83.c`.

```
static void p83_read_matrix(unsigned *matrix, size_t side, const char *path) {
    FILE *file = euler_open(path);
    for (size_t row = 0; row < side; ++row) {
        for (size_t col = 0; col < side; ++col) {
            unsigned *cell = &matrix[row * side + col];
            if (fscanf(file, "%u", cell) != 1 || *cell == 0 || *cell > 1000000)
                euler_fail("Expected positive matrix entries");
            if (col + 1 < side && fgetc(file) != ',') euler_fail("Expected matrix comma");
        }
        int separator = fgetc(file);
        if (separator == '\r') separator = fgetc(file);
        if (separator != '\n' && !(row + 1 == side && separator == EOF))
            euler_fail("Unexpected matrix row length");
    }
```

Rust. Source: `solutions/rust/src/bin/p83.rs`.

```
fn solve() -> i64 {
    use std::cmp::Reverse;
    use std::collections::BinaryHeap;
    let input = matrix(include_str!("../../../../tests/data/p083_matrix.txt"));
    let side = input.len();
    let mut cost = vec![u64::MAX; side * side];
    cost[0] = input[0][0];
    let mut queue = BinaryHeap::new();
    queue.push((Reverse(cost[0]), 0usize));
    while let Some((Reverse(d), u)) = queue.pop() {
        if d != cost[u] {
            continue;
        }
        if u == side * side - 1 {
```

Gforth. Source: `solutions/forth/p83.fs`.

```
: p83-relax 0 { vertex costs distance current candidate } ( index costs dist value -- )
  current vertex costs prefix@ + to candidate
  candidate vertex distance prefix@ < if candidate vertex distance prefix! then ;

: p83 ( -- n )
  s" tests/data/p083_matrix.txt" 6400 fixture-integers
  6400 cells allocate throw 6400 allocate throw
  0 0 0 0 0 { costs count distance used best vertex row col candidate }
  count 6400 <> abort" Matrix requires 6400 values"
  used 6400 erase
  6400 0 do 100000000 i distance prefix! loop
  0 costs prefix@ 0 distance prefix!
  6400 0 do
    100000000 to best -1 to vertex
```

Factor. Source: `solutions/factor/euler/advanced/advanced.factor`.

```
:: p83 ( -- n )
    read-matrix :> matrix
    6400 2147483647 <array> :> dist
    6400 f <array> :> visited
    0 matrix nth 0 dist set-nth
    6400 [| |
        -1 :> choice! 2147483647 :> smallest!
        0 6399 [a..b] [| index |
            index visited nth not index dist nth smallest < and [
                index choice! index dist nth smallest!
            ] when
        ] each
        choice -1 >= [
            t choice visited set-nth
```

Comparison. C exposes fixture parsing. Shared calls: `euler_calloc`, `euler_malloc`, `euler_open`. Rust exposes fixture parsing. Shared calls: `matrix`. Gforth exposes modular arithmetic. Shared calls: `fixture-integers`, `prefix!`, `prefix@`. Factor exposes modular arithmetic, and fixture parsing. All listed programs print the same exact answer string from the manifest. The excerpts can differ in

algorithm, data representation, and startup cost.

40.84 Problem 84: Monopoly Odds

Deterministic Markov calculation for cyclic decks, not a random simulation. State is (square, consecutive doubles, next CC card, next CH card), so cards return to the bottom exactly as specified. Since the initial shuffle is not supplied, use the explicit valid shuffled orders below. Fine probabilities depend on that order; the requested three-square modal string is also checked with a different ordering in tests/c_middle_examples.c. For six sided dice these two orders even exchange GO and square 19 in third place; the example percentages are not treated as universal for every shuffle. Power iteration stops only when the full distribution's L1 change is small.

C. Source: `solutions/c/monopoly_odds_84.c`.

```
enum { P84_STATES = 40 * 3 * 16 * 16, P84_MAX_OUTCOMES = 21 };
enum { P84_STAY = -1, P84_NEXT_R = -2, P84_NEXT_U = -3, P84_BACK = -4 };
typedef struct { unsigned sum, doubles, ways; } P84Outcome;

static unsigned p84_index(unsigned square, unsigned doubles, unsigned cc, unsigned ch) {
    return ((square * 3 + doubles) * 16 + cc) * 16 + ch;
}

static unsigned p84_transition(unsigned state, P84Outcome roll, const int cc_deck[16], const int
     ch_deck[16]) {
    unsigned ch = state % 16; state /= 16;
    unsigned cc = state % 16; state /= 16;
    unsigned doubles = state % 3, square = state / 3;
    if (roll.doubles && doubles == 2) return p84_index(10, 0, cc, ch);
    doubles = roll.doubles ? doubles + 1 : 0;
```

Rust. Source: `solutions/rust/src/bin/p84.rs`.

```
const STAY: i8 = -1;

const NEXT_R: i8 = -2;

const NEXT_U: i8 = -3;

const BACK: i8 = -4;

const SQUARES: usize = 40;

const STATES: usize = SQUARES * 3 * 16 * 16;

const CHEST: [i8; 16] = [
    STAY, 0, STAY, STAY, STAY, STAY, 10, STAY, STAY, STAY, STAY, STAY, STAY, STAY, STAY, STAY,
```

Gforth. Source: `solutions/forth/p84.fs`.

```
create p84-current 120 floats allot

create p84-next 120 floats allot
```

```
create p84-used 40 allot
fvariable p84-maximum

: p84-store { square doubles } ( F: probability -- )
  p84-next square 3 * doubles + floats +
  dup f@ f+ f! ;

: p84-cc { square doubles } ( F: probability -- )
  fdup 16e f/ 0 doubles p84-store
  fdup 16e f/ 10 0 p84-store
```

Factor. Source: `solutions/factor/euler/advanced/advanced.factor`.

```
:: p84 ( -- n )
    120 120.0 recip <array> :> probability!
    300 [| |
        120 0.0 <array> :> next
        0 39 [a..b] [| square |
            0 2 [a..b] [| doubles |
                square 3 * doubles + probability nth 16 / :> weight
                1 4 [a..b] [| a |
                    1 4 [a..b] [| b |
                        square doubles a b weight next p84-roll
                    ] each
                ] each
            ] each
        ] each next probability!
```

Comparison. C exposes direct bounded arithmetic. Shared calls: `euler_fail`, `euler_malloc`. Rust exposes sorting or set deduplication. Gforth exposes modular arithmetic. Factor exposes direct bounded arithmetic. All listed programs print the same exact answer string from the manifest. The excerpts can differ in algorithm, data representation, and startup cost.

40.85 Problem 85: Counting Rectangles

An m by n grid has T(m)*T(n) rectangles, where T(k)=k*(k+1)/2. Restrict m<=n. For each m, counts increase with n, so stop immediately above the target. Once the square count is too large, every later m loses.

C. Source: `solutions/c/counting_rectangles_85.c`.

```
int main(void) {
    const uint64_t target = 2000000;
    uint64_t best_difference = target, best_area = 0;
    for (uint64_t m = 1;; ++m) {
        uint64_t tm = m * (m + 1) / 2;
        if (tm * tm > target && tm * tm - target > best_difference) break;
        for (uint64_t n = m;; ++n) {
            uint64_t count = tm * n * (n + 1) / 2;
            uint64_t difference = count > target ? count - target : target - count;
            if (difference < best_difference) { best_difference = difference; best_area = m * n;
     }
```

```
            if (count >= target) break;
        }
    }
    printf("%" PRIu64 "\n", best_area);
```

Rust. Source: `solutions/rust/src/bin/p85.rs`.

```
fn solve() -> i64 {
    {
        let mut closest = u64::MAX;
        let mut best = 0;
        for w in 1..=2000u64 {
            for h in 1..=w {
                let number = w * (w + 1) * h * (h + 1) / 4;
                let diff = number.abs_diff(2_000_000);
                if diff < closest {
                    closest = diff;
                    best = w * h;
                }
                if number > 2_100_000 {
                    break;
```

Gforth. Source: `solutions/forth/p85.fs`.

```
: p85 ( -- n ) 2000000 0 0 0 0 0 { closest best width height rectangles diff }
  2001 1 do i to width
    width 1+ 1 do i to height
      width width 1+ * height height 1+ * * 4 / to rectangles
      rectangles 2000000 - abs to diff
      diff closest < if diff to closest width height * to best then
      rectangles 2100000 > if leave then
    loop
  loop best ;

[undefined] euler-testing [if] p85 . cr bye [then]
```

Factor. Source: `solutions/factor/euler/extra/extra.factor`.

```
:: p85 ( -- n )
    2000000 :> diff! 0 :> best!
    1 2000 [a..b] [| width |
        1 width [a..b] [| height |
            width width 1 + * height height 1 + * * 4 /i :> rectangles
            rectangles 2000000 - abs :> error
            error diff < [ error diff! width height * best! ] when
        ] each
    ] each best ;
```

Comparison. C exposes direct bounded arithmetic. Rust exposes direct bounded arithmetic. Gforth exposes direct bounded arithmetic. Factor exposes direct bounded arithmetic. All listed programs print the same exact answer string from the manifest. The excerpts can differ in algorithm, data representation, and startup cost.

40.86 Problem 86: Cuboid Route

With a<=b<=c the shortest unfolded route is sqrt(c*c+(a+b)^2). For a fixed c and s=a+b, valid a run from max(1,s-c) through floor(s/2). This counts unordered cuboids once without iterating over a and b separately.

C. Source: `solutions/c/cuboid_route_86.c`.

```
static uint64_t p86_layer(unsigned c) {
    uint64_t count = 0;
    for (uint64_t sum = 2; sum <= 2 * (uint64_t)c; ++sum) {
        uint64_t squared = (uint64_t)c * c + sum * sum;
        uint64_t root = euler_isqrt(squared);
        if (root * root != squared) continue;
        uint64_t low = sum > c ? sum - c : 1, high = sum / 2;
        if (high >= low) count += high - low + 1;
    }
    return count;
}

int main(void) {
    uint64_t count = 0;
```

Rust. Source: `solutions/rust/src/bin/p86.rs`.

```
fn solve() -> i64 {
    {
        let mut total = 0u64;
        for maximum in 1u64.. {
            for sum in 2..=2 * maximum {
                if is_square(maximum * maximum + sum * sum) {
                    let low = if sum > maximum { sum - maximum } else { 1 };
                    let high = (sum / 2).min(maximum);
                    if high >= low {
                        total += high - low + 1;
                    }
                }
            }
            if total > 1_000_000 {
```

Gforth. Source: `solutions/forth/p86.fs`.

```
: p86 ( -- n ) 0 0 0 0 0 0 { total largest squared root low high }
  1 to largest
  begin largest 2* 1+ 2 do
      largest largest * i i * + to squared
      squared integer-sqrt to root
      root root * squared = if
        i largest > if i largest - else 1 then to low
        i 2 / largest min to high
        high low >= if total high low - 1+ + to total then
      then
    loop
    total 1000000 > if largest exit then
    largest 1+ to largest
```

```
  again ;
```

Factor. Source: `solutions/factor/euler/extra/extra.factor`.

```
:: p86 ( -- n )
    0 :> count! 1 :> length!
    [ count 1000000 <= ] [
        2 length 2 * [a..b] [| joined |
            length sq joined sq + euler-isqrt :> root
            root sq length sq joined sq + = [
                joined length - 1 max :> low
                joined 2 /i length min :> high
                high low >= [ count high low - 1 + + count! ] when
            ] when
        ] each
        length 1 + length!
    ] while length 1 - ;
```

Comparison. C exposes direct bounded arithmetic. Shared calls: `euler_isqrt`. Rust exposes direct bounded arithmetic. Gforth exposes direct bounded arithmetic. Shared calls: `integer-sqrt`. Factor exposes direct bounded arithmetic. Shared calls: `euler-isqrt`. All listed programs print the same exact answer string from the manifest. The excerpts can differ in algorithm, data representation, and startup cost.

40.87 Problem 87: Prime Power Triples

Enumerate prime fourth powers, cubes and squares, stopping each increasing loop at fifty million. A bitset deduplicates sums reached in multiple ways.

C. Source: `solutions/c/prime_power_triples_87.c`.

```
int main(void) {
    const uint64_t limit = 50000000;
    unsigned bound = (unsigned)euler_isqrt(limit - 1);
    unsigned char *prime = euler_sieve(bound);
    unsigned *list = euler_malloc(((size_t)bound + 1) * sizeof *list);
    size_t count = 0;
    for (unsigned n = 2; n <= bound; ++n) if (prime[n]) list[count++] = n;
    unsigned char *seen = euler_calloc((size_t)((limit + 7) / 8), 1);
    unsigned answer = 0;
    for (size_t i = 0; i < count; ++i) {
        uint64_t a = list[i], fourth = a * a * a * a;
        if (fourth >= limit) break;
        for (size_t j = 0; j < count; ++j) {
            uint64_t b = list[j], partial = fourth + b * b * b;
```

Rust. Source: `solutions/rust/src/bin/p87.rs`.

```
fn solve() -> i64 {
    {
        let limit = 50_000_000u64;
```

```
        let roots = primes(isqrt(limit) as usize);
        let mut seen = vec![false; limit as usize];
        for &p in &roots {
            let a = p * p;
            if a >= limit {
                break;
            }
            for &q in &roots {
                let b = q * q * q;
                if a + b >= limit {
                    break;
```

Gforth. Source: `solutions/forth/p87.fs`.

```
: p87 ( -- n )
  7072 sieve-build
  1000 cells allocate throw
  6250000 allocate throw
  0 0 0 0 0 0 0 { primes list seen count answer x y z value mask }
  seen 6250000 erase
  7072 2 do primes i + c@ if i count list prefix! count 1+ to count then loop
  count 0 do
    i list prefix@ dup * dup * to x
    x 50000000 >= if leave then
    count 0 do
      i list prefix@ dup dup * * to y
      x y + 50000000 >= if leave then
      count 0 do
```

Factor. Source: `solutions/factor/euler/advanced/advanced.factor`.

```
:: p87 ( -- n )
    50000000 f <array> :> seen
    7072 euler-sieve :> sieve
    V{ } clone :> primes
    2 7071 [a..b] [| n | n sieve nth [ n primes push ] when ] each
    primes [| a |
        a sq :> x
        x 50000000 < [
            primes [| b |
                b 3 ^ x + :> y
                y 50000000 < [
                    primes [| c |
                        c 4 ^ y + :> z
                        z 50000000 < [ t z seen set-nth ] when
```

Comparison. C exposes prime generation or testing. Shared calls: `euler_calloc`, `euler_isqrt`, `euler_malloc`, `euler_sieve`. Rust exposes prime generation or testing. Shared calls: `isqrt`, `primes`. Gforth exposes prime generation or testing. Shared calls: `prefix!`, `prefix@`, `sieve-build`. Factor exposes prime generation or testing. Shared calls: `euler-sieve`. All listed programs print the same exact answer string from the manifest. The excerpts can differ in algorithm, data representation, and startup cost.

40.88 Problem 88: Product-sum Numbers

Enumerate nondecreasing factors >=2. Padding with ones gives k=product-sum+factor_count. Every k has a representation at 2*k, using factors 2,k and k-2 ones, so products above 2*K cannot improve any minimum. Adding factors never decreases k, which supplies another safe pruning rule.

C. Source: `solutions/c/product_sum_numbers_88.c`.

```
static void p88_search(unsigned product, unsigned sum, unsigned count, unsigned first,
                       unsigned limit, unsigned *minimum) {
    unsigned k = product - sum + count;
    if (k > limit) return;
    if (k >= 2 && product < minimum[k]) minimum[k] = product;
    for (unsigned factor = first; factor <= 2 * limit / product; ++factor)
        p88_search(product * factor, sum + factor, count + 1, factor, limit, minimum);
}

static uint64_t p88_sum(unsigned limit) {
    unsigned *minimum = euler_malloc(((size_t)limit + 1) * sizeof *minimum);
    for (unsigned k = 0; k <= limit; ++k) minimum[k] = 2 * limit;
    p88_search(1, 0, 0, 2, limit, minimum);
    bool *seen = euler_calloc(2 * (size_t)limit + 1, sizeof *seen);
```

Rust. Source: `solutions/rust/src/bin/p88.rs`.

```
fn solve() -> i64 {
    fn enumerate(product: usize, sum: usize, count: usize, first: usize, min: &mut [usize]) {
        let k = product - sum + count;
        if k >= min.len() {
            return;
        }
        if k >= 2 {
            min[k] = min[k].min(product);
        }
        for factor in first..=2 * (min.len() - 1) / product {
            enumerate(product * factor, sum + factor, count + 1, factor, min);
        }
    }
    let limit = 12000usize;
```

Gforth. Source: `solutions/forth/p88.fs`.

```
: p88-recurse { product sum count first minimum } ( product sum count first array -- )
  product sum - count + 12000 > if exit then
  product sum - count + dup 2 >= if
    dup minimum prefix@ product > if product swap minimum prefix! else drop then
  else drop then
  24000 product / 1+ first ?do
    product i * sum i + count 1+ i minimum recurse
  loop ;

: p88 ( -- n )
  12001 cells allocate throw 24001 allocate throw
  0 { minima seen answer }
  12001 0 do 24000 i minima prefix! loop
```

```
  seen 24001 erase
```

Factor. Source: `solutions/factor/euler/advanced/advanced.factor`.

```
:: p88 ( -- n )
    12001 24001 <array> :> minima
    2 1 0 0 minima p88-search
    2 12001 minima subseq members sum ;

CONSTANT: roman-map H{ { 73 1 } { 86 5 } { 88 10 } { 76 50 } { 67 100 }
      { 68 500 } { 77 1000 } }
```

Comparison. C exposes direct bounded arithmetic. Shared calls: `euler_calloc`, `euler_malloc`. Rust exposes sorting or set deduplication. Gforth exposes recursive search. Shared calls: `prefix!`, `prefix@`. Factor exposes direct bounded arithmetic. All listed programs print the same exact answer string from the manifest. The excerpts can differ in algorithm, data representation, and startup cost.

40.89 Problem 89: Roman Numerals

Decode each supplied Roman numeral by subtracting a symbol when the next symbol is larger. Greedy canonical tokens M,CM,D,CD,...,IV,I give the shortest valid form; only its length is needed to count saved characters.

C. Source: `solutions/c/roman_numerals_89.c`.

```
static unsigned p89_symbol(char symbol) {
    switch (symbol) {
        case 'I': return 1; case 'V': return 5; case 'X': return 10; case 'L': return 50;
        case 'C': return 100; case 'D': return 500; case 'M': return 1000;
        default: euler_fail("Invalid Roman symbol"); return 0;
    }
}

static unsigned p89_value(const char *text) {
    int value = 0;
    for (size_t i = 0; text[i]; ++i) {
        unsigned current = p89_symbol(text[i]);
        if (text[i + 1] && p89_symbol(text[i + 1]) > current) value -= (int)current;
        else value += (int)current;
```

Rust. Source: `solutions/rust/src/bin/p89.rs`.

```
fn solve() -> i64 {
    {
        let text = include_str!("../../../../tests/data/p089_roman.txt");
        fn value(ch: u8) -> usize {
            match ch {
                b'I' => 1,
                b'V' => 5,
                b'X' => 10,
                b'L' => 50,
```

```
                b'C' => 100,
                b'D' => 500,
                b'M' => 1000,
                _ => 0,
            }
```

Gforth. Source: `solutions/forth/p89.fs`.

```
: p89-symbol ( ascii -- numeral )
  case
    [char] I of 1 endof [char] V of 5 endof
    [char] X of 10 endof [char] L of 50 endof
    [char] C of 100 endof [char] D of 500 endof
    [char] M of 1000 endof
    abort" Invalid roman glyph"
  endcase ;

create p89-values 1000 , 900 , 500 , 400 , 100 , 90 , 50 , 40 , 10 , 9 , 5 , 4 , 1 ,

create p89-lengths 1 , 2 , 1 , 2 , 1 , 2 , 1 , 2 , 1 , 2 , 1 , 2 , 1 ,

: p89-minimum { value } ( numeral -- minimal count )
```

Factor. Source: `solutions/factor/euler/advanced/advanced.factor`.

```
:: p89 ( -- n )
    "tests/data/p089_roman.txt" utf8 file-contents "\n" split
    [ empty? not ] filter
    [| str | str length str roman-number roman-min-length - ] map sum ;
```

Comparison. C exposes direct bounded arithmetic. Shared calls: `euler_fail`, `euler_open`. Rust exposes fixture parsing. Gforth exposes modular arithmetic, and fixture parsing. Shared calls: `prefix@`, `read-fixture`. Factor exposes fixture parsing. All listed programs print the same exact answer string from the manifest. The excerpts can differ in algorithm, data representation, and startup cost.

40.90 Problem 90: Cube Digit Pairs

Enumerate the 210 six-element digit sets and unordered pairs of them. Keep 6-labelled and 9-labelled cubes distinct, but allow either symbol to display both digits when testing the nine two-digit square numbers.

C. Source: `solutions/c/cube_digit_pairs_90.c`.

```
static unsigned p90_popcount(unsigned n) {
    unsigned count = 0;
    while (n) { n &= n - 1; ++count; }
    return count;
}

static bool p90_valid(unsigned left, unsigned right) {
```

```
    static const unsigned squares[] = {1, 4, 9, 16, 25, 36, 49, 64, 81};
    const unsigned reversible = (1u << 6) | (1u << 9);
    if (left & reversible) left |= reversible;
    if (right & reversible) right |= reversible;
    for (size_t i = 0; i < sizeof squares / sizeof *squares; ++i) {
        unsigned a = 1u << (squares[i] / 10), b = 1u << (squares[i] % 10);
        if (!((left & a) && (right & b)) && !((left & b) && (right & a))) return false;
```

Rust. Source: `solutions/rust/src/bin/p90.rs`.

```
fn solve() -> i64 {
    let masks: Vec<u32> = (0..1u32 << 10).filter(|m| m.count_ones() == 6).collect();
    let mut count = 0;
    for (i, &a) in masks.iter().enumerate() {
        for &b in &masks[i..] {
            let rev = (1 << 6) | (1 << 9);
            let a = if a & rev != 0 { a | rev } else { a };
            let b = if b & rev != 0 { b | rev } else { b };
            if [1, 4, 9, 16, 25, 36, 49, 64, 81].iter().all(|&s| {
                let (x, y) = (1 << (s / 10), 1 << (s % 10));
                ((a & x != 0) && (b & y != 0)) || ((a & y != 0) && (b & x != 0))
            }) {
                count += 1;
            }
```

Gforth. Source: `solutions/forth/p90.fs`.

```
: p90-valid? 0 0 0 { a b x y squares } ( mask mask -- flag )
  1 6 lshift 1 9 lshift or to squares
  a squares and if a squares or to a then
  b squares and if b squares or to b then
  10 1 do
    i i * to x
    1 x 10 / lshift to y
    1 x 10 mod lshift to x
    a x and 0<> b y and 0<> and
    a y and 0<> b x and 0<> and or 0= if false unloop exit then
  loop true ;

: p90 ( -- n )
  210 cells allocate throw 0 0 { masks count answer }
```

Factor. Source: `solutions/factor/euler/advanced/advanced.factor`.

```
:: p90 ( -- n )
    0 :> count!
    0 1023 [a..b] [| a |
        a p90-countbits 6 = [
            a 1023 [a..b] [| b |
                b p90-countbits 6 = a b p90-pair? and [
                    count 1 + count!
                ] when
            ] each
        ] when
    ] each count ;
```

Comparison. C exposes direct bounded arithmetic. Rust exposes direct bounded arithmetic. Gforth exposes modular arithmetic. Shared calls: `bit-count`, `prefix!`, `prefix@`. Factor exposes direct bounded arithmetic. All listed programs print the same exact answer string from the manifest. The excerpts can differ in algorithm, data representation, and startup cost.

40.91 Problem 91: Right Triangles with Integer Coordinates

A perpendicular primitive step at (x,y) is (-y/g,x/g), g=gcd(x,y). Count both directions within the square; axes contribute 3*N*N.

C. Source: `solutions/c/right_triangles_with_integer_coordinates_91.c`.

```
static uint64_t triangles(unsigned limit) {
    uint64_t total = 3ULL * limit * limit;
    for (unsigned x = 1; x <= limit; ++x) {
        for (unsigned y = 1; y <= limit; ++y) {
            unsigned g = (unsigned)euler_gcd(x, y);
            unsigned dx = y / g, dy = x / g;
            unsigned left = x / dx, up = (limit - y) / dy;
            unsigned right = (limit - x) / dx, down = y / dy;
            total += left < up ? left : up;
            total += right < down ? right : down;
        }
    }
    return total;
}
```

Rust. Source: `solutions/rust/src/bin/p91.rs`.

```
fn solve() -> i64 {
    {
        let mut count = 0;
        for x1 in 0i64..=50 {
            for y1 in 0i64..=50 {
                if x1 == 0 && y1 == 0 {
                    continue;
                }
                for x2 in 0i64..=50 {
                    for y2 in 0i64..=50 {
                        if x2 == 0 && y2 == 0 || x1 == x2 && y1 == y2 {
                            continue;
                        }
                        let dot0 = x1 * x2 + y1 * y2;
```

Gforth. Source: `solutions/forth/p91.fs`.

```
: p91 ( -- n ) 0 0 0 0 0 0 0 0 { answer x1 y1 x2 y2 dot0 dot1 dot2 }
  51 0 do i to x1
    51 0 do i to y1
      x1 0<> y1 0<> or if
        51 0 do i to x2
          51 0 do i to y2
            x2 0<> y2 0<> or x1 x2 <> y1 y2 <> or and if
```

```
            x1 x2 * y1 y2 * + to dot0
            x1 x1 x2 - * y1 y1 y2 - * + to dot1
            x2 x2 x1 - * y2 y2 y1 - * + to dot2
            dot0 0= dot1 0= or dot2 0= or if answer 1+ to answer then
          then
        loop
      loop
```

Factor. Source: `solutions/factor/euler/extra/extra.factor`.

```
:: p91 ( -- n )
    0 :> answer!
    0 50 [a..b] [| ax |
        0 50 [a..b] [| ay |
            ax 0 = ay 0 = and not [
                0 50 [a..b] [| bx |
                    0 50 [a..b] [| by |
                        bx 0 = by 0 = and not
                        ax bx = ay by = and not and [
                            ax bx * ay by * + zero?
                            ax ax bx - * ay ay by - * + zero? or
                            bx bx ax - * by by ay - * + zero? or
                            [ answer 1 + answer! ] when
                        ] when
```

Comparison. C exposes direct bounded arithmetic. Shared calls: `euler_gcd`, `gcd`. Rust exposes direct bounded arithmetic. Gforth exposes direct bounded arithmetic. Factor exposes direct bounded arithmetic. All listed programs print the same exact answer string from the manifest. The excerpts can differ in algorithm, data representation, and startup cost.

40.92 Problem 92: Square Digit Chains

The program computes the exact answer for Square Digit Chains.

C. Source: `solutions/c/square_digit_chains_92.c`.

```
static long long solve(void) {
  unsigned char cache[568] = {0};
  cache[1] = 1;
  cache[89] = 89;
  const unsigned sq[] = {0, 1, 4, 9, 16, 25, 36, 49, 64, 81};
  unsigned answer = 0;
  for (unsigned n = 1; n < 10000000; n++) {
    unsigned x = digit_power_sum(n, sq);
    while (!cache[x])
      x = digit_power_sum(x, sq);
    if (cache[x] == 89)
      answer++;
  }
  return answer;
```

Rust. Source: `solutions/rust/src/bin/p92.rs`.

```
fn solve() -> i64 {
    {
        let mut cache = [0u8; 568];
        cache[1] = 1;
        cache[89] = 89;
        let squares = [0u32, 1, 4, 9, 16, 25, 36, 49, 64, 81];
        let mut count = 0;
        for n in 1u32..10_000_000 {
            let mut v = digit_power_sum(n, &squares);
            while cache[v as usize] == 0 {
                v = digit_power_sum(v, &squares);
            }
            if cache[v as usize] == 89 {
                count += 1;
```

Gforth. Source: `solutions/forth/p92.fs`.

```
create squared-digits 0 , 1 , 4 , 9 , 16 , 25 , 36 , 49 , 64 , 81 ,

: digit-squares 0 { n result } ( n -- sum )
  0 to result
  begin n 10 mod cells squared-digits + @ result + to result n 10 / to n
    n 0= if result exit then
  again ;

: chain-end { n } ( n -- 1|89 )
  begin n 1 <> n 89 <> and while n digit-squares to n repeat n ;

create ends-at-89 568 allot

: p92 ( -- answer )
```

Factor. Source: `solutions/factor/euler/extra/extra.factor`.

```
:: p92 ( -- n )
    0 :> answer!
    1 9999999 [a..b] [| n |
        n p92-end 89 = [ answer 1 + answer! ] when
    ] each answer ;
```

Comparison. C exposes dynamic programming or memoization. Shared calls: `digit_power_sum`. Rust exposes dynamic programming or memoization. Shared calls: `digit_power_sum`. Gforth exposes modular arithmetic. Factor exposes direct bounded arithmetic. All listed programs print the same exact answer string from the manifest. The excerpts can differ in algorithm, data representation, and startup cost.

40.93 Problem 93: Arithmetic Expressions

Recursively combine each pair of values using all binary operations. Every binary expression tree is represented; reduced rational numbers preserve exact equality, including intermediate fractional

results.

C. Source: `solutions/c/arithmetic_expressions_93.c`.

```
typedef struct { int64_t numerator, denominator; } Rational;
enum { REACHABLE = 10000 };

static Rational rational(int64_t n, int64_t d) {
    if (d < 0) { n = -n; d = -d; }
    uint64_t g = euler_gcd((uint64_t)(n < 0 ? -n : n), (uint64_t)d);
    return (Rational){n / (int64_t)g, d / (int64_t)g};
}
static void expressions(const Rational *values, unsigned count, bool *reached) {
    if (count == 1) {
        Rational x = values[0];
        if (x.denominator == 1 && x.numerator > 0 && x.numerator < REACHABLE)
            reached[x.numerator] = true;
        return;
```

Rust. Source: `solutions/rust/src/bin/p93.rs`.

```
#[derive(Clone, Copy)]
struct Ratio {
    n: i64,
    d: i64,
}
impl Ratio {
    fn new(mut n: i64, mut d: i64) -> Self {
        assert!(d != 0);
        if d < 0 {
            n = -n;
            d = -d;
        }
        let divisor = gcd(n.unsigned_abs(), d as u64) as i64;
        Self {
```

Gforth. Source: `solutions/forth/p93.fs`.

```
create p93-num 16 cells allot

create p93-den 16 cells allot

create p93-reached 10000 allot

: p93-n@ { depth index } ( depth index -- n ) depth 4 * index + cells p93-num + @ ;

: p93-d@ { depth index } ( depth index -- d ) depth 4 * index + cells p93-den + @ ;

: p93-set { n d depth index } ( n d depth index -- )
  n depth 4 * index + cells p93-num + !
  d depth 4 * index + cells p93-den + ! ;
```

Factor. Source: `solutions/factor/euler/advanced/advanced.factor`.

```
:: p93 ( -- n )
    0 :> bestlen! 0 :> answer!
    0 6 [a..b] [| a |
        a 1 + 7 [a..b] [| b |
            b 1 + 8 [a..b] [| c |
                c 1 + 9 [a..b] [| d |
                    a b c d p93-run :> length
                    length bestlen > [
                        length bestlen!
                        a 1000 * b 100 * + c 10 * + d + answer!
                    ] when
                ] each
            ] each
        ] each
```

Comparison. C exposes recursive search. Shared calls: `euler_fail`, `euler_gcd`. Rust exposes modular arithmetic. Shared calls: `gcd`. Gforth exposes direct bounded arithmetic. Shared calls: `integer-gcd`. Factor exposes direct bounded arithmetic. All listed programs print the same exact answer string from the manifest. The excerpts can differ in algorithm, data representation, and startup cost.

40.94 Problem 94: Almost Equilateral Triangles

With equal sides a and base a+delta, x=3*a-delta and y=2*height satisfy x*x-3*y*y=4. Multiplication by 2+sqrt(3) enumerates the positive solutions in increasing order; perimeters are x+/-2.

C. Source: `solutions/c/almost_equilateral_triangles_94.c`.

```
int main(void) {
    const uint64_t limit = UINT64_C(1000000000);
    uint64_t x = 2, y = 0, total = 0;
    while (x <= limit + 2) {
        uint64_t a, b, perimeter;
        if (x % 3 == 2) {
            a = (x + 1) / 3; b = a + 1; perimeter = x + 2;
        } else {
            a = (x - 1) / 3; b = a ? a - 1 : 0; perimeter = x - 2;
        }
        if (a > 1 && b && perimeter <= limit && b * y % 4 == 0)
            total += perimeter;
        uint64_t next_x = 2 * x + 3 * y;
        y = x + 2 * y;
```

Rust. Source: `solutions/rust/src/bin/p94.rs`.

```
fn solve() -> i64 {
    {
        let mut sum = 0i64;
        let (mut x, mut y) = (2i64, 0i64);
        loop {
            let (nx, ny) = (2 * x + 3 * y, x + 2 * y);
            x = nx;
```

```
            y = ny;
            let perimeter = if x % 3 == 2 { x + 2 } else { x - 2 };
            if perimeter > 1_000_000_000 {
                break;
            }
            let a = if x % 3 == 2 { (x + 1) / 3 } else { (x - 1) / 3 };
            let b = if x % 3 == 2 { a + 1 } else { a - 1 };
```

Gforth. Source: `solutions/forth/p94.fs`.

```
: p94 ( -- n ) 2 0 0 0 0 0 0 0 { x y nx ny perimeter a b sum }
  begin x 1000000002 <= while
    x 3 mod 2 = if x 1+ 3 / to a a 1+ to b x 2 + to perimeter
    else x 1- 3 / to a a 1- to b x 2 - to perimeter then
    a 1 > b 0> and perimeter 1000000000 <= and if
      b y * 4 mod 0= if sum perimeter + to sum then
    then
    x 2* y 3 * + to nx
    x y 2* + to ny
    nx to x ny to y
  repeat sum ;

[undefined] euler-testing [if] p94 . cr bye [then]
```

Factor. Source: `solutions/factor/euler/advanced/advanced.factor`.

```
:: p94 ( -- n )
    2 :> x! 0 :> y! 0 :> total!
    [ x 1000000002 <= ] [
        x 3 mod 2 = [
            x 1 + 3 /i :> side
            side 1 + :> base
            x 2 + :> perimeter
            side 1 > base y * 4 mod zero? and
            perimeter 1000000000 <= and
            [ total perimeter + total! ] when
        ] [
            x 1 - 3 /i :> side
            side 1 - :> base
            x 2 - :> perimeter
```

Comparison. C exposes direct bounded arithmetic. Rust exposes direct bounded arithmetic. Gforth exposes modular arithmetic. Factor exposes modular arithmetic. All listed programs print the same exact answer string from the manifest. The excerpts can differ in algorithm, data representation, and startup cost.

40.95 Problem 95: Amicable Chains

A divisor sieve supplies the successor of each integer. Each vertex is visited once. A repeated vertex from the current traversal identifies a cycle; entering an older traversal cannot discover a new cycle.

C. Source: `solutions/c/amicable_chains_95.c`.

```
int main(void) {
    enum { LIMIT = 1000000 };
    unsigned *sum = euler_calloc(LIMIT + 1, sizeof *sum);
    unsigned *visit = euler_calloc(LIMIT + 1, sizeof *visit);
    unsigned *depth = euler_calloc(LIMIT + 1, sizeof *depth);
    for (unsigned d = 1; d <= LIMIT / 2; ++d)
        for (unsigned n = 2 * d; n <= LIMIT; n += d) sum[n] += d;
    unsigned best_length = 0, best_member = 0;
    for (unsigned start = 2; start <= LIMIT; ++start) {
        if (visit[start]) continue;
        unsigned n = start, step = 0;
        while (n && n <= LIMIT && !visit[n]) {
            visit[n] = start; depth[n] = step++;
            n = sum[n];
```

Rust. Source: `solutions/rust/src/bin/p95.rs`.

```
fn solve() -> i64 {
    const LIMIT: usize = 1_000_000;
    let sums = proper_divisor_sums(LIMIT);
    let mut visited = vec![0usize; LIMIT + 1];
    let mut depths = vec![0usize; LIMIT + 1];
    let mut best = (0, usize::MAX);
    for start in 2..=LIMIT {
        if visited[start] != 0 {
            continue;
        }
        let mut n = start;
        let mut step = 0;
        while n > 0 && n <= LIMIT && visited[n] == 0 {
            visited[n] = start;
```

Gforth. Source: `solutions/forth/p95.fs`.

```
: p95 ( -- n )
  1000001 cells allocate throw 1000001 cells allocate throw 1000001 cells allocate throw
  0 0 0 0 0 0 0 0 { successors visited depth best-length best-member start number step minimum
    first cycle-start }
  successors 1000001 cells erase visited 1000001 cells erase
  500001 1 do
    1000001 i 2* do
      j i successors prefix@ + i successors prefix!
    j +loop
  loop
  1000001 2 do i to start
    start visited prefix@ 0= if
      start to number 0 to step
      begin number 0> number 1000000 <= and if number visited prefix@ 0= else false then while
        start number visited prefix!
```

Factor. Source: `solutions/factor/euler/advanced/advanced.factor`.

```
:: p95 ( -- n )
```

```
    1000000 divisor-sum-sieve :> divisor-sums
    1000001 0 <array> :> seen
    1000001 0 <array> :> position
    0 :> bestlength! 0 :> bestmember!
    2 1000000 [a..b] [| start |
        start :> n! 0 :> step!
        V{ } clone :> path
        [ n 0 > n 1000000 <= and [
            n seen nth dup start = not swap zero? and
          ] [ f ] if ] [
            start n seen set-nth
            step n position set-nth
            n path push step 1 + step!
```

Comparison. C exposes prime generation or testing. Shared calls: `euler_calloc`. Rust exposes direct bounded arithmetic. Shared calls: `proper_divisor_sums`. Gforth exposes direct bounded arithmetic. Shared calls: `prefix!`, `prefix@`. Factor exposes prime generation or testing. Shared calls: `divisor-sum-sieve`. All listed programs print the same exact answer string from the manifest. The excerpts can differ in algorithm, data representation, and startup cost.

40.96 Problem 96: Su Doku

Bit masks enforce rows, columns and boxes. Backtracking chooses the empty cell with the fewest candidates and checks solution uniqueness.

C. Source: `solutions/c/su_doku_96.c`.

```
typedef struct {
    unsigned cells[81], solution[81];
    unsigned row[9], column[9], box[9], solutions;
} Puzzle;

static unsigned bit_count(unsigned mask) {
    unsigned count = 0;
    while (mask) { mask &= mask - 1; ++count; }
    return count;
}
static void solve(Puzzle *p) {
    if (p->solutions >= 2) return;
    unsigned chosen = 81, options = 0, minimum = 10;
    for (unsigned i = 0; i < 81; ++i) if (!p->cells[i]) {
```

Rust. Source: `solutions/rust/src/bin/p96.rs`.

```
fn sudoku_solve(
    grid: &mut [u8; 81],
    row: &mut [u16; 9],
    col: &mut [u16; 9],
    box_mask: &mut [u16; 9],
    answers: &mut Vec<[u8; 81]>,
) {
    if answers.len() >= 2 {
```

```
        return;
    }
    let mut chosen = 81;
    let mut options = 0;
    let mut minimum = 10;
    for i in 0..81 {
```

Gforth. Source: `solutions/forth/p96.fs`.

```
: p96-search 0 0 0 0 0 0 0 0 0 0 0
  { board rows cols boxes choice candidates few row col box digit bit mask count old } ( board
    rows cols boxes -- solved? )
  -1 to choice 10 to few
  81 0 do
    i board prefix@ 0= if
      i 9 / to row i 9 mod to col
      row 3 / 3 * col 3 / + to box
      row rows prefix@ col cols prefix@ or box boxes prefix@ or invert 1022 and to mask
      mask bit-count to count
      count 0= if false unloop exit then
      count few < if count to few i to choice mask to candidates then
    then
  loop
  choice -1 = if true exit then
```

Factor. Source: `solutions/factor/euler/advanced/advanced.factor`.

```
:: p96 ( -- n )
    "tests/data/p096_sudoku.txt" utf8 file-contents "\n" split
    [ empty? not ] filter :> lines
    0 :> answer!
    0 49 [a..b] [| grid |
        81 0 <array> :> board
        0 8 [a..b] [| row |
            grid 10 * row + 1 + lines nth :> line
            0 8 [a..b] [| col |
                col line nth 48 - row 9 * col + board set-nth
            ] each
        ] each
        board p96-solve [
            0 board nth 100 * 1 board nth 10 * + 2 board nth +
```

Comparison. C exposes fixture parsing. Shared calls: `euler_open`. Rust exposes fixture parsing. Gforth exposes modular arithmetic, fixture parsing, and recursive search. Shared calls: `bit-count`, `prefix!`, `prefix@`, `read-fixture`. Factor exposes fixture parsing. All listed programs print the same exact answer string from the manifest. The excerpts can differ in algorithm, data representation, and startup cost.

40.97 Problem 97: Large Non-Mersenne Prime

Modular repeated squaring retains exactly the last ten decimal digits.

C. Source: `solutions/c/large_non_mersenne_prime_97.c`.

```
int main(void) {
    const uint64_t modulus = UINT64_C(10000000000);
    uint64_t power = euler_powmod(2, 7830457, modulus);
    uint64_t result = (euler_mulmod(28433, power, modulus) + 1) % modulus;
    printf("%010" PRIu64 "\n", result);
    return 0;
}
```

Rust. Source: `solutions/rust/src/bin/p97.rs`.

```
fn solve() -> i64 {
    ((28433u64 * mod_pow(2, 7_830_457, 10_000_000_000) + 1) % 10_000_000_000) as i64
}

fn main() {
    println!("{}", solve());
}
```

Gforth. Source: `solutions/forth/p97.fs`.

```
: p97 ( -- n ) 2 7830457 10000000000 power-mod 28433 * 1+ 10000000000 mod ;

[undefined] euler-testing [if] p97 . cr bye [then]
```

Factor. Source: `solutions/factor/euler/extra/extra.factor`.

```
: p97 ( -- n ) 2 7830457 10000000000 modular-pow 28433 * 1 + 10000000000 mod ;
```

Comparison. C exposes modular arithmetic. Shared calls: `euler_mulmod`, `euler_powmod`. Rust exposes prime generation or testing, and modular arithmetic. Shared calls: `mod_pow`. Gforth exposes prime generation or testing, and modular arithmetic. Shared calls: `power-mod`. Factor exposes modular arithmetic. Shared calls: `modular-pow`. All listed programs print the same exact answer string from the manifest. The excerpts can differ in algorithm, data representation, and startup cost.

40.98 Problem 98: Anagramic Squares

For each anagram pair, enumerate every square of that word length. Its digits determine the only possible letter mapping. Apply that bijection to the other word and test the result with an exact sqrt.

C. Source: `solutions/c/anagramic_squares_98.c`.

```
typedef struct { char text[32]; unsigned char counts[26]; size_t length; } Word;
static uint64_t pair_maximum(const Word *a, const Word *b) {
    if (a->length >= 19) euler_fail("Anagram pair exceeds uint64_t square range");
    uint64_t upper = 1;
    for (size_t i = 0; i < a->length; ++i) upper *= 10;
    uint64_t lower = upper / 10, first = euler_isqrt(lower);
    if (first * first < lower) ++first;
```

```
    uint64_t last = euler_isqrt(upper - 1), maximum = 0;
    for (uint64_t root = first; root <= last; ++root) {
        uint64_t square = root * root;
        char digits[32];
        snprintf(digits, sizeof digits, "%" PRIu64, square);
        int letter[26], used[10];
        for (unsigned i = 0; i < 26; ++i) letter[i] = -1;
```

Rust. Source: `solutions/rust/src/bin/p98.rs`.

```
fn solve() -> i64 {
    let words: Vec<&str> = include_str!("../../../../tests/data/p098_words.txt")
        .split(',')
        .map(|w| w.trim().trim_matches('"'))
        .collect();
    let mut groups: HashMap<Vec<u8>, Vec<&str>> = HashMap::new();
    for word in &words {
        let mut key = word.as_bytes().to_vec();
        key.sort_unstable();
        groups.entry(key).or_default().push(word);
    }
    let mut best = 0u64;
    for group in groups.values() {
        if group.len() < 2 {
```

Gforth. Source: `solutions/forth/p98.fs`.

```
create p98-words 1800 cells allot

create p98-lengths 1800 cells allot

create p98-signatures 46800 allot

create p98-letters 26 cells allot

create p98-used 10 cells allot

variable p98-count

: p98-word@ ( index -- address ) cells p98-words + @ ;
```

Factor. Source: `solutions/factor/euler/advanced/advanced.factor`.

```
:: p98 ( -- n )
    "tests/data/p098_words.txt" read-upper-words :> words
    words [ >array sort ] map :> signatures
    0 :> answer!
    0 words length 2 - [a..b] [| i |
        i 1 + words length 1 - [a..b] [| j |
            i signatures nth j signatures nth = [
                i words nth j words nth p98-pair
                answer max answer!
            ] when
        ] each
    ] each answer ;
```

Comparison. C exposes direct bounded arithmetic. Shared calls: `euler_calloc`, `euler_isqrt`, `euler_open`. Rust exposes fixture parsing, and sorting or set deduplication. Shared calls: `is_square`, `isqrt`. Gforth exposes modular arithmetic, and fixture parsing. Shared calls: `integer-sqrt`, `read-fixture`. Factor exposes fixture parsing, and sorting or set deduplication. Shared calls: `read-upper-words`. All listed programs print the same exact answer string from the manifest. The excerpts can differ in algorithm, data representation, and startup cost.

40.99 Problem 99: Largest Exponential

Compare exponent*log(base) using exact rational intervals. After binary range reduction, z=(t-1)/(t+1) <= 1/3, and log(t)=2*atanh(z). The omitted series tail is bounded by its first term divided by 1-z*z. If intervals overlap, compare the full integers with GMP.

C. Source: `solutions/c/largest_exponential_99.c`.

```
static void logarithm_series(mpq_t low, mpq_t high, unsigned long num,
                             unsigned long den) {
    enum { TERMS = 24 };
    mpq_t z, z2, power, term, divisor, gap;
    mpq_inits(z, z2, power, term, divisor, gap, NULL);
    mpq_set_ui(z, num, den); mpq_canonicalize(z);
    mpq_mul(z2, z, z); mpq_set(power, z); mpq_set_ui(low, 0, 1);
    for (unsigned j = 0; j < TERMS; ++j) {
        mpq_set_ui(divisor, 2 * j + 1, 1);
        mpq_div(term, power, divisor); mpq_add(low, low, term);
        mpq_mul(power, power, z2);
    }
    mpq_mul_2exp(low, low, 1);
    mpq_set_ui(gap, 1, 1); mpq_sub(gap, gap, z2);
```

Rust. Source: `solutions/rust/src/bin/p99.rs`.

```
fn logarithm_interval(num: u64, den: u64, scale: &BigUint) -> (BigUint, BigUint) {
    assert!(num < den && 3 * num <= den);
    let (numerator, denominator) = (BigUint::from(num), BigUint::from(den));
    let nsq = &numerator * &numerator;
    let dsq = &denominator * &denominator;
    let mut power = numerator.clone();
    let mut power_den = denominator.clone();
    let mut lower = BigUint::zero();
    const TERMS: usize = 24;
    for j in 0..TERMS {
        lower += ((&power * scale) << 1usize) / (&power_den * BigUint::from((2 * j + 1) as u64));

        power *= &nsq;
        power_den *= &dsq;
    }
```

Gforth. Source: `solutions/forth/p99.fs`.

```
: p99 ( -- n )
  s" tests/data/p099_base_exp.txt" 2000 fixture-integers
```

```
  0 0 0 { values count best-index index exponent }
  count 2000 <> abort" Invalid exponential-comparison data"
  -1e300 p99-best-score f!
  1000 0 do
    i 2* values prefix@ s>f fln
    i 2* 1+ values prefix@ s>f f*
    fdup p99-best-score f@ f> if
      i 1+ to best-index
      p99-best-score f!
    else fdrop then
  loop values free throw best-index ;
```

Factor. Source: `solutions/factor/euler/advanced/advanced.factor`.

```
:: p99 ( -- n )
    "tests/data/p099_base_exp.txt" read-numbers :> data
    -1.0e100 :> greatest! 0 :> answer!
    0 999 [a..b] [| index |
        index 2 * data nth >float log
        index 2 * 1 + data nth * :> score
        score greatest > [ score greatest! index 1 + answer! ] when
    ] each answer ;
```

Comparison. C exposes exact large-integer arithmetic. Shared calls: `euler_open`. Rust exposes modular arithmetic, exact large-integer arithmetic, and fixture parsing. Gforth exposes direct bounded arithmetic. Shared calls: `fixture-integers`, `prefix@`. Factor exposes fixture parsing. Shared calls: `read-numbers`. All listed programs print the same exact answer string from the manifest. The excerpts can differ in algorithm, data representation, and startup cost.

40.100 Problem 100: Arranged Probability

t(t-1)=2*b(b-1) is a negative Pell equation. Its positive solutions are generated in increasing t by the recurrence below.

C. Source: `solutions/c/arranged_probability_100.c`.

```
int main(void) {
    uint64_t blue = 1, total = 1;
    while (total <= UINT64_C(1000000000000)) {
        uint64_t next_blue = 3 * blue + 2 * total - 2;
        total = 4 * blue + 3 * total - 3;
        blue = next_blue;
    }
    printf("%" PRIu64 "\n", blue);
    return 0;
}
```

Rust. Source: `solutions/rust/src/bin/p100.rs`.

```
fn solve() -> i64 {
    {
```

```
        let (mut blue, mut total) = (15i64, 21i64);
        while total <= 1_000_000_000_000 {
            let next_blue = 3 * blue + 2 * total - 2;
            let next_total = 4 * blue + 3 * total - 3;
            blue = next_blue;
            total = next_total;
        }
        blue
    }
}

fn main() {
```

Gforth. Source: `solutions/forth/p100.fs`.

```
: p100 ( -- n ) 15 21 0 0 { blue total next-blue next-total }
  begin total 1000000000000 <= while
    3 blue * 2 total * + 2 - to next-blue
    4 blue * 3 total * + 3 - to next-total
    next-blue to blue next-total to total
  repeat blue ;

[undefined] euler-testing [if] p100 . cr bye [then]
```

Factor. Source: `solutions/factor/euler/extra/extra.factor`.

```
:: p100 ( -- n )
    15 :> blue! 21 :> total!
    [ total 1000000000000 <= ] [
        blue 3 * total 2 * + 2 - :> next-blue
        blue 4 * total 3 * + 3 - :> next-total
        next-blue blue! next-total total!
    ] while blue ;
```

Comparison. C exposes direct bounded arithmetic. Rust exposes direct bounded arithmetic. Gforth exposes direct bounded arithmetic. Factor exposes direct bounded arithmetic. All listed programs print the same exact answer string from the manifest. The excerpts can differ in algorithm, data representation, and startup cost.

40.101 Problem 101: Optimum Polynomial

Newton's forward-difference interpolation uses only integers. The degree-(k-1) fit is sum_j binomial(n-1,j)*Delta^j u(1).

C. Source: `solutions/c/optimum_polynomial_101.c`.

```
static int64_t sequence(unsigned n) {
    int64_t value = 1;
    for (unsigned power = 0; power < 10; ++power) value = 1 - n * value;
    return value;
}
```

```
static int64_t fit(const int64_t *first, unsigned terms, unsigned n) {
    int64_t value = 0, choose = 1;
    for (unsigned j = 0; j < terms; ++j) {
        value += first[j] * choose;
        choose = choose * (n - 1 - j) / (j + 1);
    }
    return value;
}
int main(void) {
```

Rust. Source: `solutions/rust/src/bin/p101.rs`.

```
fn solve() -> i64 {
    fn sequence(n: i128) -> i128 {
        let mut v = 1;
        for _ in 0..10 {
            v = 1 - n * v;
        }
        v
    }
    fn fit(first: &[i128], n: usize) -> i128 {
        let mut sum = 0;
        let mut choose = 1i128;
        for (j, &v) in first.iter().enumerate() {
            sum += v * choose;
            if j + 1 < first.len() {
```

Gforth. Source: `solutions/forth/p101.fs`.

```
: p101-sequence { n } ( n -- polynomial )
  1 10 0 do n * 1 swap - loop ;

: p101-fit 0 0 { differences terms at sum coefficient } ( ptr terms at -- value )
  0 to sum 1 to coefficient
  terms 0 do
    i differences prefix@ coefficient * sum + to sum
    i 1+ terms < if coefficient at 1- i - * i 1+ / to coefficient then
  loop sum ;

: p101 ( -- n )
  10 cells allocate throw 10 cells allocate throw
  0 0 0 { differences first total k at }
  11 1 do i to k
```

Factor. Source: `solutions/factor/euler/advanced/advanced.factor`.

```
: p101 ( -- n ) 1 10 [a..b] [ p101-fit ] map sum ;
```

Comparison. C exposes direct bounded arithmetic. Rust exposes direct bounded arithmetic. Gforth exposes direct bounded arithmetic. Shared calls: `prefix!`, `prefix@`. Factor exposes direct bounded arithmetic. All listed programs print the same exact answer string from the manifest. The excerpts can differ in algorithm, data representation, and startup cost.

40.102 Problem 102: Triangle Containment

The origin lies strictly inside a triangle exactly when the cross products of consecutive position vectors all have the same sign.

C. Source: `solutions/c/triangle_containment_102.c`.

```
static bool contains_origin(const long long p[6]) {
    long long a = p[0] * p[3] - p[1] * p[2];
    long long b = p[2] * p[5] - p[3] * p[4];
    long long c = p[4] * p[1] - p[5] * p[0];
    return (a > 0 && b > 0 && c > 0) || (a < 0 && b < 0 && c < 0);
}
int main(void) {
    FILE *file = euler_open("tests/data/p102_triangles.txt");
    char line[256];
    unsigned total = 0, lines = 0;
    while (fgets(line, sizeof line, file)) {
        long long p[6];
        char *cursor = line;
        for (unsigned i = 0; i < 6; ++i) {
```

Rust. Source: `solutions/rust/src/bin/p102.rs`.

```
fn solve() -> i64 {
    include_str!("../../../../tests/data/p102_triangles.txt")
        .lines()
        .filter(|line| {
            let p: Vec<i64> = line.split(',').map(|x| x.parse().expect("coord")).collect();
            assert_eq!(p.len(), 6);
            let a = p[0] * p[3] - p[1] * p[2];
            let b = p[2] * p[5] - p[3] * p[4];
            let c = p[4] * p[1] - p[5] * p[0];
            (a > 0 && b > 0 && c > 0) || (a < 0 && b < 0 && c < 0)
        })
        .count() as i64
}
```

Gforth. Source: `solutions/forth/p102.fs`.

```
: signed-fixture 0 0 0 0 0 0 0 { addr u cap raw length numbers count value negative active } ( c-
    addr u limit -- array count )
  addr u cap 16 * read-fixture to length to raw
  cap cells allocate throw to numbers
  length 0 do
    raw i + c@ dup [char] - = if
      negative 0= active 0= and if -1 to negative then
      drop
    else dup [char] 0 [char] 9 1+ within if
      [char] 0 - value 10 * + to value true to active
    else drop
      active if
        count cap >= abort" Signed fixture too long"
        value negative 0< if negate then count numbers prefix!
        count 1+ to count
```

Factor. Source: `solutions/factor/euler/advanced/advanced.factor`.

```
:: p102 ( -- n )
    "tests/data/p102_triangles.txt" read-signed-numbers :> values
    0 :> answer!
    0 999 [a..b] [| i |
        i 6 * i 6 * 6 + values subseq p102-contains?
        [ answer 1 + answer! ] when
    ] each answer ;
```

Comparison. C exposes direct bounded arithmetic. Shared calls: `euler_open`. Rust exposes fixture parsing. Gforth exposes modular arithmetic, and fixture parsing. Shared calls: `prefix!`, `prefix@`, `read-fixture`. Factor exposes fixture parsing. Shared calls: `read-signed-numbers`. All listed programs print the same exact answer string from the manifest. The excerpts can differ in algorithm, data representation, and startup cost.

40.103 Problem 103: Special Subset Sums: Optimum

The stated optimum six-element set gives a valid seven-element upper bound by the suggested middle-element construction. Then enumerate ALL increasing positive seven-element sets of smaller total, not a neighbourhood of that construction. Unique subset sums and necessary smallest-(k+1) versus largest-k inequalities prune the finite search.

C. Source: `solutions/c/special_subset_sums_optimum_103.c`.

```
enum { SIZE = 7, SUM_BOUND = 512, MASK_WORDS = SUM_BOUND / 64 };
static unsigned current[SIZE], best[SIZE], best_sum;

static bool cardinality_possible(unsigned chosen) {
    for (unsigned k = 1; 2 * k + 1 <= SIZE; ++k) {
        if (chosen < k + 1) continue;
        unsigned small = 0, large = 0;
        for (unsigned i = 0; i <= k; ++i) small += current[i];
        /* Unchosen entries take their smallest possible increasing values. */
        for (unsigned i = SIZE - k; i < SIZE; ++i)
            large += i < chosen ? current[i] : current[chosen - 1] + i - chosen + 1;
        if (small <= large) return false;
    }
    return true;
```

Rust. Source: `solutions/rust/src/bin/p103.rs`.

```
fn solve() -> i64 {
    fn cardinality_ok(values: &[u64; 7], chosen: usize) -> bool {
        for k in 1..=3 {
            if chosen < k + 1 {
                continue;
            }
            let small: usize = (0..=k).map(|i| values[i] as usize).sum();
            let large: usize = (7 - k..7)
                .map(|i| {
                    if i < chosen {
```

```
                    values[i] as usize
                } else {
                    values[chosen - 1] as usize + i - chosen + 1
                }
```

Gforth. Source: `solutions/forth/p103.fs`.

```
create p103-current 7 cells allot

create p103-best 7 cells allot

create p103-sums 1024 cells allot

create p103-seen 4096 allot

variable p103-bound

: p103-value@ { index } index cells p103-current + @ ;

: p103-cardinal? 0 0 0 0 0 { chosen k low high index last } ( chosen -- flag )
  1 to k
```

Factor. Source: `solutions/factor/euler/advanced/advanced.factor`.

```
:: p103 ( -- n )
    V{ 20 } clone :> seed
    { 11 18 19 20 22 25 } [| n | n 20 + seed push ] each
    seed clone 1array :> best
    seed sum 1array :> upper
    V{ } clone { 0 } best upper p103-search
    best first [ number>string ] map "" join decimal-cell ;
```

Comparison. C exposes direct bounded arithmetic. Shared calls: `euler_fail`. Rust exposes direct bounded arithmetic. Gforth exposes recursive search. Factor exposes direct bounded arithmetic. Shared calls: `decimal-cell`. All listed programs print the same exact answer string from the manifest. The excerpts can differ in algorithm, data representation, and startup cost.

40.104 Problem 104: Pandigital Fibonacci Ends

Keep trailing digits modulo 10^9. For every pandigital tail, compute the entire Fibonacci integer with GMP and inspect its leading digits. No logarithmic approximation can reject a valid leading-digit match.

C. Source: `solutions/c/pandigital_fibonacci_ends_104.c`.

```
static bool pandigital_number(unsigned n) {
    unsigned mask = 0;
    for (unsigned i = 0; i < 9; ++i) {
        unsigned digit = n % 10;
        if (!digit || (mask & (1U << digit))) return false;
        mask |= 1U << digit; n /= 10;
```

```
    }
    return n == 0 && mask == 0x3fe;
}
int main(void) {
    unsigned a = 1, b = 1;
    mpz_t value;
    mpz_init(value);
    for (unsigned long index = 3; ; ++index) {
```

Rust. Source: `solutions/rust/src/bin/p104.rs`.

```
fn solve() -> i64 {
    {
        let mut prev = 1u64;
        let mut curr = 1u64;
        let mut index = 2u64;
        loop {
            index += 1;
            let next = (prev + curr) % 1_000_000_000;
            prev = curr;
            curr = next;
            if pandigital_1_to_9(curr) {
                let text = fibonacci_pair(index).0.to_string();
                if text.len() >= 9 && pandigital_1_to_9(text[..9].parse().unwrap()) {
                    break index as i64;
```

Gforth. Source: `solutions/forth/p104.fs`.

```
: bceil-div10! { big } ( bigint -- )
  10 big bdiv-small! 0<> if 1 big badd-small! then ;

: bsmall-u { big } ( bigint <=9 digits -- n )
  0 big blen@ 0 do
    i big blimb@ i 4 * pow10 * +
  loop ;

: bleading-nine 0 { big temp } ( bignum -- 9-digit integer )
  12 bnew to temp big temp bcopy
  begin temp bdecimal-length 9 > while 10 temp bdiv-small! drop repeat
  temp bsmall-u temp bfree ;

: p104 ( -- n )
```

Factor. Source: `solutions/factor/euler/advanced/advanced.factor`.

```
:: p104 ( -- n )
    1 :> a! 1 :> b! 2 :> index! 0 :> answer!
    [ answer zero? ] [
        a b + 1000000000 mod :> next
        b a! next b!
        index 1 + index!
        b decimal-digits 9 = b pandigital-nine? and [
            index p104-fib-pair drop
            number>string 0 9 rot subseq decimal-cell
            pandigital-nine? [ index answer! ] when
```

```
        ] when
    ] while answer ;
```

Comparison. C exposes exact large-integer arithmetic. Shared calls: `euler_fail`. Rust exposes direct bounded arithmetic. Shared calls: `fibonacci_pair`. Gforth exposes modular arithmetic, and exact large-integer arithmetic. Shared calls: `badd`, `badd-small!`, `bcopy`, `bdecimal-length`, `bdiv-small!`, `bfree`, `blen@`, `blimb@`, `bnew`, `bset-u`, `pandigital-nine?`, `pow10`. Factor exposes modular arithmetic. Shared calls: `decimal-cell`, `decimal-digits`, `pandigital-nine?`. All listed programs print the same exact answer string from the manifest. The excerpts can differ in algorithm, data representation, and startup cost.

40.105 Problem 105: Special Subset Sums: Testing

Sorted smallest-(k+1) versus largest-k sums settle the size rule. Two equal subset sums imply equal disjoint sums after cancelling their intersection, so sorting every subset sum settles the other rule.

C. Source: `solutions/c/special_subset_sums_testing_105.c`.

```
static int compare_u64(const void *a, const void *b) {
    uint64_t x = *(const uint64_t *)a, y = *(const uint64_t *)b;
    return (x > y) - (x < y);
}
static bool special(uint64_t *values, unsigned n) {
    qsort(values, n, sizeof *values, compare_u64);
    for (unsigned i = 1; i < n; ++i) if (values[i] == values[i - 1]) return false;
    for (unsigned k = 1; 2 * k + 1 <= n; ++k) {
        uint64_t small = 0, large = 0;
        for (unsigned i = 0; i <= k; ++i) small += values[i];
        for (unsigned i = 0; i < k; ++i) large += values[n - 1 - i];
        if (small <= large) return false;
    }
    uint64_t sums[1U << 12] = {0};
```

Rust. Source: `solutions/rust/src/bin/p105.rs`.

```
fn solve() -> i64 {
    include_str!("../../../../tests/data/p105_sets.txt")
        .lines()
        .map(|line| {
            let mut values: Vec<u64> = line.split(',').map(|v| v.parse().unwrap()).collect();
            values.sort_unstable();
            if !special_sum_set(&values) {
                0
            } else {
                values.iter().sum::<u64>() as i64
            }
        })
        .sum()
}
```

Gforth. Source: `solutions/forth/p105.fs`.

```
: p105-special? 0 0 0 0 0 0 { values count sums k small large size candidate }
  count 1 do
    i values prefix@ to candidate
    i to size
    begin size 0> if size 1- values prefix@ candidate > else false then while
      size 1- values prefix@ size values prefix!
      size 1- to size
    repeat
    candidate size values prefix!
  loop
  count 1 do
    i 2* 1+ count <= if
      0 to small 0 to large
      i 1+ 0 do i values prefix@ small + to small loop
```

Factor. Source: `solutions/factor/euler/advanced/advanced.factor`.

```
:: p105 ( -- n )
    "tests/data/p105_sets.txt" utf8 file-contents "\n" split
    [ empty? not ] filter
    [| line |
        line "," split [ decimal-cell ] map :> values
        values special-sum? [ values sum ] [ 0 ] if
    ] map sum ;
```

Comparison. C exposes sorting or set deduplication. Shared calls: `euler_open`. Rust exposes fixture parsing, and sorting or set deduplication. Gforth exposes fixture parsing. Shared calls: `prefix!`, `prefix@`, `read-fixture`. Factor exposes fixture parsing. Shared calls: `decimal-cell`. All listed programs print the same exact answer string from the manifest. The excerpts can differ in algorithm, data representation, and startup cost.

40.106 Problem 106: Special Subset Sums: Meta-testing

For 2k selected elements there are C(2k,k)/2 unordered equal-size partitions. The Catalan number counts those already ordered term by term; only the remaining partitions require a sum comparison.

C. Source: `solutions/c/special_subset_sums_meta_testing_106.c`.

```
static uint64_t choose(unsigned n, unsigned k) {
    uint64_t result = 1;
    for (unsigned i = 1; i <= k; ++i) result = result * (n - k + i) / i;
    return result;
}
static uint64_t comparisons(unsigned n) {
    uint64_t total = 0;
    for (unsigned k = 2; 2 * k <= n; ++k) {
        uint64_t middle = choose(2 * k, k);
        total += choose(n, 2 * k) * (middle / 2 - middle / (k + 1));
    }
    return total;
}
```

```
int main(void) {
```

Rust. Source: `solutions/rust/src/bin/p106.rs`.

```
fn solve() -> i64 {
    {
        let n = 12u64;
        (2..=6)
            .map(|k| {
                let pairs = choose(n, 2 * k) * choose(2 * k, k) / 2;
                let comparable = choose(2 * k, k) / u128::from(k + 1);
                (pairs - choose(n, 2 * k) * comparable) as i64
            })
            .sum()
    }
}

fn main() {
```

Gforth. Source: `solutions/forth/p106.fs`.

```
: p106 ( -- n ) 0 0 0 { total middle catalan }
  7 2 do
    i 2* i choose to middle
    middle i 1+ / to catalan
    12 i 2* choose middle 2 / catalan - * total + to total
  loop total ;

[undefined] euler-testing [if] p106 . cr bye [then]
```

Factor. Source: `solutions/factor/euler/extra/extra.factor`.

```
:: p106 ( -- n )
    0 :> total!
    2 6 [a..b] [| k |
        2 k * factorial-number k factorial-number sq /i :> choices
        choices k 1 + /i :> catalan
        12 factorial-number
        2 k * factorial-number 12 2 k * - factorial-number * /i
        choices 2 /i catalan - * total + total!
    ] each total ;
```

Comparison. C exposes direct bounded arithmetic. Rust exposes direct bounded arithmetic. Shared calls: `choose`. Gforth exposes direct bounded arithmetic. Shared calls: `choose`. Factor exposes exact large-integer arithmetic. Shared calls: `factorial-number`. All listed programs print the same exact answer string from the manifest. The excerpts can differ in algorithm, data representation, and startup cost.

40.107 Problem 107: Minimal Network

A connected minimum-weight subgraph with positive edges is a spanning tree. Prim's algorithm finds its weight; subtract it from all edges.

C. Source: `solutions/c/minimal_network_107.c`.

```
int main(void) {
    enum { VERTICES = 40 };
    uint64_t weight[VERTICES][VERTICES], distance[VERTICES];
    bool chosen[VERTICES] = {false};
    FILE *file = euler_open("tests/data/p107_network.txt");
    char line[4096];
    for (unsigned r = 0; r < VERTICES; ++r) {
        if (!fgets(line, sizeof line, file)) euler_fail("Truncated network input");
        char *p = line;
        for (unsigned c = 0; c < VERTICES; ++c) {
            while (isspace((unsigned char)*p)) ++p;
            if (*p == '-') { weight[r][c] = 0; ++p; }
            else {
                char *end;
```

Rust. Source: `solutions/rust/src/bin/p107.rs`.

```
fn solve() -> i64 {
    let input: Vec<Vec<u64>> = include_str!("../../../../tests/data/p107_network.txt")
        .lines()
        .map(|line| {
            line.split(',')
                .map(|v| if v == "-" { 0 } else { v.parse().unwrap() })
                .collect()
        })
        .collect();
    let n = input.len();
    assert_eq!(n, 40);
    let mut total = 0;
    for i in 0..n {
        for j in i + 1..n {
```

Gforth. Source: `solutions/forth/p107.fs`.

```
: p107 ( -- n )
  s" tests/data/p107_network.txt" 8000 read-fixture
  1600 cells allocate throw 40 cells allocate throw 40 allocate throw
  0 0 0 0 0 0 0 0 0
  { raw length edges distance chosen count number active char total cost best vertex candidate }
  0 to count 0 to number false to active
  length 0 do
    raw i + c@ to char
    char [char] 0 [char] 9 1+ within if
      char [char] 0 - number 10 * + to number true to active
    else char [char] - = if
      true to active 0 to number
    else active if
      count 1600 >= abort" Network too large"
```

Factor. Source: `solutions/factor/euler/advanced/advanced.factor`.

```
:: p107 ( -- n )
    "tests/data/p107_network.txt" utf8 file-contents "\n" split
    [ empty? not ] filter :> lines
    1600 0 <array> :> weight
    0 :> total!
    0 39 [a..b] [| r |
        r lines nth "," split :> entries
        0 39 [a..b] [| c |
            c entries nth :> token
            token "-" = not [
                token decimal-cell :> edge
                edge r 40 * c + weight set-nth
                r c < [ edge total + total! ] when
            ] when
```

Comparison. C exposes direct bounded arithmetic. Shared calls: `euler_open`. Rust exposes fixture parsing. Gforth exposes fixture parsing. Shared calls: `prefix!`, `prefix@`, `read-fixture`. Factor exposes fixture parsing. Shared calls: `decimal-cell`. All listed programs print the same exact answer string from the manifest. The excerpts can differ in algorithm, data representation, and startup cost.

40.108 Problem 108: Diophantine Reciprocals I

(x-n)(y-n)=n^2 gives (d(n^2)+1)/2 unordered positive solutions. A minimal n uses consecutive primes with nonincreasing exponents: moving a larger exponent onto a smaller prime can only decrease n. Enumerate every such exponent vector below a constructed feasible primorial. Bounds are proven; no guessed maximum search value is used.

C. Source: `solutions/c/diophantine_reciprocals_i_108.c`.

```
static const unsigned primes[] = {2, 3, 5, 7, 11, 13, 17, 19, 23, 29, 31, 37, 41, 43, 47};
static uint64_t best, target;

static void search(unsigned index, unsigned maximum_exponent,
                   uint64_t value, uint64_t divisor_product) {
    if (divisor_product > target) {
        if (value < best) best = value;
        return;
    }
    if (index == sizeof primes / sizeof *primes) return;
    for (unsigned exponent = 1; exponent <= maximum_exponent; ++exponent) {
        if (value > (best - 1) / primes[index]) break;
        value *= primes[index];
        uint64_t factor = 2 * exponent + 1;
```

Rust. Source: `solutions/rust/src/bin/p108.rs`.

```
fn solve() -> i64 {
    least_reciprocal_denominator(1000)
}
```

```
fn main() {
    println!("{}", solve());
}
```

Gforth. Source: `solutions/forth/p108.fs`.

```
: p108 ( -- n ) 1000 reciprocal-bound ;

[undefined] euler-testing [if] p108 . cr bye [then]
```

Factor. Source: `solutions/factor/euler/advanced/advanced.factor`.

```
:: p108 ( -- n )
    1 :> n!
    [ n p108-divisors 1000 <= ] [ n 1 + n! ] while n ;

CONSTANT: p110-primes { 2 3 5 7 11 13 17 19 23 29 31 37 41 43 47 }
```

Comparison. C exposes prime generation or testing. Rust exposes direct bounded arithmetic. Gforth exposes direct bounded arithmetic. Shared calls: `reciprocal-bound`. Factor exposes prime generation or testing. All listed programs print the same exact answer string from the manifest. The excerpts can differ in algorithm, data representation, and startup cost.

40.109 Problem 109: Darts

The final dart is a double. Earlier darts are an unordered multiset of labelled regions: equal scores in different regions remain distinct.

C. Source: `solutions/c/darts_109.c`.

```
int main(void) {
    unsigned score[62], count = 0, total = 0;
    for (unsigned multiplier = 1; multiplier <= 3; ++multiplier)
        for (unsigned value = 1; value <= 20; ++value)
            score[count++] = multiplier * value;
    score[count++] = 25; score[count++] = 50;
    for (unsigned finish = 1; finish <= 21; ++finish) {
        unsigned last = finish <= 20 ? 2 * finish : 50;
        ++total; /* A checkout consisting of the finishing dart alone. */
        for (unsigned i = 0; i < count; ++i) {
            if (score[i] + last < 100) ++total;
            for (unsigned j = i; j < count; ++j)
                if (score[i] + score[j] + last < 100) ++total;
        }
```

Rust. Source: `solutions/rust/src/bin/p109.rs`.

```
fn solve() -> i64 {
    {
```

```
        let mut scores = Vec::new();
        for n in 1..=20 {
            scores.push(n);
            scores.push(2 * n);
            scores.push(3 * n);
        }
        scores.extend([25, 50]);
        let mut doubles: Vec<i64> = (1..=20).map(|n| 2 * n).collect();
        doubles.push(50);
        let mut count = 0;
        for &d in &doubles {
            if d < 100 {
```

Gforth. Source: `solutions/forth/p109.fs`.

```
: p109 ( -- n )
  62 cells allocate throw 0 0 0 0 0 { scores count total finish last value }
  4 1 do
    21 1 do i j * count scores prefix! count 1+ to count loop
  loop
  25 count scores prefix! 50 count 1+ scores prefix! 62 to count
  22 1 do
    i 21 = if 50 else i 2* then to last
    total 1+ to total
    count 0 do
      i scores prefix@ last + 100 < if total 1+ to total then
      count i do
        i scores prefix@ j scores prefix@ + last + 100 < if total 1+ to total then
      loop
```

Factor. Source: `solutions/factor/euler/advanced/advanced.factor`.

```
:: p109 ( -- n )
    V{ 0 } clone :> scores
    1 20 [a..b] [| n |
        n scores push n 2 * scores push n 3 * scores push
    ] each
    25 scores push 50 scores push
    V{ } clone :> doubles
    1 20 [a..b] [| n | n 2 * doubles push ] each
    50 doubles push
    0 :> answer!
    doubles [| finish |
        0 scores length 1 - [a..b] [| i |
            i scores length 1 - [a..b] [| j |
                i scores nth j scores nth + finish + 100 <
```

Comparison. C exposes direct bounded arithmetic. Rust exposes direct bounded arithmetic. Gforth exposes direct bounded arithmetic. Shared calls: `prefix!`, `prefix@`. Factor exposes direct bounded arithmetic. All listed programs print the same exact answer string from the manifest. The excerpts can differ in algorithm, data representation, and startup cost.

40.110 Problem 110: Diophantine Reciprocals II

(x-n)(y-n)=n^2 gives (d(n^2)+1)/2 unordered positive solutions. A minimal n uses consecutive primes with nonincreasing exponents: moving a larger exponent onto a smaller prime can only decrease n. Enumerate every such exponent vector below a constructed feasible primorial. Bounds are proven; no guessed maximum search value is used.

C. Source: `solutions/c/diophantine_reciprocals_ii_110.c`.

```
static const unsigned primes[] = {2, 3, 5, 7, 11, 13, 17, 19, 23, 29, 31, 37, 41, 43, 47};
static uint64_t best, target;

static void search(unsigned index, unsigned maximum_exponent,
                   uint64_t value, uint64_t divisor_product) {
    if (divisor_product > target) {
        if (value < best) best = value;
        return;
    }
    if (index == sizeof primes / sizeof *primes) return;
    for (unsigned exponent = 1; exponent <= maximum_exponent; ++exponent) {
        if (value > (best - 1) / primes[index]) break;
        value *= primes[index];
        uint64_t factor = 2 * exponent + 1;
```

Rust. Source: `solutions/rust/src/bin/p110.rs`.

```
fn solve() -> i64 {
    least_reciprocal_denominator(4_000_000)
}

fn main() {
    println!("{}", solve());
}
```

Gforth. Source: `solutions/forth/p110.fs`.

```
: p110 ( -- n ) 4000000 reciprocal-bound ;

[undefined] euler-testing [if] p110 . cr bye [then]
```

Factor. Source: `solutions/factor/euler/advanced/advanced.factor`.

```
:: p110 ( -- n )
    1000000000000000000 1array :> best
    0 13 1 1 best p110-search
    best first ;
```

Comparison. C exposes prime generation or testing. Rust exposes direct bounded arithmetic. Gforth exposes direct bounded arithmetic. Shared calls: `reciprocal-bound`. Factor exposes direct bounded arithmetic. All listed programs print the same exact answer string from the manifest. The excerpts can differ in algorithm, data representation, and startup cost.

40.111 Problem 111: Primes with Runs

For each repeated digit, increase the number of replacements from zero. Enumerate every placement and every different replacement digit exactly once, stopping at the first count with at least one prime.

C. Source: `solutions/c/primes_with_runs_111.c`.

```
typedef struct { unsigned length, digit; uint64_t count, sum; } Runs;
static void replacements(Runs *runs, unsigned position, unsigned left, uint64_t value) {
    if (left > runs->length - position) return;
    if (position == runs->length) {
        if (!left && euler_is_prime(value)) { ++runs->count; runs->sum += value; }
        return;
    }
    if (runs->digit || position)
        replacements(runs, position + 1, left, 10 * value + runs->digit);
    if (left) for (unsigned digit = position ? 0 : 1; digit <= 9; ++digit)
        if (digit != runs->digit)
            replacements(runs, position + 1, left - 1, 10 * value + digit);
}
static uint64_t run_sum(unsigned length) {
```

Rust. Source: `solutions/rust/src/bin/p111.rs`.

```
fn solve() -> i64 {
    fn recurse(
        position: u32,
        length: u32,
        digit: u64,
        changes: u32,
        value: u64,
        count: &mut u64,
        sum: &mut u64,
    ) {
        if changes > length - position {
            return;
        }
        if position == length {
```

Gforth. Source: `solutions/forth/p111.fs`.

```
variable p111-count variable p111-sum

: p111-replace { repeat-digit position remaining value } ( repeat-digit position remaining value
     -- )
  remaining 10 position - > if exit then
  position 10 = if
    remaining 0= if value is-prime? if
      p111-count @ 1+ p111-count !
      value p111-sum @ + p111-sum !
    then then
    exit
  then
  repeat-digit 0<> position 0<> or if
    repeat-digit position 1+ remaining value 10 * repeat-digit + recurse
```

```
  then
```

Factor. Source: `solutions/factor/euler/advanced/advanced.factor`.

```
:: p111 ( -- n )
    0 :> total!
    0 9 [a..b] [| digit |
        f :> found!
        1 10 [a..b] [| changes |
            found not [
                0 0 2array :> result
                1 1023 [a..b] [| mask |
                    mask p90-countbits changes = [
                        digit mask 0 0 result p111-build
                    ] when
                ] each
                result second 0 > [
                    result first total + total!
```

Comparison. C exposes prime generation or testing. Shared calls: `euler_is_prime`. Rust exposes prime generation or testing, and recursive search. Shared calls: `prime`. Gforth exposes prime generation or testing, and recursive search. Shared calls: `is-prime?`. Factor exposes direct bounded arithmetic. All listed programs print the same exact answer string from the manifest. The excerpts can differ in algorithm, data representation, and startup cost.

40.112 Problem 112: Bouncy Numbers

A bouncy number contains both a rising and a falling adjacent digit pair. Test the target proportion by integer cross multiplication.

C. Source: `solutions/c/bouncy_numbers_112.c`.

```
static bool bouncy(unsigned n) {
    bool rises = false, falls = false;
    unsigned previous = n % 10;
    while ((n /= 10)) {
        unsigned digit = n % 10;
        rises |= digit < previous;
        falls |= digit > previous;
        previous = digit;
    }
    return rises && falls;
}
int main(void) {
    uint64_t count = 0;
    for (unsigned n = 1; ; ++n) {
```

Rust. Source: `solutions/rust/src/bin/p112.rs`.

```
fn solve() -> i64 {
    {
```

```
        fn bouncy(n: u64) -> bool {
            let d = digits(n);
            !(d.windows(2).all(|v| v[0] <= v[1]) || d.windows(2).all(|v| v[0] >= v[1]))
        }
        let mut count = 0;
        let mut n = 99u64;
        loop {
            n += 1;
            if bouncy(n) {
                count += 1;
            }
            if 100 * count == 99 * n {
```

Gforth. Source: `solutions/forth/p112.fs`.

```
: p112-bouncy? 0 0 0 0 { n last increasing decreasing digit }
  true to increasing true to decreasing
  n 10 mod to last n 10 / to n
  begin n 0> while
    n 10 mod to digit
    digit last > if false to decreasing then
    digit last < if false to increasing then
    digit to last n 10 / to n
  repeat increasing 0= decreasing 0= and ;

: p112 ( -- n ) 0 100 { count n }
  begin n p112-bouncy? if count 1+ to count then
    count 100 * n 99 * = if n exit then
    n 1+ to n
```

Factor. Source: `solutions/factor/euler/extra/extra.factor`.

```
:: p112 ( -- n )
    100 :> n! 0 :> count!
    0 :> answer!
    [ answer zero? ] [
        n p112-bouncy? [ count 1 + count! ] when
        count 100 * n 99 * = [ n answer! ] when
        n 1 + n!
    ] while answer ;
```

Comparison. C exposes direct bounded arithmetic. Rust exposes direct bounded arithmetic. Shared calls: `digits`. Gforth exposes modular arithmetic. Factor exposes direct bounded arithmetic. All listed programs print the same exact answer string from the manifest. The excerpts can differ in algorithm, data representation, and startup cost.

40.113 Problem 113: Non-bouncy Numbers

Digit DP counts monotone positive integers at each length. Nine constant-digit integers occur in both directions at every length.

C. Source: `solutions/c/non_bouncy_numbers_113.c`.

```
static uint64_t non_bouncy(unsigned digits) {
    uint64_t up[10] = {0}, down[10] = {0}, total = 0;
    for (unsigned d = 1; d <= 9; ++d) up[d] = down[d] = 1;
    for (unsigned length = 1; length <= digits; ++length) {
        for (unsigned d = 0; d <= 9; ++d) total += up[d] + down[d];
        total -= 9;
        uint64_t next_up[10] = {0}, next_down[10] = {0};
        for (unsigned previous = 0; previous <= 9; ++previous) {
            for (unsigned d = previous; d <= 9; ++d) next_up[d] += up[previous];
            for (unsigned d = 0; d <= previous; ++d) next_down[d] += down[previous];
        }
        memcpy(up, next_up, sizeof up); memcpy(down, next_down, sizeof down);
    }
    return total;
```

Rust. Source: `solutions/rust/src/bin/p113.rs`.

```
fn solve() -> i64 {
    {
        let mut up = [0u64; 10];
        let mut down = [0u64; 10];
        for d in 1..=9 {
            up[d] = 1;
            down[d] = 1;
        }
        let mut sum = 0u64;
        for _ in 1..=100 {
            sum += up.iter().sum::<u64>() + down.iter().sum::<u64>() - 9;
            let mut next_up = [0u64; 10];
            let mut next_down = [0u64; 10];
            for old in 0..=9 {
```

Gforth. Source: `solutions/forth/p113.fs`.

```
: p113 ( -- n ) 110 10 choose 109 9 choose + 1002 - ;

[undefined] euler-testing [if] p113 . cr bye [then]
```

Factor. Source: `solutions/factor/euler/extra/extra.factor`.

```
: p113 ( -- n ) 110 10 p113-binomial 109 9 p113-binomial + 1002 - ;
```

Comparison. C exposes dynamic programming or memoization. Rust exposes direct bounded arithmetic. Gforth exposes direct bounded arithmetic. Shared calls: `choose`. Factor exposes direct bounded arithmetic. All listed programs print the same exact answer string from the manifest. The excerpts can differ in algorithm, data representation, and startup cost.

40.114 Problem 114: Counting Block Combinations I

Either begin with one black square, or a red run of length at least three, followed by a mandatory black separator unless the row ends.

C. Source: `solutions/c/counting_block_combinations_i_114.c`.

```
static uint64_t block_ways(unsigned length, unsigned minimum) {
    uint64_t *ways = euler_calloc(length + 1, sizeof *ways);
    ways[0] = 1;
    for (unsigned n = 1; n <= length; ++n) {
        ways[n] = ways[n - 1];
        for (unsigned red = minimum; red <= n; ++red)
            ways[n] += red == n ? 1 : ways[n - red - 1];
    }
    uint64_t answer = ways[length];
    free(ways);
    return answer;
}
int main(void) {
    printf("%" PRIu64 "\n", block_ways(50, 3));
```

Rust. Source: `solutions/rust/src/bin/p114.rs`.

```
fn solve() -> i64 {
    {
        let mut ways = [0u64; 51];
        ways[0] = 1;
        for n in 1..=50 {
            ways[n] = ways[n - 1];
            for red in 3..=n {
                ways[n] += if red == n { 1 } else { ways[n - red - 1] };
            }
        }
        ways[50] as i64
    }
}
```

Gforth. Source: `solutions/forth/p114.fs`.

```
: p114 ( -- n )
  51 cells allocate throw 0 { ways result }
  ways 51 cells erase 1 ways !
  51 1 do
    i 1- ways prefix@ i ways prefix!
    i 3 >= if i 1+ 3 do
      i j = if 1 else j i - 1- ways prefix@ then
      j cells ways + +!
    loop then
  loop
  50 ways prefix@ to result ways free throw result ;

[undefined] euler-testing [if] p114 . cr bye [then]
```

Factor. Source: `solutions/factor/euler/extra/extra.factor`.

```
: p114 ( -- n ) 50 3 p114-ways ;
```

Comparison. C exposes direct bounded arithmetic. Shared calls: `euler_calloc`. Rust exposes direct bounded arithmetic. Gforth exposes direct bounded arithmetic. Shared calls: `prefix!`, `prefix@`. Factor exposes direct bounded arithmetic. All listed programs print the same exact answer string from the manifest. The excerpts can differ in algorithm, data representation, and startup cost.

40.115 Problem 115: Counting Block Combinations II

The recurrence is the same separated-red-run DP as #114. Twenty disjoint blocks of 51 cells already allow 2^20 choices, proving that a row of 1020 cells exceeds one million. Counts saturate above target.

C. Source: `solutions/c/counting_block_combinations_ii_115.c`.

```
int main(void) {
    enum { MINIMUM = 50, LIMIT = 20 * (MINIMUM + 1) };
    const uint64_t target = 1000000;
    uint64_t ways[LIMIT + 1] = {1};
    for (unsigned n = 1; n <= LIMIT; ++n) {
        ways[n] = ways[n - 1];
        for (unsigned red = MINIMUM; red <= n; ++red) {
            ways[n] += red == n ? 1 : ways[n - red - 1];
            if (ways[n] > target) { ways[n] = target + 1; break; }
        }
        if (ways[n] > target) { printf("%u\n", n); return 0; }
    }
    euler_fail("Proven block-count bound was not reached");
    return 1;
```

Rust. Source: `solutions/rust/src/bin/p115.rs`.

```
fn solve() -> i64 {
    {
        let mut ways = vec![1u64];
        for n in 1.. {
            let mut count = ways[n - 1];
            for block in 50..=n {
                count += if block == n { 1 } else { ways[n - block - 1] };
            }
            ways.push(count);
            if count > 1_000_000 {
                return n as i64;
            }
        }
        unreachable!("unbounded tiling search")
```

Gforth. Source: `solutions/forth/p115.fs`.

```
: p115 ( -- n )
  256 cells allocate throw 0 0 { ways result index }
```

```
  ways 256 cells erase 1 ways !
  256 1 do
    i 1- ways prefix@ to result
    i 50 >= if i 1+ 50 do
      i j = if 1 else j i - 1- ways prefix@ then
      result + to result
    loop then
    result i ways prefix!
    result 1000000 > if ways free throw i unloop exit then
  loop ways free throw -1 ;

[undefined] euler-testing [if] p115 . cr bye [then]
```

Factor. Source: `solutions/factor/euler/extra/extra.factor`.

```
:: p115 ( -- n )
    50 :> length!
    [ length 50 p114-ways 1000000 > ] [ length 1 + length! ] until
    length ;
```

Comparison. C exposes dynamic programming or memoization. Rust exposes direct bounded arithmetic. Gforth exposes direct bounded arithmetic. Shared calls: `prefix!`, `prefix@`. Factor exposes direct bounded arithmetic. All listed programs print the same exact answer string from the manifest. The excerpts can differ in algorithm, data representation, and startup cost.

40.116 Problem 116: Red, Green or Blue Tiles

For one colour, the last tile has length one or that colour's length. Subtract the all-black arrangement before adding the three colours.

C. Source: `solutions/c/red_green_or_blue_tiles_116.c`.

```
static uint64_t coloured_ways(unsigned length, unsigned tile) {
    uint64_t ways[51] = {1};
    for (unsigned n = 1; n <= length; ++n)
        ways[n] = ways[n - 1] + (n >= tile ? ways[n - tile] : 0);
    return ways[length] - 1;
}
int main(void) {
    uint64_t total = 0;
    for (unsigned tile = 2; tile <= 4; ++tile) total += coloured_ways(50, tile);
    printf("%" PRIu64 "\n", total);
    return 0;
}
```

Rust. Source: `solutions/rust/src/bin/p116.rs`.

```
fn solve() -> i64 {
    {
        let mut sum = 0u64;
        for size in [2usize, 3, 4] {
```

```
            let mut ways = [0u64; 51];
            ways[0] = 1;
            for n in 1..=50 {
                ways[n] = ways[n - 1] + if n >= size { ways[n - size] } else { 0 };
            }
            sum += ways[50] - 1;
        }
        sum as i64
    }
}
```

Gforth. Source: `solutions/forth/p116.fs`.

```
: p116 ( -- n )
  51 cells allocate throw 0 0 0 { ways total length result }
  5 2 do
    i to length ways 51 cells erase 1 ways !
    51 1 do
      i 1- ways prefix@ to result
      i length >= if i length - ways prefix@ result + to result then
      result i ways prefix!
    loop
    50 ways prefix@ 1- total + to total
  loop ways free throw total ;

[undefined] euler-testing [if] p116 . cr bye [then]
```

Factor. Source: `solutions/factor/euler/extra/extra.factor`.

```
:: p116 ( -- n )
    0 :> answer!
    2 4 [a..b] [| tile |
        51 0 <array> :> ways
        1 0 ways set-nth
        1 50 [a..b] [| n |
            n 1 - ways nth :> count!
            n tile >= [ count n tile - ways nth + count! ] when
            count n ways set-nth
        ] each
        50 ways nth 1 - answer + answer!
    ] each answer ;
```

Comparison. C exposes direct bounded arithmetic. Rust exposes direct bounded arithmetic. Gforth exposes direct bounded arithmetic. Shared calls: `prefix!`, `prefix@`. Factor exposes direct bounded arithmetic. All listed programs print the same exact answer string from the manifest. The excerpts can differ in algorithm, data representation, and startup cost.

40.117 Problem 117: Red, Green, and Blue Tiles

Partition arrangements by their final tile, of length 1, 2, 3 or 4.

C. Source: `solutions/c/red_green_and_blue_tiles_117.c`.

```
static uint64_t mixed_ways(unsigned length) {
    uint64_t ways[51] = {1};
    for (unsigned n = 1; n <= length; ++n)
        for (unsigned tile = 1; tile <= 4 && tile <= n; ++tile)
            ways[n] += ways[n - tile];
    return ways[length];
}
int main(void) {
    printf("%" PRIu64 "\n", mixed_ways(50));
    return 0;
}
```

Rust. Source: `solutions/rust/src/bin/p117.rs`.

```
fn solve() -> i64 {
    {
        let mut ways = [0u64; 51];
        ways[0] = 1;
        for n in 1..=50 {
            for size in 1..=4 {
                if size <= n {
                    ways[n] += ways[n - size];
                }
            }
        }
        ways[50] as i64
    }
}
```

Gforth. Source: `solutions/forth/p117.fs`.

```
: p117 ( -- n )
  51 cells allocate throw 0 { ways result }
  ways 51 cells erase 1 ways !
  51 1 do
    5 1 do j i >= if j i - ways prefix@ j cells ways + +! then loop
  loop
  50 ways prefix@ to result ways free throw result ;

[undefined] euler-testing [if] p117 . cr bye [then]
```

Factor. Source: `solutions/factor/euler/extra/extra.factor`.

```
:: p117 ( -- n )
    51 0 <array> :> ways
    1 0 ways set-nth
    1 50 [a..b] [| n |
        0 :> count!
        1 4 [a..b] [| width |
            n width >= [ count n width - ways nth + count! ] when
        ] each
        count n ways set-nth
    ] each 50 ways nth ;
```

Comparison. C exposes direct bounded arithmetic. Rust exposes direct bounded arithmetic. Gforth exposes direct bounded arithmetic. Shared calls: `prefix@`. Factor exposes direct bounded arithmetic. All listed programs print the same exact answer string from the manifest. The excerpts can differ in algorithm, data representation, and startup cost.

40.118 Problem 118: Pandigital Prime Sets

Count primes for each digit subset by enumerating its permutations. Partition the remaining mask; requiring the next subset to contain its lowest digit counts every unordered set of prime blocks once.

C. Source: `solutions/c/pandigital_prime_sets_118.c`.

```
static uint64_t prime_count[512], memo[512];
static bool known[512];
static void enumerate_primes(unsigned mask, uint64_t value, unsigned digit_sum) {
    if (mask && (value == 2 || value == 3 || value == 5 || value == 7 ||
        (digit_sum % 3 && value % 2 && value % 5 && euler_is_prime(value))))
        ++prime_count[mask];
    for (unsigned digit = 1; digit <= 9; ++digit) {
        unsigned bit = 1U << (digit - 1);
        if (!(mask & bit)) enumerate_primes(mask | bit, 10 * value + digit, digit_sum + digit);
    }
}
static uint64_t partitions(unsigned mask) {
    if (!mask) return 1;
    if (known[mask]) return memo[mask];
```

Rust. Source: `solutions/rust/src/bin/p118.rs`.

```
fn solve() -> i64 {
    fn enumerate(mask: usize, value: u64, digit_sum: u32, counts: &mut [u64; 512]) {
        if mask != 0
            && (matches!(value, 2 | 3 | 5 | 7)
                || (digit_sum % 3 != 0 && value % 2 != 0 && value % 5 != 0 && prime(value)))
        {
            counts[mask] += 1;
        }
        for digit in 1..=9u64 {
            let bit = 1usize << (digit - 1);
            if mask & bit == 0 {
                enumerate(
                    mask | bit,
                    value * 10 + digit,
```

Gforth. Source: `solutions/forth/p118.fs`.

```
create p118-prime-count 512 cells allot

create p118-memo 512 cells allot

create p118-known 512 allot

: p118-enumerate { mask value sum } ( mask value digit-sum -- )
```

```
  mask 0<> if
    value 2 = value 3 = or value 5 = or value 7 = or
    sum 3 mod 0<> value 2 mod 0<> and value 5 mod 0<> and value is-prime? and or if
      mask p118-prime-count prefix@ 1+ mask p118-prime-count prefix!
    then
  then
  10 1 do
```

Factor. Source: `solutions/factor/euler/advanced/advanced.factor`.

```
:: p118 ( -- n )
    512 0 <array> :> counts
    511 511 0 counts p118-construct
    ! All submasks are generated from the same pool, not just from 511.
    1 510 [a..b] [| mask |
        mask mask 0 counts p118-construct
    ] each
    512 -1 <array> :> memory
    511 counts memory p118-solve ;
```

Comparison. C exposes prime generation or testing, and dynamic programming or memoization. Shared calls: `euler_is_prime`. Rust exposes prime generation or testing, and dynamic programming or memoization. Shared calls: `prime`. Gforth exposes prime generation or testing, modular arithmetic, dynamic programming or memoization, and recursive search. Shared calls: `is-prime?`, `prefix!`, `prefix@`. Factor exposes dynamic programming or memoization. All listed programs print the same exact answer string from the manifest. The excerpts can differ in algorithm, data representation, and startup cost.

40.119 Problem 119: Digit Power Sum

Below 10^d a digit sum is at most 9*d. Enumerate every power of every such base, sort and deduplicate, then enlarge d until 30 values exist. This decimal cutoff proves that no smaller qualifying term is missed.

C. Source: `solutions/c/digit_power_sum_119.c`.

```
static unsigned digit_sum(uint64_t n) {
    unsigned sum = 0;
    do { sum += n % 10; n /= 10; } while (n);
    return sum;
}
static int compare_u64(const void *a, const void *b) {
    uint64_t x = *(const uint64_t *)a, y = *(const uint64_t *)b;
    return (x > y) - (x < y);
}
int main(void) {
    /* At most 171 bases and 64 powers per base for any uint64_t cutoff. */
    uint64_t values[171 * 64], limit = 100;
    for (unsigned digits = 2; digits <= 19; ++digits) {
        size_t count = 0;
```

Rust. Source: `solutions/rust/src/bin/p119.rs`.

```
fn solve() -> i64 {
    for size in 2..=19u64 {
        let cutoff = 10u64.pow(size as u32);
        let mut values = Vec::new();
        for base in 2..=9 * size {
            let mut v = base * base;
            while v < cutoff {
                if v >= 10 && digits(v).iter().map(|&v| v as u64).sum::<u64>() == base {
                    values.push(v);
                }
                if v > (cutoff - 1) / base {
                    break;
                }
                v *= base;
```

Gforth. Source: `solutions/forth/p119.fs`.

```
: p119-digit-sum { number } ( n -- sum )
  0 begin number 0> while number 10 mod + number 10 / to number repeat ;

: p119-sort 0 0 { values count value position } ( array n -- )
  count 1 do
    i values prefix@ to value
    i to position
    begin position 0> if position 1- values prefix@ value > else false then while
      position 1- values prefix@ position values prefix!
      position 1- to position
    repeat
    value position values prefix!
  loop ;
```

Factor. Source: `solutions/factor/euler/advanced/advanced.factor`.

```
:: p119 ( -- n )
    V{ } clone :> found
    2 200 [a..b] [| base |
        base sq :> power!
        [ power 100000000000000000 <= ] [
            power 10 >= power digit-sum base = and [ power found push ] when
            power base * power!
        ] while
    ] each found members sort :> ordered
    29 ordered nth ;
```

Comparison. C exposes sorting or set deduplication. Rust exposes sorting or set deduplication. Shared calls: `digits`. Gforth exposes modular arithmetic, and sorting or set deduplication. Shared calls: `prefix!`, `prefix@`. Factor exposes sorting or set deduplication. Shared calls: `digit-sum`. All listed programs print the same exact answer string from the manifest. The excerpts can differ in algorithm, data representation, and startup cost.

40.120 Problem 120: Square Remainders

Binomial expansion gives remainder 2 for even n and 2*a*n mod a^2 for odd n. Its maximum is 2*a*floor((a-1)/2).

C. Source: `solutions/c/square_remainders_120.c`.

```
static uint64_t maximum_remainder(unsigned a) {
    return 2ULL * a * ((a - 1) / 2);
}
int main(void) {
    uint64_t total = 0;
    for (unsigned a = 3; a <= 1000; ++a) total += maximum_remainder(a);
    printf("%" PRIu64 "\n", total);
    return 0;
}
```

Rust. Source: `solutions/rust/src/bin/p120.rs`.

```
fn solve() -> i64 {
    (3u64..=1000)
        .map(|a| if a % 2 == 0 { a * (a - 2) } else { a * (a - 1) })
        .sum::<u64>() as i64
}

fn main() {
    println!("{}", solve());
}
```

Gforth. Source: `solutions/forth/p120.fs`.

```
: p120 ( -- n ) 0 { total }
  1001 3 do
    i 2 mod if i i 1- * else i i 2 - * then total + to total
  loop total ;

\ Further reusable enumerators and exact integer recurrences.

[undefined] euler-testing [if] p120 . cr bye [then]
```

Factor. Source: `solutions/factor/euler/extra/extra.factor`.

```
:: p120 ( -- n )
    3 1000 [a..b] [| a |
        a odd? [ a a 1 - * ] [ a a 2 - * ] if
    ] map sum ;
```

Comparison. C exposes modular arithmetic. Rust exposes direct bounded arithmetic. Gforth exposes modular arithmetic. Factor exposes direct bounded arithmetic. All listed programs print the same exact answer string from the manifest. The excerpts can differ in algorithm, data representation, and startup cost.

40.121 Problem 121: Disc Game Prize Fund

On turn t, multiplying every outcome probability by t+1 leaves integer weights: blue contributes 1 and red contributes t. ‘ways[r]‘ stores the total weight of sequences containing r red draws, updated backward so each turn is used once. The common denominator after 15 turns is 2*3*...*16. Winning means strictly more blue than red, hence fewer than eight reds. If W is the weighted winning count and D the denominator, the largest fair integral prize fund is floor(D/W). All weights fit uint64_t.

C. Source: `solutions/c/disc_game_prize_fund_121.c`.

```
static uint64_t prize_fund(unsigned turns) {
    uint64_t ways[16] = {1};
    uint64_t denominator = 1;
    for (unsigned turn = 1; turn <= turns; ++turn) {
        for (unsigned reds = turn; reds > 0; --reds)
            ways[reds] += ways[reds - 1] * turn;
        denominator *= turn + 1;
    }
    uint64_t winning = 0;
    for (unsigned reds = 0; reds < (turns + 1) / 2; ++reds) winning += ways[reds];
    return denominator / winning;
}

int main(void) {
```

Rust. Source: `solutions/rust/src/bin/p121.rs`.

```
fn solve() -> u64 {
    let mut ways = [0u64; 16];
    ways[0] = 1;
    let mut denominator = 1u64;
    for turn in 1..=15 {
        for r in (1..=turn).rev() {
            ways[r] += ways[r - 1] * turn as u64;
        }
        denominator *= (turn + 1) as u64;
    }
    denominator / ways[..8].iter().sum::<u64>()
}
fn main() {
    println!("{}", solve());
```

Factor. Source: `solutions/factor/euler/extended/extended.factor`.

```
:: p121 ( -- n )
    16 0 <array> :> ways
    1 0 ways set-nth
    1 15 [a..b] [| turn |
        turn 1 [a..b] [| reds |
            reds ways nth reds 1 - ways nth turn * +
            reds ways set-nth
        ] each
    ] each
    0 :> wins!
```

```
    0 7 [a..b] [| reds | reds ways nth wins + wins! ] each
    16 factorial-number wins /i ;

! Problem 123: for odd n, the remainder collapses to 2*n*p_n modulo p_n^2.
```

Comparison. C exposes direct bounded arithmetic. Rust exposes dynamic programming or memoization. Factor exposes exact large-integer arithmetic. Shared calls: `factorial-number`. No validated source exists for Gforth. All listed programs print the same exact answer string from the manifest. The excerpts can differ in algorithm, data representation, and startup cost.

40.122 Problem 122: Efficient Exponentiation

An addition chain starts at 1; appending a+b means multiplying already available powers x^a and x^b. We use iterative deepening, so the first chain found for each target is provably shortest. The search specializes to star chains, where every new exponent adds the current largest exponent to an earlier one; this suffices for the requested targets through 200. If doubling at every remaining level still cannot reach the target, the branch is impossible and is pruned. ‘seen‘ also merges different choices that create the same next exponent at one node. A 16-element chain is ample because the discovered depths are far below that bound.

C. Source: `solutions/c/efficient_exponentiation_122.c`.

```
static unsigned chain[16];

static bool reaches(unsigned target, unsigned depth, unsigned limit) {
    unsigned top = chain[depth];
    if (depth == limit) return top == target;
    if ((uint64_t)top << (limit - depth) < target) return false;
    bool seen[201] = {false};
    for (int i = (int)depth; i >= 0; --i) {
        unsigned next = top + chain[i];
        if (next <= top || next > target || seen[next]) continue;
        seen[next] = true;
        chain[depth + 1] = next;
        if (reaches(target, depth + 1, limit)) return true;
    }
```

Rust. Source: `solutions/rust/src/bin/p122.rs`.

```
fn reaches(target: usize, depth: usize, limit: usize, chain: &mut [usize; 16]) -> bool {
    let top = chain[depth];
    if depth == limit {
        return top == target;
    }
    if top << (limit - depth) < target {
        return false;
    }
    let mut seen = [false; 201];
    for i in (0..=depth).rev() {
        let next = top + chain[i];
        if next <= target && !seen[next] {
            seen[next] = true;
```

```
            chain[depth + 1] = next;
```

Comparison. C exposes direct bounded arithmetic. Rust exposes direct bounded arithmetic. No validated source exists for Gforth, and Factor. All listed programs print the same exact answer string from the manifest. The excerpts can differ in algorithm, data representation, and startup cost.

40.123 Problem 123: Prime Square Remainders

The program computes the exact answer for Prime Square Remainders.

C. Source: `solutions/c/prime_square_remainders_123.c`.

```
int main(void) {
    enum { LIMIT = 300000 };
    bool *composite = calloc(LIMIT, sizeof(*composite));
    if (!composite) return EXIT_FAILURE;
    unsigned index = 0;
    for (unsigned p = 2; p < LIMIT; ++p) {
        if (composite[p]) continue;
        ++index;
        if ((uint64_t)p * p < LIMIT)
            for (unsigned multiple = p * p; multiple < LIMIT; multiple += p)
                composite[multiple] = true;
        if ((index & 1U) && 2ULL * index * p > 10000000000ULL) {
            printf("%u\n", index);
            free(composite);
```

Rust. Source: `solutions/rust/src/bin/p123.rs`.

```
fn solve() -> usize {
    sieve(300_000)
        .iter()
        .enumerate()
        .filter(|(_, p)| **p)
        .enumerate()
        .find(|(i, (p, _))| {
            let n = i + 1;
            n % 2 == 1 && 2u64 * n as u64 * (*p as u64) > 10_000_000_000
        })
        .unwrap()
        .0
        + 1
}
```

Factor. Source: `solutions/factor/euler/extended/extended.factor`.

```
:: p123 ( -- n )
    1 :> index! 2 :> candidate! 0 :> answer!
    [ answer zero? ] [
        candidate 1 + candidate!
        candidate prime? [
            index 1 + index!
```

```
            index odd? index 2 * candidate * 10000000000 > and
            [ index answer! ] when
        ] when
    ] while answer ;

! Problem 125: prefix sums enumerate consecutive square sums; a hash table
! removes values having more than one representation.
```

Comparison. C exposes direct bounded arithmetic. Rust exposes prime generation or testing. Factor exposes prime generation or testing. No validated source exists for Gforth. All listed programs print the same exact answer string from the manifest. The excerpts can differ in algorithm, data representation, and startup cost.

40.124 Problem 124: Ordered Radicals

The program computes the exact answer for Ordered Radicals.

C. Source: `solutions/c/ordered_radicals_124.c`.

```c
static int compare_entries(const void *left, const void *right) {
    const struct radical_entry *a = left, *b = right;
    if (a->radical != b->radical) return a->radical < b->radical ? -1 : 1;
    return a->number < b->number ? -1 : a->number != b->number;
}

int main(void) {
    enum { LIMIT = 100000 };
    struct radical_entry *entries = malloc((LIMIT + 1) * sizeof(*entries));
    if (!entries) return EXIT_FAILURE;
    for (unsigned n = 0; n <= LIMIT; ++n) entries[n] = (struct radical_entry){1, n};
    for (unsigned p = 2; p <= LIMIT; ++p)
        if (entries[p].radical == 1)
            for (unsigned n = p; n <= LIMIT; n += p) entries[n].radical *= p;
```

Rust. Source: `solutions/rust/src/bin/p124.rs`.

```rust
fn solve() -> usize {
    const N: usize = 100_000;
    let mut rad = vec![1usize; N + 1];
    for p in 2..=N {
        if rad[p] == 1 {
            for n in (p..=N).step_by(p) {
                rad[n] *= p;
            }
        }
    }
    let mut v: Vec<(usize, usize)> = (1..=N).map(|n| (rad[n], n)).collect();
    v.sort_unstable();
    v[9999].1
}
```

Factor. Source: `solutions/factor/p124.factor`.

```
! Native Factor Project Euler 124 solver supplied by the bundled Factor image.
USING: io math.parser project-euler.124 ; IN: scratchpad
euler124 number>string print
```

Comparison. C exposes sorting or set deduplication. Rust exposes prime generation or testing, and sorting or set deduplication. Factor exposes direct bounded arithmetic. No validated source exists for Gforth. All listed programs print the same exact answer string from the manifest. The excerpts can differ in algorithm, data representation, and startup cost.

40.125 Problem 125: Palindromic Sums

The program computes the exact answer for Palindromic Sums.

C. Source: `solutions/c/palindromic_sums_125.c`.

```
static bool palindrome(unsigned value) {
    unsigned original = value, reverse = 0;
    do { reverse = reverse * 10 + value % 10; value /= 10; } while (value);
    return reverse == original;
}

static int compare_unsigned(const void *left, const void *right) {
    unsigned a = *(const unsigned *)left, b = *(const unsigned *)right;
    return a < b ? -1 : a != b;
}

int main(void) {
    enum { LIMIT = 100000000, ROOT = 10000 };
    uint64_t prefix[ROOT + 1] = {0};
```

Rust. Source: `solutions/rust/src/bin/p125.rs`.

```
fn solve() -> u64 {
    let mut pre = vec![0u64; 10_001];
    for n in 1..=10_000 {
        pre[n] = pre[n - 1] + (n * n) as u64;
    }
    let mut set = HashSet::new();
    for a in 1..10_000 {
        for b in a + 1..=10_000 {
            let s = pre[b] - pre[a - 1];
            if s >= 100_000_000 {
                break;
            }
            if palindrome(s, 10) {
                set.insert(s);
```

Factor. Source: `solutions/factor/euler/extended/extended.factor`.

```
:: p125 ( -- n )
```

```
    10001 0 <array> :> prefix
    1 10000 [a..b] [| n | n 1 - prefix nth n sq + n prefix set-nth ] each
    H{ } clone :> found
    1 9999 [a..b] [| first |
        first 1 + :> last!
        [ last 10000 <= ] [
            last prefix nth first 1 - prefix nth - :> value
            value 100000000 >= [ 10001 last! ] [
                value decimal-palindrome? [ t value found set-at ] when
                last 1 + last!
            ] if
        ] while
    ] each found keys sum ;
```

Comparison. C exposes sorting or set deduplication. Rust exposes sorting or set deduplication. Factor exposes prime generation or testing, and sorting or set deduplication. Shared calls: `decimal-palindrome?`. No validated source exists for Gforth. All listed programs print the same exact answer string from the manifest. The excerpts can differ in algorithm, data representation, and startup cost.

40.126 Problem 126: Cuboid Layers

The program computes the exact answer for Cuboid Layers.

C. Source: `solutions/c/cuboid_layers_126.c`.

```
int main(void) {
    enum { LIMIT = 30000, TARGET = 1000 };
    unsigned *count = calloc(LIMIT, sizeof(*count));
    if (!count) return EXIT_FAILURE;
    for (unsigned a = 1; 6 * a * a < LIMIT; ++a) {
        for (unsigned b = a; 2 * (a * b + a * b + b * b) < LIMIT; ++b) {
            for (unsigned c = b;; ++c) {
                unsigned first = 2 * (a * b + a * c + b * c);
                if (first >= LIMIT) break;
                for (unsigned layer = 1, cubes = first; cubes < LIMIT; ++layer) {
                    ++count[cubes];
                    cubes += 4 * (a + b + c) + 8 * (layer - 1);
                }
            }
```

Rust. Source: `solutions/rust/src/bin/p126.rs`.

```
fn solve() -> usize {
    const L: usize = 30_000;
    let mut count = vec![0u16; L];
    for a in 1.. {
        if 6 * a * a >= L {
            break;
        }
        for b in a.. {
            if 2 * (a * b + a * b + b * b) >= L {
                break;
```

```
            }
            for c in b.. {
                let first = 2 * (a * b + a * c + b * c);
                if first >= L {
```

Comparison. C exposes direct bounded arithmetic. Rust exposes direct bounded arithmetic. No validated source exists for Gforth, and Factor. All listed programs print the same exact answer string from the manifest. The excerpts can differ in algorithm, data representation, and startup cost.

40.127 Problem 127: abc-hits

The program computes the exact answer for abc-hits.

C. Source: `solutions/c/abc_hits_127.c`.

```
static int by_radical(const void *left, const void *right) {
    const struct radical_number *a = left, *b = right;
    if (a->radical != b->radical) return a->radical < b->radical ? -1 : 1;
    return a->number < b->number ? -1 : a->number != b->number;
}

int main(void) {
    enum { LIMIT = 120000 };
    unsigned *rad = malloc(LIMIT * sizeof(*rad));
    struct radical_number *ordered = malloc(LIMIT * sizeof(*ordered));
    if (!rad || !ordered) { free(rad); free(ordered); return EXIT_FAILURE; }
    for (unsigned n = 0; n < LIMIT; ++n) rad[n] = 1;
    for (unsigned p = 2; p < LIMIT; ++p)
        if (rad[p] == 1)
```

Rust. Source: `solutions/rust/src/bin/p127.rs`.

```
fn solve() -> u64 {
    const L: usize = 120_000;
    let mut rad = vec![1usize; L];
    for p in 2..L {
        if rad[p] == 1 {
            for n in (p..L).step_by(p) {
                rad[n] *= p;
            }
        }
    }
    let mut ordered: Vec<_> = (1..L).map(|n| (rad[n], n)).collect();
    ordered.sort_unstable();
    let mut total = 0;
    for c in 3..L {
```

Comparison. C exposes sorting or set deduplication. Shared calls: `euler_gcd`. Rust exposes sorting or set deduplication. No validated source exists for Gforth, and Factor. All listed programs print the same exact answer string from the manifest. The excerpts can differ in algorithm, data representation, and startup cost.

40.128 Problem 128: Hexagonal Tile Differences

The program computes the exact answer for Hexagonal Tile Differences.

C. Source: `solutions/c/hexagonal_tile_differences_128.c`.

```
int main(void) {
    unsigned found = 1;
    uint64_t answer = 1;
    for (uint64_t n = 1; found < 2000; ++n) {
        if (euler_is_prime(6 * n - 1) && euler_is_prime(6 * n + 1) &&
            euler_is_prime(12 * n + 5)) {
            answer = 3 * n * (n - 1) + 2;
            ++found;
            if (found == 2000) break;
        }
        if (n > 1 && euler_is_prime(6 * n - 1) && euler_is_prime(6 * n + 5) &&
            euler_is_prime(12 * n - 7)) {
            answer = 3 * n * (n + 1) + 1;
            ++found;
```

Rust. Source: `solutions/rust/src/bin/p128.rs`.

```
fn solve() -> u64 {
    let (mut found, mut answer) = (1, 1);
    for n in 1u64.. {
        if prime64(6 * n - 1) && prime64(6 * n + 1) && prime64(12 * n + 5) {
            answer = 3 * n * (n - 1) + 2;
            found += 1;
            if found == 2000 {
                break;
            }
        }
        if n > 1 && prime64(6 * n - 1) && prime64(6 * n + 5) && prime64(12 * n - 7) {
            answer = 3 * n * (n + 1) + 1;
            found += 1;
            if found == 2000 {
```

Factor. Source: `solutions/factor/euler/extended/extended.factor`.

```
:: p128 ( -- n )
    1 :> count! 1 :> answer! 1 :> ring!
    [ count 2000 < ] [
        ring 6 * 1 - prime? ring 6 * 1 + prime? and
        ring 12 * 5 + prime? and [
            count 1 + count!
            ring 3 * ring 1 - * 2 + answer!
        ] when
        ring 1 > [
            ring 6 * 1 - prime? ring 6 * 5 + prime? and
            ring 12 * 7 - prime? and [
                count 1 + count!
                ring 3 * ring 1 + * 1 + answer!
            ] when
```

Comparison. C exposes prime generation or testing. Rust exposes prime generation or testing. Factor exposes prime generation or testing. No validated source exists for Gforth. All listed programs print the same exact answer string from the manifest. The excerpts can differ in algorithm, data representation, and startup cost.

40.129 Problem 129: Repunit Divisibility

A(k) is found without constructing the k-digit repunit. If r is the current remainder modulo n, appending a decimal 1 changes it to (10r+1) mod n. For gcd(n,10)=1 a zero remainder must eventually occur, and the iteration count is exactly A(n). The answer exceeds one million and must be odd and not divisible by five, so the scan starts immediately above the bound, skips even values, and rejects multiples of five before running the remainder recurrence. Every remainder fits in ‘unsigned‘; 10r+1 also fits for this search range.

C. Source: `solutions/c/repunit_divisibility_129.c`.

```
static unsigned repunit_length(unsigned n) {
    unsigned remainder = 1 % n, length = 1;
    while (remainder) { remainder = (10 * remainder + 1) % n; ++length; }
    return length;
}

int main(void) {
    for (unsigned n = 1000001;; n += 2) {
        if (n % 5 && repunit_length(n) > 1000000) {
            printf("%u\n", n);
            return 0;
        }
    }
}
```

Rust. Source: `solutions/rust/src/bin/p129.rs`.

```
fn a(n: u64) -> u64 {
    let (mut r, mut k) = (1 % n, 1);
    while r != 0 {
        r = (10 * r + 1) % n;
        k += 1
    }
    k
}
fn solve() -> u64 {
    (1_000_001u64..)
        .step_by(2)
        .find(|&n| n % 5 != 0 && a(n) > 1_000_000)
        .unwrap()
}
```

Factor. Source: `solutions/factor/euler/extended/extended.factor`.

```
:: p129 ( -- n )
    1000001 :> n!
```

```
    [ n 10 gcd nip 1 = [ n repunit-length 1000000 > ] [ f ] if not ]
    [ n 1 + n! ] while n ;
```

Comparison. C exposes modular arithmetic. Shared calls: `gcd`. Rust exposes direct bounded arithmetic. Factor exposes direct bounded arithmetic. No validated source exists for Gforth. All listed programs print the same exact answer string from the manifest. The excerpts can differ in algorithm, data representation, and startup cost.

40.130 Problem 130: Composites with Prime Repunit Property

Repeatedly applying r <- (10r+1) mod n computes A(n), the least repunit length divisible by n, without storing that repunit. The requested composite analogue of Fermat's property is precisely A(n) | n-1. Only numbers coprime to 10 can divide a repunit, so the outer loop visits odd values and excludes multiples of five. Primes are excluded explicitly; qualifying composites are accumulated until the first 25 have been found.

C. Source: `solutions/c/composites_with_prime_repunit_property_130.c`.

```c
static unsigned repunit_length(unsigned n) {
    unsigned remainder = 1 % n, length = 1;
    while (remainder) { remainder = (10 * remainder + 1) % n; ++length; }
    return length;
}

int main(void) {
    unsigned count = 0, sum = 0;
    for (unsigned n = 3; count < 25; n += 2) {
        if (n % 5 == 0 || euler_is_prime(n)) continue;
        if ((n - 1) % repunit_length(n) == 0) { sum += n; ++count; }
    }
    printf("%u\n", sum);
    return 0;
```

Rust. Source: `solutions/rust/src/bin/p130.rs`.

```rust
fn a(n: u64) -> u64 {
    let (mut r, mut k) = (1 % n, 1);
    while r != 0 {
        r = (10 * r + 1) % n;
        k += 1
    }
    k
}
fn solve() -> u64 {
    let (mut found, mut sum) = (0, 0);
    for n in (3u64..).step_by(2) {
        if n % 5 != 0 && !prime64(n) && (n - 1) % a(n) == 0 {
            sum += n;
            found += 1;
```

Factor. Source: `solutions/factor/euler/extended/extended.factor`.

```
:: p130 ( -- n )
    91 :> n! 0 :> found! 0 :> total!
    [ found 25 < ] [
        n 10 gcd nip 1 = n prime? not and [
            n 1 - n repunit-length mod zero? [
                total n + total! found 1 + found!
            ] when
        ] when
        n 2 + n!
    ] while total ;

! Problem 131: consecutive cubes yield p=3n^2+3n+1.
```

Comparison. C exposes prime generation or testing, and modular arithmetic. Shared calls: `euler_is_prime`. Rust exposes prime generation or testing. Shared calls: `prime64`. Factor exposes prime generation or testing, and modular arithmetic. No validated source exists for Gforth. All listed programs print the same exact answer string from the manifest. The excerpts can differ in algorithm, data representation, and startup cost.

40.131 Problem 131: Prime Cube Partnership

From n^3+p n^2=m^3, the two cubes must be consecutive: writing m=n+k and comparing factors shows the prime case forces k=1. The possible primes are therefore p=(n+1)^3-n^3 = 3n^2+3n+1. Generate this increasing quadratic sequence, stop at one million, and test each term for primality. This replaces a three-variable search by a short one-dimensional enumeration.

C. Source: `solutions/c/prime_cube_partnership_131.c`.

```
int main(void) {
    unsigned count = 0;
    for (uint64_t n = 1;; ++n) {
        uint64_t p = 3 * n * n + 3 * n + 1;
        if (p >= 1000000) break;
        if (euler_is_prime(p)) ++count;
    }
    printf("%u\n", count);
    return 0;
}
```

Rust. Source: `solutions/rust/src/bin/p131.rs`.

```
fn solve() -> usize {
    (1u64..)
        .map(|n| 3 * n * n + 3 * n + 1)
        .take_while(|&p| p < 1_000_000)
        .filter(|&p| prime64(p))
        .count()
}
fn main() {
    println!("{}", solve());
```

```
}
```

Factor. Source: `solutions/factor/euler/extended/extended.factor`.

```
:: p131 ( -- n )
    0 :> count! 1 :> n!
    [ n sq 3 * n 3 * + 1 + 1000000 < ] [
        n sq 3 * n 3 * + 1 + prime? [ count 1 + count! ] when
        n 1 + n!
    ] while count ;

! Problems 137 and 138 are the Pell recurrences obtained after eliminating
! the square-root conditions.  The loops still derive every requested term.
```

Comparison. C exposes prime generation or testing. Shared calls: `euler_is_prime`. Rust exposes prime generation or testing. Shared calls: `prime64`. Factor exposes prime generation or testing. No validated source exists for Gforth. All listed programs print the same exact answer string from the manifest. The excerpts can differ in algorithm, data representation, and startup cost.

40.132 Problem 132: Large Repunit Factors

R(k)=(10^k-1)/9. For a prime p other than 2 or 5, p | R(k) <=> 10^k == 1 (mod 9p). The modulus 9p retains the division by nine exactly, avoiding an enormous billion-digit repunit. Binary modular exponentiation in ‘euler_powmod‘ tests each prime in O(log k) multiplications. We scan primes in increasing order, add the first 40 factors, and use 64-bit intermediates for 9p.

C. Source: `solutions/c/large_repunit_factors_132.c`.

```
int main(void) {
    unsigned found = 0, sum = 0;
    for (unsigned p = 2; found < 40; ++p) {
        if (!euler_is_prime(p) || p == 2 || p == 5) continue;
        if (euler_powmod(10, 1000000000ULL, 9ULL * p) == 1) {
            sum += p;
            ++found;
        }
    }
    printf("%u\n", sum);
    return 0;
}
```

Rust. Source: `solutions/rust/src/bin/p132.rs`.

```
fn solve() -> u64 {
    (2u64..)
        .filter(|&p| p != 2 && p != 5 && prime64(p) && mod_pow(10, 1_000_000_000, 9 * p) == 1)
        .take(40)
        .sum()
}
fn main() {
    println!("{}", solve());
```

```
}
```

Factor. Source: `solutions/factor/euler/extended/extended.factor`.

```
:: p132 ( -- n )
    0 :> found! 0 :> answer! 2 :> p!
    [ found 40 < ] [
        p 2 = p 5 = or not p prime? and [
            10 1000000000 p 9 * modular-pow 1 = [
                answer p + answer! found 1 + found!
            ] when
        ] when p 1 + p!
    ] while answer ;

! Problem 133: repeated tenth powers test whether a prime can ever divide a
! repunit whose length is a power of ten.
```

Comparison. C exposes prime generation or testing, and modular arithmetic. Shared calls: `euler_is_prime`, `euler_powmod`. Rust exposes prime generation or testing, and modular arithmetic. Shared calls: `mod_pow`, `prime64`. Factor exposes prime generation or testing, and modular arithmetic. Shared calls: `modular-pow`. No validated source exists for Gforth. All listed programs print the same exact answer string from the manifest. The excerpts can differ in algorithm, data representation, and startup cost.

40.133 Problem 133: Repunit Nonfactors

The program computes the exact answer for Repunit Nonfactors.

C. Source: `solutions/c/repunit_nonfactors_133.c`.

```
int main(void) {
    uint64_t sum = 0;
    for (unsigned p = 2; p < 100000; ++p) {
        if (!euler_is_prime(p)) continue;
        uint64_t residue = 10 % (9ULL * p);
        for (unsigned n = 0; n < 20; ++n)
            residue = euler_powmod(residue, 10, 9ULL * p);
        if (residue != 1) sum += p;
    }
    printf("%" PRIu64 "\n", sum);
    return 0;
}
```

Rust. Source: `solutions/rust/src/bin/p133.rs`.

```
fn solve() -> u64 {
    (2u64..100_000)
        .filter(|&p| prime64(p))
        .filter(|&p| {
            let m = 9 * p;
            let mut r = 10 % m;
```

```
            for _ in 0..20 {
                r = mod_pow(r, 10, m)
            }
            r != 1
        })
        .sum()
}
fn main() {
```

Factor. Source: `solutions/factor/euler/extended/extended.factor`.

```
:: p133 ( -- n )
    0 :> answer!
    2 99999 [a..b] [| p |
        p prime? [
            p 9 * :> modulus 10 modulus mod :> residue!
            20 [ residue 10 modulus modular-pow residue! ] times
            residue 1 = not [ answer p + answer! ] when
        ] when
    ] each answer ;

! Problem 157: isolate powers of two and five, then count divisor choices in
! the remaining coprime factor.  The second term covers both nonzero exponents.
```

Comparison. C exposes prime generation or testing, and modular arithmetic. Shared calls: `euler_is_prime`, `euler_powmod`. Rust exposes prime generation or testing, and modular arithmetic. Shared calls: `mod_pow`, `prime64`. Factor exposes prime generation or testing, and modular arithmetic. Shared calls: `modular-pow`. No validated source exists for Gforth. All listed programs print the same exact answer string from the manifest. The excerpts can differ in algorithm, data representation, and startup cost.

40.134 Problem 134: Prime Pair Connection

The program computes the exact answer for Prime Pair Connection.

C. Source: `solutions/c/prime_pair_connection_134.c`.

```
static uint64_t inverse_mod(uint64_t a, uint64_t modulus) {
    int64_t old_r = (int64_t)a, r = (int64_t)modulus;
    int64_t old_s = 1, s = 0;
    while (r) {
        int64_t q = old_r / r;
        int64_t next = old_r - q * r; old_r = r; r = next;
        next = old_s - q * s; old_s = s; s = next;
    }
    if (old_s < 0) old_s += (int64_t)modulus;
    return (uint64_t)old_s;
}

int main(void) {
    uint64_t total = 0;
```

Rust. Source: `solutions/rust/src/bin/p134.rs`.

```
fn inv(mut a: i64, mut m: i64) -> u64 {
    let modulus = m;
    let (mut x0, mut x1) = (1i64, 0i64);
    while m != 0 {
        let q = a / m;
        (a, m) = (m, a - q * m);
        (x0, x1) = (x1, x0 - q * x1);
    }
    x0.rem_euclid(modulus) as u64
}
fn solve() -> u64 {
    let ps: Vec<_> = sieve(1_100_000)
        .iter()
        .enumerate()
```

Factor. Source: `solutions/factor/p134.factor`.

```
! Native Factor Project Euler 134 solver; computes prime-pair suffixes.
USING: io math.parser project-euler.134 ; IN: scratchpad
euler134 number>string print
```

Comparison. C exposes prime generation or testing. Shared calls: `euler_is_prime`. Rust exposes prime generation or testing, and modular arithmetic. Shared calls: `sieve`. Factor exposes prime generation or testing. No validated source exists for Gforth. All listed programs print the same exact answer string from the manifest. The excerpts can differ in algorithm, data representation, and startup cost.

40.135 Problem 135: Same Differences

The program computes the exact answer for Same Differences.

C. Source: `solutions/c/same_differences_135.c`.

```
int main(void) {
    enum { LIMIT = 1000000 };
    unsigned char *representations = calloc(LIMIT, 1);
    if (!representations) return EXIT_FAILURE;
    for (unsigned u = 1; u < LIMIT; ++u) {
        for (unsigned v = (u + 2) / 3; (uint64_t)u * v < LIMIT; ++v) {
            if (u >= 3 * v || ((u + v) & 3U)) continue;
            unsigned n = u * v;
            if (representations[n] < 11) ++representations[n];
        }
    }
    unsigned count = 0;
    for (unsigned n = 1; n < LIMIT; ++n) count += representations[n] == 10;
    printf("%u\n", count);
```

Rust. Source: `solutions/rust/src/bin/p135.rs`.

```
fn solve() -> usize {
    const L: usize = 1_000_000;
    let mut reps = vec![0u8; L];
    for u in 1..L {
        let mut v = (u + 2) / 3;
        while u * v < L {
            if u < 3 * v && (u + v) % 4 == 0 {
                reps[u * v] = (reps[u * v] + 1).min(11)
            }
            v += 1
        }
    }
    reps.into_iter().filter(|&x| x == 10).count()
}
```

Factor. Source: `solutions/factor/euler/extended/extended.factor`.

```
:: p135 ( -- n )
    1000000 0 <array> :> representations
    1 999999 [a..b] [| u |
        u 2 + 3 /i :> v!
        [ u v * 1000000 < ] [
            u 3 v * < u v + 4 mod zero? and [
                u v * :> product
                product representations nth 1 + 11 min product representations set-nth
            ] when v 1 + v!
        ] while
    ] each representations [ 10 = ] count ;

! Problem 140: all golden nuggets lie in six Pell orbits of x^2-5y^2=44.
```

Comparison. C exposes direct bounded arithmetic. Rust exposes direct bounded arithmetic. Factor exposes modular arithmetic. No validated source exists for Gforth. All listed programs print the same exact answer string from the manifest. The excerpts can differ in algorithm, data representation, and startup cost.

40.136 Problem 136: Singleton Difference

The program computes the exact answer for Singleton Difference.

C. Source: `solutions/c/singleton_difference_136.c`.

```
int main(void) {
    enum { LIMIT = 50000000 };
    unsigned char *count = calloc(LIMIT, 1);
    if (!count) return EXIT_FAILURE;
    for (unsigned u = 1; (uint64_t)u * (u / 3 + 1) < LIMIT; ++u) {
        unsigned v = u / 3 + 1;
        v += (4 - (u + v) % 4) % 4;
        for (; (uint64_t)u * v < LIMIT; v += 4) {
            unsigned n = u * v;
            if (count[n] < 2) ++count[n];
```

```
        }
    }
    unsigned answer = 0;
    for (unsigned n = 1; n < LIMIT; ++n) answer += count[n] == 1;
```

Rust. Source: `solutions/rust/src/bin/p136.rs`.

```
fn solve() -> usize {
    const L: usize = 50_000_000;
    let mut count = vec![0u8; L];
    let mut u = 1;
    while u * (u / 3 + 1) < L {
        let mut v = u / 3 + 1;
        v += (4 - (u + v) % 4) % 4;
        while u * v < L {
            let n = u * v;
            count[n] = (count[n] + 1).min(2);
            v += 4
        }
        u += 1
    }
```

Comparison. C exposes direct bounded arithmetic. Rust exposes direct bounded arithmetic. No validated source exists for Gforth, and Factor. All listed programs print the same exact answer string from the manifest. The excerpts can differ in algorithm, data representation, and startup cost.

40.137 Problem 137: Fibonacci Golden Nuggets

Requiring x/(1-x-x^2) to be integral turns its discriminant condition into a Pell equation. Taking successive positive Pell solutions yields the nuggets directly, with recurrence A_1=2, A_2=15, A_n=7A_(n-1)-A_(n-2)+1. Advancing through the fifteenth term avoids floating-point roots and tests no non-candidates. The requested term fits uint64_t.

C. Source: `solutions/c/fibonacci_golden_nuggets_137.c`.

```
int main(void) {
    uint64_t previous = 2, current = 15;
    for (unsigned index = 2; index < 15; ++index) {
        uint64_t next = 7 * current - previous + 1;
        previous = current;
        current = next;
    }
    printf("%" PRIu64 "\n", current);
    return 0;
}
```

Rust. Source: `solutions/rust/src/bin/p137.rs`.

```
fn solve() -> u64 {
    let (mut a, mut b) = (2u64, 15u64);
    for _ in 2..15 {
```

```
        (a, b) = (b, 7 * b - a + 1)
    }
    b
}
fn main() {
    println!("{}", solve());
}
```

Factor. Source: `solutions/factor/euler/extended/extended.factor`.

```
:: p137 ( -- n )
    2 :> a! 15 :> b!
    13 [
        b :> next-a
        b 7 * a - 1 + :> next-b
        next-a a! next-b b!
    ] times b ;
```

Comparison. C exposes direct bounded arithmetic. Rust exposes direct bounded arithmetic. Factor exposes direct bounded arithmetic. No validated source exists for Gforth. All listed programs print the same exact answer string from the manifest. The excerpts can differ in algorithm, data representation, and startup cost.

40.138 Problem 138: Special Isosceles Triangles

Substituting h=b+-1 into L^2=h^2+(b/2)^2 reduces the integer-triangle condition to a Pell equation. Ordering its positive solutions gives the linear recurrence L_0=1, L_1=17, L_(n+1)=18L_n-L_(n-1). The degenerate seed 1 is used only to advance the recurrence; the twelve requested lengths begin at 17. Their sum remains below 2^64.

C. Source: `solutions/c/special_isosceles_triangles_138.c`.

```
int main(void) {
    uint64_t previous = 1, current = 17, sum = current;
    for (unsigned index = 1; index < 12; ++index) {
        uint64_t next = 18 * current - previous;
        previous = current;
        current = next;
        sum += current;
    }
    printf("%" PRIu64 "\n", sum);
    return 0;
}
```

Rust. Source: `solutions/rust/src/bin/p138.rs`.

```
fn solve() -> u64 {
    let (mut a, mut b, mut sum) = (1u64, 17u64, 17u64);
    for _ in 1..12 {
        (a, b) = (b, 18 * b - a);
        sum += b
```

```
    }
    sum
}
fn main() {
    println!("{}", solve());
}
```

Factor. Source: `solutions/factor/euler/extended/extended.factor`.

```
:: p138 ( -- n )
    1 :> a! 17 :> b! 17 :> total!
    11 [
        b :> next-a
        b 18 * a - :> next-b
        next-a a! next-b b!
        total b + total!
    ] times total ;

! Problem 139: primitive Euclidean triples tile exactly when c is divisible
! by the difference of the legs; scaled perimeters are then counted at once.
```

Comparison. C exposes direct bounded arithmetic. Rust exposes direct bounded arithmetic. Factor exposes direct bounded arithmetic. No validated source exists for Gforth. All listed programs print the same exact answer string from the manifest. The excerpts can differ in algorithm, data representation, and startup cost.

40.139 Problem 139: Pythagorean Tiles

The program computes the exact answer for Pythagorean Tiles.

C. Source: `solutions/c/pythagorean_tiles_139.c`.

```
int main(void) {
    enum { LIMIT = 100000000 };
    uint64_t count = 0;
    for (unsigned m = 2; 2ULL * m * (m + 1) < LIMIT; ++m) {
        for (unsigned n = 1; n < m; ++n) {
            if (((m - n) & 1U) == 0 || euler_gcd(m, n) != 1) continue;
            uint64_t a = (uint64_t)m * m - (uint64_t)n * n;
            uint64_t b = 2ULL * m * n;
            uint64_t c = (uint64_t)m * m + (uint64_t)n * n;
            uint64_t difference = a > b ? a - b : b - a;
            if (c % difference == 0) count += (LIMIT - 1) / (a + b + c);
        }
    }
    printf("%" PRIu64 "\n", count);
```

Rust. Source: `solutions/rust/src/bin/p139.rs`.

```
fn solve() -> u64 {
    const L: u64 = 100_000_000;
```

```
    let mut count = 0;
    for m in 2u64.. {
        if 2 * m * (m + 1) >= L {
            break;
        }
        for n in 1..m {
            if (m - n) % 2 == 0 || gcd(m, n) != 1 {
                continue;
            }
            let (a, b, c) = (m * m - n * n, 2 * m * n, m * m + n * n);
            if c % a.abs_diff(b) == 0 {
                count += (L - 1) / (a + b + c)
```

Factor. Source: `solutions/factor/euler/extended/extended.factor`.

```
:: p139 ( -- n )
    0 :> total! 2 :> m!
    [ m m 1 + * 2 * 100000000 < ] [
        1 m 1 - [a..b] [| n |
            m n - odd? m n gcd nip 1 = and [
                m sq n sq - :> a
                m n * 2 * :> b
                m sq n sq + :> c
                c a b - abs mod zero? [
                    total 99999999 a b + c + /i + total!
                ] when
            ] when
        ] each
        m 1 + m!
```

Comparison. C exposes direct bounded arithmetic. Shared calls: `euler_gcd`. Rust exposes direct bounded arithmetic. Shared calls: `gcd`. Factor exposes modular arithmetic. No validated source exists for Gforth. All listed programs print the same exact answer string from the manifest. The excerpts can differ in algorithm, data representation, and startup cost.

40.140 Problem 140: Modified Fibonacci Golden Nuggets

With x=5n+7, integral nuggets solve x^2-5y^2=44. The six reduced signed seeds generate every positive orbit under multiplication by 9+4sqrt(5).

C. Source: `solutions/c/modified_fibonacci_golden_nuggets_140.c`.

```
static int compare_u64(const void *left, const void *right) {
    uint64_t a = *(const uint64_t *)left, b = *(const uint64_t *)right;
    return a < b ? -1 : a != b;
}

int main(void) {
    const int64_t seeds[6][2] = {{7,1},{7,-1},{8,2},{8,-2},{13,5},{13,-5}};
    uint64_t values[240];
    unsigned count = 0;
    for (unsigned seed = 0; seed < 6; ++seed) {
        int64_t x = seeds[seed][0], y = seeds[seed][1];
```

```
        for (unsigned step = 0; step < 35; ++step) {
            if (x > 7 && (x - 7) % 5 == 0) values[count++] = (uint64_t)(x - 7) / 5;
            if (x > INT64_MAX / 29 || y > INT64_MAX / 29) break;
```

Rust. Source: `solutions/rust/src/bin/p140.rs`.

```
fn solve() -> u64 {
    let seeds = [(7i128, 1i128), (7, -1), (8, 2), (8, -2), (13, 5), (13, -5)];
    let mut v = Vec::new();
    for (mut x, mut y) in seeds {
        for _ in 0..35 {
            if x > 7 && (x - 7) % 5 == 0 {
                v.push(((x - 7) / 5) as u64)
            }
            (x, y) = (9 * x + 20 * y, 4 * x + 9 * y);
        }
    }
    v.sort_unstable();
    v.dedup();
    v.into_iter().take(30).sum()
```

Factor. Source: `solutions/factor/euler/extended/extended.factor`.

```
:: p140 ( -- n )
    { { 7 1 } { 7 -1 } { 8 2 } { 8 -2 } { 13 5 } { 13 -5 } } :> seeds
    V{ } clone :> values
    seeds [| seed |
        seed first :> x! seed second :> y!
        35 [
            x 7 > x 7 - 5 mod zero? and [ x 7 - 5 /i values push ] when
            x 9 * y 20 * + :> next-x
            x 4 * y 9 * + :> next-y
            next-x x! next-y y!
        ] times
    ] each values sort members 30 head sum ;

! Problem 144: reflect the incoming vector across the ellipse normal and find
```

Comparison. C exposes sorting or set deduplication. Rust exposes sorting or set deduplication. Factor exposes modular arithmetic, and sorting or set deduplication. No validated source exists for Gforth. All listed programs print the same exact answer string from the manifest. The excerpts can differ in algorithm, data representation, and startup cost.

40.141 Problem 141: Square Progressive Numbers

Coprime b>c parameterize d=a*b^2, q=a*b*c, r=a*c^2, hence n=d*q+r=a*c*(a*b^3+c).

C. Source: `solutions/c/square_progressive_numbers_141.c`.

```
static bool square141(uint64_t n) {
    uint64_t r = (uint64_t)sqrt((double)n);
```

```
    while ((r + 1) * (r + 1) <= n) ++r;
    while (r * r > n) --r;
    return r * r == n;
}
static int compare141(const void *l, const void *r) {
    uint64_t a=*(const uint64_t*)l,b=*(const uint64_t*)r;
    return a<b?-1:a!=b;
}
int main(void) {
    const uint64_t limit = 1000000000000ULL;
    uint64_t found[1024]; unsigned count=0;
    for (uint64_t b=2; b*b*b < limit; ++b) {
```

Rust. Source: `solutions/rust/src/bin/p141.rs`.

```
fn solve() -> u64 {
    let mut found = Vec::new();
    for b in 2u64.. {
        let b3 = b * b * b;
        if b3 >= 1_000_000_000_000 {
            break;
        }
        for c in 1..b {
            if gcd(b, c) != 1 {
                continue;
            }
            for a in 1u64.. {
                let Some(n) = a.checked_mul(c).and_then(|x| x.checked_mul(a * b3 + c)) else {
                    break;
```

Comparison. C exposes prime generation or testing, and sorting or set deduplication. Shared calls: `euler_gcd`. Rust exposes prime generation or testing, and sorting or set deduplication. No validated source exists for Gforth, and Factor. All listed programs print the same exact answer string from the manifest. The excerpts can differ in algorithm, data representation, and startup cost.

40.142 Problem 142: Perfect Square Collection

The program computes the exact answer for Perfect Square Collection.

C. Source: `solutions/c/perfect_square_collection_142.c`.

```
static bool square_u64(uint64_t n) {
    uint64_t root = (uint64_t)sqrt((double)n);
    while ((root + 1) * (root + 1) <= n) ++root;
    while (root * root > n) --root;
    return root * root == n;
}

int main(void) {
    uint64_t best = UINT64_MAX;
    for (uint64_t ar = 2; ar * ar < best; ++ar) {
        uint64_t a = ar * ar;
        for (uint64_t br = (ar & 1) ? 1 : 2; br < ar; br += 2) {
```

```
            uint64_t b = br * br, x = (a + b) / 2, y = (a - b) / 2;
            if (x + y >= best) continue;
```

Rust. Source: `solutions/rust/src/bin/p142.rs`.

```
fn solve() -> u64 {
    let mut best = u64::MAX;
    let mut ar = 2;
    while ar * ar < best {
        let a = ar * ar;
        for br in ((if ar % 2 == 1 { 1 } else { 2 })..ar).step_by(2) {
            let b = br * br;
            let (x, y) = ((a + b) / 2, (a - b) / 2);
            if x + y >= best {
                continue;
            }
            for dr in 1.. {
                let d = dr * dr;
                if d >= x {
```

Comparison. C exposes direct bounded arithmetic. Rust exposes direct bounded arithmetic. No validated source exists for Gforth, and Factor. All listed programs print the same exact answer string from the manifest. The excerpts can differ in algorithm, data representation, and startup cost.

40.143 Problem 143: Torricelli Triangles

Compatible Fermat distances p,q satisfy p^2+pq+q^2=square. Primitive Eisenstein triples generate the graph; Torricelli triples are its 3-cycles.

C. Source: `solutions/c/torricelli_triangles_143.c`.

```
enum { LIMIT143 = 120000 };
struct neighbors { unsigned *items, count, capacity; };
static struct neighbors graph143[LIMIT143 + 1];

static void add_neighbor(unsigned from,unsigned to){
    struct neighbors *list=&graph143[from];
    if(list->count==list->capacity){
        unsigned capacity=list->capacity?2*list->capacity:4;
        unsigned *items=realloc(list->items,capacity*sizeof(*items));
        if(!items){fputs("allocation failure\n",stderr);exit(EXIT_FAILURE);}
        list->items=items;list->capacity=capacity;
    }
    list->items[list->count++]=to;
}
```

Rust. Source: `solutions/rust/src/bin/p143.rs`.

```
fn solve() -> u64 {
    const L: usize = 120_000;
    let mut g = vec![Vec::<usize>::new(); L + 1];
```

```
    for m in 2usize.. {
        if m * m + 2 * m >= L {
            break;
        }
        for n in 1..m {
            if gcd(m as u64, n as u64) != 1 {
                continue;
            }
            let (mut p, mut q) = (m * m - n * n, 2 * m * n + n * n);
            if (m - n) % 3 == 0 {
                p /= 3;
```

Comparison. C exposes sorting or set deduplication. Shared calls: `euler_gcd`. Rust exposes sorting or set deduplication. No validated source exists for Gforth, and Factor. All listed programs print the same exact answer string from the manifest. The excerpts can differ in algorithm, data representation, and startup cost.

40.144 Problem 144: Laser Beam Reflections

The program computes the exact answer for Laser Beam Reflections.

C. Source: `solutions/c/laser_beam_reflections_144.c`.

```
int main(void) {
    double x0 = 0.0, y0 = 10.1, x = 1.4, y = -9.6;
    unsigned reflections = 0;
    for (;;) {
        double vx = x - x0, vy = y - y0;
        double nx = 4 * x, ny = y;
        double scale = 2 * (vx * nx + vy * ny) / (nx * nx + ny * ny);
        double rx = vx - scale * nx, ry = vy - scale * ny;
        double qa = 4 * rx * rx + ry * ry;
        double qb = 8 * x * rx + 2 * y * ry;
        double t = -qb / qa;
        x0 = x; y0 = y; x += t * rx; y += t * ry;
        ++reflections;
        if (fabs(x) <= 0.01 && y > 0) break;
```

Rust. Source: `solutions/rust/src/bin/p144.rs`.

```
fn solve() -> u32 {
    let (mut x0, mut y0, mut x, mut y) = (0.0f64, 10.1, 1.4, -9.6);
    let mut count = 0;
    loop {
        let (vx, vy) = (x - x0, y - y0);
        let (nx, ny) = (4.0 * x, y);
        let scale = 2.0 * (vx * nx + vy * ny) / (nx * nx + ny * ny);
        let (rx, ry) = (vx - scale * nx, vy - scale * ny);
        let t = -(8.0 * x * rx + 2.0 * y * ry) / (4.0 * rx * rx + ry * ry);
        (x0, y0) = (x, y);
        x += t * rx;
        y += t * ry;
        count += 1;
```

```
        if x.abs() <= 0.01 && y > 0.0 {
```

Factor. Source: `solutions/factor/euler/extended/extended.factor`.

```
:: p144 ( -- n )
    0.0 :> x0! 10.1 :> y0! 1.4 :> x! -9.6 :> y! 0 :> count!
    t :> running!
    [ running ] [
        x x0 - :> vx y y0 - :> vy x 4.0 * :> nx y :> ny
        vx nx * vy ny * + 2.0 * nx sq ny sq + / :> scale
        vx scale nx * - :> rx vy scale ny * - :> ry
        x rx * 8.0 * y ry * 2.0 * + neg
        rx sq 4.0 * ry sq + / :> time
        x x0! y y0! x time rx * + x! y time ry * + y!
        count 1 + count!
        x abs 0.01 <= y 0.0 > and [ f running! ] when
    ] while count ;
```

Comparison. C exposes direct bounded arithmetic. Rust exposes direct bounded arithmetic. Factor exposes direct bounded arithmetic. No validated source exists for Gforth. All listed programs print the same exact answer string from the manifest. The excerpts can differ in algorithm, data representation, and startup cost.

40.145 Problem 145: Reversible Numbers

Carry analysis by decimal length gives 20*30^(k-1) for 2k digits, 100*500^k for 4k+3 digits, and zero for 4k+1 digits.

C. Source: `solutions/c/reversible_numbers_145.c`.

```
int main(void) {
    uint64_t total = 0, even = 20;
    for (unsigned digits = 2; digits <= 8; digits += 2) {
        total += even;
        even *= 30;
    }
    uint64_t odd = 100;
    for (unsigned digits = 3; digits <= 7; digits += 4) {
        total += odd;
        odd *= 500;
    }
    printf("%" PRIu64 "\n", total);
    return 0;
}
```

Rust. Source: `solutions/rust/src/bin/p145.rs`.

```
fn solve() -> u64 {
    let (mut total, mut even) = (0, 20);
    for _ in (2..=8).step_by(2) {
        total += even;
```

```
        even *= 30
    }
    let mut odd = 100;
    for _ in (3..=7).step_by(4) {
        total += odd;
        odd *= 500
    }
    total
}
fn main() {
```

Factor. Source: `solutions/factor/euler/extended/extended.factor`.

```
:: p145 ( -- n )
    0 :> total! 20 :> even!
    4 [ total even + total! even 30 * even! ] times
    100 :> odd! 2 [ total odd + total! odd 500 * odd! ] times total ;

! Problem 149: generate the 2000^2 lagged-Fibonacci grid, then run Kadane on
! every row, column and both diagonal directions.
```

Comparison. C exposes direct bounded arithmetic. Rust exposes direct bounded arithmetic. Factor exposes direct bounded arithmetic. No validated source exists for Gforth. All listed programs print the same exact answer string from the manifest. The excerpts can differ in algorithm, data representation, and startup cost.

40.146 Problem 146: Investigating a Prime Pattern

A valid n ends in zero. Before performing expensive 64-bit primality tests, sieve each candidate against small primes: if any required n^2+offset is divisible by a filter prime, that n cannot work. Survivors must make all six specified offsets prime. The word "consecutive" also matters: offsets 19 and 21 between 13 and 27 must be composite. Checking those two values prevents accepting a set that has extra primes in the interval. Since n<150 million, n^2 fits uint64_t.

C. Source: `solutions/c/investigating_a_prime_pattern_146.c`.

```
int main(void) {
    const unsigned offsets[] = {1,3,7,9,13,27};
    const unsigned filters[] =
    {3,7,11,13,17,19,23,29,31,37,41,43,47,53,59,61,67,71,73,79,83,89,97};
    uint64_t sum=0;
    for(uint64_t n=10;n<150000000;n+=10) {
        uint64_t square=n*n; bool possible=true;
        for(unsigned i=0;i<sizeof(filters)/sizeof(filters[0])&&possible;++i)
            for(unsigned j=0;j<sizeof(offsets)/sizeof(offsets[0]);++j)
                if((square+offsets[j])%filters[i]==0){possible=false;break;}
        if(!possible) continue;
        for(unsigned j=0;j<sizeof(offsets)/sizeof(offsets[0]);++j)
            if(!euler_is_prime(square+offsets[j])){possible=false;break;}
        if(possible && !euler_is_prime(square+19) && !euler_is_prime(square+21)) sum+=n;
    }
```

Rust. Source: `solutions/rust/src/bin/p146.rs`.

```
fn solve() -> u64 {
    let offsets = [1, 3, 7, 9, 13, 27];
    let filters = [
        3, 7, 11, 13, 17, 19, 23, 29, 31, 37, 41, 43, 47, 53, 59, 61, 67, 71, 73, 79, 83, 89, 97,

    ];
    let mut sum = 0;
    for n in (10u64..150_000_000).step_by(10) {
        let s = n * n;
        if filters
            .iter()
            .any(|&p| offsets.iter().any(|&o| (s + o) % p == 0))
        {
            continue;
        }
```

Comparison. C exposes prime generation or testing. Shared calls: `euler_is_prime`. Rust exposes prime generation or testing. Shared calls: `prime64`. No validated source exists for Gforth, and Factor. All listed programs print the same exact answer string from the manifest. The excerpts can differ in algorithm, data representation, and startup cost.

40.147 Problem 147: Rectangles in Cross-hatched Grids

Axis-aligned rectangles have the closed form C(w+1,2)C(h+1,2). For tilted rectangles, scale both coordinates by two: diagonal grid edges then become ordinary integer steps. Each admissible start, parity, width, and height describes one tilted rectangle; boundary tests stop a direction as soon as all larger shapes would also leave the board. The function swaps dimensions so the longer side has one canonical role. Summing both orientations for every sub-grid 1<=w<=47, 1<=h<=43 counts the requested collection. The total is held in uint64_t.

C. Source: `solutions/c/rectangles_in_cross_hatched_grids_147.c`.

```
static uint64_t upright(unsigned w,unsigned h){return (uint64_t)w*(w+1)*h*(h+1)/4;}
static uint64_t diagonal(unsigned width,unsigned height){
    unsigned a=width,b=height;if(a<b){unsigned t=a;a=b;b=t;}
    uint64_t count=0;
    for(unsigned i=0;i<a;++i) for(unsigned j=0;j<b;++j) for(unsigned parity=0;parity<2;++parity){

        unsigned sx=2*i+1+parity, sy=2*j+2-parity;
        bool stop=false; unsigned max_height=UINT_MAX;
        for(unsigned rw=0;!stop;++rw){
            unsigned x=sx+rw;
            if(rw>=sy) break;
            unsigned y=sy-rw;
            for(unsigned rh=0;rh<max_height;++rh){
                unsigned ex=x+rh,ey=y+rh;
                if(ex>=2*a||ey>=2*b){if(max_height>rh)max_height=rh;stop=(rh==0);break;}
```

Rust. Source: `solutions/rust/src/bin/p147.rs`.

```
fn upright(w: u64, h: u64) -> u64 {
    w * (w + 1) * h * (h + 1) / 4
}
fn diagonal(width: usize, height: usize) -> u64 {
    let (mut a, mut b) = (width, height);
    if a < b {
        (a, b) = (b, a)
    }
    let mut count = 0;
    for i in 0..a {
        for j in 0..b {
            for parity in 0..2 {
                let (sx, sy) = (2 * i + 1 + parity, 2 * j + 2 - parity);
                let mut stop = false;
```

Comparison. C exposes direct bounded arithmetic. Rust exposes direct bounded arithmetic. No validated source exists for Gforth, and Factor. All listed programs print the same exact answer string from the manifest. The excerpts can differ in algorithm, data representation, and startup cost.

40.148 Problem 148: Exploring Pascal's Triangle

The program computes the exact answer for Exploring Pascal's Triangle.

C. Source: `solutions/c/exploring_pascals_triangle_148.c`.

```
static uint64_t nondivisible_rows(uint64_t rows) {
    unsigned digits[32], length = 0;
    for (uint64_t value = rows; value; value /= 7) digits[length++] = value % 7;
    uint64_t powers[32] = {1};
    for (unsigned i = 1; i <= length; ++i) powers[i] = powers[i - 1] * 28;
    uint64_t total = 0, prefix = 1;
    for (unsigned pos = length; pos-- > 0;) {
        uint64_t d = digits[pos];
        total += prefix * d * (d + 1) / 2 * powers[pos];
        prefix *= d + 1;
    }
    return total;
}
```

Rust. Source: `solutions/rust/src/bin/p148.rs`.

```
fn solve() -> u64 {
    let mut value = 1_000_000_000u64;
    let mut digits = Vec::new();
    while value > 0 {
        digits.push(value % 7);
        value /= 7
    }
    let mut powers = vec![1u64; digits.len() + 1];
    for i in 1..powers.len() {
        powers[i] = powers[i - 1] * 28
    }
    let (mut total, mut prefix) = (0, 1);
```

```
    for pos in (0..digits.len()).rev() {
        let d = digits[pos];
```

Factor. Source: `solutions/factor/p148.factor`.

```
! Native Factor Project Euler 148 solver; applies the base-seven digit count.
USING: io math.parser project-euler.148 ; IN: scratchpad
euler148 number>string print
```

Comparison. C exposes direct bounded arithmetic. Rust exposes direct bounded arithmetic. Factor exposes direct bounded arithmetic. No validated source exists for Gforth. All listed programs print the same exact answer string from the manifest. The excerpts can differ in algorithm, data representation, and startup cost.

40.149 Problem 149: Maximum-sum Subsequence

First generate all four million lagged-Fibonacci values in row-major order. Every horizontal, vertical, and both diagonal line is then scanned with Kadane's recurrence: extend a nonnegative prefix, otherwise restart at the current cell. Starting 'best' at INT64_MIN correctly handles an all-negative line rather than allowing an empty subsequence. Launching scans from each boundary covers every line once (corner lines may be harmlessly revisited). Sums use int64_t even though each grid entry is a signed 20-bit value.

C. Source: `solutions/c/maximum_sum_subsequence_149.c`.

```
enum { SIDE149 = 2000 };
static int *grid149;

static int64_t line_sum(int row, int col, int dr, int dc) {
    int64_t best = INT64_MIN, current = 0;
    while (row >= 0 && row < SIDE149 && col >= 0 && col < SIDE149) {
        current += grid149[row * SIDE149 + col];
        if (current > best) best = current;
        if (current < 0) current = 0;
        row += dr; col += dc;
    }
    return best;
}
```

Rust. Source: `solutions/rust/src/bin/p149.rs`.

```
const SIDE: i32 = 2000;
fn line(g: &[i32], mut r: i32, mut c: i32, dr: i32, dc: i32) -> i64 {
    let (mut best, mut cur) = (i64::MIN, 0);
    while (0..SIDE).contains(&r) && (0..SIDE).contains(&c) {
        cur += g[(r * SIDE + c) as usize] as i64;
        best = best.max(cur);
        cur = cur.max(0);
        r += dr;
        c += dc
```

```
    }
    best
}
fn solve() -> i64 {
    let size = (SIDE * SIDE) as usize;
```

Factor. Source: `solutions/factor/euler/extended/extended.factor`.

```
:: p149 ( -- n )
    4000000 0 <array> :> grid
    1 4000000 [a..b] [| k |
        k 55 <= [
            100003 k 200003 * - k 3 ^ 300007 * + 1000000 mod
        ] [
            k 25 - grid nth k 56 - grid nth + 1000000 + 1000000 mod
        ] if :> value
        value 500000 - k 1 - grid set-nth
    ] each
    -1000000000 :> best!
    0 1999 [a..b] [| i |
        grid i 0 0 1 p149-line best max best!
        grid 0 i 1 0 p149-line best max best!
```

Comparison. C exposes direct bounded arithmetic. Rust exposes direct bounded arithmetic. Factor exposes modular arithmetic. No validated source exists for Gforth. All listed programs print the same exact answer string from the manifest. The excerpts can differ in algorithm, data representation, and startup cost.

40.150 Problem 150: Sub-triangle Sums

Generate the 500500 pseudo-random entries exactly as specified and store a prefix sum for every row. For a fixed apex (top,col), increasing 'bottom' adds one contiguous row slice whose endpoints expand by one each time. Row prefixes make that slice O(1), so all downward triangles are examined in O(ROWS^3) arithmetic instead of summing each triangle from scratch. Individual values are signed, and both prefix and triangle sums can exceed 32 bits, hence int64_t throughout. The allocated rows include a leading zero so slice [left,right] is prefix[right+1]-prefix[left].

C. Source: `solutions/c/sub_triangle_sums_150.c`.

```
int main(void) {
    enum { ROWS = 1000 };
    int64_t **prefix = malloc(ROWS * sizeof(*prefix));
    if (!prefix) return EXIT_FAILURE;
    int64_t t = 0;
    for (unsigned row = 0; row < ROWS; ++row) {
        prefix[row] = malloc((row + 2) * sizeof(**prefix));
        if (!prefix[row]) return EXIT_FAILURE;
        prefix[row][0] = 0;
        for (unsigned col = 0; col <= row; ++col) {
            t = (615949 * t + 797807) % (1 << 20);
            prefix[row][col + 1] = prefix[row][col] + t - (1 << 19);
        }
```

```
    }
```

Rust. Source: `solutions/rust/src/bin/p150.rs`.

```
fn solve() -> i64 {
    const R: usize = 1000;
    let mut rows = Vec::with_capacity(R);
    let mut t = 0i64;
    for r in 0..R {
        let mut p = vec![0i64; r + 2];
        for c in 0..=r {
            t = (615949 * t + 797807) % (1 << 20);
            p[c + 1] = p[c] + t - (1 << 19)
        }
        rows.push(p)
    }
    let mut best = i64::MAX;
    for top in 0..R {
```

Factor. Source: `solutions/factor/p150.factor`.

```
! Native Factor Project Euler 150 solver; searches accumulated triangle rows.
USING: io math.parser project-euler.150 ; IN: scratchpad
euler150 number>string print
```

Comparison. C exposes direct bounded arithmetic. Rust exposes direct bounded arithmetic. Factor exposes direct bounded arithmetic. No validated source exists for Gforth. All listed programs print the same exact answer string from the manifest. The excerpts can differ in algorithm, data representation, and startup cost.

40.151 Problem 151: A Preference for A5

A state is the count of A2, A3, A4, and A5 sheets in the envelope. Drawing an Ai removes it and cutting the unused pieces adds one sheet of every smaller size. Each transition is weighted by count[i]/total. We add one to the expectation exactly when the envelope contains a single sheet, excluding the final lone A5 as required. The process is acyclic despite occasionally increasing the sheet count, and the reachable count bounds are [2][3][5][9]; memoization evaluates every state once.

C. Source: `solutions/c/a_preference_for_a5_151.c`.

```
static double memo151[2][3][5][9];
static bool seen151[2][3][5][9];

static double expected151(int a, int b, int c, int d) {
    if (a + b + c + d == 0) return 0;
    if (seen151[a][b][c][d]) return memo151[a][b][c][d];
    seen151[a][b][c][d] = true;
    int sheets[4] = {a, b, c, d};
    int total = a + b + c + d;
    double result = total == 1 && d == 0 ? 1.0 : 0.0;
    for (int i = 0; i < 4; ++i) {
```

```
        if (!sheets[i]) continue;
        int next[4] = {a, b, c, d};
        --next[i];
```

Rust. Source: `solutions/rust/src/bin/p151.rs`.

```
fn expected(s: [u8; 4], memo: &mut HashMap<[u8; 4], f64>) -> f64 {
    let total: u8 = s.iter().sum();
    if total == 0 {
        return 0.0;
    }
    if let Some(&v) = memo.get(&s) {
        return v;
    }
    let mut result = if total == 1 && s[3] == 0 { 1.0 } else { 0.0 };
    for i in 0..4 {
        if s[i] == 0 {
            continue;
        }
        let mut n = s;
```

Factor. Source: `solutions/factor/p151.factor`.

```
! Native Factor Project Euler 151 solver; evolves the paper-size distribution.
USING: formatting io math project-euler.151 ; IN: scratchpad
! The vocabulary deliberately returns an exact ratio; the problem requests
! six decimal places, for which Factor's shortest float rendering is exact.
euler151 >float "%.6f" printf nl
```

Comparison. C exposes dynamic programming or memoization. Rust exposes dynamic programming or memoization. Factor exposes direct bounded arithmetic. No validated source exists for Gforth. All listed programs print the same exact answer string from the manifest. The excerpts can differ in algorithm, data representation, and startup cost.

40.152 Problem 152: Sums of Square Reciprocals

Prime-denominator valuation arguments eliminate every denominator not in the fixed ‘candidates‘ list: a largest prime power occurring only once could never cancel in a sum equal to 1/2. The remaining 31 terms are handled by meet-in-the-middle rather than 2^31 direct subsets. Enumerate and sort every sum from the right 16 terms. For each subset of the left 15, binary-search the exact GMP rational ‘1/2-left_sum‘; repeated equal right sums are all counted. Exact mpq arithmetic avoids any tolerance question when many small reciprocal squares nearly coincide.

C. Source: `solutions/c/writing_one_half_as_inverse_squares_152.c`.

```
typedef struct { mpq_t value; } rational152;

static int compare152(const void *left, const void *right) {
    const rational152 *a = left, *b = right;
    return mpq_cmp(a->value, b->value);
```

```
}

static size_t lower_bound152(rational152 *values, size_t count, const mpq_t key) {
    size_t low = 0, high = count;
    while (low < high) {
        size_t mid = low + (high - low) / 2;
        if (mpq_cmp(values[mid].value, key) < 0) low = mid + 1;
        else high = mid;
    }
```

Rust. Source: `solutions/rust/src/bin/p152.rs`.

```
fn gcd(mut a: BigUint, mut b: BigUint) -> BigUint {
    while !b.is_zero() {
        let r = &a % &b;
        a = b;
        b = r
    }
    a
}
fn solve() -> u64 {
    let c = [
        2u32, 3, 4, 5, 6, 7, 8, 9, 10, 12, 13, 14, 15, 18, 20, 21, 24, 28, 30, 35, 36, 39, 40,
    42,
        45, 52, 56, 60, 63, 70, 72,
    ];
    let mut common = BigUint::one();
```

Comparison. C exposes prime generation or testing, and sorting or set deduplication. Rust exposes exact large-integer arithmetic, and sorting or set deduplication. No validated source exists for Gforth, and Factor. All listed programs print the same exact answer string from the manifest. The excerpts can differ in algorithm, data representation, and startup cost.

40.153 Problem 153: Investigating Gaussian Integers

Real Gaussian divisors contribute the ordinary divisor sum, accumulated as sum_d d*floor(N/d). A non-real primitive direction a+bi has gcd(a,b)=1 and norm a^2+b^2. Every scaled divisor k(a+bi), together with its conjugate, contributes 2ak for each multiple of k*norm not exceeding N. Enumerating coprime positive (a,b) prevents counting the same direction at several scales. The norm bound terminates both loops, and uint64_t is used for the large accumulated divisor sum.

C. Source: `solutions/c/investigating_gaussian_integers_153.c`.

```
static unsigned gcd153(unsigned a, unsigned b) {
    while (b) {
        unsigned remainder = a % b;
        a = b;
        b = remainder;
    }
    return a;
}
```

```
int main(void) {
    const unsigned limit = 100000000;
    uint64_t answer = 0;
    for (unsigned divisor = 1; divisor <= limit; ++divisor)
        answer += (uint64_t)divisor * (limit / divisor);
```

Rust. Source: `solutions/rust/src/bin/p153.rs`.

```
fn solve() -> u64 {
    const L: u64 = 100_000_000;
    let mut answer = (1..=L).map(|d| d * (L / d)).sum();
    for a in 1u64.. {
        if a * a >= L {
            break;
        }
        for b in 1u64.. {
            let norm = a * a + b * b;
            if norm > L {
                break;
            }
            if gcd(a, b) != 1 {
                continue;
```

Comparison. C exposes prime generation or testing. Shared calls: `gcd`. Rust exposes direct bounded arithmetic. No validated source exists for Gforth, and Factor. All listed programs print the same exact answer string from the manifest. The excerpts can differ in algorithm, data representation, and startup cost.

40.154 Problem 154: Exploring Pascal's Pyramid

A trinomial coefficient 200000!/(a!b!c!) is divisible by 10^12 exactly when its prime valuations v2 and v5 are both at least 12. Prefix tables store v_p(n!), allowing each coefficient's two valuations to be tested in O(1). Choose i=b+c first, then j=b. If C(200000,i) already supplies both twelve factors, every one of the i+1 splits qualifies. Otherwise only half the b,c pairs are tested and nonsymmetric successes are doubled. This symmetry and the fast outer acceptance remove most of the naive quadratic work.

C. Source: `solutions/c/exploring_pascals_pyramid_154.c`.

```
static unsigned choose154(const unsigned *factorial, unsigned n, unsigned k) {
    return factorial[n] - factorial[k] - factorial[n - k];
}

int main(void) {
    enum { LAYER = 200000 };
    unsigned *twos = calloc(LAYER + 1, sizeof(*twos));
    unsigned *fives = calloc(LAYER + 1, sizeof(*fives));
    if (!twos || !fives) return EXIT_FAILURE;
    for (unsigned value = 1; value <= LAYER; ++value) {
        unsigned current = value;
        twos[value] = twos[value - 1];
        fives[value] = fives[value - 1];
```

```
        while (current % 2 == 0) { ++twos[value]; current /= 2; }
```

Rust. Source: `solutions/rust/src/bin/p154.rs`.

```
fn choose(v: &[u32], n: usize, k: usize) -> u32 {
    v[n] - v[k] - v[n - k]
}
fn solve() -> u64 {
    const N: usize = 200_000;
    let (mut twos, mut fives) = (vec![0u32; N + 1], vec![0u32; N + 1]);
    for n in 1..=N {
        let mut x = n;
        twos[n] = twos[n - 1];
        fives[n] = fives[n - 1];
        while x % 2 == 0 {
            twos[n] += 1;
            x /= 2
        }
```

Comparison. C exposes prime generation or testing, and exact large-integer arithmetic. Rust exposes exact large-integer arithmetic. No validated source exists for Gforth, and Factor. All listed programs print the same exact answer string from the manifest. The excerpts can differ in algorithm, data representation, and startup cost.

40.155 Problem 155: Counting Capacitor Circuits

The program computes the exact answer for Counting Capacitor Circuits.

C. Source: `solutions/c/counting_capacitor_circuits_155.c`.

```
typedef struct { uint32_t numerator, denominator; } fraction155;
typedef struct { fraction155 *data; size_t size, capacity; } vector155;

static uint32_t gcd155(uint32_t a, uint32_t b) {
    while (b) { uint32_t r = a % b; a = b; b = r; }
    return a;
}

static fraction155 reduce155(uint64_t numerator, uint64_t denominator) {
    uint32_t n = numerator, d = denominator;
    uint32_t divisor = gcd155(n, d);
    return (fraction155){n / divisor, d / divisor};
}
```

Rust. Source: `solutions/rust/src/bin/p155.rs`.

```
#[derive(Clone, Copy, Hash, Eq, PartialEq)]
struct F(u64, u64);
fn f(n: u64, d: u64) -> F {
    let g = gcd(n, d);
    F(n / g, d / g)
```

```
}
fn solve() -> usize {
    let mut sets: Vec<HashSet<F>> = (0..=18).map(|_| HashSet::new()).collect();
    sets[1].insert(F(1, 1));
    for size in 2..=18 {
        let mut out = HashSet::new();
        for l in 1..=size / 2 {
            for &F(an, ad) in &sets[l] {
                for &F(bn, bd) in &sets[size - l] {
```

Comparison. C exposes sorting or set deduplication. Rust exposes sorting or set deduplication. Shared calls: `gcd`. No validated source exists for Gforth, and Factor. All listed programs print the same exact answer string from the manifest. The excerpts can differ in algorithm, data representation, and startup cost.

40.156 Problem 156: Counting Digits

‘occurrences156(n,d)‘ evaluates f(n,d) position by position. At decimal place p, complete high cycles contribute high*p occurrences, followed by a partial cycle determined by the current digit and lower suffix. To find fixed points f(n,d)=n, recursively inspect an interval [lo,hi]. Monotonicity gives a rigorous rejection: if f(lo)>hi or f(hi)<lo, its graph cannot meet the diagonal inside. Remaining intervals are bisected down to singletons. Digits 1..9 avoid the leading-zero complications of d=0, and all counts and the final sum require uint64_t.

C. Source: `solutions/c/counting_digits_156.c`.

```
static uint64_t occurrences156(uint64_t n, unsigned digit) {
    uint64_t count = 0;
    for (uint64_t place = 1; place <= n;) {
        uint64_t lower = n % place;
        uint64_t current = (n / place) % 10;
        uint64_t higher = n / place / 10;
        count += higher * place;
        if (current > digit) count += place;
        else if (current == digit) count += lower + 1;
        if (place > UINT64_MAX / 10) break;
        place *= 10;
    }
    return count;
}
```

Rust. Source: `solutions/rust/src/bin/p156.rs`.

```
fn occ(n: u64, d: u64) -> u64 {
    let (mut count, mut p) = (0, 1);
    while p <= n {
        let (lower, current, higher) = (n % p, (n / p) % 10, n / p / 10);
        count += higher * p;
        if current > d {
            count += p
        } else if current == d {
            count += lower + 1
```

```
        }
        if p > u64::MAX / 10 {
            break;
        }
        p *= 10
```

Comparison. C exposes recursive search. Rust exposes recursive search. No validated source exists for Gforth, and Factor. All listed programs print the same exact answer string from the manifest. The excerpts can differ in algorithm, data representation, and startup cost.

40.157 Problem 157: Base-10 Diophantine Reciprocal

The program computes the exact answer for Base-10 Diophantine Reciprocal.

C. Source: `solutions/c/solving_diophantine_equation_157.c`.

```
static unsigned divisors157(uint64_t value) {
    unsigned result = 1;
    for (uint64_t prime = 2; prime * prime <= value; ++prime) {
        if (value % prime) continue;
        unsigned exponent = 0;
        do { value /= prime; ++exponent; } while (value % prime == 0);
        result *= exponent + 1;
    }
    return value > 1 ? result * 2 : result;
}

int main(void) {
    uint64_t answer = 0, ten = 1;
    for (unsigned n = 1; n <= 9; ++n) {
```

Rust. Source: `solutions/rust/src/bin/p157.rs`.

```
fn solve() -> u64 {
    let (mut answer, mut ten) = (0, 1u64);
    for n in 1..=9 {
        ten *= 10;
        let mut two = 1;
        for a in 0..=n {
            let mut five = 1;
            for b in 0..=n {
                let y = two * five;
                answer += divisor_count((ten / y) * (y + 1)) as u64;
                if a > 0 && b > 0 {
                    answer += divisor_count((ten / y) * (two + five)) as u64
                }
                five *= 5
```

Factor. Source: `solutions/factor/euler/extended/extended.factor`.

```
:: p157 ( -- n )
    0 :> answer! 1 :> ten!
```

```
    1 9 [a..b] [| n |
        ten 10 * ten! 1 :> two!
        0 n [a..b] [| a |
            1 :> five!
            0 n [a..b] [| b |
                two five * :> y
                ten y /i y 1 + * divisor-count answer + answer!
                a 0 > b 0 > and [
                    ten y /i two five + * divisor-count answer + answer!
                ] when
                five 5 * five!
            ] each two 2 * two!
```

Comparison. C exposes prime generation or testing. Rust exposes direct bounded arithmetic. Shared calls: `divisor_count`. Factor exposes direct bounded arithmetic. No validated source exists for Gforth. All listed programs print the same exact answer string from the manifest. The excerpts can differ in algorithm, data representation, and startup cost.

40.158 Problem 158: Lexicographical Neighbours

The program computes the exact answer for Lexicographical Neighbours.

C. Source: `solutions/c/lexicographical_neighbours_158.c`.

```
int main(void){uint64_t choose=1,best=0,power=1;for(unsigned n=1;n<=26;++n){
    choose=choose*(27-n)/n;power*=2;uint64_t value=choose*(power-n-1);if(value>best)best=value;}
    printf("%" PRIu64 "\n",best);return 0;}
```

Rust. Source: `solutions/rust/src/bin/p158.rs`.

```
fn solve() -> u64 {
    let (mut choose, mut power, mut best) = (1u64, 1u64, 0u64);
    for n in 1..=26 {
        choose = choose * (27 - n) / n;
        power *= 2;
        best = best.max(choose * (power - n - 1))
    }
    best
}
fn main() {
    println!("{}", solve());
}
```

Factor. Source: `solutions/factor/euler/extended/extended.factor`.

```
:: p158 ( -- n )
    1 :> choose! 1 :> power! 0 :> best!
    1 26 [a..b] [| n |
        choose 27 n - * n /i choose!
        power 2 * power!
        choose power n - 1 - * best max best!
```

```
    ] each best ;

! Problem 162: inclusion-exclusion forces hexadecimal 0, 1 and A to occur.
```

Comparison. C exposes direct bounded arithmetic. Rust exposes direct bounded arithmetic. Factor exposes direct bounded arithmetic. No validated source exists for Gforth. All listed programs print the same exact answer string from the manifest. The excerpts can differ in algorithm, data representation, and startup cost.

40.159 Problem 159: Digital Root Sums of Factorisations

The program computes the exact answer for Digital Root Sums of Factorisations.

C. Source: `solutions/c/digital_root_sums_of_factorisations_159.c`.

```
int main(void){enum{LIMIT=1000000};unsigned char *m=malloc(LIMIT);if(!m)return EXIT_FAILURE;
    m[0]=0;for(unsigned n=1;n<LIMIT;++n)m[n]=1+(n-1)%9;
    for(unsigned a=2;a<LIMIT;++a)for(unsigned b=2;(uint64_t)a*b<LIMIT;++b){unsigned n=a*b,v=m[a]+
    m[b];if(v>m[n])m[n]=v;}
    uint64_t sum=0;for(unsigned n=2;n<LIMIT;++n)sum+=m[n];printf("%" PRIu64 "\n",sum);free(m);
    return 0;}
```

Rust. Source: `solutions/rust/src/bin/p159.rs`.

```
fn solve() -> u64 {
    const L: usize = 1_000_000;
    let mut m = vec![0u8; L];
    for n in 1..L {
        m[n] = (1 + (n - 1) % 9) as u8
    }
    for a in 2..L {
        for b in 2.. {
            let n = a * b;
            if n >= L {
                break;
            }
            m[n] = m[n].max(m[a] + m[b])
        }
```

Factor. Source: `solutions/factor/euler/extended/extended.factor`.

```
:: p159 ( -- n )
    1000000 0 <array> :> best
    1 999999 [a..b] [| n | n 1 - 9 mod 1 + n best set-nth ] each
    2 999999 [a..b] [| a |
        2 :> b!
        [ a b * 1000000 < ] [
            a b * :> product
            product best nth a best nth b best nth + max product best set-nth
            b 1 + b!
        ] while
```

```
    ] each best 2 tail-slice sum ;

! Problem 171: a DP tracks both the number and numeric sum of digit strings
! for each square-digit total, reduced modulo the requested nine digits.
```

Comparison. C exposes direct bounded arithmetic. Rust exposes dynamic programming or memoization. Factor exposes modular arithmetic, and dynamic programming or memoization. No validated source exists for Gforth. All listed programs print the same exact answer string from the manifest. The excerpts can differ in algorithm, data representation, and startup cost.

40.160 Problem 160: Factorial Trailing Digits

Every trailing zero is a paired factor 2*5. The range reduction uses the periodicity of the non-five residues modulo powers of ten, shrinking 10^12 to a tractable block. During multiplication, each factor of five is removed and matched by dividing an already accumulated factor of two. Keeping nine low digits provides guard digits while cancellations occur; only the final five nonzero digits are printed. The multiplication reduces every step, so uint64_t safely contains its intermediate values.

C. Source: `solutions/c/factorial_trailing_digits_160.c`.

```
int main(void) {
    uint64_t limit = 1000000000000ULL;
    while (limit % 100000 == 0) limit /= 5;
    uint64_t result = 1;
    for (uint64_t value = 1; value <= limit; ++value) {
        uint64_t current = value;
        while (current % 5 == 0) {
            current /= 5;
            result /= 2;
        }
        result *= current % 100000;
        result %= 1000000000ULL;
    }
    printf("%05" PRIu64 "\n", result % 100000);
```

Rust. Source: `solutions/rust/src/bin/p160.rs`.

```
fn solve() -> u64 {
    let mut limit = 1_000_000_000_000u64;
    while limit % 100_000 == 0 {
        limit /= 5
    }
    let mut r = 1;
    for v in 1..=limit {
        let mut x = v;
        while x % 5 == 0 {
            x /= 5;
            r /= 2
        }
        r = r * (x % 100_000) % 1_000_000_000
    }
```

Comparison. C exposes exact large-integer arithmetic. Rust exposes direct bounded arithmetic. No validated source exists for Gforth, and Factor. All listed programs print the same exact answer string from the manifest. The excerpts can differ in algorithm, data representation, and startup cost.

40.161 Problem 161: Triominoes

A profile DP fills the first empty cell of the current row. Bit masks a,b,c record occupancy in the current and next two rows—the deepest look-ahead any straight or L triomino needs. Anchoring every placement at the first empty cell ensures a tiling has exactly one construction order. The six placements below are the two straight orientations and four L orientations. A full current row shifts the masks upward. ‘(rows,a,b,c)‘ is packed into a nonzero 64-bit key and memoized in an open-addressed table; the final count fits uint64_t.

C. Source: `solutions/c/triominoes_161.c`.

```
enum { WIDTH161 = 9, CACHE161 = 1 << 20 };
static uint64_t keys161[CACHE161];
static uint64_t values161[CACHE161];

static size_t slot161(uint64_t key) {
    size_t slot = (size_t)((key * 11400714819323198485ULL) >> 44);
    while (keys161[slot] && keys161[slot] != key) slot = (slot + 1) & (CACHE161 - 1);
    return slot;
}

static uint64_t search161(unsigned rows, unsigned a, unsigned b, unsigned c) {
    if (rows == 0) return 1;
    const unsigned full = (1u << WIDTH161) - 1;
    if (a == full) return search161(rows - 1, b, c, 0);
```

Rust. Source: `solutions/rust/src/bin/p161.rs`.

```
const W: u32 = 9;
fn go(rows: u32, a: u32, b: u32, c: u32, m: &mut HashMap<(u32, u32, u32, u32), u64>) -> u64 {
    if rows == 0 {
        return 1;
    }
    let full = (1 << W) - 1;
    if a == full {
        return go(rows - 1, b, c, 0, m);
    }
    if let Some(&v) = m.get(&(rows, a, b, c)) {
        return v;
    }
    let col = a.trailing_ones();
    let x = 1 << col;
```

Comparison. C exposes dynamic programming or memoization. Rust exposes dynamic programming or memoization. No validated source exists for Gforth, and Factor. All listed programs print the same exact answer string from the manifest. The excerpts can differ in algorithm, data representation, and startup cost.

40.162 Problem 162: Hexadecimal Numbers

For each length 3..16, inclusion-exclusion enforces the presence of digits 0, 1, and A. A mask says which required symbols are absent, leaving an alphabet of 16-popcount(mask) symbols. If zero is excluded, every remaining symbol may lead; otherwise the leading position has one fewer choice. Alternating signs by subset size gives the valid count for that length. Counts fit uint64_t, while the inclusion-exclusion accumulator is signed. The requested answer is formatted directly in uppercase hexadecimal.

C. Source: `solutions/c/hexadecimal_numbers_162.c`.

```
static uint64_t power162(unsigned base,unsigned exponent){uint64_t r=1;while(exponent--)r*=base;
    return r;}
int main(void){uint64_t answer=0;for(unsigned length=3;length<=16;++length){int64_t count=0;
    for(unsigned mask=0;mask<8;++mask){unsigned removed=__builtin_popcount(mask),alphabet=16-
    removed;
        uint64_t ways=(mask&1)?power162(alphabet,length):(uint64_t)(alphabet-1)*power162(
    alphabet,length-1);
        count+=(removed&1)?-(int64_t)ways:(int64_t)ways;}answer+=count;}
    printf("%" PRIX64 "\n",answer);return 0;}
```

Rust. Source: `solutions/rust/src/bin/p162.rs`.

```
fn solve() -> u64 {
    let mut answer = 0;
    for len in 3..=16 {
        let mut count = 0i64;
        for mask in 0u32..8 {
            let removed = mask.count_ones();
            let a = 16 - removed as u64;
            let ways = if mask & 1 != 0 {
                a.pow(len)
            } else {
                (a - 1) * a.pow(len - 1)
            };
            count += if removed % 2 == 1 {
                -(ways as i64)
```

Factor. Source: `solutions/factor/euler/extended/extended.factor`.

```
:: p162 ( -- n )
    0 :> answer!
    3 16 [a..b] [| len |
        15 16 len 1 - ^ *
        15 len ^ -
        14 15 len 1 - ^ * 2 * -
        14 len ^ 2 * +
        13 14 len 1 - ^ * +
        13 len ^ -
        answer + answer!
    ] each answer ;

! Problem 182: CRT makes the unconcealed-message count a product of two gcds.
```

Comparison. C exposes direct bounded arithmetic. Rust exposes direct bounded arithmetic. Factor exposes direct bounded arithmetic. No validated source exists for Gforth. All listed programs print the same exact answer string from the manifest. The excerpts can differ in algorithm, data representation, and startup cost.

40.163 Problem 163: Cross-hatched Triangles

The program computes the exact answer for Cross-hatched Triangles.

C. Source: `solutions/c/cross_hatched_triangles_163.c`.

```
int main(void) {
    const int64_t n = 36;
    int64_t numerator = 1678 * n * n * n + 3117 * n * n + 88 * n;
    numerator -= 345 * (n % 2);
    numerator -= 320 * (n % 3);
    numerator -= 90 * (n % 4);
    numerator -= 288 * (n % 5);
    printf("%" PRId64 "\n", numerator / 240);
    return 0;
}
```

Rust. Source: `solutions/rust/src/bin/p163.rs`.

```
fn solve() -> i64 {
    let n = 36;
    let mut x = 1678 * n * n * n + 3117 * n * n + 88 * n;
    x -= 345 * (n % 2);
    x -= 320 * (n % 3);
    x -= 90 * (n % 4);
    x -= 288 * (n % 5);
    x / 240
}
fn main() {
    println!("{}", solve());
}
```

Factor. Source: `solutions/factor/euler/extended/extended.factor`.

```
:: p163 ( -- n )
    36 :> n
    n 3 ^ 1678 * n sq 3117 * + n 88 * +
    n 2 mod 345 * - n 3 mod 320 * - n 4 mod 90 * - n 5 mod 288 * -
    240 /i ;

! Problem 174: enumerate hollow square laminae and retain tile totals having
! from one through ten representations.
```

Comparison. C exposes direct bounded arithmetic. Rust exposes direct bounded arithmetic. Factor exposes modular arithmetic. No validated source exists for Gforth. All listed programs

print the same exact answer string from the manifest. The excerpts can differ in algorithm, data representation, and startup cost.

40.164 Problem 164: Three Consecutive Digital Sum Limit

Whether a new digit is legal depends only on the previous two digits, so a 10x10 table is the complete DP state. ‘dp[a][b]‘ counts prefixes ending in a,b; appending d is allowed exactly when a+b+d<=9 and moves to (b,d). Initialization uses a synthetic leading zero followed by a nonzero first digit, thereby excluding leading-zero 20-digit numbers without a special transition. After nineteen extensions the table represents length 20, and summing all terminal pairs produces the answer in uint64_t.

C. Source: `solutions/c/three_consecutive_digital_sum_limit_164.c`.

```
int main(void) {
    uint64_t dp[10][10] = {{0}}, next[10][10];
    for (unsigned d = 1; d <= 9; ++d) dp[0][d] = 1;
    for (unsigned pos = 1; pos < 20; ++pos) {
        memset(next, 0, sizeof(next));
        for (unsigned a = 0; a < 10; ++a)
            for (unsigned b = 0; b < 10; ++b)
                for (unsigned d = 0; a + b + d <= 9; ++d)
                    next[b][d] += dp[a][b];
        memcpy(dp, next, sizeof(dp));
    }
    uint64_t total = 0;
    for (unsigned a = 0; a < 10; ++a)
        for (unsigned b = 0; b < 10; ++b) total += dp[a][b];
```

Rust. Source: `solutions/rust/src/bin/p164.rs`.

```
fn solve() -> u64 {
    let mut dp = [[0u64; 10]; 10];
    for d in 1..=9 {
        dp[0][d] = 1
    }
    for _ in 1..20 {
        let mut next = [[0u64; 10]; 10];
        for a in 0..10 {
            for b in 0..10 {
                if a + b <= 9 {
                    for d in 0..=9 - a - b {
                        next[b][d] += dp[a][b]
                    }
                }
```

Factor. Source: `solutions/factor/p164.factor`.

```
! Native Factor Project Euler 164 digit-state dynamic program.
USING: io math.parser project-euler.164 ; IN: scratchpad
euler164 number>string print
```

Comparison. C exposes dynamic programming or memoization. Rust exposes dynamic programming or memoization. Factor exposes dynamic programming or memoization. No validated source exists for Gforth. All listed programs print the same exact answer string from the manifest. The excerpts can differ in algorithm, data representation, and startup cost.

40.165 Problem 165: Intersections

The program computes the exact answer for Intersections.

C. Source: `solutions/c/intersections_165.c`.

```
typedef struct { int32_t xn, xd, yn, yd; } point165;
typedef struct { int x1, y1, x2, y2; } segment165;
enum { TABLE165 = 1 << 23 };
static point165 *points165;

static int64_t gcd165(int64_t a, int64_t b) {
    if (a < 0) a = -a;
    while (b) {
        int64_t remainder = a % b;
        a = b;
        b = remainder;
    }
    return a;
}
```

Rust. Source: `solutions/rust/src/bin/p165.rs`.

```
fn gcd(mut a: i64, mut b: i64) -> i64 {
    a = a.abs();
    while b != 0 {
        (a, b) = (b, a % b)
    }
    a
}
fn solve() -> usize {
    let mut seg = Vec::with_capacity(5000);
    let mut s = 290797u64;
    for _ in 0..5000 {
        let mut p = [0i64; 4];
        for x in &mut p {
            s = s * s % 50515093;
```

Comparison. C exposes direct bounded arithmetic. Rust exposes sorting or set deduplication. No validated source exists for Gforth, and Factor. All listed programs print the same exact answer string from the manifest. The excerpts can differ in algorithm, data representation, and startup cost.

40.166 Problem 166: Criss Cross

Name the 4x4 entries a..p. Enumerating all sixteen digits would require 10^16 cases. Instead choose the first row a,b,c,d and three cells e,f,g of the second row; their common line sum determines h.

Choosing j and k then lets the row, column, and anti-diagonal equations determine i,l,m,n,o,p. Every derived value is rejected unless it is a decimal digit. All line equations except the main diagonal have then been enforced by construction, so one final diagonal comparison completes the test. This parametrization visits each valid grid once and uses only small signed integers.

C. Source: `solutions/c/criss_cross_166.c`.

```
static bool digit166(int value) { return value >= 0 && value <= 9; }

int main(void) {
    uint64_t answer = 0;
    for (int a = 0; a <= 9; ++a)
    for (int b = 0; b <= 9; ++b)
    for (int c = 0; c <= 9; ++c)
    for (int d = 0; d <= 9; ++d) {
        int sum = a + b + c + d;
        for (int e = 0; e <= 9; ++e)
        for (int f = 0; f <= 9; ++f)
        for (int g = 0; g <= 9; ++g) {
            int h = sum - e - f - g;
            if (!digit166(h)) continue;
```

Rust. Source: `solutions/rust/src/bin/p166.rs`.

```
fn digit(x: i32) -> bool {
    (0..=9).contains(&x)
}
fn solve() -> u64 {
    let mut answer = 0;
    for a in 0..=9 {
        for b in 0..=9 {
            for c in 0..=9 {
                for d in 0..=9 {
                    let sum = a + b + c + d;
                    for e in 0..=9 {
                        for f in 0..=9 {
                            for g in 0..=9 {
                                let h = sum - e - f - g;
```

Comparison. C exposes direct bounded arithmetic. Rust exposes direct bounded arithmetic. No validated source exists for Gforth, and Factor. All listed programs print the same exact answer string from the manifest. The excerpts can differ in algorithm, data representation, and startup cost.

40.167 Problem 167: Investigating Ulam Sequences

For U(2,v) with odd v, generate the short prefix until the sole even term 2(v+1) appears. Beyond it, membership of successive odd numbers follows a finite XOR recurrence whose state is the last v+1 membership bits. Store the first index of each bit state to locate the eventual cycle. A prefix count of set bits gives how many Ulam terms occur before and within the cycle, so the 10^11-th term is reached by whole-cycle arithmetic plus a short local scan. This proves the extrapolation instead of assuming a period observed for a finite sample.

C. Source: `solutions/c/investigating_ulam_sequences_167.c`.

```
static uint64_t kth167(unsigned v, uint64_t rank) {
    unsigned gap = v + 1;
    unsigned even = 2 * gap;
    bool member[128] = {false};
    unsigned char representations[256] = {0};
    unsigned terms[128] = {2, v};
    unsigned used = 2, last = v;
    member[2] = member[v] = true;
    ++representations[2 + v];
    while (last < even) {
        unsigned next = last + 1;
        while (representations[next] != 1) ++next;
        for (unsigned i = 0; i < used; ++i)
            if (representations[next + terms[i]] < 2) ++representations[next + terms[i]];
```

Rust. Source: `solutions/rust/src/bin/p167.rs`.

```
fn kth(v: usize, rank: u64) -> u64 {
    let gap = v + 1;
    let even = 2 * gap;
    let mut member = [false; 128];
    let mut reps = [0u8; 256];
    let mut terms = vec![2, v];
    member[2] = true;
    member[v] = true;
    reps[2 + v] += 1;
    let mut last = v;
    while last < even {
        let mut next = last + 1;
        while reps[next] != 1 {
            next += 1
```

Comparison. C exposes direct bounded arithmetic. Rust exposes direct bounded arithmetic. No validated source exists for Gforth, and Factor. All listed programs print the same exact answer string from the manifest. The excerpts can differ in algorithm, data representation, and startup cost.

40.168 Problem 168: Number Rotations

Write N=10q+d. Moving the final digit d to the front of an L-digit number gives d*10^(L-1)+q=mN. Rearranging yields q = d(10^L-m)/(10m-1). Thus each length, multiplier, and last digit has at most one candidate. GMP performs the exact division for lengths up to 100; a zero remainder and the bounds 10^(L-2)<=q<10^(L-1) enforce integrality and full length. Only N mod 100000 is retained in the requested modular sum.

C. Source: `solutions/c/number_rotations_168.c`.

```
int main(void) {
    mpz_t power, numerator, quotient, remainder, lower, number;
    mpz_inits(power, numerator, quotient, remainder, lower, number, NULL);
    mpz_set_ui(power, 10);
    unsigned answer = 0;
```

```
    for (unsigned digits = 2; digits <= 100; ++digits) {
        mpz_fdiv_q_ui(lower, power, 10);
        for (unsigned multiplier = 1; multiplier <= 9; ++multiplier) {
            unsigned denominator = 10 * multiplier - 1;
            for (unsigned last = 1; last <= 9; ++last) {
                mpz_mul_ui(numerator, power, last);
                mpz_sub_ui(numerator, numerator, multiplier * last);
                mpz_fdiv_qr_ui(quotient, remainder, numerator, denominator);
                if (mpz_sgn(remainder) != 0 || mpz_cmp(quotient, lower) < 0 ||
```

Rust. Source: `solutions/rust/src/bin/p168.rs`.

```
fn solve() -> u64 {
    let mut power = BigUint::from(10u32);
    let mut answer = 0;
    for _digits in 2..=100 {
        let lower = &power / 10u32;
        for m in 1u32..=9 {
            let den = 10 * m - 1;
            for last in 1u32..=9 {
                let num = &power * last - m * last;
                let q = &num / den;
                if &num % den != BigUint::zero() || q < lower || q >= power {
                    continue;
                }
                let n = q * 10u32 + last;
```

Comparison. C exposes modular arithmetic, and exact large-integer arithmetic. Rust exposes exact large-integer arithmetic. No validated source exists for Gforth, and Factor. All listed programs print the same exact answer string from the manifest. The excerpts can differ in algorithm, data representation, and startup cost.

40.169 Problem 169: Sums of Powers of Two

A representation chooses coefficient 0, 1, or 2 for every power of two. Process the binary digits of 10^25 from least significant upward. The DP state is the carry into the current bit; a coefficient is legal when (coefficient+carry) has the required parity, and its half becomes the next carry. GMP stores the input exactly, but the number of representations fits in uint64_t. When every input bit has been consumed, only zero carry denotes a complete representation, so dp[0] is the answer.

C. Source: `solutions/c/sums_of_powers_of_two_169.c`.

```
int main(void) {
    mpz_t n;
    mpz_init_set_ui(n, 10);
    mpz_pow_ui(n, n, 25);
    uint64_t dp[2] = {1, 0};
    uint64_t next[2];
    while (mpz_sgn(n) > 0) {
        unsigned bit = mpz_tstbit(n, 0);
        next[0] = next[1] = 0;
        for (unsigned carry = 0; carry < 2; ++carry) {
```

```
            for (unsigned digit = 0; digit <= 2; ++digit) {
                if (((digit + carry) & 1) == bit)
                    next[(digit + carry) >> 1] += dp[carry];
            }
```

Rust. Source: `solutions/rust/src/bin/p169.rs`.

```
fn solve() -> u64 {
    let mut n = BigUint::from(10u32).pow(25);
    let mut dp = [1u64, 0];
    while !n.is_zero() {
        let bit = (&n & BigUint::one()) == BigUint::one();
        let mut next = [0, 0];
        for carry in 0..2 {
            for digit in 0..=2 {
                if ((digit + carry) & 1 == 1) == bit {
                    next[(digit + carry) >> 1] += dp[carry]
                }
            }
        }
        dp = next;
```

Factor. Source: `solutions/factor/p169.factor`.

```
! Native Factor Project Euler 169 solver for restricted powers-of-two sums.
USING: io math.parser project-euler.169 ; IN: scratchpad
euler169 number>string print
```

Comparison. C exposes dynamic programming or memoization, and exact large-integer arithmetic. Rust exposes dynamic programming or memoization, and exact large-integer arithmetic. Factor exposes direct bounded arithmetic. No validated source exists for Gforth. All listed programs print the same exact answer string from the manifest. The excerpts can differ in algorithm, data representation, and startup cost.

40.170 Problem 170: Pandigital Concatenating Products

Generate 0-to-9 pandigital result strings from largest to smallest. For every split into left and right products, any common multiplicand must divide gcd(left,right), so only divisors of that gcd need testing. Dividing both halves by a candidate produces the two multiplicands. ‘append170‘ verifies that the common factor and both quotients together use each decimal digit exactly once. Because result permutations are visited in descending lexicographic order, the first successful split is globally maximal and the recursion can stop immediately.

C. Source: `solutions/c/pandigital_concatenating_products_170.c`.

```
static uint64_t answer170;
static unsigned digits170[10];

static uint64_t gcd170(uint64_t a, uint64_t b) {
    while (b) { uint64_t r = a % b; a = b; b = r; }
```

```
    return a;
}

static bool append170(uint64_t value, unsigned *mask, unsigned *length) {
    unsigned local[20], count = 0;
    do { local[count++] = value % 10; value /= 10; } while (value);
    for (unsigned i = count; i-- > 0;) {
        unsigned bit = 1u << local[i];
        if (*mask & bit) return false;
```

Rust. Source: `solutions/rust/src/bin/p170.rs`.

```
fn append(mut n: u64, mask: &mut u16, len: &mut usize) -> bool {
    let mut d = Vec::new();
    loop {
        d.push((n % 10) as u8);
        n /= 10;
        if n == 0 {
            break;
        }
    }
    for x in d.into_iter().rev() {
        let b = 1 << x;
        if *mask & b != 0 {
            return false;
        }
```

Comparison. C exposes direct bounded arithmetic. Rust exposes recursive search. Shared calls: `gcd`. No validated source exists for Gforth, and Factor. All listed programs print the same exact answer string from the manifest. The excerpts can differ in algorithm, data representation, and startup cost.

40.171 Problem 171: Square Sum of the Digital Squares

Digits Is a Square For each attainable square-sum s, keep both the number of fixed-width prefixes and the sum of their numeric values modulo 10^9. Appending digit d moves s to s+d^2, multiplies every old value by ten, and adds d once for every old prefix. Leading zeros are intentionally allowed: they provide a unique 20-digit padded representation of every integer below 10^20. After twenty positions, sum the value totals at square indices. The maximum index is 20*9^2=1620, so two small rolling arrays cover the entire DP.

C. Source: `solutions/c/square_sum_of_the_digital_squares_171.c`.

```
int main(void){enum{MAX=20*81,MOD=1000000000};uint64_t count[MAX+1]={1},sum[MAX+1]={0},nc[MAX+1],
    ns[MAX+1];
    for(unsigned pos=0;pos<20;++pos){memset(nc,0,sizeof(nc));memset(ns,0,sizeof(ns));for(
    unsigned s=0;s<=pos*81;++s)for(unsigned d=0;d<10;++d){unsigned t=s+d*d;
        nc[t]=(nc[t]+count[s])%MOD;ns[t]=(ns[t]+sum[s]*10+count[s]*d)%MOD;}memcpy(count,nc,
    sizeof(count));memcpy(sum,ns,sizeof(sum));}
    uint64_t answer=0;for(unsigned r=1;r*r<=MAX;++r)answer=(answer+sum[r*r])%MOD;printf("%"
    PRIu64 "\n",answer);return 0;}
```

Rust. Source: `solutions/rust/src/bin/p171.rs`.

```
fn solve() -> u64 {
    const M: u64 = 1_000_000_000;
    let mut count = vec![0u64; 1621];
    let mut sum = vec![0u64; 1621];
    count[0] = 1;
    for pos in 0..20 {
        let (mut nc, mut ns) = (vec![0u64; 1621], vec![0u64; 1621]);
        for s in 0..=pos * 81 {
            for d in 0..10 {
                let t = s + d * d;
                nc[t] = (nc[t] + count[s]) % M;
                ns[t] = (ns[t] + sum[s] * 10 + count[s] * d as u64) % M
            }
        }
```

Factor. Source: `solutions/factor/euler/extended/extended.factor`.

```
:: p171 ( -- n )
    1621 0 <array> :> counts! 1621 0 <array> :> sums!
    1 0 counts set-nth
    0 19 [a..b] [| position |
        1621 0 <array> :> next-counts
        1621 0 <array> :> next-sums
        0 position 81 * [a..b] [| square-sum |
            0 9 [a..b] [| digit |
                square-sum digit sq + :> target
                target next-counts nth square-sum counts nth + 1000000000 mod
                target next-counts set-nth
                target next-sums nth square-sum sums nth 10 * +
                square-sum counts nth digit * + 1000000000 mod
                target next-sums set-nth
```

Comparison. C exposes modular arithmetic, and dynamic programming or memoization. Rust exposes dynamic programming or memoization. Factor exposes modular arithmetic. No validated source exists for Gforth. All listed programs print the same exact answer string from the manifest. The excerpts can differ in algorithm, data representation, and startup cost.

40.172 Problem 172: Few Repeated Digits

Encode the usage count of each decimal digit in two bits of a 20-bit state. At position ‘pos‘, try every digit whose count is below three and increment its two-bit field. Zero is rejected only at the first position, exactly enforcing an 18-digit number rather than a padded shorter number. Memoizing (position,state) merges every prefix with the same remaining digit capacities. A terminal position contributes one completed number; the full count fits uint64_t.

C. Source: `solutions/c/few_repeated_digits_172.c`.

```
static uint64_t memo172[19][4*4*4*4*4*4*4*4*4*4];static bool seen172[19][1048576];
static uint64_t ways172(unsigned pos,unsigned state){if(pos==18)return 1;if(seen172[pos][state])
    return memo172[pos][state];seen172[pos][state]=true;uint64_t total=0;
```

```
    for(unsigned d=0;d<10;++d){if(pos==0&&d==0)continue;unsigned shift=2*d,c=(state>>shift)&3;if
    (c<3)total+=ways172(pos+1,state+(1u<<shift));}return memo172[pos][state]=total;}
int main(void){printf("%" PRIu64 "\n",ways172(0,0));return 0;}
```

Rust. Source: `solutions/rust/src/bin/p172.rs`.

```
fn fact(n: u64) -> u64 {
    (1..=n).product()
}
fn rec(d: usize, left: u8, c: &mut [u8; 10], ans: &mut u64) {
    if d == 10 {
        if left != 0 {
            return;
        }
        let den: u64 = c.iter().map(|&x| fact(x as u64)).product();
        let mut ways = fact(18) / den;
        if c[0] > 0 {
            let mut d0 = fact((c[0] - 1) as u64);
            for &x in &c[1..] {
                d0 *= fact(x as u64)
```

Comparison. C exposes dynamic programming or memoization. Rust exposes direct bounded arithmetic. No validated source exists for Gforth, and Factor. All listed programs print the same exact answer string from the manifest. The excerpts can differ in algorithm, data representation, and startup cost.

40.173 Problem 173: Hollow Square Laminae I

The program computes the exact answer for Hollow Square Laminae I.

C. Source: `solutions/c/hollow_square_laminae_i_173.c`.

```
int main(void) {
    unsigned count = 0;
    for (unsigned outer = 3; 4 * (outer - 1) <= 1000000; ++outer) {
        for (unsigned inner = outer - 2; inner >= 1; inner -= 2) {
            if (outer * outer - inner * inner > 1000000) break;
            ++count;
            if (inner <= 2) break;
        }
    }
    printf("%u\n", count);
    return 0;
}
```

Rust. Source: `solutions/rust/src/bin/p173.rs`.

```
fn solve() -> u64 {
    let mut z = 0;
    for outer in 3u64.. {
        if 4 * (outer - 1) > 1_000_000 {
```

```
            break;
        }
        let mut inner = outer - 2;
        loop {
            if outer * outer - inner * inner > 1_000_000 {
                break;
            }
            z += 1;
            if inner <= 2 {
                break;
```

Factor. Source: `solutions/factor/p173.factor`.

```
! Native Factor Project Euler 173 lamina enumeration.
USING: io math.parser project-euler.173 ; IN: scratchpad
euler173 number>string print
```

Comparison. C exposes direct bounded arithmetic. Rust exposes direct bounded arithmetic. Factor exposes direct bounded arithmetic. No validated source exists for Gforth. All listed programs print the same exact answer string from the manifest. The excerpts can differ in algorithm, data representation, and startup cost.

40.174 Problem 174: Hollow Square Laminae II

The program computes the exact answer for Hollow Square Laminae II.

C. Source: `solutions/c/hollow_square_laminae_ii_174.c`.

```
int main(void) {
    unsigned char *count = calloc(1000001, 1);
    if (!count) return EXIT_FAILURE;
    for (unsigned outer = 3; 4 * (outer - 1) <= 1000000; ++outer) {
        for (unsigned inner = outer - 2; inner >= 1; inner -= 2) {
            unsigned tiles = outer * outer - inner * inner;
            if (tiles > 1000000) break;
            if (count[tiles] < 11) ++count[tiles];
            if (inner <= 2) break;
        }
    }
    unsigned answer = 0;
    for (unsigned n = 1; n <= 1000000; ++n)
        answer += count[n] >= 1 && count[n] <= 10;
```

Rust. Source: `solutions/rust/src/bin/p174.rs`.

```
fn solve() -> usize {
    let mut c = vec![0u8; 1_000_001];
    for o in 3usize.. {
        if 4 * (o - 1) > 1_000_000 {
            break;
        }
```

```
        let mut i = o - 2;
        loop {
            let t = o * o - i * i;
            if t > 1_000_000 {
                break;
            }
            c[t] = (c[t] + 1).min(11);
            if i <= 2 {
```

Factor. Source: `solutions/factor/euler/extended/extended.factor`.

```
:: p174 ( -- n )
    1000001 0 <array> :> counts 3 :> outer!
    [ outer 1 - 4 * 1000000 <= ] [
        outer 2 - :> inner!
        t :> running!
        [ running ] [
            outer sq inner sq - :> tiles
            tiles 1000000 > [ f running! ] [
                tiles counts nth 1 + 11 min tiles counts set-nth
                inner 2 <= [ f running! ] [ inner 2 - inner! ] if
            ] if
        ] while outer 1 + outer!
    ] while counts [ dup 1 >= swap 10 <= and ] count ;
```

Comparison. C exposes direct bounded arithmetic. Rust exposes direct bounded arithmetic. Factor exposes direct bounded arithmetic. No validated source exists for Gforth. All listed programs print the same exact answer string from the manifest. The excerpts can differ in algorithm, data representation, and startup cost.

40.175 Problem 175: Fractions and Sum of Powers of Two

The ratio of adjacent Stern-Brocot/Calkin-Wilf counts is decoded by the subtractive Euclidean algorithm. Instead of subtracting one copy at a time, ‘(larger-1)/smaller‘ records an entire run while preserving a positive remainder. These quotients are precisely the run lengths in the requested shortened binary expansion. Euclid discovers runs from the leaf back toward the root, so they are printed in reverse order after appending the root run of length one. uint64_t easily covers both the given reduced fraction and every intermediate subtraction.

C. Source: `solutions/c/fractions_involving_the_number_of_ways_175.c`.

```
int main(void) {
    uint64_t numerator = 123456789;
    uint64_t denominator = 987654321;
    uint64_t runs[32];
    unsigned count = 0;
    while (numerator != denominator) {
        if (numerator < denominator) {
            uint64_t quotient = (denominator - 1) / numerator;
            runs[count++] = quotient;
            denominator -= quotient * numerator;
        } else {
```

```
            uint64_t quotient = (numerator - 1) / denominator;
            runs[count++] = quotient;
            numerator -= quotient * denominator;
```

Rust. Source: `solutions/rust/src/bin/p175.rs`.

```
fn solve() -> String {
    let (mut a, mut b) = (123456789u64, 987654321u64);
    let mut runs = Vec::new();
    while a != b {
        if a < b {
            let q = (b - 1) / a;
            runs.push(q);
            b -= q * a
        } else {
            let q = (a - 1) / b;
            runs.push(q);
            a -= q * b
        }
    }
```

Factor. Source: `solutions/factor/p175.factor`.

```
! Native Factor Project Euler 175 continued-fraction solver.
USING: io project-euler.175 ; IN: scratchpad
! euler175 already returns the requested comma-separated representation.
euler175 print
```

Comparison. C exposes direct bounded arithmetic. Rust exposes direct bounded arithmetic. Factor exposes direct bounded arithmetic. No validated source exists for Gforth. All listed programs print the same exact answer string from the manifest. The excerpts can differ in algorithm, data representation, and startup cost.

40.176 Problem 176: Common Cathetus Right-angled Triangles

The program computes the exact answer for Common Cathetus Right-angled Triangles.

C. Source: `solutions/c/right_angled_triangles_that_share_a_cathetus_176.c`.

```
static const uint64_t primes176[] = {2, 3, 5, 7, 11, 13, 17, 19, 23, 29};
static uint64_t best176 = UINT64_MAX;

static void search176(unsigned position, unsigned max_exponent,
                      uint64_t remaining_divisors, uint64_t value) {
    if (remaining_divisors == 1) {
        if (value < best176) best176 = value;
        return;
    }
    if (position == sizeof(primes176) / sizeof(primes176[0])) return;
    uint64_t power = 1;
    for (unsigned exponent = 1; exponent <= max_exponent; ++exponent) {
```

```
        if (power > best176 / primes176[position]) break;
        power *= primes176[position];
```

Rust. Source: `solutions/rust/src/bin/p176.rs`.

```
fn go(pos: usize, maxe: u32, remain: u64, value: u64, best: &mut u64) {
    const P: [u64; 10] = [2, 3, 5, 7, 11, 13, 17, 19, 23, 29];
    if remain == 1 {
        *best = (*best).min(value);
        return;
    }
    if pos == P.len() {
        return;
    }
    let mut power = 1u64;
    for e in 1..=maxe {
        let Some(v) = power.checked_mul(P[pos]) else {
            break;
        };
```

Comparison. C exposes prime generation or testing. Rust exposes prime generation or testing. No validated source exists for Gforth, and Factor. All listed programs print the same exact answer string from the manifest. The excerpts can differ in algorithm, data representation, and startup cost.

40.177 Problem 177: Integer Angled Quadrilaterals

The program computes the exact answer for Integer Angled Quadrilaterals.

C. Source: `solutions/c/integer_angled_quadrilaterals_177.c`.

```
typedef struct { double ratio; uint8_t a, c; } ratio177;
typedef struct { uint8_t angle[8]; } shape177;
enum { SHAPES177 = 1 << 20 };
static shape177 *shapes177;

static int compare_ratio177(const void *left, const void *right) {
    double a = ((const ratio177 *)left)->ratio;
    double b = ((const ratio177 *)right)->ratio;
    return a < b ? -1 : a > b;
}

static bool less177(const uint8_t *a, const uint8_t *b) {
    for (unsigned i = 0; i < 8; ++i) {
        if (a[i] != b[i]) return a[i] < b[i];
```

Rust. Source: `solutions/rust/src/bin/p177.rs`.

```
fn canon(x: [u8; 8]) -> [u8; 8] {
    let mut best = x;
    let mut rev = x;
    rev.reverse();
    for src in [x, rev] {
```

```
        for r in (0..8).step_by(2) {
            let mut c = [0; 8];
            for i in 0..8 {
                c[i] = src[(i + r) & 7]
            }
            best = best.min(c)
        }
    }
    best
```

Comparison. C exposes sorting or set deduplication. Rust exposes sorting or set deduplication. No validated source exists for Gforth, and Factor. All listed programs print the same exact answer string from the manifest. The excerpts can differ in algorithm, data representation, and startup cost.

40.178 Problem 178: Step Numbers

A state records the final digit and a ten-bit mask of digits already seen. The next digit of a step number must be final-1 or final+1; OR-ing that digit into the mask preserves exactly the information needed to recognize a pandigital prefix. Initial states use digits 1..9, excluding leading zero. At each length from 10 through 40, states with mask 1023 are accumulated. Two rolling 10x1024 tables avoid a length dimension, and the total fits in uint64_t.

C. Source: `solutions/c/step_numbers_178.c`.

```
int main(void) {
    uint64_t dp[10][1024] = {{0}}, next[10][1024], answer = 0;
    for (unsigned d = 1; d <= 9; ++d) dp[d][1u << d] = 1;
    for (unsigned len = 1; len <= 40; ++len) {
        if (len >= 10)
            for (unsigned d = 0; d < 10; ++d) answer += dp[d][1023];
        memset(next, 0, sizeof(next));
        for (unsigned d = 0; d < 10; ++d) {
            for (unsigned mask = 0; mask < 1024; ++mask) {
                if (d) next[d - 1][mask | (1u << (d - 1))] += dp[d][mask];
                if (d < 9) next[d + 1][mask | (1u << (d + 1))] += dp[d][mask];
            }
        }
        memcpy(dp, next, sizeof(dp));
```

Rust. Source: `solutions/rust/src/bin/p178.rs`.

```
fn solve() -> u64 {
    let mut dp = vec![vec![0u64; 1024]; 10];
    for d in 1..=9 {
        dp[d][1 << d] = 1
    }
    let mut answer = 0;
    for len in 1..=40 {
        if len >= 10 {
            answer += (0..10).map(|d| dp[d][1023]).sum::<u64>()
        }
        let mut n = vec![vec![0u64; 1024]; 10];
```

```
        for d in 0..10 {
            for m in 0..1024 {
                if d > 0 {
```

Factor. Source: `solutions/factor/euler/extended/extended.factor`.

```
:: p178 ( -- n )
    10240 0 <array> :> dp!
    1 9 [a..b] [| d | 1 d 2 d ^ p178-index dp set-nth ] each
    0 :> answer!
    1 40 [a..b] [| length |
        length 10 >= [ 0 9 [a..b] [| d |
            answer d 1023 p178-index dp nth + answer!
        ] each ] when
        10240 0 <array> :> next
        0 9 [a..b] [| d | 0 1023 [a..b] [| mask |
            d mask p178-index dp nth :> ways
            d 0 > [
                d 1 - :> nd mask 2 nd ^ bitor :> nm
                nd nm p178-index next nth ways + nd nm p178-index next set-nth
```

Comparison. C exposes dynamic programming or memoization. Rust exposes dynamic programming or memoization. Factor exposes prime generation or testing, and dynamic programming or memoization. No validated source exists for Gforth. All listed programs print the same exact answer string from the manifest. The excerpts can differ in algorithm, data representation, and startup cost.

40.179 Problem 179: Consecutive Positive Divisors

A divisor sieve computes tau(n) for every n<=10^7: divisor d increments all multiples d,2d,3d,... exactly once. The largest divisor count in this range is tiny, so unsigned short storage is ample and keeps the sieve cache-friendly. A final linear pass compares tau(n) with tau(n+1) for 2<=n<10^7. The upper array endpoint is allocated because the last comparison reads tau(10^7).

C. Source: `solutions/c/consecutive_positive_divisors_179.c`.

```
int main(void) {
    enum { LIMIT = 10000000 };
    unsigned short *divs = calloc(LIMIT + 1, sizeof(*divs));
    if (!divs) return EXIT_FAILURE;
    for (unsigned d = 1; d <= LIMIT; ++d)
        for (unsigned n = d; n <= LIMIT; n += d) ++divs[n];
    unsigned answer = 0;
    for (unsigned n = 2; n < LIMIT; ++n) answer += divs[n] == divs[n + 1];
    printf("%u\n", answer);
    free(divs);
    return 0;
}
```

Rust. Source: `solutions/rust/src/bin/p179.rs`.

```
fn solve() -> usize {
    const L: usize = 10_000_000;
    let mut d = vec![0u16; L + 1];
    for x in 1..=L {
        for n in (x..=L).step_by(x) {
            d[n] += 1
        }
    }
    (2..L).filter(|&n| d[n] == d[n + 1]).count()
}
fn main() {
    println!("{}", solve());
}
```

Comparison. C exposes prime generation or testing, and dynamic programming or memoization. Rust exposes prime generation or testing. No validated source exists for Gforth, and Factor. All listed programs print the same exact answer string from the manifest. The excerpts can differ in algorithm, data representation, and startup cost.

40.180 Problem 180: Golden Triplets

The program computes the exact answer for Golden Triplets.

C. Source: `solutions/c/golden_triplets_180.c`.

```
typedef struct { uint64_t numerator, denominator; } fraction180;
static fraction180 sums180[400000];
static size_t count180;

static uint64_t gcd180(uint64_t a, uint64_t b) {
    while (b) { uint64_t r = a % b; a = b; b = r; }
    return a;
}

static fraction180 reduce180(uint64_t numerator, uint64_t denominator) {
    uint64_t divisor = gcd180(numerator, denominator);
    return (fraction180){numerator / divisor, denominator / divisor};
}
```

Rust. Source: `solutions/rust/src/bin/p180.rs`.

```
#[derive(Clone, Copy, Hash, Eq, PartialEq)]
struct F(u64, u64);
fn f(n: u64, d: u64) -> F {
    let g = gcd(n, d);
    F(n / g, d / g)
}
fn valid(x: F) -> bool {
    x.0 > 0 && x.0 < x.1 && x.1 <= 35
}
fn add(x: F, y: F, z: F) -> F {
    f(
```

```
        x.0 * y.1 * z.1 + y.0 * x.1 * z.1 + z.0 * x.1 * y.1,
        x.1 * y.1 * z.1,
    )
```

Comparison. C exposes exact large-integer arithmetic, and sorting or set deduplication. Rust exposes exact large-integer arithmetic, and sorting or set deduplication. No validated source exists for Gforth, and Factor. All listed programs print the same exact answer string from the manifest. The excerpts can differ in algorithm, data representation, and startup cost.

40.181 Problem 181: Grouping Two Different Coloured Objects

A grouping is an unordered multiset of group types. A type (b,w) says that one group contains b black and w white objects. Iterating the types outside the usual coin-change recurrence makes every multiset appear once; iterating object totals upward allows a type to be used repeatedly.

C. Source: `solutions/c/investigating_in_how_many_ways_objects_of_two_different_colours_can_be_grouped_181.c`.

```
int main(void) {
    uint64_t ways[61][41] = {{0}};
    ways[0][0] = 1;
    for (unsigned black = 0; black <= 60; ++black) {
        for (unsigned white = 0; white <= 40; ++white) {
            if (black == 0 && white == 0) continue; /* an empty group is not a type */
            for (unsigned b = black; b <= 60; ++b)
                for (unsigned w = white; w <= 40; ++w)
                    ways[b][w] += ways[b - black][w - white];
        }
    }
    printf("%" PRIu64 "\n", ways[60][40]);
    return 0;
}
```

Rust. Source: `solutions/rust/src/bin/p181.rs`.

```
fn solve() -> u64 {
    let mut ways = [[0u64; 41]; 61];
    ways[0][0] = 1;
    for black in 0..=60 {
        for white in 0..=40 {
            if black == 0 && white == 0 {
                continue;
            }
            for b in black..=60 {
                for w in white..=40 {
                    ways[b][w] += ways[b - black][w - white];
                }
            }
        }
```

Factor. Source: `solutions/factor/euler/extended/extended.factor`.

```
:: p181 ( -- n )
    2501 0 <array> :> ways
    1 0 ways set-nth
    0 60 [a..b] [| black |
        0 40 [a..b] [| white |
            black zero? white zero? and not [
                black 60 [a..b] [| b |
                    white 40 [a..b] [| w |
                        b w p181-index ways nth
                        b black - w white - p181-index ways nth +
                        b w p181-index ways set-nth
                    ] each
                ] each
            ] when
```

Comparison. C exposes direct bounded arithmetic. Rust exposes direct bounded arithmetic. Factor exposes direct bounded arithmetic. No validated source exists for Gforth. All listed programs print the same exact answer string from the manifest. The excerpts can differ in algorithm, data representation, and startup cost.

40.182 Problem 182: RSA Encryption

For RSA exponent e, a message is fixed by encryption modulo a prime p in 1 + gcd(e-1,p-1) ways. CRT multiplies the counts for p and q. We scan the valid exponents, retain the smallest fixed-message count, and sum every e attaining it. This is only phi=(p-1)(q-1) iterations.

C. Source: `solutions/c/rsa_encryption_182.c`.

```
static uint64_t gcd182(uint64_t a, uint64_t b) {
    while (b) { uint64_t r = a % b; a = b; b = r; }
    return a;
}

int main(void) {
    const uint64_t p = 1009, q = 3643, phi = (p - 1) * (q - 1);
    uint64_t minimum = UINT64_MAX, answer = 0;
    for (uint64_t e = 2; e < phi; ++e) {
        if (gcd182(e, phi) != 1) continue;
        uint64_t unconcealed = (gcd182(e - 1, p - 1) + 1) *
                               (gcd182(e - 1, q - 1) + 1);
        if (unconcealed < minimum) { minimum = unconcealed; answer = e; }
        else if (unconcealed == minimum) answer += e;
```

Rust. Source: `solutions/rust/src/bin/p182.rs`.

```
fn gcd(mut a: u64, mut b: u64) -> u64 {
    while b != 0 {
        (a, b) = (b, a % b);
    }
    a
}
```

```
fn solve() -> u64 {
    let (p, q) = (1009u64, 3643u64);
    let phi = (p - 1) * (q - 1);
    let (mut minimum, mut answer) = (u64::MAX, 0);
    for e in 2..phi {
        if gcd(e, phi) != 1 {
            continue;
        }
```

Factor. Source: `solutions/factor/euler/extended/extended.factor`.

```
:: p182 ( -- n )
    1009 :> p 3643 :> q p 1 - q 1 - * :> phi
    1000000000 :> minimum! 0 :> answer!
    2 phi 1 - [a..b] [| e |
        e phi gcd nip 1 = [
            e 1 - p 1 - gcd nip 1 +
            e 1 - q 1 - gcd nip 1 + * :> fixed
            fixed minimum < [ fixed minimum! e answer! ] [
                fixed minimum = [ answer e + answer! ] when
            ] if
        ] when
    ] each answer ;

! Problem 183: the optimum number of parts is adjacent to n/e; termination
```

Comparison. C exposes prime generation or testing. Shared calls: `gcd`. Rust exposes direct bounded arithmetic. Factor exposes prime generation or testing. No validated source exists for Gforth. All listed programs print the same exact answer string from the manifest. The excerpts can differ in algorithm, data representation, and startup cost.

40.183 Problem 183: Maximum Product of Parts

For k equal parts, the product is (N/k)^k and its logarithm is maximized at k=N/e. Therefore only floor(N/e) and ceil(N/e) need comparison. After reducing N/k, the decimal terminates exactly when its denominator contains no prime factors other than 2 and 5.

C. Source: `solutions/c/maximum_product_of_parts_183.c`.

```
static unsigned gcd183(unsigned a, unsigned b) {
    while (b) { unsigned r = a % b; a = b; b = r; }
    return a;
}

int main(void) {
    int64_t answer = 0;
    for (unsigned n = 5; n <= 10000; ++n) {
        unsigned low = (unsigned)(n / exp(1.0));
        unsigned high = low + 1;
        unsigned k = low * log((double)n / low) > high * log((double)n / high) ? low : high;
        unsigned denominator = k / gcd183(n, k);
        while (denominator % 2 == 0) denominator /= 2;
```

```
        while (denominator % 5 == 0) denominator /= 5;
```

Rust. Source: `solutions/rust/src/bin/p183.rs`.

```
fn gcd(mut a: u32, mut b: u32) -> u32 {
    while b != 0 {
        (a, b) = (b, a % b);
    }
    a
}
fn solve() -> i64 {
    let mut answer = 0i64;
    for n in 5u32..=10_000 {
        let low = (n as f64 / std::f64::consts::E).floor() as u32;
        let high = low + 1;
        let score = |k: u32| k as f64 * (n as f64 / k as f64).ln();
        let k = if score(low) > score(high) { low } else { high };
        let mut denominator = k / gcd(n, k);
```

Factor. Source: `solutions/factor/euler/extended/extended.factor`.

```
:: p183 ( -- n )
    0 :> answer!
    5 10000 [a..b] [| n |
        n >float e / floor >integer :> low
        low 1 + :> high
        low >float n >float low / log *
        high >float n >float high / log * > [ low ] [ high ] if :> k
        k n k gcd nip /i :> denominator!
        [ denominator 2 mod zero? ] [ denominator 2 /i denominator! ] while
        [ denominator 5 mod zero? ] [ denominator 5 /i denominator! ] while
        denominator 1 = [ answer n - ] [ answer n + ] if answer!
    ] each answer ;

! Problems 197, 199 and 235 are numerical processes; formatting remains in
```

Comparison. C exposes prime generation or testing. Rust exposes prime generation or testing. Factor exposes modular arithmetic. No validated source exists for Gforth. All listed programs print the same exact answer string from the manifest. The excerpts can differ in algorithm, data representation, and startup cost.

40.184 Problem 184: Triangles Containing the Origin

The program computes the exact answer for Triangles Containing the Origin.

C. Source: `solutions/c/triangles_containing_the_origin_184.c`.

```
enum { RADIUS = 105 };

typedef struct {
    int x;
```

```
    int y;
} Point;

static int half(const Point p) {
    return p.y > 0 || (p.y == 0 && p.x > 0) ? 0 : 1;
}

static int angle_compare(const void *left, const void *right) {
    const Point a = *(const Point *)left;
    const Point b = *(const Point *)right;
```

Rust. Source: `solutions/rust/src/bin/p184.rs`.

```
#[derive(Clone, Copy)]
struct Point {
    x: i32,
    y: i32,
}
fn half(p: Point) -> bool {
    p.y < 0 || (p.y == 0 && p.x < 0)
}
fn gcd(mut a: i32, mut b: i32) -> i32 {
    while b != 0 {
        (a, b) = (b, a % b);
    }
    a
}
```

Comparison. C exposes sorting or set deduplication. Rust exposes sorting or set deduplication. No validated source exists for Gforth, and Factor. All listed programs print the same exact answer string from the manifest. The excerpts can differ in algorithm, data representation, and startup cost.

40.185 Problem 185: Number Mind

The program computes the exact answer for Number Mind.

C. Source: `solutions/c/number_mind_185.c`.

```
enum { CLUES = 22, WIDTH = 16 };

static const char guesses[CLUES][WIDTH + 1] = {
    "5616185650518293", "3847439647293047", "5855462940810587",
    "9742855507068353", "4296849643607543", "3174248439465858",
    "4513559094146117", "7890971548908067", "8157356344118483",
    "2615250744386899", "8690095851526254", "6375711915077050",
    "6913859173121360", "6442889055042768", "2321386104303845",
    "2326509471271448", "5251583379644322", "1748270476758276",
    "4895722652190306", "3041631117224635", "1841236454324589",
    "2659862637316867"
};

static const unsigned char targets[CLUES] = {
```

Rust. Source: `solutions/rust/src/bin/p185.rs`.

```
const GUESSES: [&[u8; 16]; 22] = [
    b"5616185650518293",
    b"3847439647293047",
    b"5855462940810587",
    b"9742855507068353",
    b"4296849643607543",
    b"3174248439465858",
    b"4513559094146117",
    b"7890971548908067",
    b"8157356344118483",
    b"2615250744386899",
    b"8690095851526254",
    b"6375711915077050",
    b"6913859173121360",
```

Comparison. C exposes sorting or set deduplication. Rust exposes sorting or set deduplication, and recursive search. No validated source exists for Gforth, and Factor. All listed programs print the same exact answer string from the manifest. The excerpts can differ in algorithm, data representation, and startup cost.

40.186 Problem 186: Connectedness of a Network

Generate the exact telephone stream and maintain connected components with union-find. Path compression and union by size make each successful call effectively constant time. Misdials (caller==called) are discarded and do not increment the call counter, exactly as specified.

C. Source: `solutions/c/connectedness_of_a_network_186.c`.

```
enum { USERS186 = 1000000, PM186 = 524287 };
static unsigned parent186[USERS186], size186[USERS186], sequence186[55];
static unsigned find186(unsigned value) {
    if (parent186[value] != value) parent186[value] = find186(parent186[value]);
    return parent186[value];
}
static unsigned next186(uint64_t index) {
    unsigned value;
    if (index <= 55) {
        int64_t raw = 100003 - 200003 * (int64_t)index + 300007 * (int64_t)index * index * index;

        value = (raw % USERS186 + USERS186) % USERS186;
    } else value = (sequence186[(index - 25) % 55] + sequence186[(index - 56) % 55]) % USERS186;
    sequence186[(index - 1) % 55] = value;
    return value;
```

Rust. Source: `solutions/rust/src/bin/p186.rs`.

```
fn find(parent: &mut [usize], mut x: usize) -> usize {
    let mut root = x;
    while parent[root] != root {
        root = parent[root];
    }
```

```
    while parent[x] != x {
        let next = parent[x];
        parent[x] = root;
        x = next;
    }
    root
}
fn solve() -> u64 {
    const N: usize = 1_000_000;
```

Factor. Source: `solutions/factor/p186.factor`.

```
! Native Factor Project Euler 186 union-find network simulation.
USING: io math.parser project-euler.186 ; IN: scratchpad
euler186 number>string print
```

Comparison. C exposes direct bounded arithmetic. Rust exposes direct bounded arithmetic. Factor exposes direct bounded arithmetic. No validated source exists for Gforth. All listed programs print the same exact answer string from the manifest. The excerpts can differ in algorithm, data representation, and startup cost.

40.187 Problem 187: Semiprimes

A semiprime has one unique ordered-by-size representation p*q with p<=q. Sieve all possible primes q below LIMIT/2. For each p, move a shared upper index down until p*q is below LIMIT; adding the interval length counts every prime pair once.

C. Source: `solutions/c/semiprimes_187.c`.

```
int main(void) {
    enum { LIMIT = 100000000, SIEVE = LIMIT / 2 };
    bool *composite = calloc(SIEVE, 1);
    unsigned *primes = malloc(3100000 * sizeof(*primes));
    if (!composite || !primes) return EXIT_FAILURE;
    size_t count = 0;
    for (unsigned n = 2; n < SIEVE; ++n) {
        if (composite[n]) continue;
        primes[count++] = n;
        if ((uint64_t)n * n < SIEVE)
            for (unsigned multiple = n * n; multiple < SIEVE; multiple += n) composite[multiple]
     = true;
    }
    uint64_t answer = 0;
    size_t high = count;
```

Rust. Source: `solutions/rust/src/bin/p187.rs`.

```
fn solve() -> u64 {
    const LIMIT: usize = 100_000_000;
    const SIEVE: usize = LIMIT / 2;
    let mut composite = vec![false; SIEVE];
```

```
    let mut primes = Vec::with_capacity(3_100_000);
    for n in 2..SIEVE {
        if !composite[n] {
            primes.push(n);
            if n <= SIEVE / n {
                for m in (n * n..SIEVE).step_by(n) {
                    composite[m] = true;
                }
            }
        }
```

Comparison. C exposes prime generation or testing. Rust exposes prime generation or testing. No validated source exists for Gforth, and Factor. All listed programs print the same exact answer string from the manifest. The excerpts can differ in algorithm, data representation, and startup cost.

40.188 Problem 188: Hyperexponentiation

Because 1777 is coprime to every modulus in the Euler-totient chain of 10^8, Euler's theorem lets each exponent be reduced modulo phi(m). The modulus reaches 1 after very few levels, so height 1855 costs only a short recursion, not a tower-sized integer.

C. Source: `solutions/c/hyperexponentiation_188.c`.

```
static uint64_t phi188(uint64_t n) {
    uint64_t result = n;
    for (uint64_t p = 2; p * p <= n; ++p) if (n % p == 0) {
        while (n % p == 0) n /= p;
        result -= result / p;
    }
    if (n > 1) result -= result / n;
    return result;
}
static uint64_t power188(uint64_t base, uint64_t exponent, uint64_t modulus) {
    uint64_t result = 1;
    while (exponent) {
        if (exponent & 1) result = result * base % modulus;
        base = base * base % modulus;
```

Rust. Source: `solutions/rust/src/bin/p188.rs`.

```
fn phi(mut n: u64) -> u64 {
    let mut result = n;
    let mut p = 2;
    while p * p <= n {
        if n % p == 0 {
            while n % p == 0 {
                n /= p;
            }
            result -= result / p;
        }
        p += 1;
    }
    if n > 1 {
```

```
        result -= result / n;
```

Factor. Source: `solutions/factor/p188.factor`.

```
! Native Factor Project Euler 188 modular tetration solver.
USING: io math.parser project-euler.188 ; IN: scratchpad
euler188 number>string print
```

Comparison. C exposes prime generation or testing. Rust exposes prime generation or testing, and modular arithmetic. Factor exposes direct bounded arithmetic. No validated source exists for Gforth. All listed programs print the same exact answer string from the manifest. The excerpts can differ in algorithm, data representation, and startup cost.

40.189 Problem 189: Tri-colouring a Triangular Grid

The program computes the exact answer for Tri-colouring a Triangular Grid.

C. Source: `solutions/c/tri_colouring_a_triangular_grid_189.c`.

```
enum { SIDE = 8, MAX_STATES = 6561 };

static int power3(int exponent) {
    int result = 1;
    while (exponent-- > 0) {
        result *= 3;
    }
    return result;
}

static void decode_colours(int state, int length, unsigned char *colours) {
    for (int i = 0; i < length; ++i) {
        colours[i] = (unsigned char)(state % 3);
        state /= 3;
```

Rust. Source: `solutions/rust/src/bin/p189.rs`.

```
fn pow3(n: usize) -> usize {
    3usize.pow(n as u32)
}
fn solve() -> u64 {
    const SIDE: usize = 8;
    const STATES: usize = 6561;
    let mut decoded = vec![[0u8; SIDE]; STATES];
    for state in 0..STATES {
        let mut x = state;
        for j in 0..SIDE {
            decoded[state][j] = (x % 3) as u8;
            x /= 3;
        }
    }
```

Factor. Source: `solutions/factor/euler/extended/extended.factor`.

```
:: p189 ( -- n )
    6561 0 <array> :> previous!
    1 0 previous set-nth 1 1 previous set-nth 1 2 previous set-nth
    2 8 [a..b] [| row |
        6561 0 <array> :> current
        0 3 row 1 - ^ 1 - [a..b] [| before |
            before previous nth zero? not [
                0 3 row ^ 1 - [a..b] [| after |
                    1 :> multiplicity!
                    0 row 2 - [a..b] [| j |
                        before j trit :> a after j trit :> b
                        after j 1 + trit :> c
                        1 :> distinct!
                        b a = not [ distinct 1 + distinct! ] when
```

Comparison. C exposes dynamic programming or memoization. Rust exposes direct bounded arithmetic. Factor exposes dynamic programming or memoization. No validated source exists for Gforth. All listed programs print the same exact answer string from the manifest. The excerpts can differ in algorithm, data representation, and startup cost.

40.190 Problem 190: Maximising a Weighted Product

Lagrange multipliers applied to product x_i^i with sum x_i=m give x_i=2i/(m+1). Long double has ample separation from the relevant integer boundaries; multiplying the nineteen terms directly yields floor(P_m).

C. Source: `solutions/c/maximising_a_weighted_product_190.c`.

```
int main(void) {
    uint64_t answer = 0;
    for (unsigned m = 2; m <= 15; ++m) {
        long double product = 1;
        for (unsigned i = 1; i <= m; ++i)
            product *= powl(2.0L * i / (m + 1), i);
        answer += (uint64_t)floorl(product);
    }
    printf("%" PRIu64 "\n", answer);
    return 0;
}
```

Rust. Source: `solutions/rust/src/bin/p190.rs`.

```
fn solve() -> u64 {
    (2..=15)
        .map(|m| {
            let p = (1..=m).fold(1f64, |p, i| p * (2.0 * i as f64 / (m + 1) as f64).powi(i));
            p.floor() as u64
        })
        .sum()
}
fn main() {
```

```
    println!("{}", solve());
}
```

Factor. Source: `solutions/factor/p190.factor`.

```
! Native Factor Project Euler 190 weighted-product maximization.
USING: io math.parser project-euler.190 ; IN: scratchpad
euler190 number>string print
```

Comparison. C exposes direct bounded arithmetic. Rust exposes direct bounded arithmetic. Factor exposes direct bounded arithmetic. No validated source exists for Gforth. All listed programs print the same exact answer string from the manifest. The excerpts can differ in algorithm, data representation, and startup cost.

40.191 Problem 191: Prize Strings

The DP state records whether an absence has appeared and the current run of consecutive lates (0,1,2). Appending O resets the late run, A is allowed only when 'absent==0', and L is allowed only while the run is shorter than three. Those two fields are the complete history relevant to future choices. Two rolling 2x3 tables advance exactly thirty days. Summing all six terminal states counts every valid attendance string once; uint64_t comfortably holds the result.

C. Source: `solutions/c/prize_strings_191.c`.

```
int main(void) {
    uint64_t dp[2][3] = {{1,0,0},{0,0,0}}, next[2][3];
    for (unsigned day = 0; day < 30; ++day) {
        memset(next, 0, sizeof(next));
        for (unsigned absent = 0; absent < 2; ++absent) for (unsigned late = 0; late < 3; ++late)
     {
            next[absent][0] += dp[absent][late];
            if (!absent) next[1][0] += dp[absent][late];
            if (late < 2) next[absent][late + 1] += dp[absent][late];
        }
        memcpy(dp, next, sizeof(dp));
    }
    uint64_t answer = 0;
    for (unsigned a = 0; a < 2; ++a) for (unsigned l = 0; l < 3; ++l) answer += dp[a][l];
    printf("%" PRIu64 "\n", answer);
```

Rust. Source: `solutions/rust/src/bin/p191.rs`.

```
fn solve() -> u64 {
    let mut dp = [[0u64; 3]; 2];
    dp[0][0] = 1;
    for _ in 0..30 {
        let mut next = [[0; 3]; 2];
        for a in 0..2 {
            for l in 0..3 {
                next[a][0] += dp[a][l];
                if a == 0 {
```

```
                    next[1][0] += dp[a][l];
                }
                if l < 2 {
                    next[a][l + 1] += dp[a][l];
                }
```

Factor. Source: `solutions/factor/euler/extended/extended.factor`.

```
:: p191 ( -- n )
    6 0 <array> :> dp! 1 0 dp set-nth
    30 [
        6 0 <array> :> next
        0 1 [a..b] [| absent | 0 2 [a..b] [| late |
            absent late p191-index dp nth :> ways
            absent 0 p191-index next nth ways + absent 0 p191-index next set-nth
            absent zero? [ 1 0 p191-index next nth ways + 1 0 p191-index next set-nth ] when
            late 2 < [ absent late 1 + p191-index next nth ways + absent late 1 + p191-index
    next set-nth ] when
        ] each ] each next dp!
    ] times dp sum ;

! Problem 194: chromatic polynomials for the two fixed-boundary unit graphs,
! Pascal recurrence for C(100,25), all modulo the composite 10^8.
```

Comparison. C exposes dynamic programming or memoization. Rust exposes dynamic programming or memoization. Factor exposes dynamic programming or memoization. No validated source exists for Gforth. All listed programs print the same exact answer string from the manifest. The excerpts can differ in algorithm, data representation, and startup cost.

40.192 Problem 192: Best Approximations

The program computes the exact answer for Best Approximations.

C. Source: `solutions/c/best_approximations_192.c`.

```
enum { LIMIT_N = 100000 };
static const uint64_t DENOMINATOR_LIMIT = UINT64_C(1000000000000);

static int integer_sqrt(int n) {
    int low = 0;
    int high = 317;
    while (low + 1 < high) {
        const int middle = (low + high) / 2;
        if (middle * middle <= n) {
            low = middle;
        } else {
            high = middle;
        }
    }
```

Rust. Source: `solutions/rust/src/bin/p192.rs`.

```
fn closer(n: u64, p1: u64, q1: u64, p2: u64, q2: u64) -> bool {
    if p1 * q2 == p2 * q1 {
        return false;
    }
    let first_lower = p1 * q2 < p2 * q1;
    let midpoint_num = BigUint::from(p1) * q2 + BigUint::from(p2) * q1;
    let midpoint_den = BigUint::from(2u8) * q1 * q2;
    let root_below_mid =
        BigUint::from(n) * &midpoint_den * &midpoint_den < &midpoint_num * &midpoint_num;
    // The lower fraction is nearer exactly when the root is below midpoint.
    if first_lower {
        root_below_mid
    } else {
        !root_below_mid
```

Comparison. C exposes direct bounded arithmetic. Rust exposes exact large-integer arithmetic. No validated source exists for Gforth, and Factor. All listed programs print the same exact answer string from the manifest. The excerpts can differ in algorithm, data representation, and startup cost.

40.193 Problem 193: Squarefree Numbers

Inclusion-exclusion over square divisors is exactly the Möbius identity Q(N) = sum_{d<=sqrt(N)} mu(d) floor(N/d^2). A prime sieve constructs mu: each prime flips multiples once and zeros multiples of its square. N is 2^50-1 because the bound is strict.

C. Source: `solutions/c/squarefree_numbers_193.c`.

```
int main(void) {
    const uint64_t limit = (1ULL << 50) - 1;
    const unsigned root = 1u << 25;
    int8_t *mobius = malloc((root + 1) * sizeof(*mobius));
    bool *composite = calloc(root + 1, 1);
    if (!mobius || !composite) return EXIT_FAILURE;
    memset(mobius, 1, root + 1);
    for (unsigned prime = 2; prime <= root; ++prime) {
        if (composite[prime]) continue;
        for (unsigned multiple = prime; multiple <= root; multiple += prime) {
            composite[multiple] = multiple != prime ? composite[multiple] : false;
            mobius[multiple] = -mobius[multiple];
        }
        uint64_t square = (uint64_t)prime * prime;
```

Rust. Source: `solutions/rust/src/bin/p193.rs`.

```
fn solve() -> i64 {
    let limit = (1u64 << 50) - 1;
    let root = 1usize << 25;
    let mut mu = vec![1i8; root + 1];
    let mut composite = vec![false; root + 1];
    for p in 2..=root {
        if composite[p] {
            continue;
```

```
        }
        for m in (p..=root).step_by(p) {
            mu[m] = -mu[m];
        }
        if p <= root / p {
            let square = p * p;
```

Comparison. C exposes prime generation or testing. Rust exposes prime generation or testing. No validated source exists for Gforth, and Factor. All listed programs print the same exact answer string from the manifest. The excerpts can differ in algorithm, data representation, and startup cost.

40.194 Problem 194: Coloured Configurations

The program computes the exact answer for Coloured Configurations.

C. Source: `solutions/c/coloured_configurations_194.c`.

```
static const uint64_t MODULUS = UINT64_C(100000000);

static uint64_t multiply_mod(uint64_t a, uint64_t b) {
    return (a * b) % MODULUS;
}

static uint64_t power_mod(uint64_t base, unsigned exponent) {
    uint64_t result = 1;
    while (exponent != 0) {
        if ((exponent & 1U) != 0U) {
            result = multiply_mod(result, base);
        }
        base = multiply_mod(base, base);
        exponent >>= 1U;
```

Rust. Source: `solutions/rust/src/bin/p194.rs`.

```
const M: u64 = 100_000_000;
fn powmod(mut b: u64, mut e: u32) -> u64 {
    let mut r = 1;
    while e > 0 {
        if e & 1 == 1 {
            r = r * b % M;
        }
        b = b * b % M;
        e >>= 1;
    }
    r
}
fn unit_a(c: i64) -> u64 {
    (((((c - 9) * c + 34) * c - 69) * c + 77) * c - 38) as u64
```

Factor. Source: `solutions/factor/euler/extended/extended.factor`.

```
:: p194 ( -- n )
    26 0 <array> :> row 1 0 row set-nth
    1 100 [a..b] [| i |
        i 25 min 1 [a..b] [| j |
            j row nth j 1 - row nth + 100000000 mod j row set-nth
        ] each
    ] each
    25 row nth 1984 1983 * 100000000 mod * 100000000 mod
    1984 p194-a 100000000 mod 25 100000000 modular-pow * 100000000 mod
    1984 p194-b 100000000 mod 75 100000000 modular-pow * 100000000 mod ;

! Problem 205: exact convolution of the two dice distributions.
```

Comparison. C exposes direct bounded arithmetic. Rust exposes modular arithmetic. Factor exposes modular arithmetic. Shared calls: `modular-pow`. No validated source exists for Gforth. All listed programs print the same exact answer string from the manifest. The excerpts can differ in algorithm, data representation, and startup cost.

40.195 Problem 195: 60-degree Triangle Inscribed Circles

The program computes the exact answer for 60-degree Triangle Inscribed Circles.

C. Source: `solutions/c/inscribed_circles_of_triangles_with_one_angle_of_60_degrees_195.c`.

```
static int gcd_positive(int a, int b) {
    while (b != 0) {
        const int remainder = a % b;
        a = b;
        b = remainder;
    }
    return a;
}

static int scale_count(int64_t c, int64_t q) {
    int64_t k = (int64_t)((long double)c / ((long double)q * sqrtl(3.0L)));
    const __int128 c_squared = (__int128)c * c;
    while ((__int128)3 * q * q * (k + 1) * (k + 1) <= c_squared) {
        ++k;
```

Rust. Source: `solutions/rust/src/bin/p195.rs`.

```
fn gcd(mut a: i64, mut b: i64) -> i64 {
    while b != 0 {
        (a, b) = (b, a % b);
    }
    a
}
fn scale_count(c: i64, q: i64) -> i64 {
    let mut k = (c as f64 / (q as f64 * 3f64.sqrt())) as i64;
    let c2 = c as i128 * c as i128;
    while 3i128 * q as i128 * q as i128 * (k + 1) as i128 * (k + 1) as i128 <= c2 {
```

```
        k += 1;
    }
    while 3i128 * q as i128 * q as i128 * k as i128 * k as i128 > c2 {
        k -= 1;
```

Comparison. C exposes prime generation or testing. Rust exposes prime generation or testing. No validated source exists for Gforth, and Factor. All listed programs print the same exact answer string from the manifest. The excerpts can differ in algorithm, data representation, and startup cost.

40.196 Problem 196: Prime Triplets

The program computes the exact answer for Prime Triplets.

C. Source: `solutions/c/prime_triplets_196.c`.

```
typedef struct {
    uint64_t low;
    uint64_t high;
    unsigned char *prime;
} PrimeStrip;

static uint64_t row_start(int64_t row) {
    return (uint64_t)row * (uint64_t)(row - 1) / 2 + 1;
}

static uint64_t row_end(int64_t row) {
    return (uint64_t)row * (uint64_t)(row + 1) / 2;
}
```

Rust. Source: `solutions/rust/src/bin/p196.rs`.

```
fn row_start(r: i64) -> u64 {
    r as u64 * (r - 1) as u64 / 2 + 1
}
fn row_end(r: i64) -> u64 {
    r as u64 * (r + 1) as u64 / 2
}
struct Strip {
    low: u64,
    high: u64,
    prime: Vec<bool>,
}
impl Strip {
    fn at(&self, r: i64, c: i64) -> bool {
        if r < 1 || c < 0 || c >= r {
```

Comparison. C exposes prime generation or testing, and dynamic programming or memoization. Rust exposes prime generation or testing. No validated source exists for Gforth, and Factor. All listed programs print the same exact answer string from the manifest. The excerpts can differ in algorithm, data representation, and startup cost.

40.197 Problem 197: A Recursively Defined Sequence

Direct iteration is the intended numerical process. The recurrence rapidly settles into a two-cycle, so u_n+u_(n+1) stabilizes long before n=10^12. Iterating 2000 times is deliberately excessive and printing nine places matches the requested rounding.

C. Source: `solutions/c/investigating_the_behaviour_of_a_recursively_defined_sequence_197.c`.

```
static double next197(double value) {
    return floor(pow(2.0, 30.403243784 - value * value)) * 1e-9;
}
int main(void) {
    double current = -1.0, next = 0;
    for (unsigned i = 0; i < 2000; ++i) { next = next197(current); current = next; }
    printf("%.9f\n", current + next197(current));
    return 0;
}
```

Rust. Source: `solutions/rust/src/bin/p197.rs`.

```
fn next(u: f64) -> f64 {
    (2f64.powf(30.403243784 - u * u)).floor() * 1e-9
}
fn solve() -> f64 {
    let mut u = -1.0;
    for _ in 0..2000 {
        u = next(u);
    }
    u + next(u)
}
fn main() {
    println!("{:.9}", solve());
}
```

Factor. Source: `solutions/factor/euler/extended/extended.factor`.

```
:: p197 ( -- x ) -1.0 :> x! 2000 [ x p197-next x! ] times x x p197-next + ;
```

Comparison. C exposes direct bounded arithmetic. Rust exposes recursive search. Factor exposes direct bounded arithmetic. No validated source exists for Gforth. All listed programs print the same exact answer string from the manifest. The excerpts can differ in algorithm, data representation, and startup cost.

40.198 Problem 198: Ambiguous Numbers

The program computes the exact answer for Ambiguous Numbers.

C. Source: `solutions/c/ambiguous_numbers_198.c`.

```
static const uint64_t PRODUCT_LIMIT = UINT64_C(50000000);

typedef struct {
    uint64_t a;
    uint64_t b;
    uint64_t c;
    uint64_t d;
} Interval;

static int midpoint_is_below_limit(const Interval interval) {
    return UINT64_C(100) * (interval.a * interval.d + interval.b * interval.c) <
           2 * interval.b * interval.d;
}
```

Rust. Source: `solutions/rust/src/bin/p198.rs`.

```
#[derive(Clone, Copy)]
struct I {
    a: u64,
    b: u64,
    c: u64,
    d: u64,
}
const L: u64 = 50_000_000;
fn midpoint(i: I) -> bool {
    100 * (i.a * i.d + i.b * i.c) < 2 * i.b * i.d
}
fn left(i: I) -> bool {
    100 * i.a < i.b
}
```

Comparison. C exposes direct bounded arithmetic. Rust exposes direct bounded arithmetic. No validated source exists for Gforth, and Factor. All listed programs print the same exact answer string from the manifest. The excerpts can differ in algorithm, data representation, and startup cost.

40.199 Problem 199: Iterative Circle Packing

The program computes the exact answer for Iterative Circle Packing.

C. Source: `solutions/c/iterative_circle_packing_199.c`.

```
static double filled_fraction(double k1, double k2, double k3, int depth) {
    if (depth == 0) {
        return 0.0;
    }
    const double k4 = k1 + k2 + k3 +
                      2.0 * sqrt(k1 * k2 + k2 * k3 + k3 * k1);
    return 1.0 / (k4 * k4) +
           filled_fraction(k1, k2, k4, depth - 1) +
           filled_fraction(k2, k3, k4, depth - 1) +
           filled_fraction(k3, k1, k4, depth - 1);
}
```

```
int main(void) {
    const double inner_radius = 2.0 * sqrt(3.0) - 3.0;
```

Rust. Source: `solutions/rust/src/bin/p199.rs`.

```
fn fill(a: f64, b: f64, c: f64, depth: u8) -> f64 {
    if depth == 0 {
        return 0.0;
    }
    let d = a + b + c + 2.0 * (a * b + b * c + c * a).sqrt();
    1.0 / (d * d) + fill(a, b, d, depth - 1) + fill(b, c, d, depth - 1) + fill(c, a, d, depth -
    1)
}
fn solve() -> f64 {
    let r = 2.0 * 3f64.sqrt() - 3.0;
    let k = 1.0 / r;
    1.0 - (3.0 * r * r + fill(k, k, k, 10) + 3.0 * fill(-1.0, k, k, 10))
}
fn main() {
    println!("{:.8}", solve());
```

Factor. Source: `solutions/factor/euler/extended/extended.factor`.

```
:: p199 ( -- x )
    3.0 sqrt 2.0 * 3.0 - :> radius
    1.0 radius / :> curvature
    1.0 radius sq 3.0 * -
    curvature curvature curvature 10 p199-fill -
    -1.0 curvature curvature 10 p199-fill 3.0 * - ;
```

Comparison. C exposes direct bounded arithmetic. Rust exposes direct bounded arithmetic. Factor exposes direct bounded arithmetic. No validated source exists for Gforth. All listed programs print the same exact answer string from the manifest. The excerpts can differ in algorithm, data representation, and startup cost.

40.200 Problem 200: Prime-proof Squbes

The program computes the exact answer for Prime-proof Squbes.

C. Source: `solutions/c/prime_proof_squbes_200.c`.

```
static const uint64_t SEARCH_LIMIT = UINT64_C(1000000000000);

static uint64_t modular_power(uint64_t base, uint64_t exponent, uint64_t modulus) {
    uint64_t result = 1;
    while (exponent != 0) {
        if ((exponent & 1U) != 0U) {
            result = (uint64_t)((__uint128_t)result * base % modulus);
        }
        base = (uint64_t)((__uint128_t)base * base % modulus);
        exponent >>= 1U;
```

```
    }
    return result;
}
```

Rust. Source: `solutions/rust/src/bin/p200.rs`.

```
fn powmod(mut b: u64, mut e: u64, m: u64) -> u64 {
    let mut r = 1;
    while e > 0 {
        if e & 1 == 1 {
            r = (r as u128 * b as u128 % m as u128) as u64;
        }
        b = (b as u128 * b as u128 % m as u128) as u64;
        e >>= 1;
    }
    r
}
fn prime(n: u64) -> bool {
    for p in [2, 3, 5, 7, 11, 13, 17, 19, 23, 29, 31, 37] {
        if n % p == 0 {
```

Comparison. C exposes prime generation or testing, and sorting or set deduplication. Rust exposes prime generation or testing, modular arithmetic, and sorting or set deduplication. No validated source exists for Gforth, and Factor. All listed programs print the same exact answer string from the manifest. The excerpts can differ in algorithm, data representation, and startup cost.

40.201 Problem 201: Subsets with a Unique Sum

DP[k][s] stores how many ways the processed squares choose k terms totaling s, but counts are saturated at two: the final question distinguishes only zero, unique, and non-unique. Descending k and sum prevent reusing the same square in one subset. The maximum possible sum of all 100 squares bounds a compact byte table.

C. Source: `solutions/c/subsets_with_a_unique_sum_201.c`.

```
int main(void) {
    enum { CHOOSE = 50, MAX_SUM = 338350 };
    unsigned char *ways = calloc((CHOOSE + 1) * (MAX_SUM + 1), 1);
    if (!ways) return EXIT_FAILURE;
#define W201(k,s) ways[(size_t)(k) * (MAX_SUM + 1) + (s)]
    W201(0,0) = 1;
    unsigned reachable = 0;
    for (unsigned n = 1; n <= 100; ++n) {
        unsigned square = n * n;
        reachable += square;
        unsigned top = n < CHOOSE ? n : CHOOSE;
        for (unsigned k = top; k >= 1; --k) {
            for (unsigned sum = reachable; sum >= square; --sum) {
                unsigned count = W201(k,sum) + W201(k-1,sum-square);
```

Rust. Source: `solutions/rust/src/bin/p201.rs`.

```
fn solve() -> u64 {
    const CHOOSE: usize = 50;
    const MAX: usize = 338350;
    let width = MAX + 1;
    let mut ways = vec![0u8; (CHOOSE + 1) * width];
    ways[0] = 1;
    let mut reachable = 0;
    for n in 1..=100usize {
        let sq = n * n;
        reachable += sq;
        for k in (1..=n.min(CHOOSE)).rev() {
            for s in (sq..=reachable).rev() {
                let i = k * width + s;
                ways[i] = (ways[i] + ways[(k - 1) * width + s - sq]).min(2);
```

Comparison. C exposes dynamic programming or memoization. Rust exposes direct bounded arithmetic. No validated source exists for Gforth, and Factor. All listed programs print the same exact answer string from the manifest. The excerpts can differ in algorithm, data representation, and startup cost.

40.202 Problem 202: Laserbeam

The program computes the exact answer for Laserbeam.

C. Source: `solutions/c/laserbeam_202.c`.

```
static uint64_t count_k_in_residue(uint64_t maximum, int residue) {
    if (residue == 0) {
        return maximum / 3;
    }
    if (maximum < (uint64_t)residue) {
        return 0;
    }
    return (maximum - (uint64_t)residue) / 3 + 1;
}

static uint64_t multiples_in_residue(uint64_t n, uint64_t divisor, int residue) {
    const uint64_t maximum_k = (n - 1) / divisor;
    const int divisor_mod3 = (int)(divisor % 3);
    if (divisor_mod3 == 0) {
```

Rust. Source: `solutions/rust/src/bin/p202.rs`.

```
fn residue_count(max: u64, r: u64) -> u64 {
    if r == 0 {
        max / 3
    } else if max < r {
        0
    } else {
        (max - r) / 3 + 1
    }
```

```
}
fn multiples(n: u64, d: u64, r: u64) -> u64 {
    let max = (n - 1) / d;
    match d % 3 {
        0 => {
            if r == 0 {
```

Factor. Source: `solutions/factor/euler/extended/extended.factor`.

```
:: p202 ( -- n )
    12017639150 2 /i :> n n :> remaining!
    V{ } clone :> factors 2 :> p!
    [ p sq remaining <= ] [
        remaining p mod zero? [
            p factors push [ remaining p mod zero? ] [ remaining p /i remaining! ] while
        ] when p 2 = [ 3 ] [ p 2 + ] if p!
    ] while remaining 1 > [ remaining factors push ] when
    n 3 mod 2 * 3 mod :> target 0 :> total!
    0 2 factors length ^ 1 - [a..b] [| mask |
        1 :> divisor! f :> odd!
        factors [| prime i | mask 2 i ^ /i 2 mod 1 = [
            divisor prime * divisor! odd not odd!
        ] when ] each-index
```

Comparison. C exposes prime generation or testing, and modular arithmetic. Rust exposes prime generation or testing, and modular arithmetic. Shared calls: `gcd`. Factor exposes prime generation or testing, and modular arithmetic. No validated source exists for Gforth. All listed programs print the same exact answer string from the manifest. The excerpts can differ in algorithm, data representation, and startup cost.

40.203 Problem 203: Squarefree Binomial Coefficients

Generate Pascal rows 0..50 exactly in 64 bits, collect all coefficients, sort away duplicates, and reject a value if any prime square divides it. Trial division only reaches sqrt(C(50,25)), which is tiny for this set.

C. Source: `solutions/c/squarefree_binomial_coefficients_203.c`.

```
static int compare203(const void *a, const void *b) {
    uint64_t x = *(const uint64_t *)a, y = *(const uint64_t *)b;
    return x < y ? -1 : x > y;
}
static bool squarefree203(uint64_t value) {
    for (uint64_t p = 2; p * p <= value; ++p)
        if (value % (p * p) == 0) return false;
    return true;
}
int main(void) {
    uint64_t values[1326]; size_t count = 0;
    for (unsigned n = 0; n <= 50; ++n) {
        uint64_t coefficient = 1;
        for (unsigned k = 0; k <= n; ++k) {
```

Rust. Source: `solutions/rust/src/bin/p203.rs`.

```
fn squarefree(v: u64) -> bool {
    let mut p = 2;
    while p <= v / p {
        if v % (p * p) == 0 {
            return false;
        }
        p += 1;
    }
    true
}
fn solve() -> u64 {
    let mut values = Vec::new();
    for n in 0..=50u64 {
        let mut c = 1;
```

Factor. Source: `solutions/factor/p203.factor`.

```
! Native Factor Project Euler 203 squarefree-binomial solver.
USING: io math.parser project-euler.203 ; IN: scratchpad
euler203 number>string print
```

Comparison. C exposes prime generation or testing, and sorting or set deduplication. Rust exposes prime generation or testing, and sorting or set deduplication. Factor exposes direct bounded arithmetic. No validated source exists for Gforth. All listed programs print the same exact answer string from the manifest. The excerpts can differ in algorithm, data representation, and startup cost.

40.204 Problem 204: Generalised Hamming Numbers

Every type-100 Hamming number has a unique nondecreasing prime factorization using primes <=100. The DFS chooses an exponent for one prime, then moves to the next, so each number is counted exactly once.

C. Source: `solutions/c/generalised_hamming_numbers_204.c`.

```
static const unsigned primes204[] =
    {2,3,5,7,11,13,17,19,23,29,31,37,41,43,47,53,59,61,67,71,73,79,83,89,97};
static uint64_t count204(unsigned position, uint64_t value) {
    if (position == sizeof(primes204) / sizeof(primes204[0])) return 1;
    uint64_t result = 0;
    do {
        result += count204(position + 1, value);
        if (value > 1000000000ULL / primes204[position]) break;
        value *= primes204[position];
    } while (true);
    return result;
}
int main(void) { printf("%" PRIu64 "\n", count204(0, 1)); return 0; }
```

Rust. Source: `solutions/rust/src/bin/p204.rs`.

```
const PS: [u64; 25] = [
    2, 3, 5, 7, 11, 13, 17, 19, 23, 29, 31, 37, 41, 43, 47, 53, 59, 61, 67, 71, 73, 79, 83, 89,
    97,
];
fn count(i: usize, mut value: u64) -> u64 {
    if i == PS.len() {
        return 1;
    }
    let mut result = 0;
    loop {
        result += count(i + 1, value);
        if value > 1_000_000_000 / PS[i] {
            break;
        }
        value *= PS[i];
```

Factor. Source: `solutions/factor/euler/extended/extended.factor`.

```
: p204 ( -- n ) 0 1 p204-count ;

! Problem 207: perfect partitions occur exactly at powers-of-two m.
```

Comparison. C exposes prime generation or testing. Rust exposes prime generation or testing. Factor exposes direct bounded arithmetic. No validated source exists for Gforth. All listed programs print the same exact answer string from the manifest. The excerpts can differ in algorithm, data representation, and startup cost.

40.205 Problem 205: Dice Game

Convolution builds the exact frequency distribution for each player's dice. Summing Peter's frequency times Colin's cumulative lower frequency gives the favorable outcomes; division occurs only for final formatting.

C. Source: `solutions/c/dice_game_205.c`.

```
static void distribution205(unsigned dice, unsigned sides, uint64_t *result) {
    uint64_t current[64] = {1}, next[64];
    for (unsigned d = 0; d < dice; ++d) {
        memset(next, 0, sizeof(next));
        for (unsigned sum = 0; sum < 63; ++sum)
            for (unsigned face = 1; face <= sides; ++face) next[sum + face] += current[sum];
        memcpy(current, next, sizeof(current));
    }
    memcpy(result, current, sizeof(current));
}
int main(void) {
    uint64_t peter[64], colin[64]; distribution205(9, 4, peter); distribution205(6, 6, colin);
    uint64_t favorable = 0, lower = 0;
    for (unsigned sum = 0; sum < 64; ++sum) { favorable += peter[sum] * lower; lower += colin[
    sum]; }
```

Rust. Source: `solutions/rust/src/bin/p205.rs`.

```
fn distribution(dice: usize, sides: usize) -> [u64; 64] {
    let mut cur = [0u64; 64];
    cur[0] = 1;
    for _ in 0..dice {
        let mut next = [0; 64];
        for s in 0..64 - sides {
            for f in 1..=sides {
                next[s + f] += cur[s];
            }
        }
        cur = next;
    }
    cur
}
```

Factor. Source: `solutions/factor/euler/extended/extended.factor`.

```
:: p205 ( -- x )
    9 4 dice-distribution :> peter 6 6 dice-distribution :> colin
    0 :> favorable! 0 :> lower!
    0 63 [a..b] [| total |
        favorable total peter nth lower * + favorable!
        lower total colin nth + lower!
    ] each favorable >float 4 9 ^ 6 6 ^ * >float / ;

! Problem 208: profile DP over four direction counts and current heading; the
! step number determines the fifth count.
```

Comparison. C exposes direct bounded arithmetic. Rust exposes direct bounded arithmetic. Factor exposes dynamic programming or memoization. No validated source exists for Gforth. All listed programs print the same exact answer string from the manifest. The excerpts can differ in algorithm, data representation, and startup cost.

40.206 Problem 206: Concealed Square

The square pattern ends in 900, so its root ends in 30 or 70. We scan only those two residue classes between the square-root bounds and test alternate decimal digits from right to left.

C. Source: `solutions/c/concealed_square_206.c`.

```
static bool pattern206(uint64_t square) {
    static const unsigned expected[] = {0,9,8,7,6,5,4,3,2,1};
    for (unsigned i = 0; i < 10; ++i) { if (square % 10 != expected[i]) return false; square /=
    100; }
    return true;
}
int main(void) {
    for (uint64_t root = 1010101030; root <= 1389026630; root += root % 100 == 30 ? 40 : 60) {
        if (pattern206(root * root)) { printf("%" PRIu64 "\n", root); return 0; }
    }
    return EXIT_FAILURE;
```

```
}
```

Rust. Source: `solutions/rust/src/bin/p206.rs`.

```
fn pattern(mut s: u64) -> bool {
    for expected in [0, 9, 8, 7, 6, 5, 4, 3, 2, 1] {
        if s % 10 != expected {
            return false;
        }
        s /= 100;
    }
    true
}
fn solve() -> u64 {
    let mut r = 1_010_101_030;
    while r <= 1_389_026_630 {
        if pattern(r * r) {
            return r;
```

Factor. Source: `solutions/factor/p206.factor`.

```
! Native Factor Project Euler 206 concealed-square search.
USING: io math.parser project-euler.206 ; IN: scratchpad
euler206 number>string print
```

Comparison. C exposes direct bounded arithmetic. Rust exposes direct bounded arithmetic. Factor exposes direct bounded arithmetic. No validated source exists for Gforth. All listed programs print the same exact answer string from the manifest. The excerpts can differ in algorithm, data representation, and startup cost.

40.207 Problem 207: Integer Partition Equations

Writing k=m(m-1) turns 4^t=2^t+k into m=2^t. Up through a given m there are m-1 partitions and exactly floor(log2(m)) perfect ones. Scan until the strict ratio bound is crossed, then convert m back to k.

C. Source: `solutions/c/integer_partition_equations_207.c`.

```
int main(void) {
    uint64_t perfect = 0, next_power = 2;
    for (uint64_t m = 2;; ++m) {
        if (m == next_power) { ++perfect; next_power *= 2; }
        if (perfect * 12345 < m - 1) { printf("%" PRIu64 "\n", m * (m - 1)); return 0; }
    }
}
```

Rust. Source: `solutions/rust/src/bin/p207.rs`.

```
fn solve() -> u64 {
    let (mut perfect, mut next) = (0u64, 2u64);
```

```
    for m in 2u64.. {
        if m == next {
            perfect += 1;
            next *= 2;
        }
        if perfect * 12345 < m - 1 {
            return m * (m - 1);
        }
    }
    unreachable!()
}
fn main() {
```

Factor. Source: `solutions/factor/euler/extended/extended.factor`.

```
:: p207 ( -- n )
    0 :> perfect! 2 :> next! 2 :> m! 0 :> answer!
    [ answer zero? ] [
        m next = [ perfect 1 + perfect! next 2 * next! ] when
        perfect 12345 * m 1 - < [ m m 1 - * answer! ] when
        m 1 + m!
    ] while answer ;

! Problem 209: decompose the 64-state Boolean permutation into cycles.  A
! cycle of length L permits F(L-1)+F(L+1) independent sets.
```

Comparison. C exposes direct bounded arithmetic. Rust exposes direct bounded arithmetic. Factor exposes direct bounded arithmetic. No validated source exists for Gforth. All listed programs print the same exact answer string from the manifest. The excerpts can differ in algorithm, data representation, and startup cost.

40.208 Problem 208: Robot Walks

Every arc changes the heading by +/-72 degrees. The five possible movement directions are the fifth roots of unity; a nonnegative integer combination of them sums to zero only when all five direction counts are equal. Thus a 70-arc closed walk uses every direction 14 times. The DP records four counts and the current heading; the fifth count follows from the step index. For a closed 70-arc journey the accumulated turn also restores heading zero, so only that terminal heading belongs to the closed-state class.

C. Source: `solutions/c/robot_walks_208.c`.

```
enum { SIDE208 = 15, STATES208 = SIDE208*SIDE208*SIDE208*SIDE208*5 };
static size_t index208(unsigned a,unsigned b,unsigned c,unsigned d,unsigned h) {
    return (((((size_t)a*SIDE208+b)*SIDE208+c)*SIDE208+d)*5+h);
}
int main(void) {
    uint64_t *current = calloc(STATES208, sizeof(*current));
    uint64_t *next = calloc(STATES208, sizeof(*next));
    if (!current || !next) return EXIT_FAILURE;
    current[index208(0,0,0,0,0)] = 1;
    for (unsigned step = 0; step < 70; ++step) {
```

```
        memset(next, 0, STATES208 * sizeof(*next));
        for (unsigned a=0;a<=14;++a) for(unsigned b=0;b<=14;++b)
        for (unsigned c=0;c<=14;++c) for(unsigned d=0;d<=14;++d) {
            unsigned used = a+b+c+d;
```

Rust. Source: `solutions/rust/src/bin/p208.rs`.

```
const SIDE: usize = 15;
const STATES: usize = SIDE * SIDE * SIDE * SIDE * 5;
fn ix(a: usize, b: usize, c: usize, d: usize, h: usize) -> usize {
    ((((a * SIDE + b) * SIDE + c) * SIDE + d) * 5) + h
}
fn solve() -> u64 {
    let mut cur = vec![0u64; STATES];
    cur[ix(0, 0, 0, 0, 0)] = 1;
    for step in 0..70 {
        let mut next = vec![0u64; STATES];
        for a in 0..=14 {
            for b in 0..=14 {
                for c in 0..=14 {
                    for d in 0..=14 {
```

Factor. Source: `solutions/factor/euler/extended/extended.factor`.

```
:: p208 ( -- n )
    253125 0 <array> :> current!
    1 0 0 0 0 0 p208-index current set-nth
    0 69 [a..b] [| step |
        253125 0 <array> :> next
        0 14 [a..b] [| a | 0 14 [a..b] [| b |
        0 14 [a..b] [| c | 0 14 [a..b] [| d |
            a b + c + d + :> used
            used step <= step used - 14 <= and [
                0 4 [a..b] [| heading |
                    a b c d heading p208-index current nth :> ways
                    ways zero? not [
                        { 4 1 } [| turn |
                            heading turn + 5 mod :> h
```

Comparison. C exposes dynamic programming or memoization. Rust exposes direct bounded arithmetic. Factor exposes modular arithmetic. No validated source exists for Gforth. All listed programs print the same exact answer string from the manifest. The excerpts can differ in algorithm, data representation, and startup cost.

40.209 Problem 209: Circular Logic

The condition couples truth-table entries x and T(x), where T(a,b,c,d,e,f)=(b,c,d,e,f,a XOR (b AND c)). T is a permutation of the 64 inputs, hence decomposes into disjoint cycles. On each cycle the truth values form a circular binary string with no adjacent ones. A length-L cycle has F_(L-1)+F_(L+1) such strings; independent cycles multiply.

C. Source: `solutions/c/circular_logic_209.c`.

```
static unsigned transition209(unsigned value) {
    unsigned a = (value >> 5) & 1, b = (value >> 4) & 1, c = (value >> 3) & 1;
    return ((value << 1) & 63) | (a ^ (b & c));
}

int main(void) {
    uint64_t fibonacci[66] = {0, 1};
    for (unsigned i = 2; i < 66; ++i) fibonacci[i] = fibonacci[i - 1] + fibonacci[i - 2];
    bool visited[64] = {false};
    uint64_t answer = 1;
    for (unsigned start = 0; start < 64; ++start) {
        if (visited[start]) continue;
        unsigned length = 0, current = start;
        do { visited[current] = true; ++length; current = transition209(current); } while (
    current != start);
```

Rust. Source: `solutions/rust/src/bin/p209.rs`.

```
fn transition(v: usize) -> usize {
    let (a, b, c) = ((v >> 5) & 1, (v >> 4) & 1, (v >> 3) & 1);
    ((v << 1) & 63) | (a ^ (b & c))
}
fn solve() -> u64 {
    let mut f = [0u64; 66];
    f[1] = 1;
    for i in 2..66 {
        f[i] = f[i - 1] + f[i - 2];
    }
    let mut seen = [false; 64];
    let mut answer = 1;
    for start in 0..64 {
        if seen[start] {
```

Factor. Source: `solutions/factor/euler/extended/extended.factor`.

```
:: p209 ( -- n )
    66 0 <array> :> fib 1 1 fib set-nth
    2 65 [a..b] [| i | i 1 - fib nth i 2 - fib nth + i fib set-nth ] each
    64 f <array> :> visited 1 :> answer!
    0 63 [a..b] [| start |
        start visited nth not [
            0 :> length! start :> current!
            [ current visited nth not ] [
                t current visited set-nth length 1 + length!
                current p209-transition current!
            ] while
            answer length 1 - fib nth length 1 + fib nth + * answer!
        ] when
    ] each answer ;
```

Comparison. C exposes direct bounded arithmetic. Rust exposes direct bounded arithmetic. Factor exposes direct bounded arithmetic. No validated source exists for Gforth. All listed programs

print the same exact answer string from the manifest. The excerpts can differ in algorithm, data representation, and startup cost.

40.210 Problem 210: Obtuse Angled Triangles

The program computes the exact answer for Obtuse Angled Triangles.

C. Source: `solutions/c/obtuse_angled_triangles_210.c`.

```
static uint64_t integer_sqrt(uint64_t value) {
    uint64_t root = (uint64_t)sqrtl((long double)value);
    while ((root + 1) * (root + 1) <= value) {
        ++root;
    }
    while (root * root > value) {
        --root;
    }
    return root;
}

static uint64_t circle_points(uint64_t k) {
    const uint64_t limit = (k * k - 1) / 2;
    const uint64_t radius = integer_sqrt(limit);
```

Rust. Source: `solutions/rust/src/bin/p210.rs`.

```
fn isqrt(n: u64) -> u64 {
    let mut r = (n as f64).sqrt() as u64;
    while (r + 1) <= n / (r + 1) {
        r += 1;
    }
    while r > n / r {
        r -= 1;
    }
    r
}
fn circle(k: u64) -> u64 {
    let limit = (k * k - 1) / 2;
    let radius = isqrt(limit);
    let diagonal = isqrt(limit / 2);
```

Comparison. C exposes direct bounded arithmetic. Rust exposes direct bounded arithmetic. No validated source exists for Gforth, and Factor. All listed programs print the same exact answer string from the manifest. The excerpts can differ in algorithm, data representation, and startup cost.

40.211 Problem 211: Divisor Square Sum

The program computes the exact answer for Divisor Square Sum.

C. Source: `solutions/c/divisor_square_sum_211.c`.

```
enum { LIMIT = 64000000, BLOCK = 1000000, PRIME_LIMIT = 8000 };

static uint64_t integer_sqrt(uint64_t value) {
    uint64_t root = (uint64_t)sqrtl((long double)value);
    while ((root + 1) * (root + 1) <= value) {
        ++root;
    }
    while (root * root > value) {
        --root;
    }
    return root;
}

int main(void) {
```

Rust. Source: `solutions/rust/src/bin/p211.rs`.

```
fn isqrt(n: u64) -> u64 {
    let mut r = (n as f64).sqrt() as u64;
    while (r + 1) <= n / (r + 1) {
        r += 1;
    }
    while r > n / r {
        r -= 1;
    }
    r
}
fn solve() -> u64 {
    const LIMIT: usize = 64_000_000;
    const BLOCK: usize = 1_000_000;
    let mut comp = [false; 8001];
```

Comparison. C exposes prime generation or testing. Rust exposes prime generation or testing. No validated source exists for Gforth, and Factor. All listed programs print the same exact answer string from the manifest. The excerpts can differ in algorithm, data representation, and startup cost.

40.212 Problem 212: Combined Volume of Cuboids

The program computes the exact answer for Combined Volume of Cuboids.

C. Source: `solutions/c/combined_volume_of_cuboids_212.c`.

```
enum { CUBOIDS = 50000, MAX_COORD = 10400, SEQUENCE_TERMS = 6 * CUBOIDS };

typedef struct {
    int x0, x1;
    int y0, y1;
    int z0, z1;
    int next_start;
    int next_end;
    int active_position;
} Cuboid;
```

```
typedef struct {
    int y;
    int z0;
```

Rust. Source: `solutions/rust/src/bin/p212.rs`.

```
const BOXES: usize = 50_000;
const MAX: usize = 10_400;
#[derive(Clone, Copy, Default)]
struct Box3 {
    x0: usize,
    x1: usize,
    y0: usize,
    y1: usize,
    z0: usize,
    z1: usize,
    next_start: isize,
    next_end: isize,
    pos: isize,
}
```

Comparison. C exposes sorting or set deduplication. Rust exposes sorting or set deduplication. No validated source exists for Gforth, and Factor. All listed programs print the same exact answer string from the manifest. The excerpts can differ in algorithm, data representation, and startup cost.

40.213 Problem 213: Flea Circus

The program computes the exact answer for Flea Circus.

C. Source: `solutions/c/flea_circus_213.c`.

```
enum { SIDE = 30, SQUARES = SIDE * SIDE, STEPS = 50 };

static int degree(int row, int column) {
    return 4 - (row == 0) - (row == SIDE - 1) - (column == 0) - (column == SIDE - 1);
}

int main(void) {
    double log_empty[SQUARES] = {0.0};
    double current[SQUARES];
    double next[SQUARES];

    for (int start = 0; start < SQUARES; ++start) {
        memset(current, 0, sizeof(current));
        current[start] = 1.0;
```

Rust. Source: `solutions/rust/src/bin/p213.rs`.

```
fn degree(r: usize, c: usize) -> f64 {
    (4 - (r == 0) as usize - (r == 29) as usize - (c == 0) as usize - (c == 29) as usize) as f64
}
fn solve() -> f64 {
```

```
    let mut logs = vec![0f64; 900];
    for start in 0..900 {
        let mut cur = vec![0f64; 900];
        cur[start] = 1.0;
        for _ in 0..50 {
            let mut next = vec![0f64; 900];
            for r in 0..30 {
                for c in 0..30 {
                    let i = r * 30 + c;
                    let q = cur[i] / degree(r, c);
```

Comparison. C exposes dynamic programming or memoization. Rust exposes direct bounded arithmetic. No validated source exists for Gforth, and Factor. All listed programs print the same exact answer string from the manifest. The excerpts can differ in algorithm, data representation, and startup cost.

40.214 Problem 214: Totient Chains

The program computes the exact answer for Totient Chains.

C. Source: `solutions/c/totient_chains_214.c`.

```
enum { LIMIT = 40000000, TARGET_LENGTH = 25 };

int main(void) {
    uint32_t *phi = malloc((size_t)LIMIT * sizeof(*phi));
    unsigned char *length = malloc((size_t)LIMIT);
    if (phi == NULL || length == NULL) {
        free(length);
        free(phi);
        return 1;
    }
    for (uint32_t n = 0; n < LIMIT; ++n) {
        phi[n] = n;
    }
    for (uint32_t p = 2; p < LIMIT; ++p) {
```

Rust. Source: `solutions/rust/src/bin/p214.rs`.

```
fn solve() -> u64 {
    const N: usize = 40_000_000;
    let mut phi: Vec<u32> = (0..N as u32).collect();
    for p in 2..N {
        if phi[p] == p as u32 {
            for m in (p..N).step_by(p) {
                phi[m] -= phi[m] / p as u32;
            }
        }
    }
    let mut len = vec![0u8; N];
    len[1] = 1;
    let mut sum = 0;
    for n in 2..N {
```

Comparison. C exposes prime generation or testing. Rust exposes prime generation or testing. No validated source exists for Gforth, and Factor. All listed programs print the same exact answer string from the manifest. The excerpts can differ in algorithm, data representation, and startup cost.

40.215 Problem 215: Crack-free Walls

The program computes the exact answer for Crack-free Walls.

C. Source: `solutions/c/crack_free_walls_215.c`.

```
enum { WIDTH = 32, HEIGHT = 10, MAX_ROWS = 10000 };

static uint32_t masks[MAX_ROWS];
static int mask_count;

static void generate_rows(int position, uint32_t mask) {
    if (position == WIDTH) {
        masks[mask_count++] = mask;
        return;
    }
    for (int brick = 2; brick <= 3; ++brick) {
        const int end = position + brick;
        if (end <= WIDTH) {
            const uint32_t next_mask = end == WIDTH ? mask : mask | (UINT32_C(1) << end);
```

Rust. Source: `solutions/rust/src/bin/p215.rs`.

```
fn rows(pos: u32, mask: u32, out: &mut Vec<u32>) {
    if pos == 32 {
        out.push(mask);
        return;
    }
    for brick in [2, 3] {
        let end = pos + brick;
        if end <= 32 {
            rows(end, if end == 32 { mask } else { mask | 1 << end }, out);
        }
    }
}
fn solve() -> u64 {
    let mut masks = Vec::new();
```

Factor. Source: `solutions/factor/p215.factor`.

```
! Native Factor Project Euler 215 crack-free-wall dynamic program.
USING: io math.parser project-euler.215 ; IN: scratchpad
euler215 number>string print
```

Comparison. C exposes dynamic programming or memoization, and recursive search. Rust exposes dynamic programming or memoization. Factor exposes dynamic programming or memoization. No

validated source exists for Gforth. All listed programs print the same exact answer string from the manifest. The excerpts can differ in algorithm, data representation, and startup cost.

40.216 Problem 216: The Primality of 2n^2 - 1

The program computes the exact answer for The Primality of 2n^2 - 1.

C. Source: `solutions/c/the_primality_of_2n2_minus_1_216.c`.

```
enum { N_LIMIT = 50000000, PRIME_LIMIT = 70710678 };

static uint64_t power_mod(uint64_t base, uint64_t exponent, uint64_t modulus) {
    uint64_t result = 1;
    while (exponent != 0) {
        if ((exponent & 1U) != 0U) {
            result = result * base % modulus;
        }
        base = base * base % modulus;
        exponent >>= 1U;
    }
    return result;
}
```

Rust. Source: `solutions/rust/src/bin/p216.rs`.

```
fn powmod(mut b: u64, mut e: u64, m: u64) -> u64 {
    let mut r = 1;
    while e > 0 {
        if e & 1 == 1 {
            r = r * b % m;
        }
        b = b * b % m;
        e >>= 1;
    }
    r
}
fn sqrt_mod(v: u64, p: u64) -> u64 {
    if p % 4 == 3 {
        return powmod(v, (p + 1) / 4, p);
```

Comparison. C exposes prime generation or testing, and modular arithmetic. Rust exposes prime generation or testing, and modular arithmetic. No validated source exists for Gforth, and Factor. All listed programs print the same exact answer string from the manifest. The excerpts can differ in algorithm, data representation, and startup cost.

40.217 Problem 217: Balanced Numbers

The program computes the exact answer for Balanced Numbers.

C. Source: `solutions/c/balanced_numbers_217.c`.

```
enum { MAX_HALF = 23, MAX_SUM = 9 * MAX_HALF, MODULUS = 14348907 };

static void append_digit_layer(const int64_t *old_count, const int64_t *old_sum,
                               int old_max_sum, int64_t *new_count, int64_t *new_sum) {
    memset(new_count, 0, (MAX_SUM + 1) * sizeof(*new_count));
    memset(new_sum, 0, (MAX_SUM + 1) * sizeof(*new_sum));
    for (int sum = 0; sum <= old_max_sum; ++sum) {
        for (int digit = 0; digit <= 9; ++digit) {
            const int next_sum = sum + digit;
            new_count[next_sum] = (new_count[next_sum] + old_count[sum]) % MODULUS;
            new_sum[next_sum] = (new_sum[next_sum] +
                                 10 * old_sum[sum] + digit * old_count[sum]) % MODULUS;
        }
    }
```

Rust. Source: `solutions/rust/src/bin/p217.rs`.

```
const MAX: usize = 207;
const MOD: i64 = 14_348_907;
fn append(count: &[i64], sum: &[i64], old_max: usize) -> (Vec<i64>, Vec<i64>) {
    let (mut nc, mut ns) = (vec![0; MAX + 1], vec![0; MAX + 1]);
    for s in 0..=old_max {
        for d in 0..=9 {
            nc[s + d] = (nc[s + d] + count[s]) % MOD;
            ns[s + d] = (ns[s + d] + 10 * sum[s] + d as i64 * count[s]) % MOD;
        }
    }
    (nc, ns)
}
fn solve() -> i64 {
    let (mut rc, mut rs, mut lc, mut ls) = (
```

Factor. Source: `solutions/factor/euler/extended/extended.factor`.

```
:: p217 ( -- n )
    208 0 <array> :> rc! 208 0 <array> :> rs!
    208 0 <array> :> lc! 208 0 <array> :> ls!
    1 0 rc set-nth
    48 0 <array> :> p10 1 0 p10 set-nth
    1 47 [a..b] [| i | i 1 - p10 nth 10 * 14348907 mod i p10 set-nth ] each
    45 :> answer!
    1 23 [a..b] [| h |
        rc rs h 1 - 9 * append-layer rs! rc!
        h 1 = [ 1 9 [a..b] [| d | 1 d lc set-nth d d ls set-nth ] each ] [
            lc ls h 1 - 9 * append-layer ls! lc!
        ] if
        0 :> even! 0 :> odd!
        0 h 9 * [a..b] [| s |
```

Comparison. C exposes dynamic programming or memoization. Rust exposes modular arithmetic. Factor exposes modular arithmetic. No validated source exists for Gforth. All listed programs print the same exact answer string from the manifest. The excerpts can differ in algorithm, data representation, and startup cost.

40.218 Problem 218: Perfect Right-angled Triangles

The program computes the exact answer for Perfect Right-angled Triangles.

C. Source: `solutions/c/perfect_right_angled_triangles_218.c`.

```
int main(void) {
    int violations = 0;
    for (int u = 0; u < 84; ++u) {
        for (int v = 0; v < 84; ++v) {
            if (((u ^ v) & 1) == 0 ||
                (u % 3 == 0 && v % 3 == 0) ||
                (u % 7 == 0 && v % 7 == 0)) {
                continue;
            }
            const int64_t u2 = (int64_t)u * u;
            const int64_t v2 = (int64_t)v * v;
            const int64_t m = u2 - v2;
            const int64_t n = 2 * (int64_t)u * v;
            int64_t area_mod = m * n % 84 * ((m * m - n * n) % 84) % 84;
```

Rust. Source: `solutions/rust/src/bin/p218.rs`.

```
fn solve() -> u32 {
    let mut bad = 0;
    for u in 0i64..84 {
        for v in 0i64..84 {
            if (u ^ v) & 1 == 0 || (u % 3 == 0 && v % 3 == 0) || (u % 7 == 0 && v % 7 == 0) {
                continue;
            }
            let (m, n) = (u * u - v * v, 2 * u * v);
            if (m * n % 84 * (m * m - n * n) % 84).rem_euclid(84) != 0 {
                bad += 1;
            }
        }
    }
    bad
```

Factor. Source: `solutions/factor/euler/extended/extended.factor`.

```
:: p218 ( -- n )
    0 :> violations!
    0 83 [a..b] [| u | 0 83 [a..b] [| v |
        u v bitxor odd?
        u 3 mod zero? v 3 mod zero? and not and
        u 7 mod zero? v 7 mod zero? and not and [
            u sq v sq - :> m u v * 2 * :> n
            m n * 84 mod m sq n sq - 84 mod * 84 mod zero? not
            [ violations 1 + violations! ] when
        ] when
    ] each ] each violations ;

! Problem 219: cost-layer multiplicities replace a billion-leaf priority queue.
```

Comparison. C exposes prime generation or testing. Rust exposes modular arithmetic. Factor exposes modular arithmetic. No validated source exists for Gforth. All listed programs print the same exact answer string from the manifest. The excerpts can differ in algorithm, data representation, and startup cost.

40.219 Problem 219: Skew-cost Coding

The program computes the exact answer for Skew-cost Coding.

C. Source: `solutions/c/skew_cost_coding_219.c`.

```
int main(void) {
    const uint64_t leaves = UINT64_C(1000000000);
    const uint64_t needed = leaves - 1;
    uint64_t multiplicity[128] = {0};
    multiplicity[0] = 1;

    uint64_t used = 0;
    uint64_t internal_cost = 0;
    for (int cost = 0; used < needed; ++cost) {
        if (cost > 0) {
            multiplicity[cost] = multiplicity[cost - 1];
            if (cost >= 4) {
                multiplicity[cost] += multiplicity[cost - 4];
            }
```

Rust. Source: `solutions/rust/src/bin/p219.rs`.

```
fn solve() -> u64 {
    let needed = 999_999_999u64;
    let mut f = [0u64; 128];
    f[0] = 1;
    let (mut used, mut cost) = (0, 0);
    for c in 0..128 {
        if c > 0 {
            f[c] = f[c - 1] + if c >= 4 { f[c - 4] } else { 0 };
        }
        let take = f[c].min(needed - used);
        used += take;
        cost += c as u64 * take;
        if used == needed {
            break;
```

Factor. Source: `solutions/factor/euler/extended/extended.factor`.

```
:: p219 ( -- n )
    128 0 <array> :> multiplicity 1 0 multiplicity set-nth
    999999999 :> needed 0 :> used! 0 :> internal! 0 :> cost!
    [ used needed < ] [
        cost 0 > [
            cost 1 - multiplicity nth
            cost 4 >= [ cost 4 - multiplicity nth + ] when
            cost multiplicity set-nth
```

```
        ] when
        cost multiplicity nth needed used - min :> take
        used take + used! internal cost take * + internal!
        cost 1 + cost!
    ] while internal needed 5 * + ;
```

Comparison. C exposes direct bounded arithmetic. Rust exposes direct bounded arithmetic. Factor exposes direct bounded arithmetic. No validated source exists for Gforth. All listed programs print the same exact answer string from the manifest. The excerpts can differ in algorithm, data representation, and startup cost.

40.220 Problem 220: Heighway Dragon

The program computes the exact answer for Heighway Dragon.

C. Source: `solutions/c/heighway_dragon_220.c`.

```
enum { ORDER = 50 };

typedef struct {
    int64_t x;
    int64_t y;
    int direction;
    uint64_t steps;
} State;

typedef State Summary;

static Summary a_summary[ORDER + 1];
static Summary b_summary[ORDER + 1];
```

Rust. Source: `solutions/rust/src/bin/p220.rs`.

```
#[derive(Clone, Copy, Default)]
struct State {
    x: i64,
    y: i64,
    dir: i32,
    steps: u64,
}
fn rotate(x: i64, y: i64, d: i32) -> (i64, i64) {
    match d & 3 {
        0 => (x, y),
        1 => (y, -x),
        2 => (-x, -y),
        _ => (-y, x),
    }
```

Comparison. C exposes recursive search. Rust exposes recursive search. No validated source exists for Gforth, and Factor. All listed programs print the same exact answer string from the manifest. The excerpts can differ in algorithm, data representation, and startup cost.

40.221 Problem 221: Alexandrian Integers

The program computes the exact answer for Alexandrian Integers.

C. Source: `solutions/c/alexandrian_integers_221.c`.

```
enum { P_LIMIT = 500000, FACTOR_SLOTS = 12, DIVISOR_SLOTS = 8192, TARGET = 150000 };

typedef struct {
    uint32_t prime;
    unsigned char exponent;
} Factor;

static uint64_t power_mod(uint64_t base, uint64_t exponent, uint64_t modulus) {
    uint64_t result = 1;
    while (exponent != 0) {
        if ((exponent & 1U) != 0U) result = result * base % modulus;
        base = base * base % modulus;
        exponent >>= 1U;
    }
```

Rust. Source: `solutions/rust/src/bin/p221.rs`.

```
fn powmod(mut b: u64, mut e: u64, m: u64) -> u64 {
    let mut r = 1;
    while e > 0 {
        if e & 1 == 1 {
            r = r * b % m;
        }
        b = b * b % m;
        e >>= 1;
    }
    r
}
fn root_minus_one(p: u64) -> u64 {
    let mut g = 2;
    while powmod(g, (p - 1) / 2, p) != p - 1 {
```

Comparison. C exposes prime generation or testing, modular arithmetic, and sorting or set deduplication. Rust exposes prime generation or testing, modular arithmetic, and sorting or set deduplication. No validated source exists for Gforth, and Factor. All listed programs print the same exact answer string from the manifest. The excerpts can differ in algorithm, data representation, and startup cost.

40.222 Problem 222: Sphere Packing

The program computes the exact answer for Sphere Packing.

C. Source: `solutions/c/sphere_packing_222.c`.

```
enum { BALLS = 21 };
```

```
int main(void) {
    const uint32_t states = UINT32_C(1) << BALLS;
    const size_t entries = (size_t)states * BALLS;
    float *dp = malloc(entries * sizeof(*dp));
    if (dp == NULL) {
        return 1;
    }
    for (size_t i = 0; i < entries; ++i) {
        dp[i] = FLT_MAX;
    }

    double separation[BALLS][BALLS];
```

Rust. Source: `solutions/rust/src/bin/p222.rs`.

```
fn solve() -> i64 {
    const N: usize = 21;
    let states = 1usize << N;
    let mut sep = [[0f32; N]; N];
    for i in 0..N {
        for j in 0..N {
            sep[i][j] = (200.0 * ((30 + i + 30 + j - 50) as f32)).sqrt();
        }
    }
    let mut dp = vec![f32::MAX; states * N];
    for i in 0..N {
        dp[(1 << i) * N + i] = (30 + i) as f32;
    }
    for mask in 1..states {
```

Comparison. C exposes dynamic programming or memoization. Rust exposes dynamic programming or memoization. No validated source exists for Gforth, and Factor. All listed programs print the same exact answer string from the manifest. The excerpts can differ in algorithm, data representation, and startup cost.

40.223 Problem 223: Almost Right-angled Triangles I

The program computes the exact answer for Almost Right-angled Triangles I.

C. Source: `solutions/c/almost_right_angled_triangles_i_223.c`.

```
enum { PERIMETER = 25000000, MAX_A = PERIMETER / 3 };

typedef struct {
    uint32_t prime;
    int exponent;
} Factor;

static Factor factors[20];
static int factor_count;
static uint64_t product_n;
static uint64_t minimum_v;
static uint64_t maximum_v;
```

```
static uint64_t integer_sqrt(uint64_t value) {
```

Rust. Source: `solutions/rust/src/bin/p223.rs`.

```
fn isqrt(n: u64) -> u64 {
    let mut r = (n as f64).sqrt() as u64;
    while (r + 1) <= n / (r + 1) {
        r += 1;
    }
    while r > n / r {
        r -= 1;
    }
    r
}
fn factor(mut v: usize, spf: &[u32], fs: &mut Vec<(u64, u32)>) {
    while v > 1 {
        let p = if spf[v] == 0 { v } else { spf[v] as usize };
        let mut e = 0;
```

Comparison. C exposes prime generation or testing, and recursive search. Rust exposes direct bounded arithmetic. No validated source exists for Gforth, and Factor. All listed programs print the same exact answer string from the manifest. The excerpts can differ in algorithm, data representation, and startup cost.

40.224 Problem 224: Almost Right-angled Triangles II

The program computes the exact answer for Almost Right-angled Triangles II.

C. Source: `solutions/c/almost_right_angled_triangles_ii_224.c`.

```
static const int64_t PERIMETER_LIMIT = INT64_C(75000000);

static uint64_t visit(int64_t a, int64_t b, int64_t c) {
    if (a + b + c > PERIMETER_LIMIT) {
        return 0;
    }
    uint64_t count = a <= b;

    count += visit(a - 2 * b + 2 * c,
                   2 * a - b + 2 * c,
                   2 * a - 2 * b + 3 * c);
    count += visit(a + 2 * b + 2 * c,
                   2 * a + b + 2 * c,
                   2 * a + 2 * b + 3 * c);
```

Rust. Source: `solutions/rust/src/bin/p224.rs`.

```
fn visit(a: i64, b: i64, c: i64) -> u64 {
    if a + b + c > 75_000_000 {
        return 0;
```

```
    }
    let mut n = (a <= b) as u64;
    n += visit(a - 2 * b + 2 * c, 2 * a - b + 2 * c, 2 * a - 2 * b + 3 * c);
    n += visit(a + 2 * b + 2 * c, 2 * a + b + 2 * c, 2 * a + 2 * b + 3 * c);
    n + visit(
        -a + 2 * b + 2 * c,
        -2 * a + b + 2 * c,
        -2 * a + 2 * b + 3 * c,
    )
}
fn solve() -> u64 {
```

Comparison. C exposes direct bounded arithmetic. Rust exposes direct bounded arithmetic. No validated source exists for Gforth, and Factor. All listed programs print the same exact answer string from the manifest. The excerpts can differ in algorithm, data representation, and startup cost.

40.225 Problem 225: Tribonacci Non-divisors

The program computes the exact answer for Tribonacci Non-divisors.

C. Source: `solutions/c/tribonacci_non_divisors_225.c`.

```
static int never_divides(int modulus) {
    int a = 1;
    int b = 1;
    int c = 1;
    do {
        const int next = (a + b + c) % modulus;
        a = b;
        b = c;
        c = next;
        if (c == 0) return 0;
    } while (a != 1 || b != 1 || c != 1);
    return 1;
}
```

Rust. Source: `solutions/rust/src/bin/p225.rs`.

```
fn never(m: u32) -> bool {
    let (mut a, mut b, mut c) = (1, 1, 1);
    loop {
        let n = (a + b + c) % m;
        (a, b, c) = (b, c, n);
        if c == 0 {
            return false;
        }
        if (a, b, c) == (1, 1, 1) {
            return true;
        }
    }
}
fn solve() -> u32 {
```

Factor. Source: `solutions/factor/euler/extended/extended.factor`.

```
:: p225 ( -- n )
    0 :> found! 1 :> candidate! 0 :> answer!
    [ answer zero? ] [
        candidate never-divides? [
            found 1 + found! found 124 = [ candidate answer! ] when
        ] when candidate 2 + candidate!
    ] while answer ;

! Problem 228: reduced edge-normal denominator d contributes phi(d) exactly
! when d divides one of the polygon sizes in the interval.
```

Comparison. C exposes direct bounded arithmetic. Rust exposes direct bounded arithmetic. Factor exposes direct bounded arithmetic. No validated source exists for Gforth. All listed programs print the same exact answer string from the manifest. The excerpts can differ in algorithm, data representation, and startup cost.

40.226 Problem 226: A Scoop of Blancmange

The program computes the exact answer for A Scoop of Blancmange.

C. Source: `solutions/c/a_scoop_of_blancmange_226.c`.

```
static double blancmange(double x) {
    double sum = 0.0;
    double weight = 1.0;
    double scaled = x;
    for (int n = 0; n < 53; ++n) {
        const double nearest = floor(scaled + 0.5);
        sum += fabs(scaled - nearest) * weight;
        scaled *= 2.0;
        weight *= 0.5;
    }
    return sum;
}

static double lower_circle(double x) {
```

Rust. Source: `solutions/rust/src/bin/p226.rs`.

```
fn blanc(mut x: f64) -> f64 {
    let (mut sum, mut weight) = (0.0, 1.0);
    for _ in 0..53 {
        sum += (x - x.round()).abs() * weight;
        x *= 2.0;
        weight *= 0.5;
    }
    sum
}
fn diff(x: f64) -> f64 {
    blanc(x) - (0.5 - (0.0625 - (x - 0.25).powi(2)).sqrt())
}
```

```
fn solve() -> f64 {
    let (mut low, mut high) = (0.0, 0.25);
```

Comparison. C exposes direct bounded arithmetic. Rust exposes direct bounded arithmetic. No validated source exists for Gforth, and Factor. All listed programs print the same exact answer string from the manifest. The excerpts can differ in algorithm, data representation, and startup cost.

40.227 Problem 227: The Chase

The program computes the exact answer for The Chase.

C. Source: `solutions/c/the_chase_227.c`.

```
enum { PLAYERS = 100, UNKNOWNS = PLAYERS - 1 };

int main(void) {
    double matrix[UNKNOWNS][UNKNOWNS + 1] = {{0.0}};
    static const int shift[5] = {-2, -1, 0, 1, 2};
    static const double probability[5] = {1.0/36.0, 2.0/9.0, 0.5, 2.0/9.0, 1.0/36.0};
    for (int k = 1; k < PLAYERS; ++k) {
        const int row = k - 1;
        matrix[row][row] = 1.0;
        matrix[row][UNKNOWNS] = 1.0;
        for (int i = 0; i < 5; ++i) {
            int next = (k + shift[i]) % PLAYERS;
            if (next < 0) next += PLAYERS;
            if (next != 0) matrix[row][next - 1] -= probability[i];
```

Rust. Source: `solutions/rust/src/bin/p227.rs`.

```
fn solve() -> f64 {
    const N: usize = 99;
    let mut a = vec![[0f64; 100]; N];
    let shifts = [-2, -1, 0, 1, 2];
    let probs = [1.0 / 36.0, 2.0 / 9.0, 0.5, 2.0 / 9.0, 1.0 / 36.0];
    for k in 1..100 {
        let r = k - 1;
        a[r][r] = 1.0;
        a[r][N] = 1.0;
        for i in 0..5 {
            let next = (k as i32 + shifts[i]).rem_euclid(100) as usize;
            if next != 0 {
                a[r][next - 1] -= probs[i];
            }
```

Factor. Source: `solutions/factor/euler/extended/extended.factor`.

```
:: p227 ( -- x )
    9900 0.0 <array> :> matrix
    { -2 -1 0 1 2 } :> shifts
    { 0.027777777777777776 0.2222222222222222 0.5 0.2222222222222222 0.027777777777777776 } :>
    probs
```

```
    1 99 [a..b] [| k |
        k 1 - :> row 1 row row p227-index matrix set-nth
        1.0 row 99 p227-index matrix set-nth
        0 4 [a..b] [| i |
            k i shifts nth + 100 + 100 mod :> next
            next zero? not [
                row next 1 - p227-index :> index
                index matrix nth i probs nth - index matrix set-nth
            ] when
        ] each
```

Comparison. C exposes direct bounded arithmetic. Rust exposes direct bounded arithmetic. Factor exposes modular arithmetic. No validated source exists for Gforth. All listed programs print the same exact answer string from the manifest. The excerpts can differ in algorithm, data representation, and startup cost.

40.228 Problem 228: Minkowski Sums

The program computes the exact answer for Minkowski Sums.

C. Source: `solutions/c/minkowski_sums_228.c`.

```
enum { FIRST = 1864, LAST = 1909 };

int main(void) {
    int present[LAST + 1] = {0};
    int phi[LAST + 1];
    for (int n = 0; n <= LAST; ++n) phi[n] = n;
    for (int p = 2; p <= LAST; ++p) {
        if (phi[p] == p) {
            for (int multiple = p; multiple <= LAST; multiple += p) {
                phi[multiple] -= phi[multiple] / p;
            }
        }
    }
    present[1] = 1;
```

Rust. Source: `solutions/rust/src/bin/p228.rs`.

```
fn solve() -> u32 {
    let mut phi: Vec<u32> = (0..=1909).collect();
    for p in 2..=1909 {
        if phi[p] == p as u32 {
            for m in (p..=1909).step_by(p) {
                phi[m] -= phi[m] / p as u32;
            }
        }
    }
    let mut present = [false; 1910];
    present[1] = true;
    for n in 1864..=1909 {
        let mut d = 1;
        while d * d <= n {
```

Factor. Source: `solutions/factor/euler/extended/extended.factor`.

```
:: p228 ( -- n )
    1909 phi-sieve :> phi 1910 f <array> :> present
    t 1 present set-nth
    1864 1909 [a..b] [| n |
        1 n sqrt >integer [a..b] [| d | n d mod zero? [
            t d present set-nth t n d /i present set-nth
        ] when ] each
    ] each
    0 :> answer! 1 1909 [a..b] [| d |
        d present nth [ answer d phi nth + answer! ] when
    ] each answer ;

! Problem 234: sum arithmetic progressions in each consecutive-prime-square
! interval, subtracting common multiples twice.
```

Comparison. C exposes prime generation or testing. Rust exposes direct bounded arithmetic. Factor exposes prime generation or testing, and modular arithmetic. Shared calls: `phi-sieve`. No validated source exists for Gforth. All listed programs print the same exact answer string from the manifest. The excerpts can differ in algorithm, data representation, and startup cost.

40.229 Problem 229: Four Representations Using Squares

The program computes the exact answer for Four Representations Using Squares.

C. Source: `solutions/c/four_representations_using_squares_229.c`.

```
enum { LIMIT = 2000000000, ROOT_LIMIT = 44721, MAX_FACTORS = 12, MAX_DIVISORS = 4096 };

typedef struct { uint32_t prime; int exponent; } Factor;

static unsigned char *odd_composite;
static uint32_t *good_primes;
static int good_count;

static int uint32_compare(const void *left, const void *right) {
    const uint32_t a = *(const uint32_t *)left;
    const uint32_t b = *(const uint32_t *)right;
    return (a > b) - (a < b);
}
```

Rust. Source: `solutions/rust/src/bin/p229.rs`.

```
const LIMIT: u64 = 2_000_000_000;
const ROOT: usize = 44_721;
fn isqrt(n: u64) -> u64 {
    let mut r = (n as f64).sqrt() as u64;
    while (r + 1) <= n / (r + 1) {
        r += 1;
    }
    while r > n / r {
        r -= 1;
```

```
    }
    r
}
fn bit_get(bits: &[u8], v: u64) -> bool {
    let i = v >> 1;
```

Comparison. C exposes prime generation or testing, and sorting or set deduplication. Rust exposes prime generation or testing, modular arithmetic, and sorting or set deduplication. No validated source exists for Gforth, and Factor. All listed programs print the same exact answer string from the manifest. The excerpts can differ in algorithm, data representation, and startup cost.

40.230 Problem 230: Fibonacci Words

The program computes the exact answer for Fibonacci Words.

C. Source: `solutions/c/fibonacci_words_230.c`.

```
static const char A[] =
    "14159265358979323846264338327950288419716939937510"
    "58209749445923078164062862089986280348253421170679";
static const char B[] =
    "82148086513282306647093844609550582231725359408128"
    "48111745028410270193852110555964462294895493038196";

static int digit_at(uint64_t position) {
    uint64_t length[100] = {100, 100};
    int term = 1;
    while (length[term] < position) {
        ++term;
        if (UINT64_MAX - length[term - 1] < length[term - 2]) {
            length[term] = UINT64_MAX;
```

Rust. Source: `solutions/rust/src/bin/p230.rs`.

```
const A:&[u8]=b
    "14159265358979323846264338327950288419716939937510582097494459230781640628620899862803482534211706 79";

const B:&[u8]=b
    "82148086513282306647093844609550582231725359408128481111745028410270193852110555964462294895493038196";

fn digit(mut pos: u64) -> u64 {
    let mut lengths = [0u64; 100];
    lengths[0] = 100;
    lengths[1] = 100;
    let mut term = 1;
    while lengths[term] < pos {
        term += 1;
        lengths[term] = lengths[term - 2].saturating_add(lengths[term - 1]);
    }
    while term >= 2 {
        if pos <= lengths[term - 2] {
            term -= 2
```

Factor. Source: `solutions/factor/euler/extended/extended.factor`.

```
:: p230 ( -- n )
    0 :> answer! 1 :> power7! 1 :> place10!
    0 17 [a..b] [| n |
        n 19 * 127 + power7 * p230-digit place10 * answer + answer!
        power7 7 * power7! place10 10 * place10!
    ] each answer ;

! Problem 232: descending score order solves the zero-score stochastic loop
! algebraically for each of player two's eight useful choices.
```

Comparison. C exposes direct bounded arithmetic. Rust exposes direct bounded arithmetic. Factor exposes direct bounded arithmetic. No validated source exists for Gforth. All listed programs print the same exact answer string from the manifest. The excerpts can differ in algorithm, data representation, and startup cost.

40.231 Problem 231: Prime Factorisation of Binomial Coefficients

The program computes the exact answer for Prime Factorisation of Binomial Coefficients.

C. Source: `solutions/c/prime_factorisation_of_binomial_coefficients_231.c`.

```
enum { N = 20000000, K = 15000000 };

static int factorial_exponent(int n, int prime) {
    int exponent = 0;
    while (n != 0) {
        n /= prime;
        exponent += n;
    }
    return exponent;
}

int main(void) {
    unsigned char *composite = calloc(N + 1, 1);
    if (composite == NULL) return 1;
```

Rust. Source: `solutions/rust/src/bin/p231.rs`.

```
fn exponent(mut n: usize, p: usize) -> usize {
    let mut e = 0;
    while n > 0 {
        n /= p;
        e += n;
    }
    e
}
fn solve() -> u64 {
    const N: usize = 20_000_000;
    let mut comp = vec![false; N + 1];
    for p in 2..=((N as f64).sqrt() as usize) {
        if !comp[p] {
```

```
            for m in (p * p..=N).step_by(p) {
```

Comparison. C exposes prime generation or testing, and exact large-integer arithmetic. Rust exposes prime generation or testing. No validated source exists for Gforth, and Factor. All listed programs print the same exact answer string from the manifest. The excerpts can differ in algorithm, data representation, and startup cost.

40.232 Problem 232: The Race

The program computes the exact answer for The Race.

C. Source: `solutions/c/the_race_232.c`.

```
int main(void) {
    enum { TARGET = 100, MAX_GAIN = 128 };
    double p1_turn[TARGET][TARGET + MAX_GAIN];
    double p2_turn[TARGET + 1][TARGET + MAX_GAIN];

    /* Scores at or above 100 are wins for player 2, regardless of whose turn
       the notation says it is.  Zero initialization handles all live cells. */
    for (int a = 0; a < TARGET; ++a) {
        for (int b = 0; b < TARGET + MAX_GAIN; ++b) {
            p1_turn[a][b] = b >= TARGET ? 1.0 : 0.0;
            p2_turn[a][b] = b >= TARGET ? 1.0 : 0.0;
        }
    }
    /* Row a=100 means player 1 has already won. */
```

Rust. Source: `solutions/rust/src/bin/p232.rs`.

```
fn solve() -> f64 {
    let (mut p1, mut p2) = (vec![vec![0f64; 228]; 100], vec![vec![0f64; 228]; 101]);
    for a in 0..100 {
        for b in 100..228 {
            p1[a][b] = 1.0;
            p2[a][b] = 1.0;
        }
    }
    for a in (0..100).rev() {
        for b in (0..100).rev() {
            let head = p2[a + 1][b];
            let (mut best, mut gain, mut probability) = (0.0, 1, 0.5);
            for _ in 1..=8 {
                let failure = 1.0 - probability;
```

Factor. Source: `solutions/factor/euler/extended/extended.factor`.

```
:: p232 ( -- x )
    22800 0.0 <array> :> p1
    23028 0.0 <array> :> p2
    0 99 [a..b] [| a | 100 227 [a..b] [| b |
```

```
        1.0 a b p232-index p1 set-nth 1.0 a b p232-index p2 set-nth
    ] each ] each
    99 0 [a..b] [| a | 99 0 [a..b] [| b |
        a 1 + b p232-index p2 nth :> after-head
        0.0 :> best! 1 :> gain! 0.5 :> probability!
        8 [
            1.0 probability - :> failure
            probability a b gain + p232-index p1 nth *
            failure 0.5 * after-head * + 1.0 failure 0.5 * - /
            best max best!
```

Comparison. C exposes dynamic programming or memoization. Rust exposes dynamic programming or memoization. Factor exposes dynamic programming or memoization. No validated source exists for Gforth. All listed programs print the same exact answer string from the manifest. The excerpts can differ in algorithm, data representation, and startup cost.

40.233 Problem 233: Lattice Points on a Circle

The program computes the exact answer for Lattice Points on a Circle.

C. Source: `solutions/c/lattice_points_on_a_circle_233.c`.

```
enum { PRIME_LIMIT = 4800000, COFACTOR_LIMIT = 278454 };
static const uint64_t LIMIT = UINT64_C(100000000000);

static int *primes;
static int prime_count;
static uint64_t cofactor_sum[COFACTOR_LIMIT + 1];
static uint64_t answer;

/* Return base^exponent, or LIMIT+1 as an overflow/too-large sentinel. */
static uint64_t bounded_power(uint64_t base, int exponent, uint64_t bound) {
    uint64_t result = 1;
    for (int i = 0; i < exponent; ++i) {
        if (result > bound / base)
            return bound + 1;
```

Rust. Source: `solutions/rust/src/bin/p233.rs`.

```
const LIMIT: u64 = 100_000_000_000;
fn bounded(base: u64, e: u32, bound: u64) -> Option<u64> {
    let mut r = 1;
    for _ in 0..e {
        if r > bound / base {
            return None;
        }
        r *= base;
    }
    Some(r)
}
fn cores(
    exps: &[u32],
    depth: usize,
```

Comparison. C exposes prime generation or testing, and modular arithmetic. Rust exposes prime generation or testing, and modular arithmetic. No validated source exists for Gforth, and Factor. All listed programs print the same exact answer string from the manifest. The excerpts can differ in algorithm, data representation, and startup cost.

40.234 Problem 234: Semidivisible Numbers

The program computes the exact answer for Semidivisible Numbers.

C. Source: `solutions/c/semidivisible_numbers_234.c`.

```
static const uint64_t LIMIT = UINT64_C(999966663333);

/* Sum multiples of d in the inclusive interval [low, high].  The largest
   intermediate is about LIMIT^2/d, so unsigned __int128 prevents overflow. */
static unsigned __int128 multiple_sum(uint64_t d, uint64_t low,
                                      uint64_t high) {
    const uint64_t first = (low + d - 1) / d;
    const uint64_t last = high / d;
    if (first > last)
        return 0;
    const unsigned __int128 count = last - first + 1;
    return (unsigned __int128)d * (first + last) * count / 2;
}
```

Rust. Source: `solutions/rust/src/bin/p234.rs`.

```
fn multiples(d: u64, low: u64, high: u64) -> u128 {
    let first = low.div_ceil(d);
    let last = high / d;
    if first > last {
        0
    } else {
        d as u128 * (first + last) as u128 * (last - first + 1) as u128 / 2
    }
}
fn solve() -> u64 {
    const LIM: u64 = 999_966_663_333;
    let n = 1_010_000;
    let mut comp = vec![false; n + 1];
    let mut ps = Vec::new();
```

Factor. Source: `solutions/factor/euler/extended/extended.factor`.

```
:: p234 ( -- n )
    1010000 euler-sieve :> sieve V{ } clone :> primes
    2 1010000 [a..b] [| n | n sieve nth [ n primes push ] when ] each
    0 :> answer!
    0 primes length 2 - [a..b] [| i |
        i primes nth :> p i 1 + primes nth :> q p sq 1 + :> low
        low 999966663333 <= [
            q sq 1 - 999966663333 min :> high
            answer p low high multiple-sum + q low high multiple-sum +
```

```
            p q * low high multiple-sum 2 * - answer!
        ] when
    ] each answer ;

! Problem 239: normalized inclusion-exclusion terms avoid constructing 100!.
```

Comparison. C exposes prime generation or testing. Rust exposes prime generation or testing. Factor exposes prime generation or testing. Shared calls: `euler-sieve`. No validated source exists for Gforth. All listed programs print the same exact answer string from the manifest. The excerpts can differ in algorithm, data representation, and startup cost.

40.235 Problem 235: An Arithmetic Geometric Sequence

The program computes the exact answer for An Arithmetic Geometric Sequence.

C. Source: `solutions/c/an_arithmetic_geometric_sequence_235.c`.

```
static double sequence_sum(double r) {
    double power = 1.0;
    double sum = 0.0;
    for (int k = 1; k <= 5000; ++k) {
        sum += (900.0 - 3.0 * k) * power;
        power *= r;
    }
    return sum;
}

int main(void) {
    const double target = -600000000000.0;
    double low = 1.0;
    double high = 1.01; /* brackets the root while keeping all powers finite */
```

Rust. Source: `solutions/rust/src/bin/p235.rs`.

```
fn sum(r: f64) -> f64 {
    let (mut power, mut total) = (1.0, 0.0);
    for k in 1..=5000 {
        total += (900.0 - 3.0 * k as f64) * power;
        power *= r;
    }
    total
}
fn solve() -> f64 {
    let (mut low, mut high) = (1.0, 1.01);
    for _ in 0..200 {
        let mid = (low + high) / 2.0;
        if sum(mid) > -600_000_000_000.0 {
            low = mid
```

Factor. Source: `solutions/factor/euler/extended/extended.factor`.

```
:: p235 ( -- x )
    1.0 :> low! 1.01 :> high!
    200 [
        low high + 2.0 / :> middle
        middle p235-sum -600000000000.0 >
        [ middle low! ] [ middle high! ] if
    ] times low high + 2.0 / ;

! Problem 204: exponent choices for the 25 allowed primes give each Hamming
! number one unique nondecreasing factorization.
CONSTANT: hamming-primes { 2 3 5 7 11 13 17 19 23 29 31 37 41 43 47 53 59 61 67 71 73 79 83 89 97
     }
```

Comparison. C exposes direct bounded arithmetic. Rust exposes direct bounded arithmetic. Factor exposes prime generation or testing. No validated source exists for Gforth. All listed programs print the same exact answer string from the manifest. The excerpts can differ in algorithm, data representation, and startup cost.

40.236 Problem 236: Luxury Hampers

The program computes the exact answer for Luxury Hampers.

C. Source: `solutions/c/luxury_hampers_236.c`.

```
static const int supplied_a[5] = {5248, 1312, 2624, 5760, 3936};
static const int supplied_b[5] = {640, 1888, 3776, 3776, 5664};
enum { TOTAL_A = 18880, TOTAL_B = 15744 };

static int64_t gcd64(int64_t a, int64_t b) {
    while (b != 0) {
        const int64_t remainder = a % b;
        a = b;
        b = remainder;
    }
    return a;
}

int main(void) {
```

Rust. Source: `solutions/rust/src/bin/p236.rs`.

```
fn gcd(mut a: i64, mut b: i64) -> i64 {
    while b != 0 {
        (a, b) = (b, a % b);
    }
    a
}
fn solve() -> String {
    let aa = [5248, 1312, 2624, 5760, 3936];
    let bb = [640, 1888, 3776, 3776, 5664];
    let (mut valid, mut bu, mut bv) = (0, 0i64, 1i64);
    for s in 1..=aa[1] {
```

```
        for r in 1..=bb[1] {
            if gcd(r, s) != 1 || 41 * r <= 59 * s {
                continue;
```

Comparison. C exposes prime generation or testing. Shared calls: `gcd`. Rust exposes prime generation or testing. No validated source exists for Gforth, and Factor. All listed programs print the same exact answer string from the manifest. The excerpts can differ in algorithm, data representation, and startup cost.

40.237 Problem 237: Tours on a 4 \times N Playing Board

The program computes the exact answer for Tours on a 4 \times N Playing Board.

C. Source: `solutions/c/tours_on_a_4_x_n_playing_board_237.c`.

```
enum { SIZE = 4 };
static const uint64_t MODULUS = UINT64_C(100000000);

typedef struct {
    uint64_t cell[SIZE][SIZE];
} Matrix;

static Matrix multiply(Matrix left, Matrix right) {
    Matrix result = {{{0}}};
    for (int row = 0; row < SIZE; ++row) {
        for (int column = 0; column < SIZE; ++column) {
            uint64_t value = 0;
            for (int k = 0; k < SIZE; ++k)
                value += left.cell[row][k] * right.cell[k][column];
```

Rust. Source: `solutions/rust/src/bin/p237.rs`.

```
const M: u64 = 100_000_000;
type Mat = [[u64; 4]; 4];
fn mul(a: Mat, b: Mat) -> Mat {
    let mut c = [[0; 4]; 4];
    for i in 0..4 {
        for j in 0..4 {
            for k in 0..4 {
                c[i][j] = (c[i][j] + a[i][k] * b[k][j]) % M;
            }
        }
    }
    c
}
fn power(mut a: Mat, mut e: u64) -> Mat {
```

Factor. Source: `solutions/factor/euler/extended/extended.factor`.

```
:: p237 ( -- n )
    { 0 1 0 0 0 0 1 0 0 0 0 1 1 99999998 2 2 }
```

```
    999999999996 matrix-power :> jump
    { 1 1 4 8 } :> initial 0 :> answer!
    0 3 [a..b] [| i |
        answer 12 i + jump nth i initial nth * + 100000000 mod answer!
    ] each answer ;

! Problem 251: Cardano parameterization turns each coprime (q,r) pair into an
! arithmetic progression in p, counted by its exact linear sum bound.
```

Comparison. C exposes modular arithmetic. Rust exposes direct bounded arithmetic. Factor exposes prime generation or testing, and modular arithmetic. No validated source exists for Gforth. All listed programs print the same exact answer string from the manifest. The excerpts can differ in algorithm, data representation, and startup cost.

40.238 Problem 238: Infinite String Tour

The program computes the exact answer for Infinite String Tour.

C. Source: `solutions/c/infinite_string_tour_238.c`.

```c
enum { MODULUS = 20300713, INITIAL = 14025256 };
static const uint64_t TARGET = UINT64_C(2000000000000000);

static void append_number(uint32_t value, uint32_t **prefixes, size_t *length,
                          size_t *capacity, uint32_t *digit_sum) {
    uint32_t divisor = 1;
    while (divisor <= value / 10)
        divisor *= 10;

    do {
        const uint32_t digit = value / divisor;
        value %= divisor;
        divisor /= 10;
```

Rust. Source: `solutions/rust/src/bin/p238.rs`.

```rust
fn append(mut v: u32, prefix: &mut Vec<u32>, sum: &mut u32) {
    let mut divisor = 1;
    while divisor <= v / 10 {
        divisor *= 10;
    }
    loop {
        let digit = v / divisor;
        v %= divisor;
        prefix.push(*sum);
        *sum += digit;
        if divisor == 1 {
            break;
        }
        divisor /= 10;
```

Comparison. C exposes modular arithmetic. Rust exposes modular arithmetic. No validated source exists for Gforth, and Factor. All listed programs print the same exact answer string from the manifest. The excerpts can differ in algorithm, data representation, and startup cost.

40.239 Problem 239: Twenty-two Foolish Primes

The program computes the exact answer for Twenty-two Foolish Primes.

C. Source: `solutions/c/twenty_two_foolish_primes_239.c`.

```
int main(void) {
    const long double choose_three = (25.0L * 24.0L * 23.0L) / 6.0L;
    long double term = 1.0L / (100.0L * 99.0L * 98.0L);
    long double sum = term;

    for (int j = 0; j < 22; ++j) {
        term *= -(long double)(22 - j)
                / ((long double)(j + 1) * (97 - j));
        sum += term;
    }

    printf("%.12Lf\n", choose_three * sum);
    return 0;
}
```

Rust. Source: `solutions/rust/src/bin/p239.rs`.

```
fn solve() -> f64 {
    let mut term = 1.0 / (100.0 * 99.0 * 98.0);
    let mut sum = term;
    for j in 0..22 {
        term *= -(22 - j) as f64 / ((j + 1) as f64 * (97 - j) as f64);
        sum += term;
    }
    25.0 * 24.0 * 23.0 / 6.0 * sum
}
fn main() {
    println!("{:.12}", solve());
}
```

Factor. Source: `solutions/factor/euler/extended/extended.factor`.

```
:: p239 ( -- x )
    1.0 100.0 99.0 * 98.0 * / :> term!
    term :> total!
    0 21 [a..b] [| j |
        term 22 j - neg >float *
        j 1 + >float 97 j - >float * / term!
        total term + total!
    ] each total 25.0 24.0 * 23.0 * 6.0 / * ;

! Problem 242: digit DP for sum 2^popcount(t) up to the binary bound.
```

Comparison. C exposes prime generation or testing, and exact large-integer arithmetic. Rust exposes prime generation or testing, and exact large-integer arithmetic. Factor exposes dynamic programming or memoization. No validated source exists for Gforth. All listed programs print the same exact answer string from the manifest. The excerpts can differ in algorithm, data representation, and startup cost.

40.240 Problem 240: Top Dice

The program computes the exact answer for Top Dice.

C. Source: `solutions/c/top_dice_240.c`.

```
enum { DICE = 20, TOP = 10, TARGET_SUM = 70 };
typedef unsigned __int128 Wide;

static Wide dp[DICE + 1][TOP + 1][TARGET_SUM + 1];
static Wide next_dp[DICE + 1][TOP + 1][TARGET_SUM + 1];
static uint64_t choose[DICE + 1][DICE + 1];

static void print_wide(Wide value) {
    char digits[64];
    int length = 0;
    do {
        digits[length++] = (char)('0' + value % 10);
        value /= 10;
    } while (value != 0);
```

Rust. Source: `solutions/rust/src/bin/p240.rs`.

```
fn solve() -> u128 {
    let mut choose = [[0u128; 21]; 21];
    for n in 0..=20 {
        choose[n][0] = 1;
        choose[n][n] = 1;
        for k in 1..n {
            choose[n][k] = choose[n - 1][k - 1] + choose[n - 1][k];
        }
    }
    let size = 21 * 11 * 71;
    let ix = |u: usize, t: usize, s: usize| (u * 11 + t) * 71 + s;
    let mut dp = vec![0u128; size];
    dp[ix(0, 0, 0)] = 1;
    for face in (1..=12).rev() {
```

Factor. Source: `solutions/factor/euler/extended/extended.factor`.

```
:: p240 ( -- n )
    21 21 * 0 <array> :> choose
    0 20 [a..b] [| n |
        1 n 21 * choose set-nth 1 n 21 * n + choose set-nth
        n 1 > [ 1 n 1 - [a..b] [| k |
            n 1 - 21 * k 1 - + choose nth n 1 - 21 * k + choose nth +
            n 21 * k + choose set-nth
```

```
        ] each ] when
    ] each
    16401 0 <array> :> dp! 1 0 dp set-nth
    12 1 [a..b] [| face |
        16401 0 <array> :> next
        0 20 [a..b] [| used | 0 10 [a..b] [| top |
        0 70 [a..b] [| total |
```

Comparison. C exposes dynamic programming or memoization. Rust exposes dynamic programming or memoization. Factor exposes dynamic programming or memoization. No validated source exists for Gforth. All listed programs print the same exact answer string from the manifest. The excerpts can differ in algorithm, data representation, and startup cost.

40.241 Problem 241: Perfection Quotients

The program computes the exact answer for Perfection Quotients.

C. Source: `solutions/c/perfection_quotients_241.c`.

```
typedef uint64_t U64;
typedef unsigned __int128 U128;
static const U64 LIMIT = UINT64_C(1000000000000000000);
static U128 total;

static U64 gcd64(U64 a, U64 b) {
    while (b) { U64 t = a % b; a = b; b = t; }
    return a;
}

static U128 gcd128(U128 a, U128 b) {
    while (b) { U128 t = a % b; a = b; b = t; }
    return a;
}
```

Rust. Source: `solutions/rust/src/bin/p241.rs`.

```
fn gcd(mut a: u64, mut b: u64) -> u64 {
    while b != 0 {
        (a, b) = (b, a % b);
    }
    a
}
fn gcd128(mut a: u128, mut b: u128) -> u128 {
    while b != 0 {
        (a, b) = (b, a % b);
    }
    a
}
fn mul(a: u64, b: u64, m: u64) -> u64 {
    (a as u128 * b as u128 % m as u128) as u64
```

Comparison. C exposes prime generation or testing, and modular arithmetic. Rust exposes prime generation or testing, and recursive search. No validated source exists for Gforth, and Factor. All listed programs print the same exact answer string from the manifest. The excerpts can differ in algorithm, data representation, and startup cost.

40.242 Problem 242: Odd Triplets

The program computes the exact answer for Odd Triplets.

C. Source: `solutions/c/odd_triplets_242.c`.

```
int main(void) {
    const uint64_t upper = (UINT64_C(1000000000000) - 1) / 4;
    /* upper < 2^38, and 3^39 still fits in uint64_t. */
    uint64_t powers_of_three[40];
    powers_of_three[0] = 1;
    for (int i = 1; i < 40; ++i)
        powers_of_three[i] = powers_of_three[i - 1] * 3;

    uint64_t answer = 0;
    uint64_t prefix_weight = 1;
    for (int bit = 39; bit >= 0; --bit) {
        if ((upper >> bit) & 1U) {
            answer += prefix_weight * powers_of_three[bit];
            prefix_weight *= 2;
```

Rust. Source: `solutions/rust/src/bin/p242.rs`.

```
fn solve() -> u64 {
    let upper = (1_000_000_000_000u64 - 1) / 4;
    let mut p3 = [1u64; 40];
    for i in 1..40 {
        p3[i] = p3[i - 1] * 3;
    }
    let (mut answer, mut weight) = (0, 1);
    for bit in (0..40).rev() {
        if upper >> bit & 1 == 1 {
            answer += weight * p3[bit];
            weight *= 2;
        }
    }
    answer + weight
```

Factor. Source: `solutions/factor/euler/extended/extended.factor`.

```
:: p242 ( -- n )
    999999999999 4 /i :> upper
    40 1 <array> :> powers
    1 39 [a..b] [| i | i 1 - powers nth 3 * i powers set-nth ] each
    0 :> answer! 1 :> weight!
    39 0 [a..b] [| bit |
        upper 2 bit ^ /i 2 mod 1 = [
            answer weight bit powers nth * + answer!
```

```
            weight 2 * weight!
        ] when
    ] each answer weight + ;

! Problem 243: primorial construction plus exact totient comparison.
```

Comparison. C exposes dynamic programming or memoization. Rust exposes dynamic programming or memoization. Factor exposes modular arithmetic. No validated source exists for Gforth. All listed programs print the same exact answer string from the manifest. The excerpts can differ in algorithm, data representation, and startup cost.

40.243 Problem 243: Resilience

The program computes the exact answer for Resilience.

C. Source: `solutions/c/resilience_243.c`.

```
static uint64_t totient(uint64_t n) {
    uint64_t result = n;
    uint64_t remaining = n;
    for (uint64_t p = 2; p * p <= remaining; ++p) {
        if (remaining % p != 0)
            continue;
        result -= result / p;
        do { remaining /= p; } while (remaining % p == 0);
    }
    if (remaining > 1)
        result -= result / remaining;
    return result;
}
```

Rust. Source: `solutions/rust/src/bin/p243.rs`.

```
fn phi(n: u64) -> u64 {
    let (mut result, mut rem) = (n, n);
    let mut p = 2;
    while p <= rem / p {
        if rem % p == 0 {
            result -= result / p;
            while rem % p == 0 {
                rem /= p;
            }
        }
        p += 1;
    }
    if rem > 1 {
        result -= result / rem;
```

Factor. Source: `solutions/factor/euler/extended/extended.factor`.

```
:: p243 ( -- n )
```

```
    { 2 3 5 7 11 13 17 19 23 29 31 37 } :> primes
    1 :> base! 1 :> phi! f :> crossed!
    primes [| p | crossed not [
        base p * base! phi p 1 - * phi!
        phi 94744 * base 15499 * < [ t crossed! ] when
    ] when ] each
    1 :> multiplier! 0 :> answer!
    [ answer zero? ] [
        base multiplier * :> d
        d p243-phi 94744 * d 1 - 15499 * < [ d answer! ] when
        multiplier 1 + multiplier!
    ] while answer ;
```

Comparison. C exposes prime generation or testing. Rust exposes direct bounded arithmetic. Factor exposes prime generation or testing. No validated source exists for Gforth. All listed programs print the same exact answer string from the manifest. The excerpts can differ in algorithm, data representation, and startup cost.

40.244 Problem 244: Sliders

The program computes the exact answer for Sliders.

C. Source: `solutions/c/sliders_244.c`.

```
enum { STATE_COUNT = 1 << 20 };
static const uint64_t CHECK_MOD = UINT64_C(100000007);

static uint32_t make_state(uint16_t red, int blank) {
    return (uint32_t)red | ((uint32_t)blank << 16);
}

int main(void) {
    /* Red positions read row-major from the two diagrams. */
    const uint16_t start_red = (uint16_t)((1U << 1) | (1U << 4) | (1U << 5)
        | (1U << 8) | (1U << 9) | (1U << 12) | (1U << 13));
    const uint16_t target_red = (uint16_t)((1U << 2) | (1U << 5) | (1U << 7)
        | (1U << 8) | (1U << 10) | (1U << 13) | (1U << 15));
    const uint32_t start = make_state(start_red, 0);
```

Rust. Source: `solutions/rust/src/bin/p244.rs`.

```
fn solve() -> u64 {
    const N: usize = 1 << 20;
    const MOD: u64 = 100_000_007;
    let start_red = (1 << 1) | (1 << 4) | (1 << 5) | (1 << 8) | (1 << 9) | (1 << 12) | (1 << 13);

    let target_red = (1 << 2) | (1 << 5) | (1 << 7) | (1 << 8) | (1 << 10) | (1 << 13) | (1 <<
    15);
    let start = start_red;
    let target = target_red;
    let mut distance = vec![u8::MAX; N];
    let (mut count, mut checks) = (vec![0u64; N], vec![0u64; N]);
    let mut q = std::collections::VecDeque::new();
```

```
    q.push_back(start);
    distance[start] = 0;
    count[start] = 1;
    while let Some(state) = q.pop_front() {
```

Comparison. C exposes modular arithmetic. Rust exposes modular arithmetic. No validated source exists for Gforth, and Factor. All listed programs print the same exact answer string from the manifest. The excerpts can differ in algorithm, data representation, and startup cost.

40.245 Problem 245: Coresilience

The program computes the exact answer for Coresilience.

C. Source: `solutions/c/coresilience_245.c`.

```
typedef uint64_t U64;
typedef unsigned __int128 U128;
enum { KMAX = 447213, MAX_FACTORS = 8 };
static const U64 LIMIT = UINT64_C(200000000000);
static int *primes, prime_count;
static U64 answer;
static U64 *remaining;
static uint32_t *factor;
static unsigned char *exponent, *factor_count;

static U64 mulmod(U64 a,U64 b,U64 m){return (U64)((U128)a*b%m);}
static U64 powmod(U64 a,U64 e,U64 m){U64 r=1;while(e){if(e&1)r=mulmod(r,a,m);a=mulmod(a,a,m);e
    >>=1;}return r;}
static int prime64(U64 n){
    if(n<2)return 0;
```

Rust. Source: `solutions/rust/src/bin/p245.rs`.

```
fn mul(a: u64, b: u64, m: u64) -> u64 {
    (a as u128 * b as u128 % m as u128) as u64
}
fn pow(mut a: u64, mut e: u64, m: u64) -> u64 {
    let mut r = 1;
    while e > 0 {
        if e & 1 == 1 {
            r = mul(r, a, m);
        }
        a = mul(a, a, m);
        e >>= 1;
    }
    r
}
```

Comparison. C exposes prime generation or testing, and modular arithmetic. Rust exposes prime generation or testing. No validated source exists for Gforth, and Factor. All listed programs print the

same exact answer string from the manifest. The excerpts can differ in algorithm, data representation, and startup cost.

40.246 Problem 246: Tangents to an Ellipse

The program computes the exact answer for Tangents to an Ellipse.

C. Source: `solutions/c/tangents_to_an_ellipse_246.c`.

```
typedef __int128 I128;

static int qualifies(int64_t x,int64_t y){
    I128 xx=(I128)x*x, yy=(I128)y*y, radius=xx+yy;
    const I128 a2=56250000,b2=31250000;
    /* Tangents require strict exteriority.  Up through the director circle
       their included angle is at least 90 degrees and needs no tan test. */
    if(b2*xx+a2*yy<=a2*b2)return 0;
    if(radius<=a2+b2)return 1;
    I128 value=radius*radius-(I128)300000000*xx
        -(I128)400000000*yy+(I128)14687500000000000LL;
    return value<0;
}
```

Rust. Source: `solutions/rust/src/bin/p246.rs`.

```
fn qualifies(x: i64, y: i64) -> bool {
    let (xx, yy) = (x as i128 * x as i128, y as i128 * y as i128);
    let (a2, b2) = (56_250_000i128, 31_250_000i128);
    if b2 * xx + a2 * yy <= a2 * b2 {
        return false;
    }
    let radius = xx + yy;
    if radius <= a2 + b2 {
        return true;
    }
    radius * radius - 300_000_000i128 * xx - 400_000_000i128 * yy + 14_687_500_000_000_000i128 <
     0
}
fn solve() -> u64 {
    let mut count = 0;
```

Comparison. C exposes direct bounded arithmetic. Rust exposes direct bounded arithmetic. No validated source exists for Gforth, and Factor. All listed programs print the same exact answer string from the manifest. The excerpts can differ in algorithm, data representation, and startup cost.

40.247 Problem 247: Squares Under a Hyperbola

The program computes the exact answer for Squares Under a Hyperbola.

C. Source: `solutions/c/squares_under_a_hyperbola_247.c`.

```
typedef struct { double x,y,side; int left,below; } Square;
static Square *heap;static size_t heap_size,heap_capacity;

static double side_at(double x,double y){double d=x-y;return (sqrt(d*d+4.0)-x-y)/2.0;}
static void push(Square value){
    if(heap_size==heap_capacity){heap_capacity=heap_capacity?heap_capacity*2:1024;Square *grown=
    realloc(heap,heap_capacity*sizeof(*heap));if(!grown)exit(1);heap=grown;}
    size_t i=heap_size++;while(i){size_t parent=(i-1)/2;if(heap[parent].side>=value.side)break;
    heap[i]=heap[parent];i=parent;}heap[i]=value;
}
static Square pop_max(void){
    Square result=heap[0],last=heap[--heap_size];size_t i=0;
    while(2*i+1<heap_size){size_t child=2*i+1;if(child+1<heap_size&&heap[child+1].side>heap[
    child].side)++child;if(heap[child].side<=last.side)break;heap[i]=heap[child];i=child;}
    if(heap_size)heap[i]=last;
    return result;
}
```

Rust. Source: `solutions/rust/src/bin/p247.rs`.

```
#[derive(Clone)]
struct S {
    x: f64,
    y: f64,
    side: f64,
    left: u32,
    below: u32,
}
impl Eq for S {}
impl PartialEq for S {
    fn eq(&self, o: &Self) -> bool {
        self.side == o.side
    }
}
```

Comparison. C exposes direct bounded arithmetic. Rust exposes direct bounded arithmetic. No validated source exists for Gforth, and Factor. All listed programs print the same exact answer string from the manifest. The excerpts can differ in algorithm, data representation, and startup cost.

40.248 Problem 248: Euler's Totient Function Equals 13!

The program computes the exact answer for Euler's Totient Function Equals 13!.

C. Source: `solutions/c/euler_s_totient_function_equals_13_factorial_248.c`.

```
typedef uint64_t U64;
static const U64 TARGET=UINT64_C(6227020800);
static U64 divisors[1584];static int divisor_count;
static U64 candidates[1584];static int candidate_count;
static U64 *solutions;static size_t solution_count,solution_capacity;
static int small_primes[8000],small_count;
```

```
static int compare_u64(const void *a,const void *b){U64 x=*(const U64*)a,y=*(const U64*)b;return
     x<y?-1:x>y;}
static int is_prime(U64 n){if(n<2)return 0;for(int i=0;i<small_count&&(U64)small_primes[i]*
    small_primes[i]<=n;++i)if(n%(U64)small_primes[i]==0)return n==(U64)small_primes[i];return 1;}


static void make_divisors(int at,U64 value){
    static const U64 prime[]={2,3,5,7,11,13};static const int max_exp[]={10,5,2,1,1,1};
    if(at==6){divisors[divisor_count++]=value;return;}U64 power=1;for(int e=0;e<=max_exp[at];++e)
    {make_divisors(at+1,value*power);power*=prime[at];}
}
```

Rust. Source: `solutions/rust/src/bin/p248.rs`.

```
const TARGET: u64 = 6_227_020_800;
fn prime(n: u64, ps: &[u64]) -> bool {
    n >= 2 && !ps.iter().take_while(|&&p| p <= n / p).any(|p| n % p == 0)
}
fn divisors(at: usize, value: u64, out: &mut Vec<u64>) {
    let ps = [2, 3, 5, 7, 11, 13];
    let es = [10, 5, 2, 1, 1, 1];
    if at == 6 {
        out.push(value);
        return;
    }
    let mut power = 1;
    for _ in 0..=es[at] {
        divisors(at + 1, value * power, out);
```

Comparison. C exposes prime generation or testing, sorting or set deduplication, and recursive search. Rust exposes prime generation or testing, sorting or set deduplication, and recursive search. No validated source exists for Gforth, and Factor. All listed programs print the same exact answer string from the manifest. The excerpts can differ in algorithm, data representation, and startup cost.

40.249 Problem 249: Prime Subset Sums

The program computes the exact answer for Prime Subset Sums.

C. Source: `solutions/c/prime_subset_sums_249.c`.

```
static const uint64_t MODULUS=UINT64_C(10000000000000000);

int main(void){
    enum{PRIME_LIMIT=5000};unsigned char composite[PRIME_LIMIT]={0};int primes[700],count=0,
    total_sum=0;
    for(int p=2;p<PRIME_LIMIT;++p)if(!composite[p]){primes[count++]=p;total_sum+=p;if(p*p<
    PRIME_LIMIT)for(int m=p*p;m<PRIME_LIMIT;m+=p)composite[m]=1;}
    uint64_t *dp=calloc((size_t)total_sum+1,sizeof(*dp));unsigned char *sum_composite=calloc((
    size_t)total_sum+1,1);if(!dp||!sum_composite)return 1;
    dp[0]=1;int current=0;
    for(int i=0;i<count;++i){int p=primes[i];for(int s=current;s>=0;--s)if(dp[s])dp[s+p]=(dp[s+p
    ]+dp[s])%MODULUS;current+=p;}
```

```
    sum_composite[0]=sum_composite[1]=1;for(int p=2;p*p<=total_sum;++p)if(!sum_composite[p])for(
    int m=p*p;m<=total_sum;m+=p)sum_composite[m]=1;
    uint64_t answer=0;for(int s=2;s<=total_sum;++s)if(!sum_composite[s])answer=(answer+dp[s])%
    MODULUS;
    printf("%016llu\n",(unsigned long long)answer);free(sum_composite);free(dp);return 0;
}
```

Rust. Source: `solutions/rust/src/bin/p249.rs`.

```
fn solve() -> u64 {
    const MOD: u64 = 10_000_000_000_000_000;
    let mut comp = [false; 5000];
    let mut ps = Vec::new();
    let mut total = 0;
    for p in 2..5000 {
        if !comp[p] {
            ps.push(p);
            total += p;
            if p < 5000 / p {
                for m in (p * p..5000).step_by(p) {
                    comp[m] = true;
                }
            }
```

Comparison. C exposes prime generation or testing, and dynamic programming or memoization. Rust exposes prime generation or testing, modular arithmetic, and dynamic programming or memoization. No validated source exists for Gforth, and Factor. All listed programs print the same exact answer string from the manifest. The excerpts can differ in algorithm, data representation, and startup cost.

40.250 Problem 250: 250250

The program computes the exact answer for 250250.

C. Source: `solutions/c/250250_250.c`.

```
static const uint64_t MODULUS=UINT64_C(10000000000000000);
static int power_mod_250(int base,int exponent){int result=1;base%=250;while(exponent){if(
    exponent&1)result=result*base%250;base=base*base%250;exponent>>=1;}return result;}

int main(void){
    uint64_t dp[250]={0},next[250];dp[0]=1;
    for(int i=1;i<=250250;++i){int value=power_mod_250(i,i);for(int r=0;r<250;++r)next[r]=dp[r];
    for(int r=0;r<250;++r){int target=(r+value)%250;next[target]=(next[target]+dp[r])%MODULUS;}
    for(int r=0;r<250;++r)dp[r]=next[r];}
    uint64_t answer=(dp[0]+MODULUS-1)%MODULUS;printf("%016llu\n",(unsigned long long)answer);
    return 0;
}
```

Rust. Source: `solutions/rust/src/bin/p250.rs`.

```
fn pow250(mut b: u32, mut e: u32) -> usize {
    let mut r = 1;
    b %= 250;
    while e > 0 {
        if e & 1 == 1 {
            r = r * b % 250;
        }
        b = b * b % 250;
        e >>= 1;
    }
    r as usize
}
fn solve() -> u64 {
    const M: u64 = 10_000_000_000_000_000;
```

Comparison. C exposes modular arithmetic, and dynamic programming or memoization. Rust exposes modular arithmetic, and dynamic programming or memoization. No validated source exists for Gforth, and Factor. All listed programs print the same exact answer string from the manifest. The excerpts can differ in algorithm, data representation, and startup cost.

40.251 Problem 251: Cardano Triplets

The program computes the exact answer for Cardano Triplets.

C. Source: `solutions/c/cardano_triplets_251.c`.

```
static int64_t gcd64(int64_t a,int64_t b){while(b){int64_t t=a%b;a=b;b=t;}return a;}
static int64_t inverse_mod(int64_t a,int64_t m){
    int64_t old_r=a,r=m,old_s=1,s=0;while(r){int64_t q=old_r/r,t=old_r-q*r;old_r=r;r=t;t=old_s-q*
    s;old_s=s;s=t;}old_s%=m;if(old_s<0)old_s+=m;return old_s;
}

int main(void){
    const int64_t limit=110000000,L=limit+1;uint64_t total=0;
    int64_t qmax=(int64_t)sqrt((double)limit);while((qmax+1)*(qmax+1)<=limit)++qmax;while(qmax*
    qmax>limit)--qmax;
    for(int64_t q=1;q<=qmax;++q){
        int64_t q2=q*q;
        int64_t denominator=8*q*q2+3*q+1;
        total+=(uint64_t)((L+3*q2)/denominator); /* r=1, p=1..pmax */

        int64_t rmax=(int64_t)cbrt((double)(8*q*L))+3;if(!(rmax&1))++rmax;
```

Rust. Source: `solutions/rust/src/bin/p251.rs`.

```
fn gcd(mut a: i64, mut b: i64) -> i64 {
    while b != 0 {
        (a, b) = (b, a % b);
    }
    a
}
fn inverse(a: i64, m: i64) -> i64 {
```

```
    let (mut r0, mut r, mut s0, mut s) = (a, m, 1, 0);
    while r != 0 {
        let q = r0 / r;
        (r0, r) = (r, r0 - q * r);
        (s0, s) = (s, s0 - q * s);
    }
    s0.rem_euclid(m)
```

Factor. Source: `solutions/factor/euler/extended/extended.factor`.

```
:: p251 ( -- n )
    110000001 :> limit 110000000 sqrt >integer :> qmax 0 :> total!
    1 qmax [a..b] [| q |
        q sq :> q2
        total limit q2 3 * + q q2 * 8 * q 3 * + 1 + /i + total!
        q limit * 8 * >float 1.0 3.0 / ^ >integer 3 + :> rmax!
        rmax even? [ rmax 1 + rmax! ] when
        3 rmax 2 <range> [| r |
            q r gcd nip 1 = [
                r sq :> r2
                q 8 * r2 mod r2 inverse-mod 3 * r2 mod :> p0!
                p0 zero? [ r2 p0! ] when
                p0 q * 8 * 3 - r2 /i :> s0
                p0 q 3 * r + * s0 q2 * + :> base
```

Comparison. C exposes modular arithmetic. Rust exposes prime generation or testing. Factor exposes modular arithmetic. No validated source exists for Gforth. All listed programs print the same exact answer string from the manifest. The excerpts can differ in algorithm, data representation, and startup cost.

40.252 Problem 252: Convex Holes

The program computes the exact answer for Convex Holes.

C. Source: `solutions/c/convex_holes_252.c`.

```
static Point point[N];static uint64_t *left_bits;static unsigned char *inner_collinear;
static int sort_anchor;
static int64_t cross3(int a,int b,int c){return(int64_t)(point[b].x-point[a].x)*(point[c].y-
    point[a].y)-(int64_t)(point[b].y-point[a].y)*(point[c].x-point[a].x);}
static int polar_compare(const void *pa,const void *pb){int a=*(const int*)pa,b=*(const int*)pb;
    int64_t c=cross3(sort_anchor,a,b);if(c)return c>0?-1:1;int64_t da=(int64_t)(point[a].x-point
    [sort_anchor].x)*(point[a].x-point[sort_anchor].x)+(int64_t)(point[a].y-point[sort_anchor].y
    )*(point[a].y-point[sort_anchor].y);int64_t db=(int64_t)(point[b].x-point[sort_anchor].x)*(
    point[b].x-point[sort_anchor].x)+(int64_t)(point[b].y-point[sort_anchor].y)*(point[b].y-
    point[sort_anchor].y);return da<db?-1:da>db;}
static uint64_t *edge_bits(int a,int b){return left_bits+((size_t)a*N+b)*WORDS;}
static int empty_triangle(int a,int b,int c){uint64_t *ab=edge_bits(a,b),*bc=edge_bits(b,c),*ca=
    edge_bits(c,a);for(int w=0;w<WORDS;++w)if(ab[w]&bc[w]&ca[w])return 0;return 1;}

int main(void){
    uint64_t state=290797;for(int i=0;i<N;++i){state=state*state%50515093;point[i].x=(int)(state
    %2000)-1000;state=state*state%50515093;point[i].y=(int)(state%2000)-1000;}
```

```
    left_bits=calloc((size_t)N*N*WORDS,sizeof(*left_bits));inner_collinear=calloc((size_t)N*N,1);
    if(!left_bits||!inner_collinear)return 1;
    for(int a=0;a<N;++a)for(int b=0;b<N;++b)if(a!=b){uint64_t *bits=edge_bits(a,b);for(int p=0;p<
    N;++p)if(p!=a&&p!=b){int64_t c=cross3(a,b,p);if(c>0)bits[p>>6]|=UINT64_C(1)<<(p&63);else if(
    c==0){int64_t dot=(int64_t)(point[p].x-point[a].x)*(point[p].x-point[b].x)+(int64_t)(point[p
    ].y-point[a].y)*(point[p].y-point[b].y);if(dot<0)inner_collinear[(size_t)a*N+b]=1;}}}

    int64_t global_best=0;int vertices[N];int64_t *dp=malloc((size_t)N*N*sizeof(*dp));if(!dp)
    return 1;
    for(int anchor=0;anchor<N;++anchor){int m=0;for(int v=0;v<N;++v)if(point[v].y>point[anchor].y
    ||(point[v].y==point[anchor].y&&point[v].x>point[anchor].x))vertices[m++]=v;if(m<2)continue;
    sort_anchor=anchor;qsort(vertices,(size_t)m,sizeof(*vertices),polar_compare);memset(dp,0,(
    size_t)m*m*sizeof(*dp));
```

Rust. Source: `solutions/rust/src/bin/p252.rs`.

```
#[derive(Clone, Copy)]
struct P {
    x: i32,
    y: i32,
}
const N: usize = 500;
const W: usize = 8;
fn cross(ps: &[P], a: usize, b: usize, c: usize) -> i64 {
    (ps[b].x - ps[a].x) as i64 * (ps[c].y - ps[a].y) as i64
        - (ps[b].y - ps[a].y) as i64 * (ps[c].x - ps[a].x) as i64
}
fn solve() -> String {
    let mut state = 290797u64;
    let mut ps = Vec::new();
```

Comparison. C exposes dynamic programming or memoization, and sorting or set deduplication. Rust exposes dynamic programming or memoization, and sorting or set deduplication. No validated source exists for Gforth, and Factor. All listed programs print the same exact answer string from the manifest. The excerpts can differ in algorithm, data representation, and startup cost.

40.253 Problem 253: Tidying Up A

The program computes the exact answer for Tidying Up A.

C. Source: `solutions/c/tidying_up_a_253.c`.

```
enum { PIECES = 40, MAX_SEGMENTS = 20, HASH_SIZE = 1 << 20 };

typedef struct {
    uint64_t key[3];
    double expectation;
    unsigned char best;
    unsigned char used;
} Entry;

static Entry *memo;
```

```
static void sort_lengths(unsigned char *lengths, int count) {
    for (int i = 1; i < count; ++i) {
        unsigned char value = lengths[i];
```

Rust. Source: `solutions/rust/src/bin/p253.rs`.

```
fn expected(state: &[u8], best: u8, memo: &mut HashMap<(Vec<u8>, u8), f64>) -> f64 {
    if state.is_empty() {
        return best as f64;
    }
    let key = (state.to_vec(), best);
    if let Some(&v) = memo.get(&key) {
        return v;
    }
    let remaining: usize = state.iter().map(|&x| x as usize).sum();
    let mut weighted = 0.0;
    let mut first = 0;
    while first < state.len() {
        let mut after = first + 1;
        while after < state.len() && state[after] == state[first] {
```

Comparison. C exposes dynamic programming or memoization, and sorting or set deduplication. Rust exposes dynamic programming or memoization, and sorting or set deduplication. No validated source exists for Gforth, and Factor. All listed programs print the same exact answer string from the manifest. The excerpts can differ in algorithm, data representation, and startup cost.

40.254 Problem 254: Sums of Digit Factorials

The program computes the exact answer for Sums of Digit Factorials.

C. Source: `solutions/c/sums_of_digit_factorials_254.c`.

```
enum { LIMIT = 150, FACT9 = 362880, BUCKET_SIZE = FACT9 / 9 };

typedef struct {
    uint32_t remainder;
    uint16_t digit_sum;
    unsigned char length;
    unsigned char count[8];
} Remainder;

typedef struct {
    uint64_t nines;
    uint16_t lower_digit_sum;
    unsigned char lower_length;
    unsigned char count[8];
```

Rust. Source: `solutions/rust/src/bin/p254.rs`.

```
#[derive(Clone)]
struct R {
```

```
    r: u32,
    digit_sum: u16,
    length: u8,
    count: [u8; 8],
}
fn decode(r: u32) -> R {
    let facts = [1, 1, 2, 6, 24, 120, 720, 5040, 40320];
    let mut value = r;
    let (mut length, mut sum, mut count) = (0, 0, [0; 8]);
    for d in (1..=8).rev() {
        let c = (value / facts[d]) as u8;
        value %= facts[d];
```

Comparison. C exposes modular arithmetic, exact large-integer arithmetic, and sorting or set deduplication. Rust exposes exact large-integer arithmetic, and sorting or set deduplication. No validated source exists for Gforth, and Factor. All listed programs print the same exact answer string from the manifest. The excerpts can differ in algorithm, data representation, and startup cost.

40.255 Problem 255: Rounded Square Roots

The program computes the exact answer for Rounded Square Roots.

C. Source: `solutions/c/rounded_square_roots_255.c`.

```
static uint64_t ceiling_division(uint64_t numerator, uint64_t denominator) {
    return numerator / denominator + (numerator % denominator != 0);
}

static uint64_t iteration_total(uint64_t lo, uint64_t hi, uint64_t x,
                                unsigned completed_iterations) {
    uint64_t first_q = ceiling_division(lo, x);
    uint64_t last_q = ceiling_division(hi, x);
    uint64_t first_y = (x + first_q) / 2;
    uint64_t last_y = (x + last_q) / 2;
    uint64_t total = 0;

    for (uint64_t y = first_y; y <= last_y; ++y) {
        /* floor((x+q)/2)=y exactly for the following one or two q values. */
```

Rust. Source: `solutions/rust/src/bin/p255.rs`.

```
fn total(lo: u64, hi: u64, x: u64, done: u64) -> u64 {
    let first = lo.div_ceil(x);
    let last = hi.div_ceil(x);
    let (first_y, last_y) = ((x + first) / 2, (x + last) / 2);
    let mut answer = 0;
    for y in first_y..=last_y {
        let mut qlo = 2 * y - x;
        let mut qhi = qlo + 1;
        qlo = qlo.max(first);
        qhi = qhi.min(last);
        let child_lo = ((qlo - 1) * x + 1).max(lo);
        let child_hi = (qhi * x).min(hi);
```

```
        if y == x {
            answer += (child_hi - child_lo + 1) * (done + 1)
```

Factor. Source: `solutions/factor/p255.factor`.

```
! Native Factor Project Euler 255 interval-aggregated rounded-root solver.
USING: io math.parser project-euler.255 sequences ; IN: scratchpad
! Preserve the statement's required ten digits after the decimal point.
euler255 number>string "0" append print
```

Comparison. C exposes recursive search. Rust exposes direct bounded arithmetic. Factor exposes direct bounded arithmetic. No validated source exists for Gforth. All listed programs print the same exact answer string from the manifest. The excerpts can differ in algorithm, data representation, and startup cost.

41 Mathematical visualization atlas

This atlas supports algorithm explanations with actual data where available, and with explicitly labelled reduced examples elsewhere. Plots are generated by `make plots-math` and embedded only when the corresponding PDF exists. Every figure has an SVG, PNG and PDF counterpart under `build/math_plots/`. The complete topic-to-file map is recorded in `docs/visualization_index.md`. None of these figures is a measurement of algorithm runtime.

41.1 Integer geometry and spirals

41.2 Grids and dynamic programming

41.3 Discrete counting and networks

41.4 Pascal's pyramid

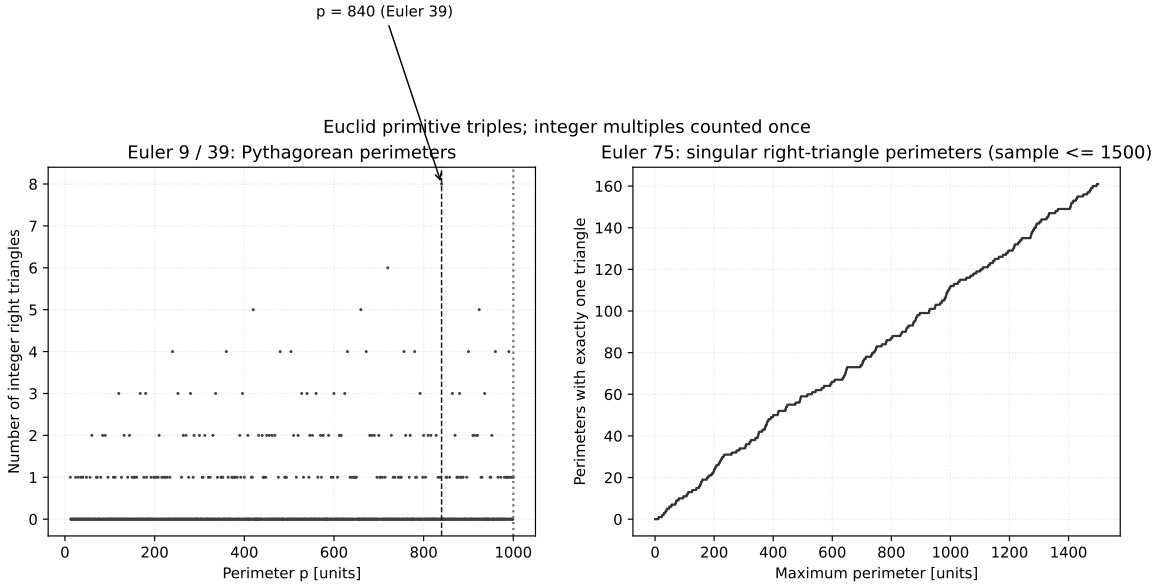


Figure 3: Project Euler 9, 39, 75: Euclid's formula enumerates right-triangle perimeter counts; the dashed marker gives the full problem 39 winner $p = 840$. Singular-count growth is sampled only to $p = 1500$.

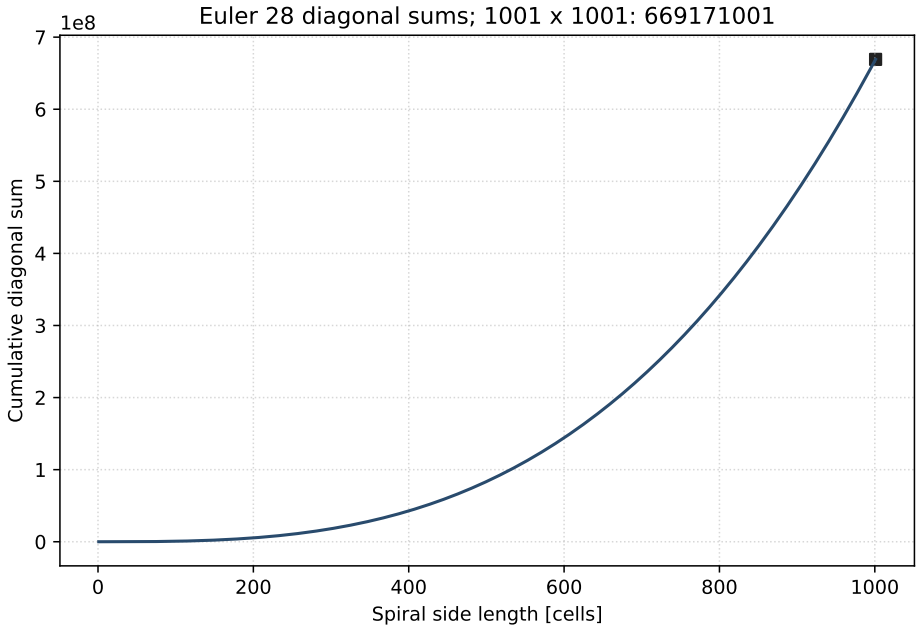


Figure 4: Project Euler 28: Sum of the four corners of each odd-width ring through the full 1001×1001 target.

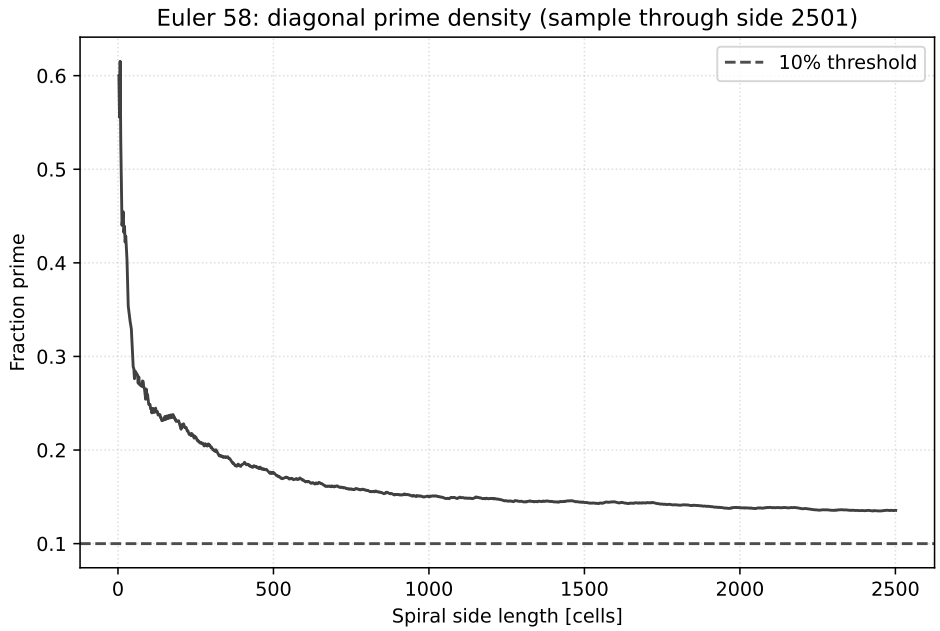


Figure 5: Project Euler 58: Prime density on spiral diagonals sampled through side length 2501, with a 10-percent reference line.

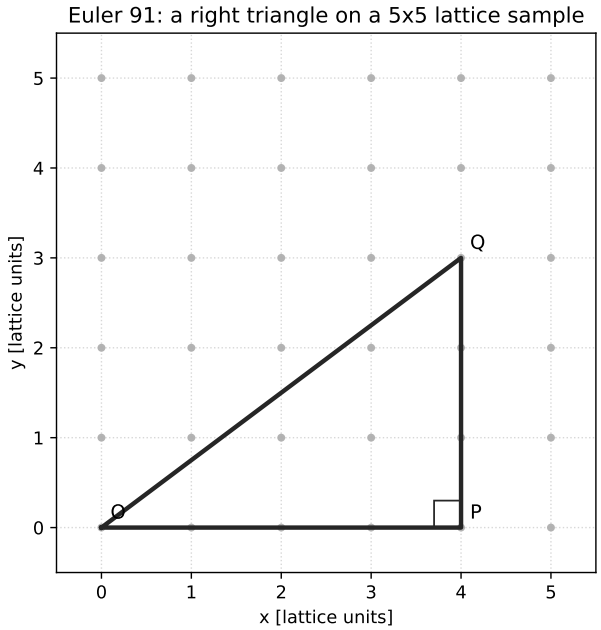


Figure 6: Project Euler 91: Lattice-coordinate geometry on a small grid; the full counting problem is not implemented here.

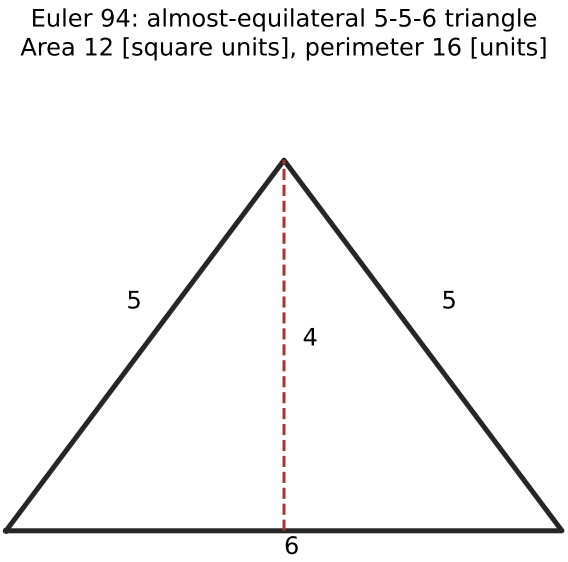


Figure 7: Project Euler 94: The $5, 5, 6$ example has an integral area and differs from an equilateral triangle by one unit.

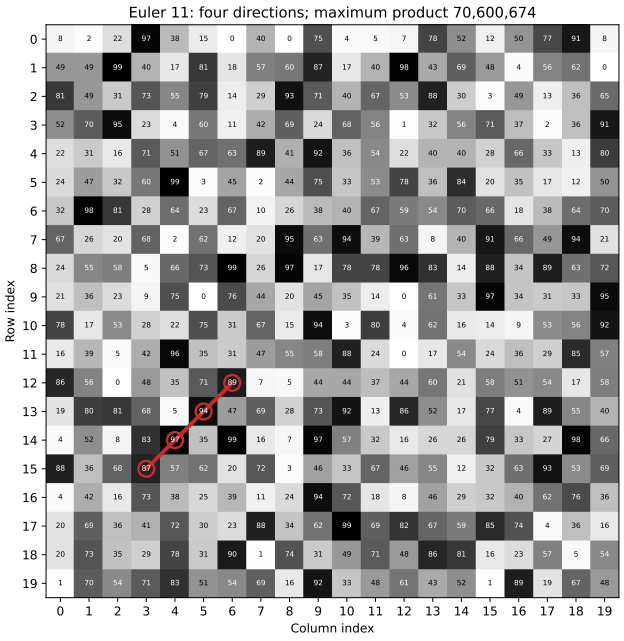


Figure 8: Project Euler 11: Actual 20×20 input grid from the repository. The highlighted window produces the verified maximum product $70,600,674$.

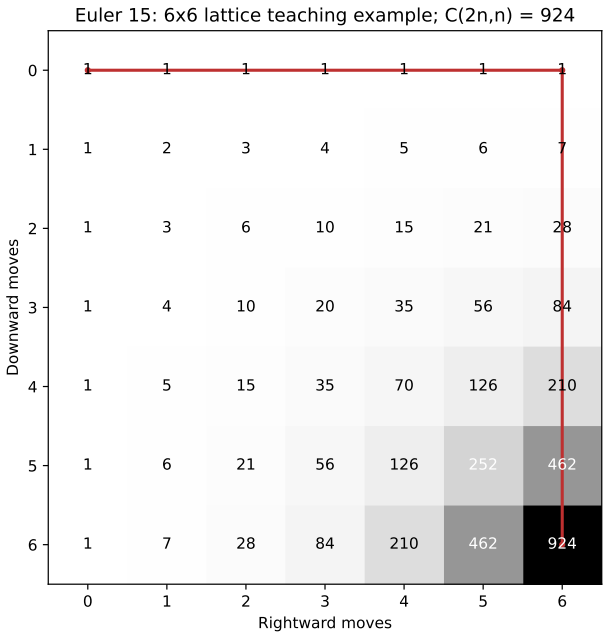


Figure 9: Project Euler 15: Binomial coefficients and a sample six-by-six lattice path. The chart is not the full problem input.

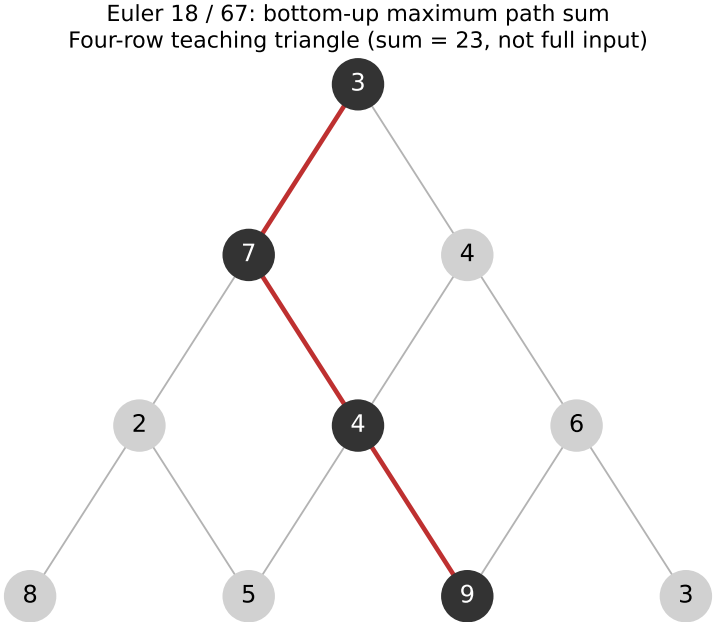


Figure 10: Project Euler 18, 67: A reduced four-row triangle shows how bottom-up dynamic programming identifies the maximum path sum.

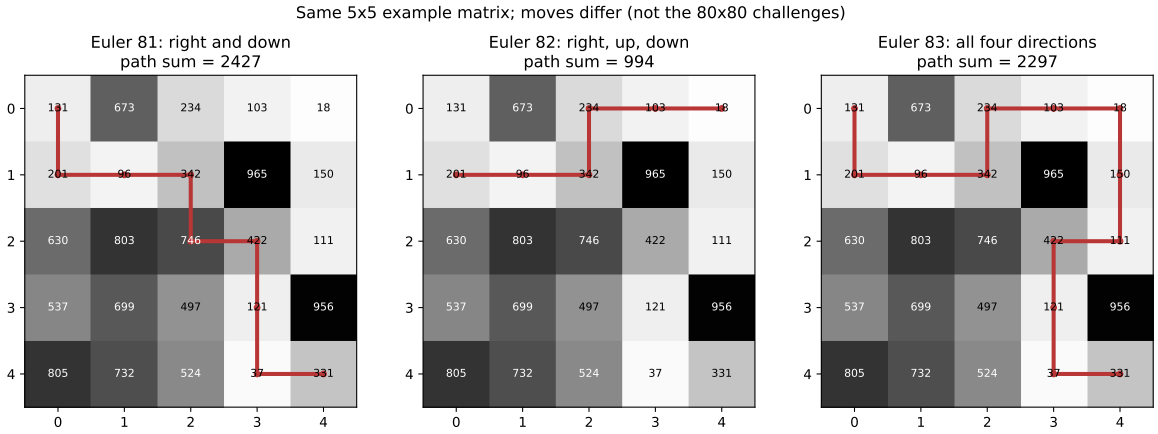


Figure 11: Project Euler 81, 82, 83: Three shortest-path models share one five-by-five example matrix but allow different moves. The full-sized input matrices are not supplied.

Euler 96: Sudoku teaching example
Bold given digits; gray digits filled by backtracking

4	8	**3**	9	**2**	1	**6**	5	7
9	6	7	**3**	4	**5**	8	2	**1**
2	5	**1**	**8**	7	**6**	**4**	9	3
5	4	**8**	**1**	3	**2**	**9**	7	6
7	2	9	5	6	4	1	3	**8**
1	3	**6**	**7**	9	**8**	**2**	4	5
3	7	**2**	**6**	8	**9**	**5**	1	4
8	1	4	**2**	5	**3**	7	6	**9**
6	9	**5**	4	**1**	7	**3**	8	2

Figure 12: Project Euler 96: A solved nine-by-nine example, distinguishing givens from cells filled by backtracking.

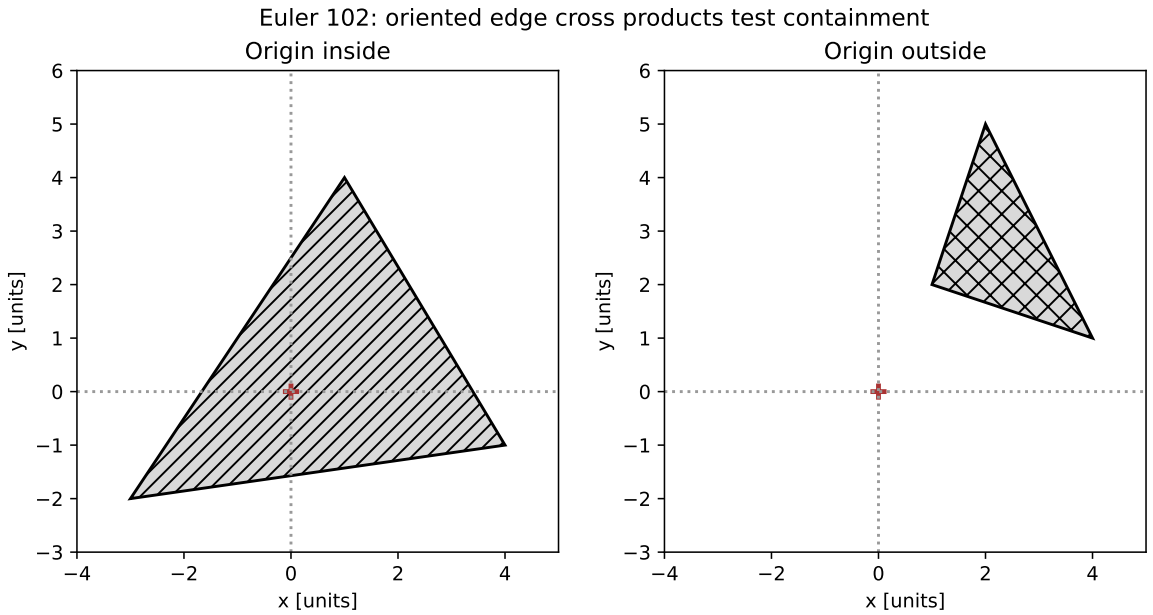


Figure 13: Project Euler 102: The signs of three oriented cross products decide whether the origin is inside a triangle.

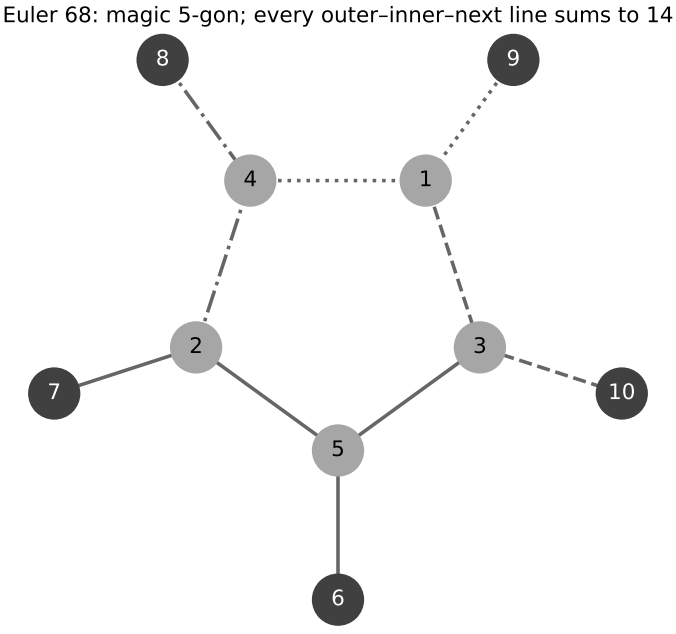


Figure 14: Project Euler 68: One valid five-gon configuration whose five triples have the same sum.

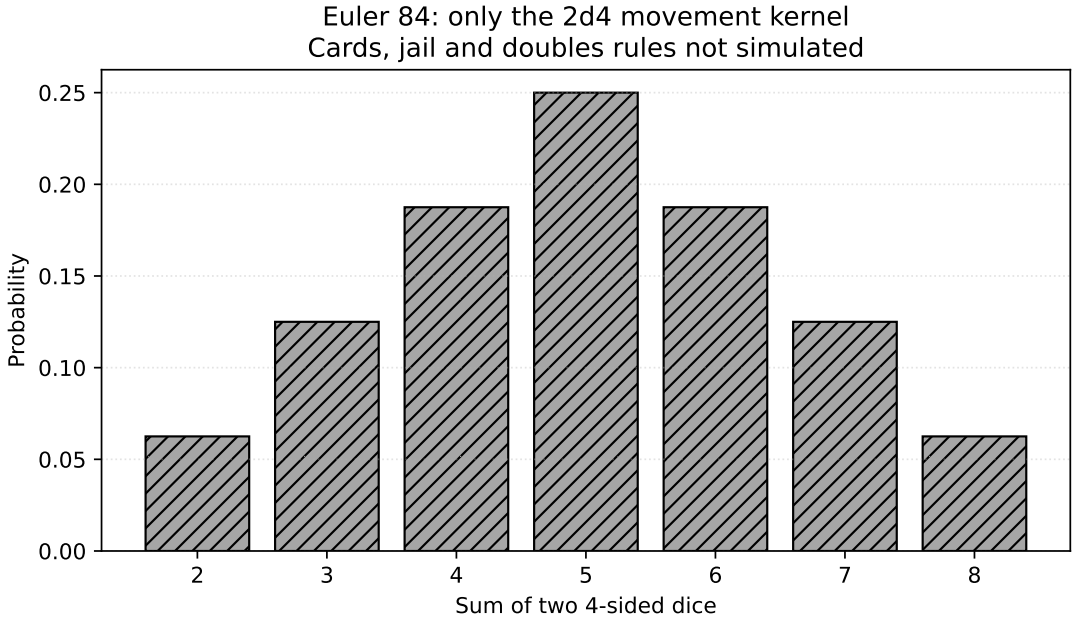


Figure 15: Project Euler 84: The probability mass function of two four-sided dice. This is only the movement kernel, not Monopoly's complete Markov model with jail and card rules.

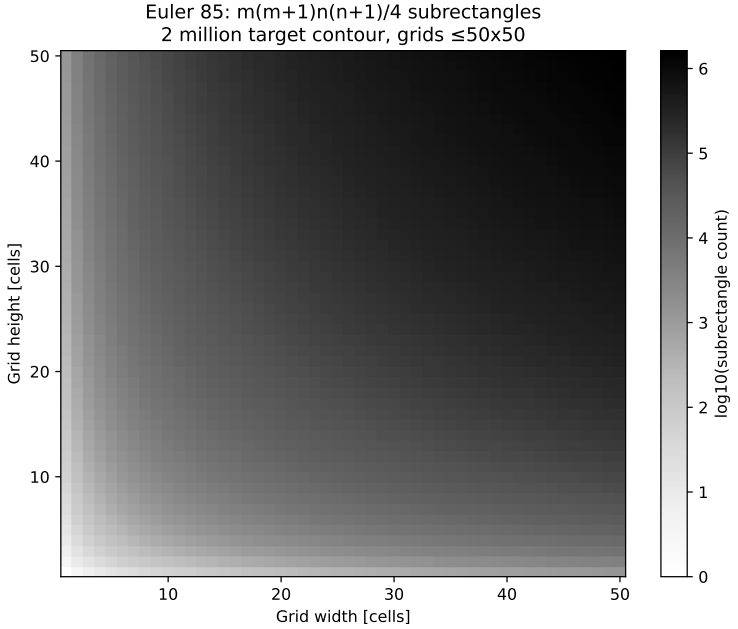


Figure 16: Project Euler 85: The number of rectangular subgrids across width and height, with a target contour at two million.

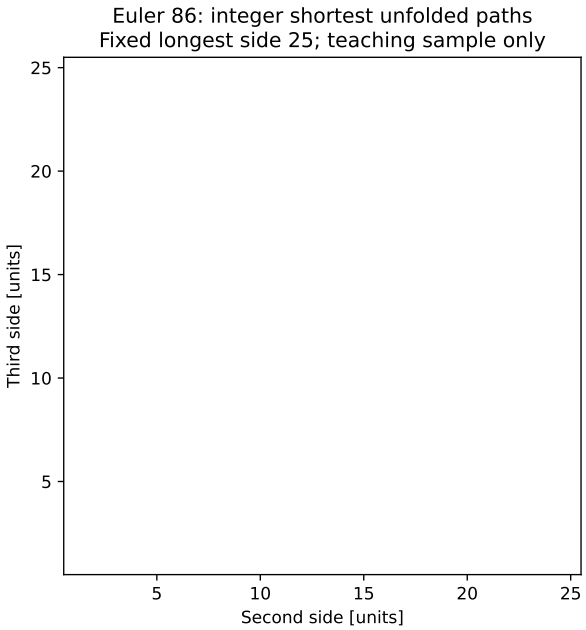


Figure 17: Project Euler 86: Integer-length unfolded surface routes for a restricted set of cuboids.

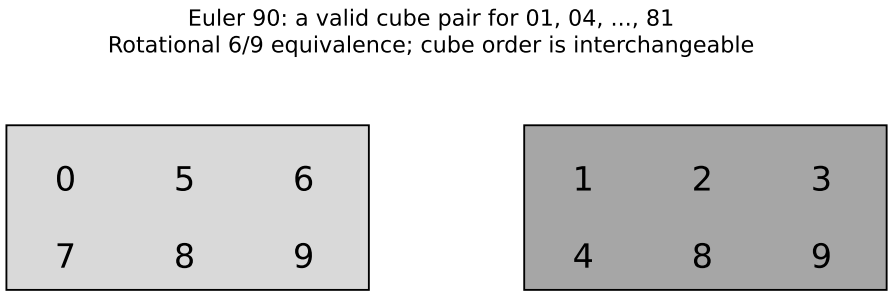


Figure 18: Project Euler 90: A valid digit placement on two cubes, where six and nine can be exchanged by rotating the cube.

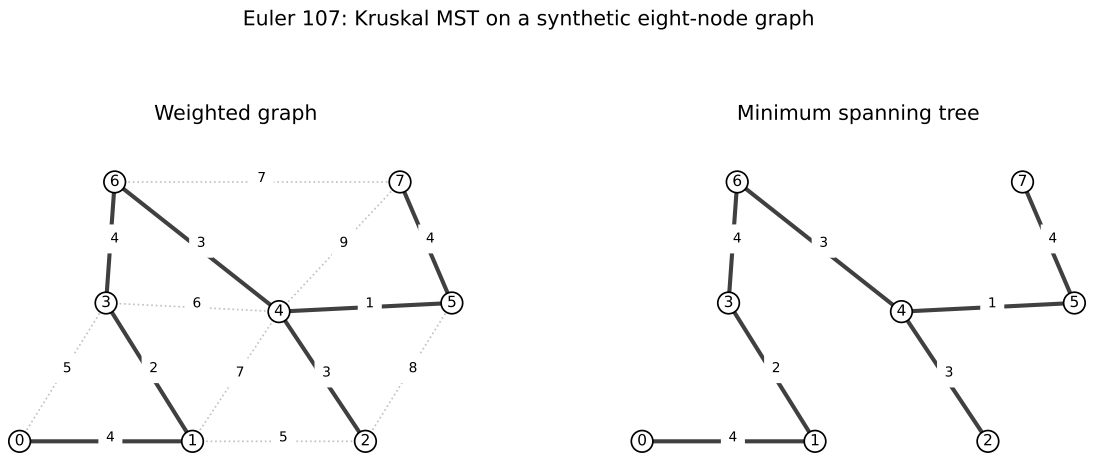


Figure 19: Project Euler 107: Kruskal's minimum-spanning-tree procedure, shown on an explicitly synthetic graph.

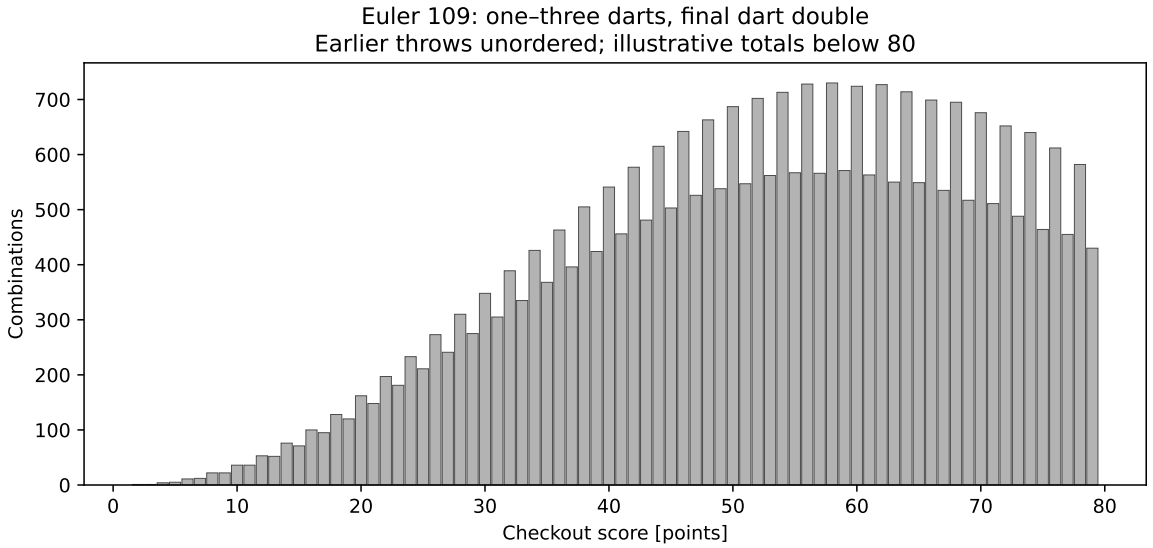


Figure 20: Project Euler 109: Distinct checkout combinations under score 80, with an obligatory final double.

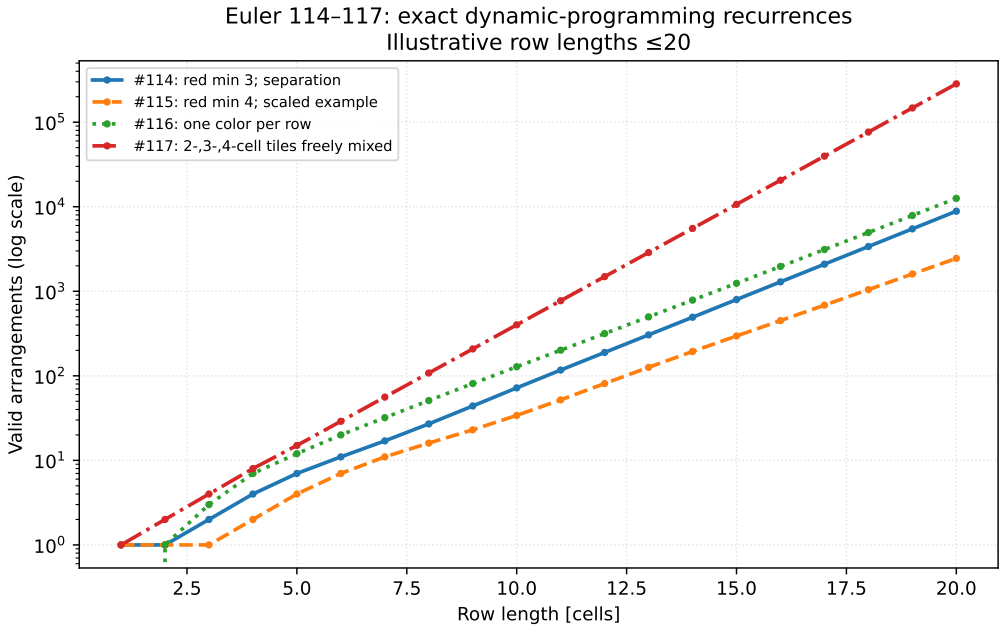


Figure 21: Project Euler 114, 115, 116, 117: Four exact dynamic-programming recurrences on short rows; the second curve is a scaled illustration, not the full problem-115 threshold.

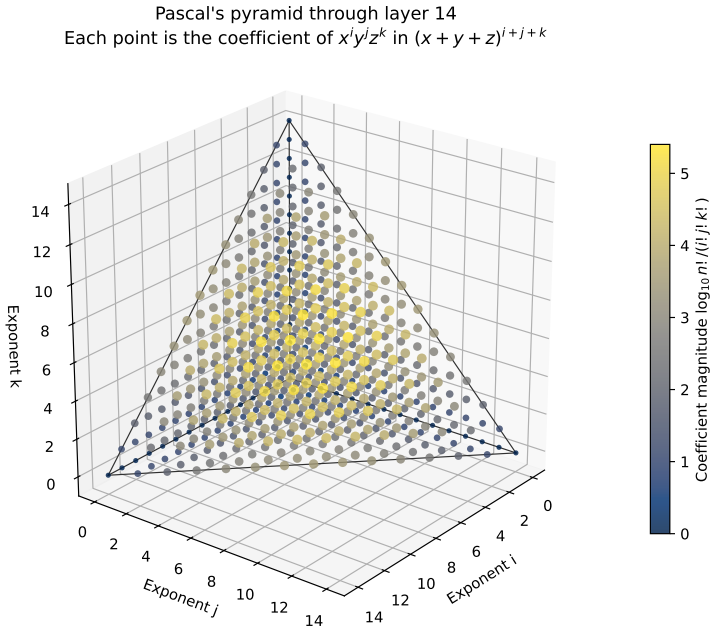


Figure 22: Every trinomial coefficient through layer 14. The integer coordinates (i, j, k) identify the coefficient of $x^i y^j z^k$ and fill a tetrahedron bounded by $i + j + k \leq 14$.

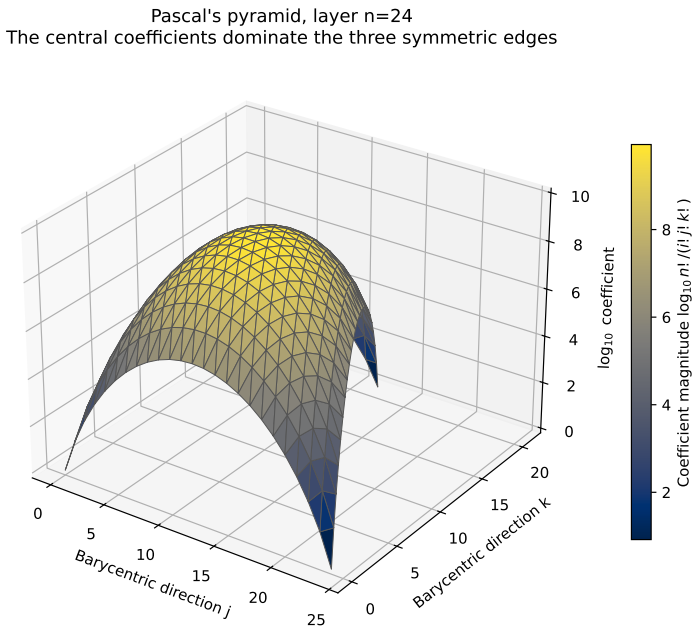


Figure 23: Layer 24 drawn in barycentric coordinates. Height and color encode the base-10 logarithm of the exact coefficient, so the three permutation-symmetric edges remain visible.

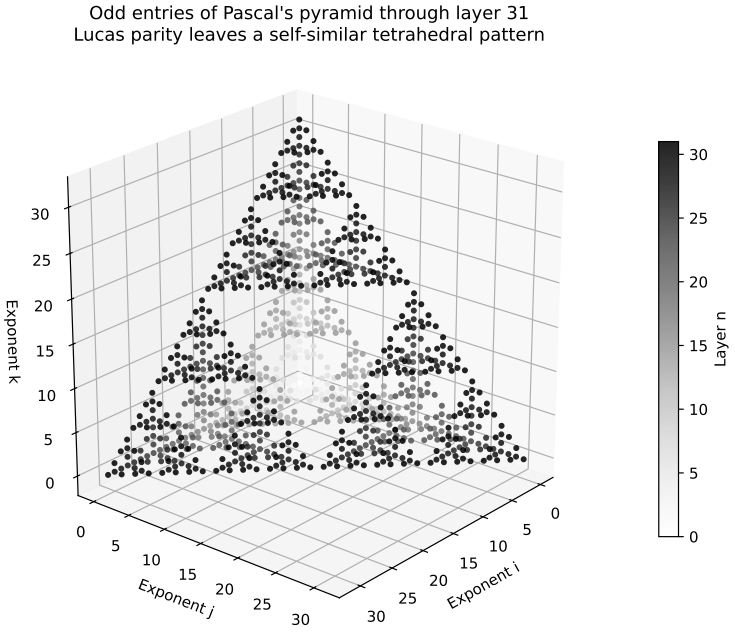


Figure 24: Odd coefficients through layer 31. Removing the even coefficients exposes the self-similar mod-2 tetrahedron.

A Shared Project Euler library API

This appendix is a generated reference. The problem programs own their algorithms. These modules provide arithmetic, input, and collection operations that more than one solver can reuse. Stack comments are the signatures for Gforth and Factor words.

A.1 C API

Source: `support/c/euler_common.h`, `support/c/euler_extra.h`.

`prime` `int prime(uint64_t n)`. Tests primality by trial division.

`gcd` `unsigned gcd(unsigned a, unsigned b)`. Computes a greatest common divisor.

`count_divisors` `unsigned count_divisors(uint64_t n)`. Provides a number-theory operation.

`divisor_sum` `unsigned divisor_sum(unsigned n)`. Provides a number-theory operation.

`palindrome` `int palindrome(unsigned n)`. Provides a reusable mathematical operation.

`sieve` `unsigned char *sieve(unsigned limit)`. Builds primality flags through an inclusive bound.

`digit_sum` `long long digit_sum(const mpz_t z)`. Provides a decimal-digit operation.

`namecmp` `int namecmp(const void *a, const void *b)`. Provides a reusable mathematical operation.

`factorial` `unsigned factorial(unsigned n)`. Provides a number-theory operation.

`digit_power_sum` `unsigned digit_power_sum(unsigned n, const unsigned table[10])`. Provides a decimal-digit operation.

`pentagonal` `int pentagonal(uint64_t n)`. Provides a reusable mathematical operation.

`distinct_factors` `unsigned distinct_factors(unsigned n)`. Provides a number-theory operation.

`euler_fail` `static inline void euler_fail(const char *message)`. Checks a C allocation, file, or failure boundary.

`euler_malloc` `static inline void *euler_malloc(size_t size)`. Checks a C allocation, file, or failure boundary.

`euler_calloc` `static inline void *euler_calloc(size_t count, size_t size)`. Checks a C allocation, file, or failure boundary.

`euler_open` `static inline FILE *euler_open(const char *path)`. Checks a C allocation, file, or failure boundary.

`euler_gcd` `static inline uint64_t euler_gcd(uint64_t a, uint64_t b)`. Computes a greatest common divisor.

`euler_isqrt` `static inline uint64_t euler_isqrt(uint64_t n)`. Returns the floor of an integer square root.

`euler_sieve` `static inline unsigned char *euler_sieve(size_t limit)`. Builds primality flags through an inclusive bound.

`euler_mulmod` `static inline uint64_t euler_mulmod(uint64_t a, uint64_t b, uint64_t m)`. Provides a reusable mathematical operation.

`euler_powmod` `static inline uint64_t euler_powmod(uint64_t a, uint64_t exponent, uint64_t m)`. Computes an integer power modulo a modulus.

`euler_is_prime` `static inline bool euler_is_prime(uint64_t n)`. Tests every 64-bit input with deterministic Miller–Rabin witnesses.

A.2 Rust API

Source: `solutions/rust/src/extra_helpers.rs`.

`gcd` `pub fn gcd(mut a: u64, mut b: u64) -> u64`. Computes a greatest common divisor.

`isqrt` `pub fn isqrt(n: u64) -> u64`. Returns the floor of an integer square root.

`prime` `pub fn prime(n: u64) -> bool`. Tests primality by trial division.

`sieve` `pub fn sieve(limit: usize) -> Vec<bool>`. Builds primality flags through an inclusive bound.

`primes` `pub fn primes(limit: usize) -> Vec<u64>`. Provides a reusable mathematical operation.

`decimal_signature` `pub fn decimal_signature(mut n: u64) -> u64`. Provides a decimal-digit operation.

`palindrome` `pub fn palindrome(mut n: u64, base: u64) -> bool`. Provides a reusable mathematical operation.

`digits` `pub fn digits(mut n: u64) -> Vec<u8>`. Provides a decimal-digit operation.

`digits_sum_big` `pub fn digits_sum_big(n: &BigUint) -> usize`. Provides a decimal-digit operation.

`parse_numbers` `pub fn parse_numbers(data: &str) -> Vec<u64>`. Reads or parses a committed problem fixture.

`matrix` `pub fn matrix(data: &str) -> Vec<Vec<u64»`. Reads or parses a committed problem fixture.

`triangle_max` `pub fn triangle_max(mut rows: Vec<Vec<u64») -> u64`. Provides a combinatoric or path operation.

`proper_divisor_sums` `pub fn proper_divisor_sums(limit: usize) -> Vec<usize>`. Provides a number-theory operation.

`totients` `pub fn totients(limit: usize) -> Vec<u32>`. Provides a number-theory operation.

`is_square` `pub fn is_square(n: u64) -> bool`. Provides a reusable mathematical operation.

`pandigital_1_to_9` `pub fn pandigital_1_to_9(mut n: u64) -> bool`. Provides a decimal-digit operation.

`next_permutation` `pub fn next_permutation<T: Ord>(items: &mut [T]) -> bool`. Provides a reusable mathematical operation.

`choose` `pub fn choose(n: u64, k: u64) -> u128`. Computes a binomial coefficient exactly.

`mod_pow` `pub fn mod_pow(mut a: u64, mut e: u64, m: u64) -> u64`. Computes an integer power modulo a modulus.

`big_isqrt` `pub fn big_isqrt(n: &BigUint) -> BigUint`. Provides a reusable mathematical operation.

`fibonacci_pair` `pub fn fibonacci_pair(n: u64) -> (BigUint, BigUint)`. Provides a reusable mathematical operation.

`prime64` `pub fn prime64(n: u64) -> bool`. Tests every 64-bit input with deterministic Miller–Rabin witnesses.

`divisor_count` `pub fn divisor_count(mut n: u64) -> u32`. Provides a number-theory operation.

`divisor_sum` `pub fn divisor_sum(n: u32) -> u32`. Provides a number-theory operation.

`is_palindrome` `pub fn is_palindrome(n: u32) -> bool`. Provides a reusable mathematical operation.

`digit_sum` `pub fn digit_sum(n: &BigUint) -> i64`. Provides a decimal-digit operation.

`digit_power_sum` `pub fn digit_power_sum(mut n: u32, table: &[u32; 10]) -> u32`. Provides a decimal-digit operation.

`distinct_factors` `pub fn distinct_factors(mut n: u32) -> u32`. Provides a number-theory operation.

`pentagonal` `pub fn pentagonal(n: u64) -> bool`. Provides a reusable mathematical operation.

`factorial` `pub fn factorial(n: u32) -> u32`. Provides a number-theory operation.

`least_reciprocal_denominator` `pub fn least_reciprocal_denominator(limit: u64) -> i64`. Provides a reusable mathematical operation.

A.3 Gforth API

Source: `solutions/forth/math.fs`, `solutions/forth/big.fs`.

`require-64bit` `( - )`. Provides a reusable mathematical operation.

`is-prime?` `( n - flag )`. Tests primality by trial division.

`integer-gcd` `( a b - gcd )`. Computes a greatest common divisor.

`divisor-count` `( n - count )`. Provides a number-theory operation.

proper-divisor-sum (n - sum). Provides a number-theory operation.

factorial (n - n!). Provides a number-theory operation.

digit-pow-sum (n power - sum). Provides a decimal-digit operation.

read-fixture (caddr u capacity - buf n). Reads or parses a committed problem fixture.

sieve-build (limit - ptr). Builds primality flags through an inclusive bound.

integer-sqrt (n - root). Returns the floor of an integer square root.

pentagonal? (n - flag). Provides a reusable mathematical operation.

multiply-mod (a b modulo - remainder). Provides a reusable mathematical operation.

power-mod (b exp m - residue). Computes an integer power modulo a modulus.

prefix@ (index address - value). Provides a reusable mathematical operation.

prefix! (value index address -). Provides a reusable mathematical operation.

decimal-digits (n - digits). Provides a decimal-digit operation.

pow10 (n - 10^n). Provides a reusable mathematical operation.

append-number (a b - ab). Provides a reusable mathematical operation.

digit-signature (n - signature). Provides a decimal-digit operation.

pandigital-nine? (n - flag). Provides a decimal-digit operation.

choose (n k - binomial). Computes a binomial coefficient exactly.

triangle-offset (r c - index). Provides a combinatoric or path operation.

fixture-integers (c-addr u capacity - values count). Reads or parses a committed problem fixture.

triangle-path-max (c-addr u rows - maximum). Provides a combinatoric or path operation.

bit-count (n - population). Provides a reusable mathematical operation.

reciprocal-search (index maxexp n divisors -). Provides a reusable mathematical operation.

reciprocal-bound (solutions - minimum n). Provides a reusable mathematical operation.

bsign@ (b - n). Provides base-10000 arbitrary-precision storage or arithmetic.

bsign! (n b -). Provides base-10000 arbitrary-precision storage or arithmetic.

blen@ (b - u). Provides base-10000 arbitrary-precision storage or arithmetic.

blen! (u b -). Provides base-10000 arbitrary-precision storage or arithmetic.

bcap@ (b - u). Provides base-10000 arbitrary-precision storage or arithmetic.

bcap! `( u b - )`. Provides base-10000 arbitrary-precision storage or arithmetic.

bptr@ `( b - a )`. Provides base-10000 arbitrary-precision storage or arithmetic.

bptr! `( a b - )`. Provides base-10000 arbitrary-precision storage or arithmetic.

bnew `( u - b )`. Provides base-10000 arbitrary-precision storage or arithmetic.

bfree `( b - )`. Provides base-10000 arbitrary-precision storage or arithmetic.

bclear `( b - )`. Provides base-10000 arbitrary-precision storage or arithmetic.

blimb@ `( i b - u )`. Provides base-10000 arbitrary-precision storage or arithmetic.

blimb! `( u i b - )`. Provides base-10000 arbitrary-precision storage or arithmetic.

bensure `( u b - )`. Provides base-10000 arbitrary-precision storage or arithmetic.

bnormalize `( b - )`. Provides base-10000 arbitrary-precision storage or arithmetic.

bset-u `( u b - )`. Provides base-10000 arbitrary-precision storage or arithmetic.

bcopy `( a b - )`. Provides base-10000 arbitrary-precision storage or arithmetic.

babscompare `( a b - -1|0|1 )`. Provides base-10000 arbitrary-precision storage or arithmetic.

baddabs! `( a b - )`. Provides base-10000 arbitrary-precision storage or arithmetic.

bsubabs! `( a b - )`. Provides base-10000 arbitrary-precision storage or arithmetic.

badd `( a b out - )`. Provides base-10000 arbitrary-precision storage or arithmetic.

bmul-small! `( u b - )`. Provides base-10000 arbitrary-precision storage or arithmetic.

bmul `( a b out - )`. Provides base-10000 arbitrary-precision storage or arithmetic.

bdiv-small! `( u b - r )`. Provides base-10000 arbitrary-precision storage or arithmetic.

badd-small! `( u b - )`. Provides base-10000 arbitrary-precision storage or arithmetic.

bfrom-dec `( c-addr u b - )`. Provides base-10000 arbitrary-precision storage or arithmetic.

bdecimal-length `( b - u )`. Provides a decimal-digit operation.

bdigitsum `( b - u )`. Provides a decimal-digit operation.

.padded4 `( 0..9999 - )`. Provides a reusable mathematical operation.

bprint `( b - )`. Provides base-10000 arbitrary-precision storage or arithmetic.

A.4 Factor API

Source: `solutions/factor/euler/shared/shared.factor`.

`decimal-digits ( n - digits )`. Provides a decimal-digit operation.

`digit-sum ( n - sum )`. Provides a decimal-digit operation.

`factorial-number ( n - n! )`. Provides a number-theory operation.

`decimal-palindrome? ( n - ? )`. Provides a decimal-digit operation.

`digit-signature ( n - seq )`. Provides a decimal-digit operation.

`pandigital-nine? ( n - ? )`. Provides a decimal-digit operation.

`pandigital-seven? ( n - ? )`. Provides a decimal-digit operation.

`digit-squares ( n - sum )`. Provides a decimal-digit operation.

`pow10 ( n - x )`. Provides a reusable mathematical operation.

`concat-number ( a b - ab )`. Provides a reusable mathematical operation.

`decimal-cell ( str - n )`. Provides a decimal-digit operation.

`whitespace? ( ch - ? )`. Provides a reusable mathematical operation.

`separator? ( ch - ? )`. Provides a reusable mathematical operation.

`read-numbers ( path - seq )`. Reads or parses a committed problem fixture.

`read-signed-numbers ( path - seq )`. Reads or parses a committed problem fixture.

`euler-sieve ( bound - flags )`. Builds primality flags through an inclusive bound.

`divisor-sum-sieve ( bound - arr )`. Provides a number-theory operation.

`phi-sieve ( bound - arr )`. Provides a number-theory operation.

`euler-isqrt ( n - root )`. Returns the floor of an integer square root.

`polygonal? ( n sides - ? )`. Provides a reusable mathematical operation.

`modular-pow ( base exponent modulus - residue )`. Computes an integer power modulo a modulus.

`read-upper-words ( path - words )`. Reads or parses a committed problem fixture.

`max-triangle-path ( path - max )`. Provides a combinatoric or path operation.